

PrimeTime[®]

User Guide

中文学习译本

原版版本：T-2022.03，March 2022

Synopsys, Inc.

原文版权归 Synopsys, Inc. 所有（Version T-2022.03, March 2022）。
本文件为非官方个人学习译本，仅供学习参考，不得用于商业传播。
命令名、选项名、文件名等技术标识符保持英文原文不译。

目录

关于本用户指南	xviii
本版本新增内容	xviii
相关产品、出版物与商标	xviii
排版约定	xix
客户支持	xix
 第一章 PrimeTime 简介	 1
1.1 PrimeTime 能力	1
1.2 PrimeTime 产品层级与许可证	2
1.3 PrimeTime 附加工具	2
1.4 在实现流程中使用 PrimeTime	4
1.5 与 Synopsys 实现工具的兼容性	4
1.6 静态时序分析概览	5
1.6.1 时序路径	5
1.6.2 延时计算	7
1.6.3 约束检查	7
1.6.4 时序例外	9
 第二章 入门	 11
2.1 安装 PrimeTime 软件与许可证	11
2.2 配置 PrimeTime 工作环境	12
2.3 启动 PrimeTime 会话	12
2.4 许可证检出	13
2.5 输入 pt_shell 命令	13
2.6 在命令行获取帮助	14
2.7 PrimeTime 静态时序分析流程	14
2.8 在 PrimeTime 中使用 Tcl/Tk	17
2.8.1 Tcl 包与 Autoload	17
2.8.2 对 incr Tcl 扩展的支持	17
2.9 用 TclPro Toolkit 检查与编译脚本	17
2.9.1 安装 TclPro 工具	18
2.9.2 用 TclPro Checker 检查脚本语法	18
2.9.3 用 TclPro Compiler 创建字节码编译脚本	20

2.9.4	用 TclPro Debugger 调试脚本	20
2.10	限制系统消息	21
2.11	保存与审阅系统消息	21
2.12	结束 PrimeTime 会话	22
2.12.1	保存 PrimeTime 会话	23
2.12.2	退出 PrimeTime 会话	23
2.12.3	命令日志文件	23
第三章	许可证	24
3.1	许可证如何工作	24
3.1.1	许可证链	24
3.1.2	本地进程许可证	25
3.1.3	分布式分析中的许可证管理	26
3.1.4	基于场景的分布式许可证	27
3.1.5	基于核的分布式许可证	27
3.1.6	SMVA 许可证	28
3.2	控制许可证行为	29
3.2.1	手动控制许可证使用	29
3.2.2	增量许可证处理	30
3.2.3	许可证池化	30
3.2.4	许可证排队	30
3.2.5	许可证自动缩减	31
第四章	管理性能与容量	32
4.1	高容量模式选项	32
4.2	高级数据管理	33
4.3	多线程多核分析	33
4.3.1	配置多线程多核分析	33
4.3.2	并行执行命令	34
4.3.3	多线程多核寄生参数读取	37
4.3.4	多线程多核基于路径的分析	37
4.4	分布式多场景分析	37
4.4.1	术语定义	38
4.4.2	DMSA 流程概述	39
4.4.3	分布式处理设置	41
4.4.4	DMSA 批处理模式脚本示例	47
4.4.5	基线映像生成与存储	48
4.4.6	主机资源亲和性	49
4.4.7	场景变量与属性	50
4.4.8	合并报告	51
4.4.9	将场景设计数据加载到管理进程	53
4.4.10	保存与恢复会话	54
4.4.11	许可证资源管理	54

4.4.12	工作进程故障处理	55
4.4.13	消息与日志文件	56
4.4.14	DMSA 变量与命令	58
4.4.15	DMSA 限制	60
4.5	HyperGrid 分布式分析	60
4.5.1	HyperGrid 流程概述	60
4.5.2	准备分布式分析	61
4.5.3	运行分布式分析	62
4.5.4	在 HyperGrid 分析中查询属性	62
4.5.5	保存分布式会话	62
4.5.6	支持 HyperGrid 分布式分析的命令	63
4.5.7	HyperGrid 分布式分析的限制	63
4.6	报告 CPU 时间与内存使用	63
4.7	流程摘要报告	65
4.7.1	流程摘要报告结构	65
4.7.2	生成流程摘要报告	66
4.7.3	定义自定义流程段	67
4.7.4	写入多个流程摘要报告	67
4.7.5	限制	67
4.8	分析 Tcl 脚本性能	67
第五章	处理设计数据	69
5.1	逻辑库	69
5.2	读取与链接设计	70
5.2.1	识别与验证所需设计文件	71
5.2.2	按实例链接库路径	72
5.3	处理设计对象	73
5.4	设置当前设计与当前实例	75
5.5	处理属性	75
5.6	保存属性	76
5.7	保存与恢复会话	76
5.7.1	保存会话	76
5.7.2	保存版本兼容会话	77
5.7.3	在保存的会话中包含 ECO 物理数据	78
5.7.4	恢复会话	79
5.7.5	恢复版本兼容会话	79
5.8	创建测试用例	79
5.8.1	控制保留哪些设计逻辑	80
5.8.2	包含逻辑库	81
5.8.3	指定输出目录	81
5.8.4	创建多实例模块测试用例	81
5.8.5	限制	82

第六章 约束设计	83
6.1 时序约束	83
6.2 输入延迟	84
6.2.1 输入端口同时作为时钟与数据	85
6.3 输出延迟	85
6.4 输入端口驱动特性	86
6.4.1 设置端口驱动单元	86
6.4.2 设置端口驱动电阻	86
6.4.3 设置固定端口转换时间	87
6.4.4 显示驱动信息	87
6.4.5 从端口移除驱动信息	87
6.5 端口电容	87
6.6 线负载模型	88
6.6.1 手动设置线负载模型	88
6.6.2 自动线负载模型选择	89
6.6.3 设置线负载模式	89
6.6.4 报告线负载模型	90
6.7 转换时间传播	90
6.8 设计规则约束	91
6.8.1 最大转换时间	91
6.8.2 库中多阈值与 Derating 值	91
6.8.3 指定最大转换时间约束	92
6.8.4 评估最大转换时间约束	93
6.8.5 最小电容	93
6.8.6 最大电容	93
6.8.7 约束上升与下降最大电容	93
6.8.8 基于频率的最大电容检查	94
6.8.9 含 Case Analysis 的最大电容检查	94
6.8.10 最大扇出负载	94
6.8.11 输出端口扇出负载值	95
6.9 理想网络	95
6.9.1 传播理想网络属性	95
6.9.2 使用理想网络	96
6.9.3 使用理想延迟	96
6.9.4 使用理想转换时间	96
6.9.5 全时钟展开模式中的理想时钟报告	97
6.10 检查约束	97
6.10.1 设计报告	97
6.10.2 使用 check_timing 命令进行约束检查	97
6.10.3 详细约束检查与调试	99

第七章 时钟	101
7.1 时钟概述	101
7.2 指定时钟	102
7.2.1 创建时钟	102
7.2.2 创建虚拟时钟	103
7.2.3 选择时钟对象	103
7.2.4 将命令应用于所有时钟	103
7.2.5 移除时钟对象	103
7.3 指定时钟特性	104
7.3.1 设置时钟延迟	104
7.3.2 设置时钟不确定性	106
7.3.3 设置时钟抖动	107
7.3.4 移除时钟抖动	107
7.3.5 报告时钟抖动	107
7.3.6 估计时钟引脚转换时间	107
7.3.7 检查最小脉冲宽度	108
7.3.8 最小周期检查	109
7.4 使用多个时钟	110
7.4.1 同步时钟	110
7.4.2 异步时钟	111
7.4.3 互斥时钟	111
7.4.4 从分析中移除时钟	113
7.5 时钟极性	114
7.6 指定脉冲时钟	115
7.6.1 约束 Pulse Generator Cell 扇出中的脉冲宽度	116
7.6.2 约束 Pulse Generator Cell 转换时间	116
7.7 PLL 设计时序分析	117
7.7.1 PLL 时序用法	117
7.7.2 PLL 漂移与抖动	118
7.7.3 PLL 路径 CRPR 计算	118
7.7.4 报告与时序检查	118
7.7.5 PLL 库 Cell 要求	118
7.8 指定时钟门控 Setup 与 Hold 检查	118
7.8.1 禁用或恢复时钟门控检查	119
7.9 指定内部生成时钟	119
7.9.1 指定二分频生成时钟	120
7.9.2 基于边沿创建生成时钟	120
7.9.3 为脉冲发生器创建时钟极性	120
7.9.4 基于下降沿创建分频时钟	121
7.9.5 偏移生成时钟边沿	122
7.9.6 源引脚处的多个时钟	122
7.9.7 选择生成时钟对象	122
7.9.8 报告时钟信息	122

7.9.9 移除生成时钟对象	122
7.10 生成时钟边沿特定源延迟传播	123
7.11 时钟网格分析	123
7.11.1 时钟网格分析概述	123
7.11.2 执行时钟网格分析	124
第八章 时序路径与例外	125
8.1 时序路径组	125
8.2 路径时序报告	126
8.3 路径时序计算	127
8.4 指定时序路径	127
8.4.1 多个 Through 参数	127
8.4.2 Rise/Fall From/To Clock	127
8.4.3 无效 Startpoint 或 Endpoint 的报告	129
8.5 时序例外	129
8.5.1 时序例外概述	130
8.5.2 单周期（默认）路径延迟约束	130
8.5.3 设置 False Path	132
8.5.4 设置最大与最小路径延迟	133
8.5.5 设置多周期路径	133
8.5.6 指定时序裕量	135
8.5.7 高效指定例外	135
8.5.8 例外优先级顺序	136
8.5.9 报告例外	137
8.5.10 报告时序例外	137
8.5.11 报告例外源文件与行号信息	138
8.5.12 检查被忽略的例外	138
8.5.13 移除例外	139
8.6 保存与恢复时序路径集合	139
第九章 工作条件	140
9.1 工作条件	140
9.1.1 互连模型类型	141
9.1.2 设置工作条件	142
9.1.3 创建工作条件	142
9.1.4 工作条件信息命令	142
9.2 工作条件分析模式	143
9.3 最小与最大延时计算	144
9.3.1 最小-最大单元与 Net 延时值	145
9.4 指定分析模式	147
9.4.1 单一工作条件分析	147
9.4.2 片上变异分析	148
9.5 用两个库进行分析	149

9.6	延时降额	150
9.6.1	降额选项	150
9.6.2	冲突的降额设置	151
9.6.3	负延时降额	151
9.7	时钟再汇聚悲观移除	151
9.7.1	片上变异示例	152
9.7.2	再汇聚逻辑示例	152
9.7.3	最小脉宽检查示例	153
9.7.4	CRPR 报告	153
9.7.5	公共点不同跳变的 CRPR 降额	153
9.7.6	CRPR 合并阈值	154
9.7.7	CRPR 与串扰分析	154
9.7.8	CRPR 与动态时钟到达	155
9.7.9	透明锁存器边沿考虑	155
9.7.10	报告 CRPR 计算	155
9.8	时钟片上变异悲观缩减	156
第十章	延时计算	157
10.1	延时计算概述	157
10.2	非线性延时模型	158
10.3	复合电流源时序模型	159
10.3.1	Pin 电容报告	159
10.3.2	表征设计规则约束的指南	160
10.3.3	解决 CCS 外推警告 (RC-011)	160
10.4	跨库电压与温度缩放	161
10.4.1	多 rail 电平转换器单元的缩放	161
10.4.2	电压缩放流中的时序降额缩放	161
10.4.3	从多 rail 缩放库组排除 Rail	162
10.5	波形传播	162
10.6	表征触发电平	163
10.7	快速多驱动延时分析	163
10.8	并行驱动器缩减	164
10.8.1	调用并行驱动器缩减	164
10.8.2	使用缩减后的驱动器	165
10.9	多输入开关分析	165
10.9.1	多输入开关分析模式	165
10.9.2	定义库时序弧系数	166
10.9.3	配置多输入开关分析	166
10.9.4	多输入开关分析示例	167
10.9.5	到达窗口重叠检查	167
10.9.6	在 MIS 分析中包含或排除特定单元	168
10.9.7	缩放工具计算的高级 MIS 降额	168
10.10	单位延时分析	168

第十一章 反标	169
11.1 SDF 反标	169
11.1.1 读入 SDF 文件	170
11.2 反标延时、时序检查与转换时间	172
11.3 写出 SDF 文件	173
11.3.1 SDF 映射文件	174
11.3.2 写出压缩 SDF	174
11.3.3 写出无 Setup/Hold 违例的 SDF	174
11.4 设置集总寄生电阻与电容	174
11.4.1 设置 Net 电容	175
11.4.2 设置 Net 电阻	175
11.5 详细寄生	175
11.6 读入寄生文件	176
11.6.1 读入多个寄生文件	176
11.6.2 对寄生数据应用位置变换	176
11.6.3 读入多物理 Pin 的寄生	177
11.6.4 从多 corner 寄生数据读入单 corner	177
11.6.5 检查反标 Net	177
11.6.6 缩放寄生值	177
11.6.7 增量时序分析的寄生读入	178
11.6.8 寄生文件限制	178
11.7 不完整反标寄生	178
11.7.1 为不完整 Net 选择 Wire Load 模型	178
11.7.2 补全 Net 上缺失段	179
11.8 反标优先级	179
11.9 报告反标寄生	179
11.10 移除反标数据	179
11.11 GPD 寄生浏览器	180
11.11.1 使能寄生浏览器	180
11.11.2 寄生电阻与电容集合	180
11.11.3 查询磁盘上的 GPD 数据	180
11.11.4 写出 SDF 的常用选项	181
11.11.5 SDF 映射函数	181
11.11.6 寄生读入缩放与增量分析	181
11.11.7 反标命令示例	181
第十二章 Case 与 Mode 分析	182
12.1 Case 分析	182
12.1.1 控制逻辑值的常量传播	183
12.1.2 配置 Case 分析与常量传播	184
12.1.3 设置与移除 Case 分析值	184
12.1.4 使能经时序单元的 Case 分析传播	185
12.1.5 用 Case 分析评估条件弧	186

12.1.6 用 Case 分析禁用扫描链	187
12.1.7 执行 Case 分析	187
12.1.8 报告被禁用弧并追踪常量传播	188
12.2 Mode 分析	189
12.2.1 设置单元 Mode	189
12.2.2 报告 Mode	192
12.3 用于场景缩减的 Mode 合并	192
12.3.1 运行 Mode 合并	193
12.3.2 调试 Mode 合并冲突	193
第十三章 片上变异	195
13.1 高级片上变异	195
13.1.1 AOCV 流程	196
13.1.2 配置高级片上变异分析	197
13.1.3 导入 AOCV 信息	197
13.1.4 查询时序路径上的 AOCV 降额	201
13.2 参数化片上变异 (POCV)	201
13.2.1 POCV 变量与命令	202
13.2.2 准备 POCV 输入数据	202
13.2.3 导入带物理位置的 SPEF 文件	204
13.2.4 启用 POCV 分析	204
13.2.5 加载 POCV 输入数据	204
13.2.6 指定 Guard Banding	204
13.2.7 缩放 POCV 系数	204
13.2.8 启用约束与 Slew 变异	205
13.2.9 启用基于矩的建模分析	205
13.2.10 报告 POCV 结果	206
13.2.11 统计图合并悲观度	206
13.2.12 查询 POCV Slack 与到达时间属性	206
第十四章 多电压设计流程	207
14.1 多电压分析要求	207
14.2 指定供电连接	208
14.2.1 UPF 指定的供电连接	208
14.2.2 网表推断的供电连接	209
14.3 UPF 命令	209
14.3.1 UPF 供电集 (Supply Sets)	211
14.3.2 UPF 供电集句柄	211
14.3.3 虚拟供电网络	211
14.3.4 设置电压与温度	211
14.3.5 多电压分析的库支持	211
14.3.6 多电压报告与检查	212
14.4 Golden UPF 流程	212

14.5 多供电电压分析	213
14.5.1 多电压分析概述	213
14.5.2 同时多电压分析 (SMVA)	214
14.5.3 IR Drop 标注	214
14.5.4 在时序分析中使用 Ansys RedHawk-SC	215
第十五章 SMVA 基于图的同时多电压分析	216
15.1 SMVA 概述	216
15.1.1 SMVA 要求	217
15.1.2 SMVA 电源域分析	217
15.2 配置 SMVA 分析	217
15.2.1 定义命名 SMVA 电压参考	218
15.3 在 SMVA 分析中报告时序路径	218
15.4 DVFS 场景	219
15.4.1 DVFS 场景概念	219
15.4.2 DVFS 场景集合对象	219
15.4.3 传播的 DVFS 场景	219
15.4.4 用 DVFS 场景控制命令作用域	220
15.4.5 查询 DVFS 场景	220
15.4.6 将 DVFS 场景应用于命令与属性	221
15.5 将 SMVA 与其他 PrimeTime 特性配合使用	222
15.5.1 与 OCV/AOCV/POCV 配合	222
15.5.2 SMVA 流程中的电压缩放时序降额	222
15.5.3 与多场景分析 (DMSA) 配合	222
15.5.4 与电平转换器、隔离与保持	223
15.5.5 与 IR Drop 标注配合	223
15.5.6 与信号完整性分析配合	223
15.5.7 与路径分析配合	223
15.6 使用示例	223
15.6.1 两域双电压 SMVA	223
15.6.2 带 DVFS 场景的报告	224
15.6.3 跨域路径过滤	224
15.7 支持 DVFS 场景的命令与属性	224
15.8 特性兼容性	225
第十六章 信号完整性分析	226
16.1 信号完整性与串扰概述	226
16.1.1 串扰延时效应	226
16.1.2 Delta 延时与扇出级效应	227
16.1.3 串扰噪声效应	227
16.2 Aggressor 与 Victim 网	228
16.2.1 时序窗口与串扰延时分析	228
16.2.2 交叉耦合模型	229

16.3 串扰延时分析	229
16.3.1 执行串扰延时分析	229
16.3.2 PrimeTime SI 如何工作	230
16.3.3 PrimeTime SI 变量	231
16.3.4 逻辑相关性	231
16.3.5 电气滤波	232
16.3.6 使用指南	232
16.3.7 包含或排除特定网的串扰分析	232
16.3.8 Victim-Aggressor 对齐与窗口模式	234
16.3.9 「All Paths」与「All Path Edges」重叠模式	234
16.3.10 时钟组	235
16.3.11 数据网络跨时钟域 Delta 延时	236
16.3.12 时序报告	236
16.3.13 报告串扰设置	236
16.3.14 双开关检测	236
16.4 静态噪声分析	237
16.4.1 静态噪声分析概述	237
16.4.2 噪声凸起特性	238
16.4.3 PrimeTime SI 噪声分析流程	238
16.4.4 噪声分析命令	239
16.4.5 使用 CCS 噪声数据的噪声建模	239
16.4.6 使用非线性延时模型 (NLDM) 的噪声建模	239
16.4.7 噪声免疫与 Slack 计算	239
16.4.8 传播噪声特性	240
16.4.9 多 Aggressor 噪声合成	241
16.4.10 噪声端点过滤与传播限制	241
16.4.11 报告与调试噪声	241
16.4.12 修复噪声违例	241
第十七章 高级分析技术	243
17.1 并行弧路径追踪	243
17.2 对 Retain 弧的支持	244
17.3 异步逻辑分析	244
17.4 三态总线分析	246
17.5 数据到数据检查	249
17.6 基于锁存器设计中的时间借用	252
17.7 高级锁存器分析	254
第十八章 约束一致性	258
18.1 约束一致性概述	258
18.2 设计一致性检查	265
18.3 层次一致性检查	266
18.4 相关性一致性检查	267

18.5 附加分析特性	268
18.6 图形用户界面	270
18.7 教程	272
第十九章 报告与调试分析结果	275
19.1 全局时序摘要报告	275
19.2 路径时序报告	277
19.3 结果质量报告	278
19.4 约束报告	278
19.5 瓶颈报告	279
19.6 全局 slack 报告	280
19.7 分析覆盖率报告	280
19.8 时钟网络时序报告	281
19.9 时钟门控与 recovery/removal 检查	282
19.10 时序更新效率	282
19.11 基于路径的时序分析	283
第二十章 图形用户界面	286
20.1 打开与关闭 GUI	286
20.1.1 打开 GUI	287
20.1.2 关闭 GUI	287
20.2 快速入门指南	287
20.3 GUI 窗口	288
20.3.1 视图窗口	288
20.3.2 视图设置面板	289
20.3.3 工具栏与面板	289
20.3.4 层次浏览器	289
20.3.5 控制台	290
20.3.6 对象选择	291
20.3.7 选择时序路径	291
20.3.8 设置 GUI 首选项	291
20.4 分析时序路径集合	291
20.4.1 加载路径集合	291
20.4.2 对时序路径分类	292
20.4.3 将路径类别保存到文件	292
20.4.4 从文件加载路径类别	292
20.4.5 标记块并应用块分类规则	293
20.4.6 创建自定义分类规则	293
20.5 在抽象时钟图中检查时钟路径	293
20.5.1 打开抽象时钟图视图	293
20.5.2 显示与隐藏时钟路径元素	294
20.5.3 撤销与重做更改	294
20.5.4 折叠选定侧分支	294

20.5.5 查看时钟延迟随时间变化	294
20.5.6 层次化显示时钟树	295
20.6 分析时钟域与时钟间关系	295
20.7 在原理图中检查时序路径与设计逻辑	297
20.7.1 检查层次单元	297
20.7.2 在设计层次中下移或上移	298
20.7.3 显示或隐藏缓冲器与反相器	298
20.7.4 展开或折叠总线	298
20.8 查看与修改原理图	298
20.9 检查路径元素	298
20.10 查看对象属性	299
20.10.1 查看当前选择	299
20.10.2 管理属性组	299
20.11 重新计算时序路径	299
20.11.1 比较正常与重新计算路径	300
20.11.2 比较正常与重新计算路径 pin	300
20.12 布局视图与 GUI 中的 ECO	300
20.13 时序路径的交互式多场景分析	302
20.13.1 交互式多场景分析流程	303
20.14 GUI Tcl 命令	305
第二十一章 ECO 流程	306
21.1 ECO 修复概述	306
21.1.1 DRC、串扰与单元电迁移违例修复	307
21.1.2 时序违例修复	308
21.1.3 功耗回收修复	308
21.1.4 ECO 修复步骤顺序	308
21.2 设置 ECO 选项	308
21.2.1 多库配置	309
21.2.2 电源管理单元尺寸调整	309
21.3 物理感知 ECO	309
21.3.1 执行物理感知 ECO	310
21.3.2 站点感知物理 ECO 修复	310
21.3.3 多电压物理约束	311
21.4 ECO 修复方法	311
21.4.1 负载缓冲与负载屏蔽	311
21.4.2 时钟网络 ECO 修复	312
21.4.3 TNS 驱动的时钟网络时序 ECO	312
21.4.4 使用负载电容单元进行 Hold 修复	313
21.4.5 基于库单元名的功耗回收	313
21.4.6 基于库单元字符串属性的功耗回收	313
21.4.7 用户脚本 ECO 机器学习	313
21.4.8 MIM 的 ECO	313

21.5 报告不可修复违例与不可用单元	313
21.6 HyperTrace ECO 修复	314
21.7 写入变更列表	314
21.8 在 Synopsys 布局工具中实现 ECO 更改	315
21.9 使用 Synopsys 工具的增量 ECO 流程	315
21.9.1 初始化增量 ECO 流程	315
21.9.2 增量 ECO 迭代	316
21.9.3 层次化增量 ECO 流程	316
21.10 使用缩减资源的 ECO 流程	316
21.10.1 缩减资源非 DMSA ECO 流程	317
21.11 Freeze Silicon ECO 流程	317
21.11.1 准备 Freeze Silicon ECO 流程	317
21.11.2 PrimeTime 中的 Freeze Silicon ECO	317
21.12 在较少主机上运行 ECO 场景	317
21.12.1 优化 DMSA ECO 脚本以加快周转	317
21.13 手动网表编辑	317
21.13.1 GUI 中的单元尺寸调整与缓冲器插入	318
21.13.2 fix_eco_timing 选项详解	318
21.13.3 fix_eco_drc 选项详解	318
21.13.4 fix_eco_power 选项详解	319
21.13.5 set_eco_options 物理选项	319
21.13.6 物理 ECO 条件变量	319
21.13.7 DEF 到 LEF Site 名称转换	319
21.13.8 层次块缺失 LEF 文件	319
21.13.9 高级间距标签与规则	320
21.13.10 StarRC 寄生数据文件	320
21.13.11 单场景与多场景功耗回收示例	320
21.13.12 功耗回收操作统计	320
21.13.13 estimate_eco 命令	320
21.13.14 write_changes 格式选项	320
21.13.15 ECO 变更列表版本与解析	321
21.13.16 层次化增量 ECO	321
21.13.17 缩减资源 DMSA ECO	321
21.13.18 Freeze Silicon 技术规则	321
21.13.19 ICC II Freeze Silicon 准备	321
21.13.20 手动插入缓冲器示例	321
21.13.21 ECO 与 UPF	322
第二十二章 层次化分析	323
22.1 层次化分析概述	323
22.1.1 何时使用扁平与层次化分析	324
22.2 HyperScale 分析	324
22.2.1 HyperScale 关键技术特性	325

22.2.2	HyperScale 使用流程	325
22.2.3	选择自顶向下或自底向上流程	328
22.2.4	自顶向下 HyperScale 流程	328
22.2.5	自底向上 HyperScale 流程	329
22.2.6	HyperScale 脚本与块 Context 自动生成	329
22.2.7	HyperScale 配置	330
22.2.8	HyperScale 会话数据	330
22.2.9	HyperScale 使用细节	330
22.2.10	模型与 Context 不匹配处理	330
22.2.11	块级分析	331
22.2.12	链接设计	331
22.2.13	控制裕量应用方式	331
22.2.14	层次化约束提取	331
22.2.15	Context 覆盖	332
22.2.16	控制块边界调整	332
22.2.17	HyperScale 属性	333
22.2.18	HyperScale 数据目录结构	333
22.2.19	为 HyperScale 块写出网表与约束	333
22.2.20	HyperScale ECO 收敛方法	333
22.2.21	HyperScale 中的时序分析	333
22.2.22	路径特定例外建模	334
22.3	Context 特征化	334
22.3.1	将 Context 用于块级时序分析	334
22.3.2	Context 特征化命令与选项	335
22.4	Context 预算	335
22.4.1	Budgeting 期间如何调整 Context	335
22.4.2	报告 Context 预算	335
22.4.3	自定义 Context 预算	336
22.4.4	写出预算后的 Context	336
22.4.5	在 HyperScale 流程中使用 Context 预算	336
22.4.6	预算规范优先级	337
22.5	提取时序模型 (ETM)	337
22.5.1	ETM 使用流程	337
22.5.2	ETM 流程概述	338
22.5.3	生成模型	338
22.5.4	时序模型提取细节	338
22.5.5	控制提取表大小	339
22.5.6	指定模型索引与裕量	340
22.5.7	抽头输出端口	340
22.5.8	时序例外与生成时钟	340
22.5.9	提取内部 Pin 与检查 Pin	340
22.5.10	跨块内电源域合并模型	340
22.5.11	保守块时序分析与模型提取	341

22.5.12	ETM 的 SI 与串扰建模	342
22.5.13	模型验证	342
22.5.14	ETM 的噪声特性	343
22.5.15	块内部路径与接口路径	343
22.6	快速时序模型 (QTM)	343
22.6.1	创建 QTM	343
22.6.2	创建 QTM 的脚本示例	344
22.6.3	在顶层使用 QTM	344
22.6.4	QTM 与 ETM 对比	344
22.6.5	从 ETM 层次化流程迁移	345
22.6.6	Context 特征化 workflow 详解	346
22.6.7	Context 文件格式	346
22.6.8	多块 Context 特征化	346
22.6.9	Budgeting 流程示例	347
22.6.10	Budgeting 与层次化块模型	347
22.6.11	ETM 分析流程详解	347
22.6.12	ETM 表与变量控制	348
22.6.13	ETM 裕量与角	348
22.6.14	工作条件与分析模式	349
22.6.15	包装设计与验证	349
22.6.16	ETM 的 Derating 与 AOCV	349
22.6.17	层次化 Signoff 使用 ETM 的最佳实践	349
22.6.18	定义快速时序模型	349
22.6.19	在设计中实例化 QTM	350
22.6.20	QTM 全局参数命令	350
22.6.21	HyperScale ECO 流程细节	350
22.6.22	HyperScale 路径类型标记	351
22.6.23	跨块内电源域合并 ETM	351
22.6.24	QTM 约束弧类型	351
22.6.25	Context 预算模式	351
22.6.26	ETM 与 CCS 数据提取	351
22.6.27	层次化分析选型指南	351
22.6.28	HyperScale 块 Context 数据类型	352
22.6.29	时钟映射检查与修复	352
22.6.30	ETM 输入到寄存器与寄存器到输出路径	352
22.6.31	ETM 透明锁存器与时间借用	353
22.6.32	ETM 模型验证流程	353
22.6.33	QTM 驱动类型与负载	353
22.6.34	Context 预算自定义示例	353
22.6.35	HyperScale 与 SI 分析	354
22.6.36	report_hier_analysis 报告	354
22.6.37	块边界自动调整机制	354
22.6.38	ETM 噪声与功耗选项	354

22.6.39 QTM 与 HyperScale/ETM 选型	354
第二十三章 将 PrimeTime 与 SPICE 配合使用	355
23.1 Simulation Link: 关联 PrimeTime 与 SPICE 结果	355
23.1.1 Simulation Link 命令摘要	356
23.1.2 关联基于路径的非耦合 PrimeTime 与 SPICE 分析	356
23.1.3 关联基于弧的耦合 PrimeTime SI 与 SPICE 分析	358
23.1.4 关联 PrimeTime SI 与 SPICE 噪声分析	360
23.1.5 在多台机器上分布仿真	360
23.2 使用 write_spice_deck 命令生成 SPICE 网表	361
23.2.1 为时序路径编写 SPICE 网表	361
23.2.2 为时序弧编写 SPICE 网表	362
23.2.3 write_spice_deck 中的库激励	363
23.2.4 库驱动波形	364
23.2.5 SPICE 仿真所需的其他信息	365
23.2.6 write_spice_deck 输出示例	365
23.2.7 使用 write_spice_deck 进行 SPICE 关联的限制	367
第二十四章 将 PrimeTime 与 Design Compiler 配合使用	368
24.1 PrimeTime 特有功能	368
24.2 与 Design Compiler 的时序分析差异	368
24.2.1 路径	369
24.2.2 转换时间	369
24.2.3 current_design 命令	369
24.3 共享 Design Compiler 与 PrimeTime 脚本	370
24.4 Synopsys Design Constraints Format (SDC) 脚本文件	371
24.4.1 检查 SDC 文件语法	371
24.4.2 检查单位一致性	371
24.5 Design Compiler 与 PrimeTime 中的路径组	371
附录 A 已弃用功能	372
A.1 HyperScale 分布式分析	372
A.1.1 从全芯片 Flat 分析到 HyperScale 分布式分析	372
A.1.2 使用恢复会话的类 Flat 顶层报告	376
A.1.3 从分布式分析到块级分析	377
A.1.4 分布式 HyperScale 分析中的 MIM 合并	378
A.1.5 HyperScale 属性	379
A.2 旧版基于路径的 SMVA	379
附录 B 术语表	384
B.1 术语条目	384

关于本用户指南

About This User Guide

Synopsys PrimeTime Suite 在单一集成环境中提供全面的全芯片静态时序分析、信号完整性分析与全芯片功耗分析。

本指南面向使用 PrimeTime 工具进行静态时序与信号完整性分析的工程师。读者应对静态时序分析原理有一定了解。

PrimeTime Suite 文档包括以下文档：

- *PrimeTime User Guide*（本用户指南）——使用 PrimeTime、PrimeTime SI、PrimeTime ADV 与 PrimeTime ADVP 工具进行时序分析。
- *PrimeECO User Guide*——基于签核质量时序进行 ECO 时序、功耗与 DRC 修复。
- *PrimePower User Guide*——静态与动态全芯片功耗分析。
- *PrimeShield User Guide*——参数化时序鲁棒性分析、单元鲁棒性分析，以及关键路径快速蒙特卡罗 HSPICE 仿真。

本前言包括以下小节：

- 本版本新增内容
- 相关产品、出版物与商标
- 排版约定
- 客户支持

本版本新增内容

关于新功能、增强与变更、已知限制，以及已解决的 Synopsys Technical Action Requests（STAR）信息，请参阅 SolvNetPlus 站点上的 *PrimeTime Release Notes*。

相关产品、出版物与商标

有关 PrimeTime 工具的更多信息，请参阅 Synopsys SolvNetPlus 支持站点上的文档：<https://solvnetplus.synopsys.com>

您可能还希望参阅以下相关 Synopsys 产品文档：

- Design Compiler®
- IC Compiler™ 与 IC Compiler™ II

- Library Compiler™
- NanoTime
- SiliconSmart®
- StarRC™

排版约定

Synopsys 文档使用下列约定。

表 1: 文档排版约定

约定	说明
Courier	表示语法，例如 <code>write_file</code>
Courier italic	表示语法中的用户自定义值，例如 <code>write_file design_list</code>
Courier bold	表示示例中用户原样输入的内容，例如 <code>prompt> write_file top</code>
紫色	• 在示例中表示特别关注的信息； • 在命令语法中表示默认值，例如 <code>include_enclosing = true false</code>
[]	表示语法中的可选参数，例如 <code>write_file [-format fmt]</code>
...	表示参数可按需重复，例如 <code>pin1 pin2 ... pinN</code>
	表示在多个备选中选择其一，例如 <code>low medium high</code>
\	表示命令行续行
/	表示目录层次
粗体	表示带有关联操作的图形用户界面（GUI）元素
Edit > Copy	表示菜单命令路径，例如打开 Edit 菜单并选择 Copy
Ctrl+C	表示组合键，例如按住 Ctrl 再按 C

客户支持

客户支持通过 SolvNetPlus 提供。

访问 SolvNetPlus

SolvNetPlus 站点包含技术文章知识库以及对 Synopsys 工具常见问题的解答，并可访问软件下载、文档与技术支持等在线服务。

访问地址：<https://solvnetplus.synopsys.com>

如提示，请输入用户名与密码。若尚无 Synopsys 账号，请按页面说明注册。若使用 SolvNetPlus 时需要帮助，请点击右上角菜单栏中的 **REGISTRATION HELP**。

联系客户支持

请访问 <https://solvnetplus.synopsys.com>。

Chapter 1

PrimeTime 简介

Introduction to PrimeTime

PrimeTime Suite 执行全芯片、门级静态时序分析（static timing analysis, STA），这是芯片设计与分析流程中的关键环节。该工具通过检查所有路径是否存在时序违例，对设计的时序性能进行穷尽验证，无需逻辑仿真或测试向量。要了解 PrimeTime 工具的基础内容，请参阅：

- PrimeTime 能力
- PrimeTime 产品层级与许可证
- PrimeTime 附加工具
- 在实现流程中使用 PrimeTime
- 与 Synopsys 实现工具的兼容性
- 静态时序分析概览

1.1 PrimeTime 能力

PrimeTime Capabilities

在 PrimeTime 中，可执行多种设计检查与分析：

- 设计检查（**Design Checks**）
 - Setup、hold、recovery 与 removal 约束
 - 用户指定的 data-to-data 时序约束
 - 时钟门控（clock-gating）的 setup 与 hold 约束
 - 时钟的最小周期（minimum period）与最小脉冲宽度（minimum pulse width）
 - 设计规则（最小/最大 transition time、capacitance 与 fanout）
- 分析功能（**Analysis Features**）
 - 多时钟与多时钟频率

- 伪路径（false path）与多周期路径（multicycle path）时序例外
- 透明锁存器（transparent latch）分析与时间借用（time borrowing）
- 对 setup 与 hold 约束进行同时的最小与最大延时分析
- 在工艺、电压与温度（PVT）条件存在片上变异（on-chip variation, OCV）时进行分析
- 对指定输入施加常量或特定跳变的 case analysis
- 按模块特定工作模式（如 RAM 模块的读模式或写模式）进行的 mode analysis
- 瓶颈分析（bottleneck analysis），报告导致最多时序违例的单元
- 物理相邻网络之间的串扰（crosstalk）分析
- 使用 HyperScale 技术的高效层次化分析
- 全面的约束一致性检查（constraint consistency checking）
- 工程变更指令（ECO）分析：通过插入缓冲器、调整单元尺寸与交换单元来纠正违例并优化功耗

1.2 PrimeTime 产品层级与许可证

PrimeTime Product Tiers and Licenses

PrimeTime 产品提供三个层级：PrimeTime Base、PrimeTime Elite 与 PrimeTime Apex。

表 1.1 列出各产品层级所包含的许可证。较新功能使用 PrimeTime-ELT 与 PrimeTime-APX 许可证；较旧功能使用其他许可证。

表 1.1: PrimeTime 产品打包层级所含许可证

PrimeTime Base	PrimeTime Elite	PrimeTime Apex
PrimeTime	PrimeTime	PrimeTime × 2
PrimeTime-SI	PrimeTime-SI	PrimeTime-SI × 2
	PrimeTime-ADV	PrimeTime-ADV × 2
	PrimeTime-ADV-PLUS	PrimeTime-ADV-PLUS × 2
	PrimeTime-ELT	PrimeTime-ELT × 2
		PrimeTime-APX

表 1.2 列出各 PrimeTime 产品层级所含功能，以及启用该功能所需的许可证。

1.3 PrimeTime 附加工具

PrimeTime Add-On Tools

PrimeTime Suite 支持以下附加功能：

- **PrimeECO**——支持工程变更指令（ECO）实现与分析：

表 1.2: PrimeTime 产品层级与所含功能

功能（及许可证）	Base	Elite	Apex
多核处理（Multicore processing）	X（16 核）	X（32 核）	X（64 核）
信号完整性分析（PrimeTime-SI）	X	X	X
静态噪声分析（PrimeTime-SI）	X	X	X
约束一致性（PrimeTime-SI）	X	X	X
以二进制格式写入与读取 Context（PrimeTime-SI）	X	X	X
IR Drop Annotation（PrimeTime-SI）	X	X	X
用 write_spice_deck 生成 SPICE Deck（PrimeTime-SI）	X	X	X
参数化片上变异 POCV（PrimeTime-ADV）		X	X
多输入开关分析 Multi-Input Switching（PrimeTime-ADV）		X	X
物理感知 ECO（PrimeTime-ADV）		X	X
基于核的分布式许可证 Distributed Core-Based Licensing（PrimeTime-ADV）		X	X
在 PrimeTime 中回放 ECO Change List（PrimeTime-ADV）		X	X
采用机器学习的穷尽基于路径分析（PrimeTime-ADV-PLUS）		X	X
HyperTrace 加速基于路径分析（PrimeTime-ADV-PLUS）		X	X
同时多电压分析 SMVA（PrimeTime-ADV-PLUS）		X	X
采用基于矩（moment-based）建模的 POCV（PrimeTime-ADV-PLUS）		X	X
含 Via Variation 的 POCV（PrimeTime-ADV-PLUS）		X	X
多输入开关分析——lib_arc 与高级模式（PrimeTime-ADV-PLUS）		X	X
采用机器学习的加速功耗回收（PrimeTime-ADV-PLUS）		X	X
GUI 中的 Layout View 与 ECO（PrimeTime-ADV-PLUS）		X	X
GPD Parasitic Explorer（PrimeTime-ADV-PLUS）		X	X
将 Ansys RedHawk-SC 用于时序分析（PrimeTime-ADV-PLUS）		X	X
HyperGrid 分布式分析（PrimeTime-ADV-PLUS）		X	X
高级数据管理 Advanced Data Management（PrimeTime-ELT）		X	X
Context Budgeting（PrimeTime-APX）			X

- PrimeECO 附加工具将 PrimeTime、IC Compiler II 与 StarRC 的时序分析、物理实现与寄生提取功能整合为单一集成工具。它可执行多轮 ECO 修复迭代，无需在迭代之间暂停。更多信息见 *PrimeECO User Guide*。
- **PrimePower**——支持高级功耗分析：
 - PrimePower 附加工具分析基于单元设计的全芯片功耗耗散（power dissipation）。支持无向量与基于向量的峰值功耗及平均功耗分析。更多信息见 *PrimePower User Guide*。
- **PrimeShield**——支持参数化鲁棒性分析：
 - 在先进工艺节点下，面对日益增大的工艺与电压变异性，PrimeShield 提供设计鲁棒性分析与修复，帮助设计者减少悲观、过度裕量与过度设计，从而有效降低功耗并提升最高工作频率。更多信息见 *PrimeShield User Guide*。

1.4 在实现流程中使用 PrimeTime

Using PrimeTime in the Implementation Flow

PrimeTime 工具可与 Design Compiler、IC Compiler II、Fusion Compiler 与 StarRC 等其他 Synopsys 工具良好协同。这些工具共享许多相同的库、数据库与命令，如下图所示。

（原书 *Figure 1: Implementation Flow With PrimeTime Analysis*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 1.1: 含 PrimeTime 分析的实现流程

从 RTL 设计出发，Design Compiler 生成门级设计。基于门级网表与物理库信息，物理实现工具（IC Compiler II）执行布局与布线。StarRC 寄生提取工具从芯片版图数据库提取寄生 RC 数据。

PrimeTime 读取门级网表与寄生数据，并利用逻辑库（.db）中的信息验证设计时序。若发现违例，可生成工程变更指令（ECO），指导物理实现工具修复违例并优化功耗。

PrimeTime 也可在其他设计流程中作为独立静态时序分析器运行。

1.5 与 Synopsys 实现工具的兼容性

Compatibility With Synopsys Implementation Tools

PrimeTime 静态时序分析工具旨在与 Design Compiler 综合工具以及 IC Compiler II 布局布线工具良好协同。兼容性体现在：

- 使用相同的逻辑库，并读取相同的设计数据文件；
- 支持 Synopsys Design Constraints（SDC）格式以指定设计约束（含时序与面积约束）；

- 共享许多命令（如 `create_clock`、`set_input_delay` 与 `report_timing`），操作相同或相似；
- 共享相同的延时计算算法，产生相似的延时结果；
- 生成相似的时序报告；
- PrimeTime 可通过生成 change list 向 IC Compiler II 提供 ECO 指导，以修复时序违例并优化功耗。

尽管 Design Compiler 与 IC Compiler II 自身具备内置静态时序分析能力，PrimeTime 在速度、精度、容量与灵活性方面更优，并提供其他工具所不支持的许多功能。

参见

在 Design Compiler 中使用 PrimeTime，见第 二十四 章。

1.6 静态时序分析概览

Overview of Static Timing Analysis

静态时序分析是一种通过检查所有可能路径是否存在时序违例，来验证设计时序性能的方法。PrimeTime 将设计分解为时序路径，计算每条路径上的信号传播延时，并检查设计内部以及输入/输出接口处的时序约束违例。

另一种时序分析方法是动态仿真（dynamic simulation）：对给定输入激励向量集合，确定电路的完整行为。与动态仿真相比，静态时序分析快得多，因为不必仿真电路的逻辑操作；也更彻底，因为它检查所有时序路径，而不仅是特定测试向量所敏化的逻辑条件。但静态时序分析只能检查电路设计的时序，不能检查功能。

1.6.1 时序路径

Timing Paths

进行时序分析时，PrimeTime 首先将设计分解为时序路径。每条时序路径由以下元素组成：

- **Startpoint**（起点）——时序路径起点，数据由时钟边沿发射，或数据必须在特定时刻可用。每个起点必须是输入端口或寄存器时钟引脚。
- **Combinational logic network**（组合逻辑网络）——无存储或内部状态的元件。可包含 AND、OR、XOR 与反相器等，但不能包含触发器、锁存器、寄存器或 RAM。
- **Endpoint**（终点）——时序路径终点，数据由时钟边沿捕获，或数据必须在特定时刻可用。每个终点必须是寄存器数据输入引脚或输出端口。

下图给出简单设计示例中的时序路径。

示例中，每个逻辑云表示一个组合逻辑网络。每条路径从数据发射点出发，经过若干组合逻辑，止于数据捕获点。

一个组合逻辑云可能包含多条路径，如下图所示。PrimeTime 用最长路径计算最大延时，用最短路径计算最小延时。

PrimeTime 还考虑以下类型的路径用于时序分析：

- **Clock path**（时钟路径）——从时钟输入端口或单元引脚出发，经过一个或多个缓冲器或反相器，到达时序元件时钟引脚的路径；用于数据 setup 与 hold 检查。

(原书 *Figure 2: Timing paths*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 1.2: 时序路径

表 1.3: 时序路径示例

路径	起点	终点
Path 1	输入端口	时序元件的数据输入
Path 2	时序元件的时钟引脚	时序元件的数据输入
Path 3	时序元件的时钟引脚	输出端口
Path 4	输入端口	输出端口

(原书 *Figure 3: Multiple paths through combinational logic*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 1.3: 组合逻辑中的多条路径

- **Clock-gating path**（时钟门控路径）——从输入端口到时钟门控元件的路径；用于时钟门控 setup 与 hold 检查。
- **Asynchronous path**（异步路径）——从输入端口到时序元件异步置位或清零引脚的路径；用于 recovery 与 removal 检查。

（原书 *Figure 4: Types of paths considered for timing analysis*）
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 1.4: 时序分析所考虑的路径类型

1.6.2 延时计算

Delay Calculation

将设计分解为时序路径集合后，PrimeTime 计算每条路径上的延时。路径总延时为路径中所有单元延时与网络延时之和。

单元延时

Cell Delay 单元延时是路径中逻辑门从输入到输出的延时量。在没有来自 SDF 文件的反标延时信息时，工具根据该单元逻辑库中的延时表计算单元延时。

通常，延时表将延时量列为一个或多个变量（如输入 **transition time** 与输出负载电容）的函数。工具根据这些表项计算各单元延时。必要时，PrimeTime 对表值进行插值或外推，以得到当前设计条件下指定的延时值。

网络延时

Net Delay 网络延时是时序路径中从一个单元输出到下一单元输入的延时量。该延时由两单元之间互连的寄生电容、网络电阻以及驱动该网络的单元有限驱动能力共同引起。

PrimeTime 用以下方法计算网络延时：

- 根据线负载模型（wire load model）估算延时——布局前芯片拓扑未知时使用；
- 使用从 Standard Delay Format（SDF）文件反标的具体时间值；
- 使用从 Galaxy Parasitic Database（GPD）、Standard Parasitic Exchange Format（SPEF）、Detailed Standard Parasitic Format（DSPF）或 Reduced Standard Parasitic Format（RSPF）文件反标的详细寄生电阻与电容数据。

1.6.3 约束检查

Constraint Checks

确定时序路径并计算路径延时后，PrimeTime 检查时序约束违例，例如 setup 与 hold 约束：

- **Setup** 约束规定：在捕获数据的时钟边沿之前，数据必须提前多少时间在时序器件输入端可用。该约束对数据路径相对于时钟边沿施加最大延时限制。
- **Hold** 约束规定：在捕获数据的时钟边沿之后，数据必须在时序器件输入端保持稳定多久。该约束对数据路径相对于时钟边沿施加最小延时限制。

除 setup 与 hold 外，PrimeTime 还可检查 recovery 与 removal 约束、data-to-data 约束、时钟门控 setup 与 hold 约束，以及时钟信号的最小脉冲宽度。

避免违例的时间裕量称为 **slack**（时序裕量）。例如，对 setup 约束，若信号须不晚于 8 ns 到达单元输入，而实际到达时间为 5 ns，则 slack 为 3 ns。Slack 为 0 表示约束刚好满足；负 slack 表示存在时序违例。

触发器的 Setup 与 Hold 检查示例

Example of Setup and Hold Checks for Flip-Flops 下例说明 PrimeTime 如何对触发器检查 setup 与 hold 约束。

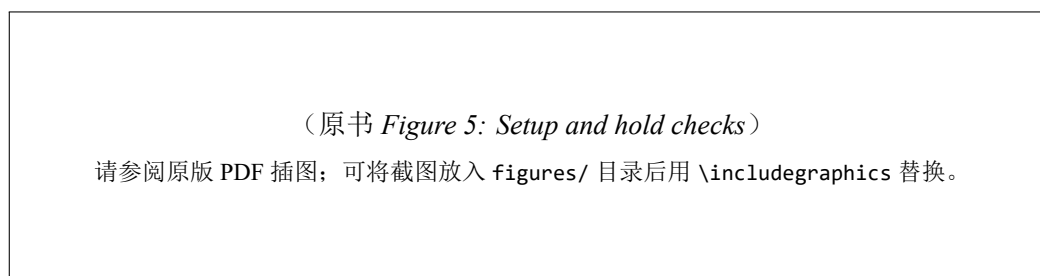


图 1.5: Setup 与 hold 检查

本例假设逻辑库中定义触发器最小 setup 时间为 1.0 时间单位，最小 hold 时间为 0.0 时间单位。工具中定义时钟周期为 10 时间单位。时间单位大小（如 ns 或 ps）在逻辑库中指定。

默认情况下，工具假定信号在一个时钟周期内沿每条数据路径传播。因此，执行 setup 检查时，工具验证从 FF1 发射的数据在一个时钟周期内到达 FF2，并至少提前 1.0 时间单位到达，以便被 FF2 的下一时钟边沿捕获。若数据路径延时过长，则报告为时序违例。对该 setup 检查，工具考虑数据路径上可能的最长延时，以及 FF1 与 FF2 之间时钟路径上可能的最短延时。

执行 hold 检查时，工具验证从 FF1 发射的数据不早于上一时钟周期的捕获时钟边沿到达 FF2。这确保已存在于 FF2 输入端的数据在捕获上一周期数据的时钟边沿之后保持足够稳定。对该 hold 检查，工具考虑数据路径上可能的最短延时，以及 FF1 与 FF2 之间时钟路径上可能的最长延时。若时钟路径延时过长，可能发生 hold 违例。

锁存器的 Setup 与 Hold 检查示例

Example of Setup and Hold Checks for Latches 基于锁存器的设计通常使用两相、非重叠时钟控制数据路径中的相继寄存器。此时，PrimeTime 可利用时间借用（time borrowing）减轻对相继路径的约束。

例如，考虑下图所示的两相、基于锁存器的路径。三个锁存器均为电平敏感，当 G 输入为高电平时门有效。L1 与 L3 由 PH1 控制，L2 由 PH2 控制。上升沿从锁存器输出发射数据，下降沿在锁存器输入捕获数据。本例中假设锁存器 setup 时间与延时时间均为零。

(原书 *Figure 6: Latch-based paths*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 1.6: 基于锁存器的路径

对从 L1 到 L2 的路径，PH1 上升沿发射数据。数据必须在时间 = 20 的 PH2 下降沿之前到达 L2。该时序要求标记为 **Setup 1**。取决于 L1 与 L2 之间的延时量，数据可能在时间 = 10 的 PH2 上升沿之前或之后到达，时序图中用虚线箭头“Arrival a”与“Arrival b”表示。在时间 = 20 之后到达即为时序违例。

若数据在时间 = 10 的 PH2 上升沿之前到达 L2 (Arrival a)，则下一条从 L2 到 L3 的路径数据由时间 = 10 的 PH2 上升沿发射，其工作方式如同同步触发器。此时不从第二条路径借用时间。L2 到 L3 的该时序要求标记为 **Setup 2a**。

若数据在 PH2 上升沿之后到达 (Arrival b)，则第一条路径 (L1 到 L2) 从第二条路径 (L2 到 L3) 借用时间。此时第二条路径的数据发射不发生在上升沿，而发生在 L2 的数据到达时刻，介于 PH2 上升沿与下降沿之间。该时序要求标记为 **Setup 2b**。发生借用时，路径起点为 L2 的 D 引脚而非 G 引脚。

对第一条路径 (L1 到 L2)，若发生借用，PrimeTime 报告 **setup slack** 为零；若数据在时间 = 10 的上升沿之前到达，**slack** 为正；若在时间 = 20 的下降沿之后到达，**slack** 为负 (违例)。

执行 hold 检查时，PrimeTime 相对于 **setup** 检查考虑发射与捕获边沿，如下图所示。它验证起点发射的数据不会过快到达终点，从而确保上一周期发射的数据已被锁存，不会被新数据覆盖。

(原书 *Figure 7: Hold checks in latch-based paths*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 1.7: 基于锁存器路径中的 hold 检查

1.6.4 时序例外

Timing Exceptions

若某些路径并非按 PrimeTime 默认假定的 **setup/hold** 行为工作，需将这些路径指定为时序例外 (timing exceptions)；否则工具可能错误地将这些路径报告为时序违例。

PrimeTime 允许指定以下类型的例外：

- **False path** (伪路径) —— 因逻辑配置、预期数据序列或工作模式而永不被敏化的路径。
- **Multicycle path** (多周期路径) —— 设计为从发射到捕获需要超过一个时钟周期的路径。

- **Minimum or maximum delay path**（最小/最大延时路径）——必须满足用户以时间值显式指定的延时约束的路径。

更多信息见第 八 章「时序路径与例外」。

Chapter 2

入门

Getting Started

要开始用 PrimeTime 进行静态时序分析，请参阅：

- 安装 PrimeTime 软件与许可证
- 配置 PrimeTime 工作环境
- 启动 PrimeTime 会话
- 许可证检出（License Checkouts）
- 输入 `pt_shell` 命令
- 在命令行获取帮助
- PrimeTime 静态时序分析流程
- 在 PrimeTime 中使用 Tcl/Tk
- 用 TclPro Toolkit 检查与编译脚本
- 限制系统消息
- 保存与审阅系统消息
- 结束 PrimeTime 会话

2.1 安装 PrimeTime 软件与许可证

Installing the PrimeTime Software and Licenses

使用 PrimeTime 之前，必须为站点安装软件与许可证。相关信息见：

- *Synopsys Installation Guide*: <http://www.synopsys.com/install>
- *Synopsys Licensing Quickstart Guide*: <http://www.synopsys.com/licensing>

2.2 配置 PrimeTime 工作环境

Configuring the PrimeTime Work Environment

启动 PrimeTime 会话时，工具会执行 PrimeTime setup 文件中的命令。可在这些文件中初始化参数、设置变量、指定设计环境，并配置偏好的工作选项。

Setup 文件同名均为 `.synopsys_pt.setup`，但位于不同目录。PrimeTime 按以下顺序从三个目录读取文件：

1. Synopsys 根目录（`$SYNOPSYS/admin/setup`）

该系统级 setup 文件包含：

- Synopsys 定义的系统变量
- 面向站点所有用户的通用 PrimeTime setup 信息

仅系统管理员可修改此文件。

2. 用户主目录（home directory）

在该用户自定义 setup 文件中，可指定对 PrimeTime 工作环境的偏好。此文件中的变量设置会覆盖系统级 setup 文件中的对应设置。

3. 启动 PrimeTime 时的当前工作目录

在该设计相关 setup 文件中，可指定项目或设计环境。此文件中的变量设置会覆盖用户自定义与系统级 setup 文件中的对应设置。

要禁止执行所有 `.synopsys_pt.setup` 文件，请在使用 `pt_shell` 命令启动工具时指定 `-no_init` 选项。

2.3 启动 PrimeTime 会话

Starting a PrimeTime Session

可用命令行界面或图形用户界面启动 PrimeTime 会话：

- 命令行界面：在 Linux shell 提示符下输入：

```
% pt_shell
```

工具显示类似如下的启动信息后进入 `pt_shell>` 提示符：

```
PrimeTime (R)

Version M-2018.06 for linux64 - May 16, 2018

Copyright (c) 1988 - 201 Synopsys, Inc.
This software and the associated documentation are proprietary ...

pt_shell>
```

- 图形用户界面（GUI）：输入：

```
% pt_shell -gui
```

(原书 *Figure 8: PrimeTime GUI window*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 2.1: PrimeTime GUI 窗口

参见

图形用户界面，见第 二十 章。

2.4 许可证检出

License Checkouts

启动 PrimeTime 会话时，工具会自动检出一枚 PrimeTime 许可证。

分析过程中，工具按所用功能自动按需检出额外许可证，并给出信息消息，例如：

```
Information: Checked out license 'PrimeTime-SI' (PT-019)
```

查看当前已检出许可证数量，使用 `list_licenses` 命令：

```
pt_shell> list_licenses
Licenses in use:
    PrimeTime      (1)
    PrimeTime-SI   (1)
1
```

还有许多附加功能可用于控制许可证使用方式。详见第 三 章「控制许可证行为」。

2.5 输入 pt_shell 命令

Entering pt_shell Commands

通过基于 Tool Command Language (Tcl) 的 `pt_shell` 命令与工具交互。PrimeTime 命令语言提供类似 Linux 命令 shell 的能力，包括变量、命令条件执行与控制流命令。运行 PrimeTime 时，可以：

- 在 `pt_shell>` 提示符下交互输入单条命令；
- 在 GUI 控制台命令行交互输入单条命令；
- 运行一个或多个 Tcl 命令脚本（包含 `pt_shell` 命令的文本文件）。

关于 Tcl 命令行界面的一般信息，见 SolvNetPlus 上的 *Using Tcl With Synopsys Tools*。

2.6 在命令行获取帮助

Getting Help on the Command Line

运行 PrimeTime 时，可使用以下帮助资源：

- 命令帮助
 - 按命令组列出所有 PrimeTime 命令：

```
pt_shell> help
```

- 显示某命令的简要说明：

```
pt_shell> help command_name
```

- 显示某命令的所有选项与参数：

```
pt_shell> help -verbose command_name
```

- Man 页
 - 显示命令、变量或消息的 man 页：

```
pt_shell> man command_variable_or_message_name
```

使用 man 命令的 -html 选项可在 HTML Web 浏览器中打开 man 页。

2.7 PrimeTime 静态时序分析流程

The PrimeTime Static Timing Analysis Flow

执行 PrimeTime 静态时序分析时，遵循表 2.1 所列典型流程。

表 2.1: 典型 PrimeTime 静态时序分析流程

步骤	任务	典型命令	相关主题
1	读入设计数据（门级网表与相关逻辑库）	set search_path set link_path read_verilog link_design	处理设计数据

表 2.1 (续)			
步骤	任务	典型命令	相关主题
2	指定时序与设计规则约束	set_input_delay set_output_delay set_min_pulse_width set_max_capacitance set_min_capacitance set_max_fanout set_max_transition	约束设计
3	指定时钟特性	create_clock set_clock_uncertainty set_clock_latency set_clock_transition	时钟
4	指定时序例外	set_multicycle_path set_false_path set_disable_timing	时序路径与例外
5	指定环境与分析条件（如工作条件与延时模型）	set_operating_conditions set_driving_cell set_load set_wire_load_model	工作条件、延时计算
6	指定 case 与 mode 分析设置	set_case_analysis set_mode	Case 与 Mode 分析
7	反标延时与寄生参数	read_sdf read_parasitics	反标
8	应用 variation（可选）	read_ocvm set_aocvm_coefficient set_aocvm_table_group set_ocvm_table_group set_timing_derate	Variation
9	指定功耗信息	load_upf create_power_domain create_supply_net create_supply_set create_supply_port connect_supply_net set_voltage	多电压设计流程
10	指定信号完整性分析选项与数据	set si_enable_analysis true read_parasitics -keep_capacitive_coupling	信号完整性分析

表 2.1 (续)			
步骤	任务	典型命令	相关主题
11	应用特定设计技术选项	set_latch_loop_breaker set_multi_input_switching_ coefficient define_scaling_lib_group 等	高级分析技术、高级锁存器分析、多输入开关分析、多轨电平移位器单元缩放、快速多驱动延时分析、并行驱动器缩减
12	检查设计数据与分析设置	check_timing check_constraints report_design report_port report_net report_clock report_wire_load report_path_group report_cell report_hierarchy report_reference report_lib	检查约束
13	执行完整时序分析并查看结果	report_global_timing report_timing report_constraint report_bottleneck report_analysis_coverage report_delay_calculation update_timing	报告与调试分析结果、图形用户界面
14	生成 ECO 以修复时序违例或回收功耗	set_eco_options fix_eco_drc fix_eco_timing fix_eco_power write_changes	ECO 流程
15	保存 PrimeTime 会话	save_session	保存 PrimeTime 会话

有关可作为起点来实现 PrimeTime 静态时序分析流程的脚本，请参阅 Synopsys Reference Methodology System: <https://solvnet.synopsys.com/rmgen>。

2.8 在 PrimeTime 中使用 Tcl/Tk

Using Tcl/Tk in PrimeTime

PrimeTime 命令界面基于 Tool Command Language (Tcl) 与 Tk 工具包, 与许多其他 Synopsys 工具相同。关于 Tcl 及其在 Synopsys 命令 shell 中的用法, 见 SolvNetPlus 上的 *Using Tcl With Synopsys Tools*。

查看当前 PrimeTime 版本所用的 Tcl/Tk 版本:

```
pt_shell> printvar tcl_version
tcl_version    = "8.6"
```

2.8.1 Tcl 包与 Autoload

Tcl Packages and Autoload

PrimeTime 支持标准 Tcl package 与 autoload 机制。但不支持 load 命令, 因此需要共享库的包无法使用。工具随 Tcl 发行版中标准实现的包一同提供; 可在 Synopsys 安装根目录下的 auxx/tcllib/lib/tcl8.6 目录中找到这些包。

向 PrimeTime 添加新 Tcl 包, 可执行以下操作之一:

- 将包安装到 Synopsys 目录树中;
- 在 .synopsys_pt.setup 中向 auto_path 变量添加新目录。

PrimeTime 提供以下默认位置用于加载包:

- 应用相关 Tcl 包: auxx/tcllib/primetime
- 适用于所有基于 Tcl 的 Synopsys 工具的包: auxx/tcllib/snps_tcl

例如, 若有名为 mycompanyPT 的 Tcl 包 (包含 mycompany 所用的 PrimeTime 报告功能), 可在 auxx/tcllib/primetime 下创建 mycompanyPT 目录, 并将该包的 pkgIndex.tcl 与 Tcl 源文件放入其中。在脚本中用以下命令使用该包:

```
package require ecompanyPT
```

2.8.2 对 incr Tcl 扩展的支持

Support of the incr Tcl Extension

PrimeTime 支持 incr Tcl (itcl) 扩展, 为 Tcl 增加面向对象编程构造。关于 incr Tcl, 见 Tcl Developer Xchange 网站: <http://tcl.tk>。

2.9 用 TclPro Toolkit 检查与编译脚本

Checking and Compiling Scripts With the TclPro Toolkit

TclPro 是面向 Tcl 编程的开源工具包, 支持:

- 用 TclPro Checker 检查脚本语法

- 用 TclPro Compiler 创建字节码编译脚本
- 用 TclPro Debugger 调试脚本

更多信息见 <http://tcl.sourceforge.net>。

2.9.1 安装 TclPro 工具

Installing TclPro Tools

使用 TclPro 工具之前，需要在系统上安装 TclPro。安装后，通过 `SNPS_TCLPRO_HOME` 环境变量指定其安装路径。例如，若 TclPro 1.5 安装在 `/u/tclpro1.5`，则将该环境变量指向此目录。PrimeTime 以该变量为启动部分 TclPro 工具的基准路径；其他 Synopsys 应用也使用该变量链接 TclPro 工具。

2.9.2 用 TclPro Checker 检查脚本语法

Checking the Syntax in Scripts With the TclPro Checker

基于 TclPro Checker (`procheck`) 的 Synopsys Syntax Checker 可帮助发现 Tcl 脚本中的语法与语义错误。语法与语义检查所需内容均随 PrimeTime 提供，不必另行下载 TclPro 即可进行语法检查。

Synopsys Syntax Checker (`snps_checker`) 检查以下内容：

- 未知选项
- 歧义的选项缩写
- 互斥选项的使用
- 缺少必需选项
- 对字面选项值的数值范围校验 (`range`、`≤`、`≥`)
- 对 `one-of-string` (关键字) 选项的校验
- 递归进入带有脚本参数的构造 (如 `redirect` 与 `foreach_in_collection`)
- 重复选项覆盖先前值的警告

运行 Synopsys Syntax Checker

Running the Synopsys Syntax Checker 在 Synopsys 环境中运行 `snps_checker` 有两种方式：从 PrimeTime 工具内启动，或独立运行。

独立运行时，须使用 PrimeTime 安装提供的 `wrapper` 脚本；直接运行 `snps_checker` 无效。对于 linux64 平台，脚本位于安装目录 `linux64/syn/bin/ptprocheck`。

从 PrimeTime 内启动时，需加载安装提供的包：

```
package require snpsTclPro
```

这将使 `check_script` 命令可用。将待检查脚本名传给该命令即可。

下列示例脚本含有错误，可用于测试 `snps_checker`：

```
create_clock [get_ports CLK] -period
create_clock [get_ports CLK] -period -12.2
sort_collection
```

```

set paths [get_timing_paths -nworst 10 -delay_type mx_fall -r]
my_report -from [sort_collection \
  [sort_collection $a b] {b c d} -x]
foreach_in_collection x $objects {
  query_objects $x
  report_timing -through $x -through $y -from a -from b -to z > r.out
}
all_fanout -from X -clock_tree
puts [pwd xyz]

```

该脚本会加载本版本随附的扩展。以下输出示例显示已加载的 Synopsys 扩展；输出中的每一处均指出检测到语法或语义错误的位置，并在无效语法起始字符下方用脱字符 ^ 标示。

```

% /synopsys/linux64/syn/bin/ptprocheck ex1.tcl
Synopsys Tcl Syntax Checker - Version 1.0

Loading snps_tcl.pcx...
Loading primetime.pcx...
scanning: /disk/scripts/ex1.tcl
checking: /disk/scripts/ex1.tcl
/disk/scripts/ex1.tcl:1 (warnUndefProc) undefined procedure:
get_ports
get_ports CLK
^
/disk/scripts/ex1.tcl:1 (SnpsE-MisVal) Value not specified for
'create_clock -period'
create_clock [get_ports CLK] -period
               ^
/disk/scripts/ex1.tcl:2 (SnpsE-BadRange) Value -12.2 for
'create_clock -period' must be >= 0.000000
create_clock [get_ports CLK] -period -12.2
               ^
/disk/scripts/ex1.tcl:3 (SnpsE-MisReq) Missing required
positional options for sort_collection: collection criteria
sort_collection
^
/disk/scripts/ex1.tcl:4 (badKey) invalid keyword "mx_fall"
must be: max min min_max max_rise max_fall min_rise min_fall
get_timing_paths -nworst 10 -delay_type mx_fall -r
               ^
/disk/scripts/ex1.tcl:4 (SnpsE-AmbOpt) Ambiguous option
'get_timing_paths -r'
get_timing_paths -nworst 10 -delay_type mx_fall -r
               ^
/disk/scripts/ex1.tcl:5 (warnUndefProc) undefined procedure:
my_report
my_report -from [sort_collection \
^
/disk/scripts/ex1.tcl:5 (SnpsE-UnkOpt) Unknown option
'sort_collection -x'

```



```

sort_collection \
[sort_collection $a b] {b c d} -x
                        ^
/disk/scripts/ex1.tcl:9 (SnpsW-DupOver) Duplicate option
'report_timing -from' overrides previous value
report_timing -through $x -through $y -from a -from b -to z > r.out
                        ^

```

代码 2.1: snps_checker 输出示例

Synopsys Syntax Checker 的限制

Limitations of the Synopsys Syntax Checker

- 不检查命令缩写；缩写命令会显示为未定义过程。
- 由 `alias` 创建的别名不会展开，也会显示为未定义过程。
- 应用运行时进行的少量检查可能不会被覆盖，例如某些「某选项要求另一选项」的情况。
- 无法检查极大脚本。
- PrimeTime 允许在命令行以非严格引号形式指定 Verilog 风格总线名（如 `A[0]`）。索引 0–255 的该格式会被检查；通配符 `*` 与 `%` 也会被检查。其他形式（含范围如 `A[15:0]`）会显示为未定义过程，除非写成 `{A[15:0]}`。
- 由 `define_proc_attributes` 增强的用户自定义过程不被检查；此类过程以 `args` 声明，不会报告语义错误。

2.9.3 用 TclPro Compiler 创建字节码编译脚本

Creating Bytecode-Compiled Scripts With the TclPro Compiler

可用 TclPro Compiler (`procomp`) 创建字节码编译脚本。相对 ASCII 脚本，其优势包括：

- 加载高效
- 内容不可读，更安全
- 编译后的 Tcl 过程体被隐藏

用 `source` 命令加载字节码编译脚本即可；除应用程序外无需其他文件。

2.9.4 用 TclPro Debugger 调试脚本

Debugging Scripts With the TclPro Debugger

TclPro debugger (`prodebug`) 是该工具包中最复杂的工具，类似大多数源代码调试器：可单步、步入过程、设置断点等。它不是独立运行的，调试脚本时需要应用程序正在运行。

运行前通过 `SNPS_TCLPRO_HOME` 指定 TclPro 安装路径。用 `debug_script` 命令调试脚本。从 PrimeTime 启动 `prodebug` 需先加载：

```
package require snpsTclPro
```

这将使 `debug_script` 命令可用。将待调试脚本名传给该命令后，命令会启动 `prodebug` 并连接 PrimeTime；脚本经插装后即可开始调试。后续可通过 `source` 或 `debug_script` 再次调用来插装脚本。更多信息见调试器随附的 TclPro 文档。

2.10 限制系统消息

Limiting System Messages

触发警告消息的条件可能在单次操作中出现多次，导致数百条重复消息。为控制会话日志大小，工具会自动限制每类条件生成的消息数量。

可用以下方法控制消息限制：

- 对整个会话中特定消息类型设置限制，使用 `set_message_info`：

```
pt_shell> set_message_info -id PARA-020 -limit 200
1
```

- 对 `read_parasitics`、`read_sdf`、`report_annotated_parasitics -check` 与 `update_timing` 生成的消息指定限制，设置 `sh_message_limit` 变量：

```
pt_shell> set_app_var sh_message_limit 50
50
pt_shell> printvar sh_limited_messages
sh_limited_messages = "DES-002 RC-002 RC-004 RC-005 RC-006 RC-009 ..."
```

该限制分别适用于每种消息类型，以及每次运行命令（如 `read_sdf`）。默认每次命令执行为每类消息 100 条。

- 指定适用于所有消息类型的总体限制，设置 `sh_global_per_message_limit`。默认在整个 PrimeTime 会话中每类 10,000 条：

```
pt_shell> set_app_var sh_global_per_message_limit 800
800
```

第一种方法优先于后两种。第二种与第三种方法之间，取较低限制。

2.11 保存与审阅系统消息

Saving and Reviewing System Messages

可保存工具报告的系统消息，随时查看，并查询消息中报告的数值与字符串。例如，PTE-064 信息消息报告路径相关时钟：

```
Information: Related clock set 0 includes clock 'SDRAM_CLK' with period
7.500. (PTE-064)
```

保存会话中生成的 PTE-064 消息，使用 `set_message_info` 的 `-save_limit` 选项：

```
pt_shell> set_message_info -id PTE-064 -save_limit 100
```

这指示工具保存接下来生成的 100 条 PTE-064 消息。默认 `-save_limit` 为 `-1`（不保存）；设为 `0` 表示无限制。

查询消息格式与保存限制，使用 `get_message_info`：

```
pt_shell> get_message_info -id PTE-064
id PTE-064 severity Information limit 10000 save_limit 100 occurrences
22 suppressed 0 message {Related clock set %d includes clock '%s' with
period %.3f.}
```

创建已保存消息的集合并查询内容，使用 `get_message_instances` 与 `get_attribute`：

```
pt_shell> set my_msgs [get_message_instances PTE-064]
...
```

`message_instance` 对象类的属性可用于查询参数（消息中报告的值与字符串）、完整消息体与消息 ID：

```
pt_shell> get_attribute -class message_instance $my_msgs arguments
0,SDRAM_CLK,7.500 0,SD_DDR_CLK,7.500 1,SYS_CLK,8.000 ...

pt_shell> get_attribute -class message_instance $my_msgs message_body
{Related clock set 0 includes clock 'SDRAM_CLK' with period 7.500.}
{Related clock set 0 includes clock 'SD_DDR_CLK' with period 7.500.}
...

pt_shell> get_attribute -class message_instance $my_msgs message_id
PTE-064 PTE-064 PTE-064 PTE-064 PTE-064 PTE-064 ...
```

从集合中每条消息取第二个参数，可用类似脚本：

```
foreach_in_collection x [get_message_instances PTE-064] {
    # To replay the message
    echo [get_attribute $x message_body]
    # To fetch arguments and process them
    # Get arguments string delimited by ','
    set y [get_attribute $x arguments]
    set z [split $y ',']
    # The second argument (with index 1) is the clock
    set my_clock [get_clock [lindex $z 1]]
    # This helps report the clock fetched from the second argument
    # in each case
    query_objects $my_clock
}
```

2.12 结束 PrimeTime 会话

Ending a PrimeTime Session

可随时结束会话并退出工具。详见：

- 保存 PrimeTime 会话
- 退出 PrimeTime 会话

- 命令日志文件

2.12.1 保存 PrimeTime 会话

Saving a PrimeTime Session

结束工作会话前，可能希望保存当前会话数据。若需稍后查看分析结果，用 `save_session` 保存会话；稍后用 `restore_session` 恢复到分析中的同一时点。

保存与恢复会话适用于以下情形：

- 用脚本过夜运行 PrimeTime；脚本在最终 `update_timing` 后使用 `save_session`。之后可恢复会话，并用 `gui_start`、`report_delay_calculation`、`report_timing` 等查看结果。
- 将当前分析状态保存为检查点（checkpoint），以便在出错或分析环境意外变化时恢复。
- 保存当前会话并多次恢复，以应用不同芯片工作模式。这是以 SDF 格式保存并应用时序数据的替代方案。

更多信息见「保存与恢复会话」。

2.12.2 退出 PrimeTime 会话

Exiting a PrimeTime Session

在 `pt_shell>` 提示符下输入 `quit` 或 `exit` 退出会话：

```
pt_shell> exit
Timing updates: 2 (1 implicit, 1 explicit)
  (0 incremental, 1 full, 1 logical)
Noise updates: 0 (0 implicit, 0 explicit) (0 incremental, 0 full)
Maximum memory usage for this session: 318.43 MB
CPU usage for this session: 2 seconds (2.06 seconds aggregate)
Elapsed time for this session: 47 seconds
Diagnostics summary: 2 errors, 1 warning, 3 informationals

Thank you for using pt_shell!
```

在 PrimeTime GUI 主窗口中，**File > Exit** 菜单命令效果相同，关闭整个 PrimeTime 工具。若要关闭 GUI 窗口但保持 `pt_shell` 窗口打开，使用 **File > Close GUI**，或点击 GUI 窗口角上的关闭按钮。

2.12.3 命令日志文件

Command Log File

结束会话时，工具将会话历史保存到命令日志文件。该文件包含会话期间执行的所有命令，作为工作记录。之后可用 `source` 将该文件作为脚本运行，以重复整个会话。

工具在当前工作目录创建 `pt_shell_command.log`；同名已有日志会被覆盖。启动新会话前，请重命名需保留的日志文件。

要为命令日志指定不同文件名，在 `setup` 文件中设置 `sh_command_log_file` 变量。工作会话期间不能更改该变量。

Chapter 3

许可证

Licensing

以下主题说明 PrimeTime 工具中许可证如何检出与管理：

- 许可证如何工作
- 控制许可证行为

3.1 许可证如何工作

How Licenses Work

以下主题说明许可证如何检出，以及许可证需求如何计算：

- 许可证链（License Chains）
- 本地进程许可证（Local-Process Licensing）
- 分布式分析中的许可证管理
- 基于场景的分布式许可证（Distributed Scenario-Based Licensing）
- 基于核的分布式许可证（Distributed Core-Based Licensing）
- SMVA 许可证

3.1.1 许可证链

License Chains

许可证链是有序的许可证序列。后续每个更高层级许可证都是其前面较低层级许可证的超集。
PrimeTime Suite 产品的完整许可证链为（从低到高）：

PrimeTime → PrimeTime-SI → PrimeTime-ADV → PrimeTime-ADV-PLUS → PrimeTime-ELT →
PrimeTime-APX

要检出某一特定许可证，必须先检出其前面的所有较低层级许可证（且数量相同）。例如，若某功能需要 PrimeTime-ADV-PLUS，则必须检出：PrimeTime、PrimeTime-SI、PrimeTime-ADV 与 PrimeTime-ADV-PLUS。

当使用需要不同许可证的多个功能时，会检出至任一功能所需的最高层级许可证。若多个功能需要同一许可证，该同一许可证即可满足这些功能。

功能完成后，其许可证仍保持由工具检出状态，以确保后续使用该功能时已具备所需许可证。关于手动签入/签出许可证，见「手动控制许可证使用」。

各 PrimeTime 层级所含许可证详情，见第一章「PrimeTime 产品层级与许可证」。

3.1.2 本地进程许可证

Local-Process Licensing

在本地进程许可证模式下，本地（非分布式）PrimeTime 进程的许可证需求由其使用的功能决定。这是默认许可证行为。

使用多核分析时：

```
pt_shell> set_host_options -max_cores N
```

适用以下行为：

- 每份已购 PrimeTime Base 产品最多启用 16 核；
- 每份已购 PrimeTime Elite 产品最多启用 32 核；
- 每份已购 PrimeTime Apex 产品最多启用 64 核。

为实现上述行为，所需许可证数量按 N 计算如下：

- 1 至 16 核：无需额外许可证，无需调整许可证数量。
- 17 至 32 核：需要 PrimeTime-ADV 许可证（由 PrimeTime Elite 产品层级提供），无需调整许可证数量。
- 33 核及以上：需要 PrimeTime-APX 许可证（由 PrimeTime Apex 产品层级提供）。所需许可证数量按下列除数规则计算（ $\text{ceil}()$ 向上取整）：

表 3.1: 33 核及以上时按核数计算的许可证数量

许可证	由 N 计算的许可证数量
PrimeTime	$\lceil N/32 \rceil$
PrimeTime-SI	$\lceil N/32 \rceil$
PrimeTime-ADV	$\lceil N/32 \rceil$
PrimeTime-ADV-PLUS	$\lceil N/32 \rceil$
PrimeTime-ELT	$\lceil N/32 \rceil$
PrimeTime-APX	$\lceil N/64 \rceil$

注意

尽管 PrimeTime-APX 之下较低层级许可证的除数为 32 而非 64，但每份 PrimeTime-APX 许可证附带两份各较低层级许可证以作补偿。详见「PrimeTime 产品层级与许可证」。

例如，本地分析使用 `-max_cores 80` 时，许可证需求计算为：

许可证	数量
PrimeTime	$\lceil 80/32 \rceil = 3$
PrimeTime-SI	3
PrimeTime-ADV	3
PrimeTime-ADV-PLUS	3
PrimeTime-ELT	3
PrimeTime-APX	$\lceil 80/64 \rceil = 2$

3.1.3 分布式分析中的许可证管理

License Management in Distributed Analysis

分布式分析允许将分析工作分布到由管理进程（manager process）控制的工作进程（worker processes）。PrimeTime 提供两类分布式分析：

- 分布式多场景分析（Distributed Multi-Scenario Analysis, DMSA）
- HyperGrid 分布式分析

在分布式分析中，管理进程执行全部许可证管理：从中央许可证服务器获取工作进程所需的任何许可证，再动态分配给工作进程。

（原书 *Figure 9: Manager Process License Management in Distributed Analysis*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 3.1: 分布式分析中管理进程的许可证管理

管理进程与工作进程共享其许可证，因此管理进程实际上不占用许可证。

默认情况下，管理进程保留其从中央许可证服务器检出的许可证，以确保工作进程具备持续工作所需许可证。但在以下情况下，管理进程可将许可证归还中央服务器：

- 启用了许可证自动缩减（license autoreduction）；
- 手动减少了许可证数量。

分布式多场景分析（DMSA）

Distributed Multi-Scenario Analysis (DMSA) 在 DMSA 中，管理进程动态向工作进程分配许可证。

工作进程仅在主动执行任务时使用许可证。若工作进程数多于许可证数，部分工作进程执行任务（使用管理进程分配的许可证），其余保持空闲（等待可用许可证）。工作进程完成任务后，其许可证可供其他工作进程使用。

每次管理进程向工作进程分配任务时：

- 若管理进程已有所需许可证，则分配给该工作进程以开始任务；
- 否则，管理进程尝试从中央许可证服务器获取；成功则分配并开始任务；
- 否则，该工作进程保持空闲，直到另一工作进程完成任务并释放许可证。

这种动态许可证分配可在许可证受限情况下尽量减少昂贵的场景镜像交换（scenario image-swapping）。

HyperGrid 分布式分析

HyperGrid Distributed Analysis 在 HyperGrid 分布式分析中，管理进程在分析开始时获取分区工作进程所需的许可证。若无法获得所需许可证，管理进程退出并报错：

Error: Insufficient license resources available to analyze 8 partitions.
(DA-020)

3.1.4 基于场景的分布式许可证

Distributed Scenario-Based Licensing

在基于场景的分布式许可证模式下：

- 每个场景的许可证需求分别按「本地进程许可证」所述确定；
- 总许可证需求为各场景许可证需求之和。

该模式仅适用于 DMSA，且为 DMSA 的默认许可证行为。

例如，两个 DMSA 场景各使用 `-max_cores 80` 时，基于场景的许可证需求为：

许可证	场景 A	场景 B	合计
PrimeTime	3	3	6
PrimeTime-SI	3	3	6
PrimeTime-ADV	3	3	6
PrimeTime-ADV-PLUS	3	3	6
PrimeTime-ELT	3	3	6
PrimeTime-APX	2	2	4

3.1.5 基于核的分布式许可证

Distributed Core-Based Licensing

在基于核的分布式许可证模式下：

- 对各许可证，将分布式场景或工作进程中需要该许可证的核数求和；
- 对每种许可证，按需要该许可证的总核数 N ，用下列除数规则计算需求（`ceil()` 向上取整）：

表 3.2: 基于核的分布式许可证除数规则

许可证	由 N 计算的许可证数量
PrimeTime	$\lceil N/32 \rceil$
PrimeTime-SI	$\lceil N/32 \rceil$
PrimeTime-ADV	$\lceil N/32 \rceil$
PrimeTime-ADV-PLUS	$\lceil N/32 \rceil$
PrimeTime-ELT	$\lceil N/32 \rceil$
PrimeTime-APX	$\lceil N/64 \rceil$

- 启用基于核的许可证需要 PrimeTime-ADV 许可证。若分析所用其他功能尚未要求该许可证，仍会按上表由 N 计算数量并检出。

HyperGrid 分布式分析始终使用基于核的许可证模式。对 DMSA，该模式可选；在管理进程上设置：

```
pt_shell> set_app_var multi_scenario_license_mode core
```

两个场景各使用 `-max_cores 80` 时，基于核的许可证需求示例为：

$\lceil (80 + 80)/32 \rceil = 5$ (PrimeTime 至 PrimeTime-ELT)， $\lceil (80 + 80)/64 \rceil = 3$ (PrimeTime-APX)。

3.1.6 SMVA 许可证

SMVA Licensing

PrimeTime 中多数功能由单一许可证启用，再按需乘以核数调整。

但同时多电压分析 (Simultaneous Multi-Voltage Analysis, SMVA) 需要 PrimeTime-ADV-PLUS 许可证，其数量取决于分析特性。SMVA 许可证需求按以下计算：

$$\lceil \max(\text{传播的 DVFS 场景数}, \text{供电组电压层级数})/8 \rceil$$

其中：

- 「传播的 DVFS 场景数」是跨电源域传播时序信息时实际发现的 DVFS 场景数量；详情见第十五章「传播的 DVFS 场景」；
- 「供电组电压层级数」是某供电组中定义的电压层级数量。

换言之，许可证需求由 DVFS 场景数与任一供电组中电压层级数（取较大者）除以 8 决定，小数部分一律向上取整。

该 SMVA 派生需求适用于 PrimeTime-ADV-PLUS（因而也适用于链中较低层级许可证）。对每种许可证，最终需求取 `-max_cores` 派生需求与 SMVA 派生需求的 `max()`。

例如，`-max_cores 80` 且有 50 个 SMVA 场景时：

$\max(\lceil 80/32 \rceil = 3, \lceil 50/8 \rceil = 7) = 7$ （适用于 PrimeTime 至 PrimeTime-ADV-PLUS）；PrimeTime-ELT 仍为 3，PrimeTime-APX 仍为 2。

在含 SMVA 的 DMSA 运行中，总体 SMVA 派生需求始终取所有场景特定 SMVA 需求的 $\max()$ 。这对两种分布式许可证模式均成立；最终许可证需求再与由 `-max_cores` 派生的需求取 $\max()$ ：

- 基于场景的许可证模式：先按场景分别计算 `-max_cores` 需求，再将各场景需求相加。例如，两个场景各有 50 个 SMVA 场景且均使用 `-max_cores 80` 时，对于 PrimeTime 至 PrimeTime-ADV-PLUS：

$$\max(\lceil 50/8 \rceil, \lceil 50/8 \rceil, \lceil 80/32 \rceil + \lceil 80/32 \rceil) = 7.$$

PrimeTime-ELT 与 PrimeTime-APX 不受 SMVA 派生需求影响，分别需要 6 份和 4 份。

- 基于核的许可证模式：按所有场景的总核数计算 `-max_cores` 需求。对同一示例，PrimeTime 至 PrimeTime-ADV-PLUS：

$$\max(\lceil 50/8 \rceil, \lceil 50/8 \rceil, \lceil (80 + 80)/32 \rceil) = 7.$$

PrimeTime-ELT 与 PrimeTime-APX 分别需要 $\lceil (80 + 80)/32 \rceil = 5$ 份和 $\lceil (80 + 80)/64 \rceil = 3$ 份。

3.2 控制许可证行为

Controlling License Behaviors

以下主题说明如何在分析期间管理 PrimeTime 签入与签出许可证的方式：

- 手动控制许可证使用
- 增量许可证处理
- 许可证池化（License Pooling）
- 许可证排队（License Queuing）
- 许可证自动缩减（License Autoreduction）

3.2.1 手动控制许可证使用

Manually Controlling License Usage

默认情况下，工具自动检出所需数量的许可证，并在退出时签回。若站点可用许可证短缺，可用下表命令控制许可证使用。

下例手动检出、报告，再签回部分许可证：

```
pt_shell> # check out four PrimeTime and PrimeTime SI licenses
pt_shell> get_license -quantity 4 {PrimeTime PrimeTime-SI}
1
pt_shell> # view the licenses currently checked out
pt_shell> list_licenses
Licenses in use:
    PrimeTime      (4)
    PrimeTime-SI   (4)
1
pt_shell> # check in all PrimeTime SI licenses except one
pt_shell> remove_license -keep 1 {PrimeTime-SI}
1
```

表 3.3: 手动控制许可证使用的命令

目的	命令
检出一份（或可选更多）许可证	<code>get_license [-quantity number] feature_list</code>
显示工具当前持有的许可证	<code>list_licenses</code>
将许可证签回服务器（可选保留一定数量）	<code>remove_license [-keep number] feature_list</code>
限制特定许可证的最大检出数量	<code>set_license_limit -quantity number feature_list</code>
移除特定许可证的限制	<code>remove_license_limit feature_list</code>
报告特定许可证的限制	<code>report_license_limit feature_list</code>

```
pt_shell> # verify that the PrimeTime SI licenses were returned
pt_shell> list_licenses
Licenses in use:
    PrimeTime      (4)
    PrimeTime-SI   (1)
1
```

3.2.2 增量许可证处理

Incremental License Handling

增量许可证处理允许在分析期间更改某功能的许可证数量。该功能始终启用。

许可证可按如下方式增量签入与签出：

- 分别用 `get_license` 或 `remove_license` 增加或减少某功能已检出的许可证数量；
- 用 `set_license_limit` 更改最大检出数量。若降低限制，PrimeTime 会签回足够数量的该功能许可证以满足新目标限制。

3.2.3 许可证池化

License Pooling

许可证池化允许从不同许可证服务器检出不同功能的许可证。该功能始终启用。

同一功能的所有许可证必须由单一许可证服务器提供。许可证跨越（license spanning，即从多个服务器为同一功能检出许可证）当前不受支持。

3.2.4 许可证排队

License Queuing

许可证排队允许在所有许可证当前被占用时等待可用许可证。默认禁用。启用方法：在调用工具之前，在用户环境中将环境变量 `SNPSLMD_QUEUE` 设为 `true`。启动工具时会看到：

```
Information: License queuing is enabled. (PT-018)
```

若启用许可证排队，可能出现两进程彼此等待对方所持许可证的死锁。为防止此情况，同时设置：

- **SNPS_MAX_WAITTIME**——进程等待获取启动 PrimeTime 会话所需初始许可证的最长时间。默认 259,200 秒（72 小时）。超时后显示：

```
Information: Timeout while waiting for feature 'PrimeTime'.  
(PT-017)
```

- **SNPS_MAX_QUEUE TIME**——在成功检出启动 `pt_shell` 的初始许可证后，同一 `pt_shell` 进程内检出后续功能许可证的最长排队时间。默认 28,800 秒（8 小时）。超时后显示类似 PT-017 消息。

注意

不建议在 DMSA 流程中设置 **SNPS_MAX_QUEUE TIME**。DMSA 管理进程已在许可证受限情况下执行动态许可证获取。若同时设置该变量，管理进程每次获取新许可证失败都会阻塞至超时，从而阻止工作进程在超时完成前启动新作业。

分析流程中，排队功能还可能显示其他状态消息，例如：

```
Information: Started queuing for feature 'PrimeTime SI'. (PT-015)  
Information: Still waiting for feature 'PrimeTime SI'. (PT-016)  
Information: Successfully checked out feature 'PrimeTime SI'. (PT-014)
```

3.2.5 许可证自动缩减

License Autoreduction

许可证自动缩减是 DMSA 功能，允许空闲场景工作进程暂时将其许可证归还中央服务器。默认禁用。可随时在 DMSA 管理进程中将全局变量 `enable_license_auto_reduction` 设为 `true/false` 以启用或禁用。启用后，若某场景任务已完成且没有其他场景在等待该许可证，则许可证归还中央许可证池。

须谨慎使用该功能，原因如下：

- 尽管可将许可证归还中央服务器，但不保证后续分析能重新获取这些许可证。若管理进程无法或部分无法重新获取，DMSA 并发度会降低，可能影响性能与出结果时间。
- 反复签入/签出许可证本身会显著拖慢交互工作（如合并报告，或与场景大量通信的 Tcl 脚本/过程）。此时可在初始时序更新期间启用自动缩减，在交互工作期间禁用：

```
# get all scenarios to update their timing  
set enable_license_auto_reduction true  
remote_execute {update_timing}  
set enable_license_auto_reduction false  
  
# generate merged reports  
report_timing ...  
report_constraint ...
```

Chapter 4

管理性能与容量

Managing Performance and Capacity

要了解如何管理 PrimeTime 工具的性能与容量，请参阅：

- 高容量模式选项（High Capacity Mode Options）
- 高级数据管理（Advanced Data Management）
- 多线程多核分析（Threaded Multicore Analysis）
- 分布式多场景分析（Distributed Multi-Scenario Analysis, DMSA）
- HyperGrid 分布式分析
- 报告 CPU 时间与内存使用
- 流程摘要报告（Flow Summary Reports）
- 分析 Tcl 脚本性能（Profiling）

4.1 高容量模式选项

High Capacity Mode Options

默认情况下，PrimeTime 以高容量模式（high capacity mode）运行，可降低峰值内存占用，并在性能与容量之间进行运行时权衡，且不影响最终结果。高容量模式兼容所有分析流程，包括 Flat 与层次化流程；但在启用时钟重汇聚悲观度移除（CRPR, Clock Reconvergence Pessimism Removal）时最为有用。

在高容量模式下，工具将数据临时存储到由 `pt_tmp_dir` 变量指定的本地磁盘分区；默认为 `/tmp` 目录。退出会话时，占用的磁盘空间会自动释放。为有效使用该模式，主机本地磁盘可用容量应尽可能接近物理 RAM。但实际磁盘使用量取决于设计、约束与工具设置。

要在高容量模式下指定容量与性能之间的权衡，将 `sh_high_capacity_effort` 变量设为 `low`、`medium`、`high` 或 `default`（`default` 等价于 `medium`）。`effort` 设置越高，峰值内存越低，但运行时间越长。必须在调用 `set_program_options` 命令之前设置该变量。

要禁用高容量模式，使用 `-disable_high_capacity` 选项运行 `set_program_options` 命令。只读变量 `sh_high_capacity_enabled` 显示高容量模式是否启用：

```
pt_shell> set_program_options -disable_high_capacity
Information: high capacity analysis mode disabled. (PTHC-001)
pt_shell> printvar sh_high_capacity_enabled
sh_high_capacity_enabled = "false"
```

4.2 高级数据管理

Advanced Data Management

PrimeTime 提供更高效的设计数据存储方法——高级数据管理（advanced data management），可降低内存消耗并允许分析更大设计。

建议用于超过 1 亿实例的设计，尤其是使用基于矩建模（moment-based modeling）的参数化片上变异（POCV）分析时。启用此功能时，运行时间与磁盘占用会略有增加。

要使用此功能，在加载任何设计数据之前设置以下变量：

```
pt_shell> set_app_var timing_enable_advanced_data_management true
```

为方便起见，将此变量设为 `true` 时，工具还会将 `sh_high_capacity_effort` 变量设为 `high`。（高容量 `effort` 默认级别为 `medium`。）

此功能需要 PrimeTime-ELT 许可证。

4.3 多线程多核分析

Threaded Multicore Analysis

PrimeTime 可执行多线程多核分析（threaded multicore analysis），通过在可用核上运行多个线程来提升共享内存服务器上的性能。工具并行执行多项任务，以加快以下命令的周转时间：

```
get_timing_paths    read_parasitics    read_verilog    report_analysis_coverage
                    report_attribute    report_constraint
report_timing    save_session    update_timing    update_noise    write_sdf
```

要了解如何使用多线程多核分析，请参阅：

- 配置多线程多核分析
- 并行执行命令
- 多线程多核寄生参数读取
- 多线程多核基于路径的分析（PBA）

4.3.1 配置多线程多核分析

Configuring Threaded Multicore Analysis

默认情况下，工具使用多线程多核分析以提升性能。此功能允许多个同时执行的线程并行工作。

默认情况下，每个进程使用的最大 CPU 核数为 4。`set_host_options` 命令的 `-max_cores` 选项可指定此限制，例如：

```
pt_shell> set_host_options -max_cores 32
Information: Checked out license 'PrimeTime-ADV' (PT-019)
```

大于 16 的值需要 PrimeTime-ADV 许可证，大于 32 的值需要一个或多个 PrimeTime-APX 许可证。即使之后将最大核数降至阈值以下，工具仍保留这些许可证。详情见「本地进程许可证」(Local-Process Licensing)。

若限制超过当前机器提供的 CPU 核数，将降至可用核数。（此调整仅考虑核的存在；不考虑机器负载。）

超额线程 (overthreading) 允许同时线程数在短时间内暂时超过最大核数。默认启用。若计算环境不允许超额线程，通过以下变量禁用：

```
pt_shell> set_app_var multi_core_allow_overthreading false
```

禁用超额线程可能降低多核性能。

4.3.2 并行执行命令

Executing Commands in Parallel

可将并行命令执行用于 Tcl 过程以及更新后报告与分析，但不能用于更新设计的命令，如 ECO、时序更新或噪声更新。

使用并行命令执行时，同时执行的并行命令最大数量由 `set_host_options` 命令的 `-max_cores` 选项决定。

并行命令执行在首先启动运行时间最长的命令时最为有效。

要并行执行命令，在脚本中使用以下命令：

- `parallel_execute` 命令
- `redirect -bg` (后台) 命令
- `parallel_foreach_in_collection` 命令

`parallel_execute` 命令

The `parallel_execute` Command 要并行运行一组命令，使用 `parallel_execute` 命令，例如：

```
parallel_execute {
  {save_session design_dir/my_session} /dev/null
  {report_timing -max 10000 -nworst 10} report_timing.log
  {report_constraints -all_violators} report_constraints.log
  {report_qor} report_qor.log
}
```

所有并行运行的命令完成后，工具返回 `pt_shell>` 提示符。

`parallel_execute` 命令在命令运行期间动态分配核。当给定混合类型命令时，这可提高处理效率：

- 指定多核与非多核命令混合时，处理核动态分配给能使用它们的命令。

- 命令完成时，其处理核可重新分配给仍在运行的其他命令。

可包含多条命令的命令块。这对彼此有依赖关系的命令，或并行化更复杂代码结构很有用，例如：

```
parallel_execute {
{
  set paths [get_timing_paths ...]
  foreach_in_collection path $paths {
    ...
  }
} paths.txt
...
}
```

若指定 `set_host_options -max_cores 1`，工具以单核分析模式运行，并按指定顺序串行执行 `parallel_execute` 内的所有命令。

redirect -bg 命令

The redirect -bg (Background) Command 要在后台运行更新后命令，使用 `redirect -bg` 命令：

```
update_timing -full
redirect -bg -file rpt_file1 {report_cmd1}
redirect -bg -file rpt_file2 {report_cmd2}
redirect -bg -file rpt_file3 {report_cmd3}
...
```

主 `pt_shell` 继续运行，`redirect -bg` 将作业排队并等待可用核。

若指定 `set_host_options -max_cores 1`，工具以单核分析模式运行；`redirect` 命令忽略 `-bg` 选项并在前台运行命令。

若存在至少一个活动的后台 `redirect -bg` 命令时发出 `exit` 命令，工具会阻塞退出请求，直至所有后台命令完成。

parallel_foreach_in_collection 命令

The parallel_foreach_in_collection Command 要加速对对象集合的迭代，使用 `parallel_foreach_in_collection` 命令。该命令类似 `foreach_in_collection`，但增加了在多核上并行处理。可与 `post_eval` 命令配合使用，后者指定由管理进程按顺序执行的脚本：

```
parallel_foreach_in_collection iterator_variable collections {
  body_script_executed_by_workers_in_parallel
  ...
  post_eval {
    script_executed_by_manager_in_sequence
    ...
  }
}
```

以下示例展示如何使用 `parallel_foreach_in_collection` 与 `post_eval` 查找并报告违例路径最多的端点：

(原书 *Figure 10: Example of parallel_foreach_in_collection and post_eval commands*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 4.1: parallel_foreach_in_collection 与 post_eval 命令示例

```
set endpoints [add_to_collection [all_registers -data_pins -async_pins] \
    [all_outputs]]
set most_violators 0
set ep_with_most_violators {}
parallel_foreach_in_collection endpoint $endpoints {
    set paths [get_timing_paths -to $endpoint -nw 1000 \
        -slack_lesser_than 0]
    set num_violators [sizeof_collection $paths]
    post_eval {
        if { $num_violators > $most_violators } {
            set most_violators $num_violators
            set ep_with_most_violators $endpoint
        }
    }
}
echo "endpoint [get_object_name $ep_with_most_violators] \
    has $most_violators violators"
```

(原书 *Figure 11: Execution of parallel_foreach_in_collection*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 4.2: parallel_foreach_in_collection 的执行时序

parallel_foreach_in_collection 命令自动调度并平衡 N 次迭代执行，以优化可用核的利用率。执行 body 脚本时，工作进程将 post_eval 指定的脚本提交给管理进程，然后各工作进程继续下一次循环迭代而不等待。管理进程按顺序执行 post_eval 命令。因此，为获得最佳性能，post_eval 命令应尽可能快，且仅限于无法并行化的部分，如更改约束、设置用户定义属性、修改持久变量以聚合结果。还应避免过多的变量传递，尤其是处理大列表与数组时。

有关将现有 foreach_in_collection 循环转换为 parallel_foreach_in_collection 循环的信息，请参阅后者命令的 man page。

4.3.3 多线程多核寄生参数读取

Threaded Multicore Parasitics Reading

默认情况下，`read_parasitics` 命令在工具启动的独立进程中运行，可与 `read_sdc` 等命令并行执行。为获得最大运行时收益，应在链接设计后立即使用 `read_parasitics` 命令。可使用 `set_host_options -max_cores 1` 命令禁用并行寄生数据读取。

寄生读取过程中写入的临时文件是存储在 `pt_tmp_dir` 目录中的大型寄生文件。会话结束时工具会自动删除这些文件。默认情况下，工具将寄生命令日志写入当前工作目录的 `parasitics_command.log` 文件；要指定不同日志文件，设置 `parasitics_log_file` 变量。

4.3.4 多线程多核基于路径的分析

Threaded Multicore Path-Based Analysis

当使用 `-pba_mode path` 或 `-pba_mode exhaustive` 选项运行 `get_timing_paths` 或 `report_timing` 命令时，PrimeTime 对基于路径的分析（PBA，Path-Based Analysis）运行多线程多核分析。此功能支持路径特定与穷举重算的所有命令变体，例如：

- `get_timing_paths -pba_mode exhaustive ...`
- `get_timing_paths -pba_mode path ...`
- `get_timing_paths -pba_mode path $path_collection`
- `report_timing -pba_mode exhaustive ...`
- `report_timing -pba_mode path ...`
- `report_timing -pba_mode path $path_collection`

若无可用多核，PrimeTime 运行标准单核基于路径的分析。

4.4 分布式多场景分析

Distributed Multi-Scenario Analysis

验证芯片设计需要在不同工作条件与不同工作模式下进行多次 PrimeTime 运行。给定芯片设计的特定工作条件与工作模式组合称为场景（scenario）。设计的场景数为：

$$\text{场景数} = [\text{工作条件集}] \times [\text{模式}]$$

PrimeTime 可通过分布式多场景分析（DMSA）并行分析多个场景。DMSA 使用管理 PrimeTime 进程设置、执行并控制多个工作进程——每个场景一个——而非按顺序分析各场景。可将场景处理分布到不同主机上并行运行，缩短总周转时间。在不同场景间共享公共数据时，总运行时间会减少。

单个脚本可控制多个场景，便于设置与管理不同分析集。此能力不使用多线程（将单进程拆分为可并行执行的片段），而是提供高效方式指定给定设计的不同工作条件与模式分析，并将这些分析任务分布到不同主机。

要了解如何使用 DMSA，请参阅：

- 术语定义

- DMSA 流程概述
- 分布式处理设置
- DMSA 批处理模式脚本示例
- 基线映像生成与存储
- 主机资源亲和性
- 场景变量与属性
- 合并报告
- 将场景设计数据加载到管理进程
- 保存与恢复会话
- 许可证资源管理
- 工作进程故障处理
- 消息与日志文件
- DMSA 变量与命令
- DMSA 限制

4.4.1 术语定义

Definition of Terms

以下术语描述分布式处理的各个方面：

baseline image（基线映像） 由网表映像与场景的公共数据文件组合产生的映像。

command focus（命令焦点） 分析命令所应用的当前场景集。命令焦点可包含会话中所有场景或仅子集。

current image（当前映像） 当场景数超过主机数且工作进程必须切换处理另一场景时，自动保存到磁盘的映像。

manager（管理进程） 管理场景分析进程分布的进程。

scenario（场景） 工作条件与工作模式的特定组合。

session（会话） 选定进行分析的当前场景集。

task（任务） 由管理进程为工作进程执行而定义的自包含工作单元。

worker（工作进程） 由管理进程启动并控制、对一个场景执行时序分析的进程；亦称 **worker process**。

场景描述配置所指定设计分析时使用的特定工作条件与工作模式组合。可创建的场景数量无限制（受管理进程内存限制）。使用 `create_scenario` 命令创建场景，该命令指定场景名称及应用场景分析条件与模式设置的脚本名称。

(原书 *Figure 12: Relationships Between Scenarios, Session, and Command Focus*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.3: 场景、会话与命令焦点之间的关系

脚本分为两组：公共数据脚本（`common-data scripts`）与特定数据脚本（`specific-data scripts`）。公共数据脚本在两个或多个场景间共享；特定数据脚本仅属于特定场景且不共享。此分组帮助管理进程管理任务并在不同场景间共享信息，最小化不同场景的重复工作。

会话描述要在并行中分析的场景集。使用 `current_session` 命令选择当前会话的场景。当前会话可包含所有已定义场景或仅子集。

命令焦点是受管理进程 PrimeTime 提示符处输入的分析命令影响的场景集。命令焦点可包含当前会话中所有场景或 `current_scenario` 命令指定的子集。默认情况下，当前会话的所有场景均在命令焦点中。

4.4.2 DMSA 流程概述

Overview of the DMSA Flow

要了解多场景流程基础，请参阅：

- 准备运行 DMSA
- DMSA 使用流程

准备运行 DMSA

Preparing to Run DMSA 开始多场景分析之前，必须设置搜索路径并创建 `.synopsys_pt.setup` 文件：

- 设置搜索路径
- `.synopsys_pt.setup` 文件

设置搜索路径 在多场景分析中，只能在管理进程设置 `search_path` 变量。读取搜索路径时，管理进程在管理进程上下文中解析所有相对路径，然后自动将完全解析的搜索路径设置到工作进程。例如，管理进程可能在 `/remote1/test/ms` 目录启动，并用以下命令设置 `search_path`：

```
set search_path ". . . ./scripts"
```

管理进程自动将工作进程的搜索路径设为：

```
/remote1/test/ms /remote1/test /remote1/ms/scripts
```

多场景分析推荐流程是将搜索路径设为指定：

- 所有场景与配置文件的位置

- 在工作进程上下文中读入的所有 Tcl 脚本及网表、SDF、库与寄生文件

为获得最佳结果，避免在工作进程上下文脚本中使用相对路径。

.synopsys_pt.setup 文件 管理进程与工作进程按以下顺序 source 同一组 **.synopsys_pt.setup** 文件：

1. PrimeTime 安装目录 setup 文件：install_dir/admin/setup/.synopsys_pt.setup
2. 主目录 setup 文件：~/ .synopsys_pt.setup
3. 管理进程启动目录 setup 文件：\$sh_launch_dir/.synopsys_pt.setup

要仅在特定标量或分布式分析模式下有条件地执行命令，检查 sh_host_mode 变量是否设为 scalar、manager 或 worker，例如：

```
if { $sh_host_mode eq "manager" } {
    set multi_scenario_working_directory "./work"
}
```

DMSA 使用流程

DMSA Usage Flow DMSA 能力可用于 PrimeTime 与 PrimeTime SI 的时序分析，以及 PrimePower 的功耗分析，显著缩短周转时间。有关在 PrimePower 中使用 DMSA 的更多信息，请参阅 PrimePower User Guide 中「Distributed Peak Power Analysis」一节。

从管理 PrimeTime 进程的 pt_shell> 提示符，可启动并控制最多 256 个工作进程（见「DMSA 限制」）。

（原书 Figure 13: Typical Multi-Scenario Analysis Flow）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 4.4: 典型多场景分析流程

典型多场景分析步骤如下：

1. 使用 -multi_scenario 选项运行 pt_shell 命令启动多场景分析模式。或从普通 PrimeTime 会话将 multi_scenario_enable_analysis 变量设为 true。
2. 使用 create_scenario 命令创建场景。每个 create_scenario 命令指定场景名称及应用场景条件的 PrimeTime 脚本文件。
3. 运行 set_host_options 命令配置用于时序更新与报告的算力资源。此命令不启动主机，但设置该主机的 host 选项。例如，host 选项名为 ptopt030，使用 rsh 连接，每进程最多 3 核：

```
pt_shell> set_host_options -name my_opts -max_cores 3 \
    -num_processes 4 -submit_command "/usr/bin/rsh -n" ptopt030
```

在管理脚本中使用 set_host_options 命令指定远程核数。

4. 运行 `report_host_usage` 命令验证 `host` 选项。此命令还报告本地进程及所有已上线分布式进程的峰值内存与 CPU 使用。报告显示指定的 `host` 选项、分布式进程状态、各进程使用的 CPU 核数及分布式主机使用的许可证。
5. 运行 `start_hosts` 命令请求算力资源并使主机上线。

注意

若设置 `multi_scenario_working_directory` 与 `multi_scenario_merged_error_log` 变量，须在启动算力资源之前设置。

必须先完成上述第 1–5 步，才能在下一步运行 `current_session` 命令。

6. 使用 `current_session` 命令为会话选择场景。该命令指定先前用 `create_scenario` 创建的场景列表。
7. (可选) 使用 `current_scenario` 命令更改当前会话中处于命令焦点的场景。默认情况下，当前会话的所有场景均在命令焦点中。
8. 查看分析报告并修复验证问题：
 - (a) 执行 `remote_execute` 命令或在管理进程执行合并报告命令以开始处理场景。有关合并报告，见「合并报告」。
 - (b) 工作进程完成所有场景处理后，可查看分析报告。在 `multi_scenario_working_directory` 变量指定目录下查找 `remote_execute` 生成的报告。若使用 `-verbose` 选项，所有信息直接显示在管理进程控制台。所有合并报告命令的输出直接显示在管理进程控制台。
 - (c) 使用 `ECO` 命令修复时序与设计规则违例。

4.4.3 分布式处理设置

Distributed Processing Setup

要对多个场景进行分布式处理，须首先以 `DMSA` 模式调用 `PrimeTime` 工具，然后管理算力资源、定义工作目录、创建场景并指定当前会话与命令焦点。详情见：

- 启动分布式多场景分析模式
- 管理算力资源
- 创建场景
- 指定当前会话与命令焦点
- 远程执行命令

启动分布式多场景分析模式

Starting the Distributed Multi-Scenario Analysis Mode 要以 `DMSA` 模式启动 `PrimeTime`，启动 `PrimeTime shell` 时使用 `-multi_scenario` 选项：

```
% pt_shell -multi_scenario
```

PrimeTime 启动并显示 `pt_shell>` 提示符，与单场景会话相同。分布式多场景分析由一个管理 PrimeTime 进程与多个 PrimeTime 工作进程执行。管理进程与工作进程通过全双工网络通信交互。

管理进程生成分析任务并管理这些任务到工作进程的分布。它不执行时序分析，也不保存网表与输入的多场景数据以外的数据。因此管理进程的命令集仅限于执行管理功能的命令。

使用 `multi_scenario_working_directory` 变量指定存储分布式处理工作文件的目录。

使用 `multi_scenario_merged_error_log` 变量指定写入工作进程发出的所有 `error`、`warning` 与 `information` 消息的文件。多个工作进程发出的同一消息在合并错误日志中合并为单条记录。详见「合并错误日志」。

`multi_scenario_merged_error_log` 变量限制每任务写入日志的特定类型消息数量。默认值为 100。

启动时，每个 PrimeTime 进程（管理进程与工作进程）均会查找并 `source .synopsys_pt.setup` 文件，与正常模式相同。各进程还写入各自的日志文件。更多信息见「日志文件」。

管理算力资源

Managing Compute Resources 在分布式分析中，管理进程使用一个或多个工作进程执行工作。这些工作进程可配置、在多种计算环境中启动、查询状态并按需终止。

管理进程为工作进程生成要执行的任务。任一任务仅适用于一个场景，任一工作进程一次仅可配置为一个场景。若场景数多于运行它们的工作进程，部分工作进程需要换出一个场景以执行所有场景。

避免在主机上指定超过可用 CPU 数量的进程，否则大量时间花在换入换出工作进程上。为获得最佳 DMSA 系统性能，使工作进程数、命令焦点中的场景数与可用 CPU 数匹配，使每个场景在各自 CPU 上的各自工作进程中执行。

设置分布式 Host 选项 使用 `set_host_options` 命令分配工作进程作为算力资源。管理进程调用的每个工作进程是仅管理进程可访问的单 PrimeTime 会话。无法手动以 `worker` 模式启动 PrimeTime 进程。

使用 `set_host_options -num_processes` 命令指定工作进程：

```
pt_shell> set_host_options -num_processes process_count ...
```

`-num_processes` 选项表示规范适用于工作进程而非本地进程。

`set_host_options` 命令提供额外选项配置工作进程。例如，`-protocol` 选项指定进程使用的计算环境：

托管算力资源使工具在资源可用时分配池化资源。`set_host_options` 还提供多种额外选项配置作业提交命令、为工作进程定义名称及定义负载共享特性。详情及命令示例见 `set_host_options man page`。

可使用多个 `set_host_options -num_processes` 命令创建多组待启动进程，每组可有各自的 `host` 选项，并可选使用 `-name` 选项命名。

以下命令配置四个工作进程：两个通过 SSH 到特定机器，两个在 LSF 计算农场：

```
set_host_options \
  -num_processes 2 \
  -name my_local \
  -max_cores 2 \
  -protocol ssh \
```

表 4.1: set_host_options -protocol 选项

-protocol 值	类型	说明
auto (默认)	自动	从 -submit_command 值推断协议
rsh	非托管 (直接登录)	远程 shell 登录
ssh	非托管 (直接登录)	Secure Shell 登录
lsf	托管 (计算农场)	Load Sharing Facility
netbatch	托管 (计算农场)	NetBatch
pbs	托管 (计算农场)	PBS Professional
rtda	托管 (计算农场)	RunTime Design Automation
sge	托管 (计算农场)	Grid Engine
slurm	托管 (计算农场)	Slurm Workload Manager
custom	自定义	自定义作业提交命令或脚本

```
workstation23.internal.mycompany.com

set_host_options \
  -num_processes 2 \
  -name my_farm \
  -max_cores 2 \
  -protocol lsf \
  -submit_command "[sh which bsub] -n 2 -R rusage" \
  -terminate_command "[sh which bkill]"
```

可与 -num_processes 选项一起指定 -max_cores 选项，配置每个工作进程用于多线程多核分析的最大核数。

还可通过设置以下变量进一步配置管理进程与工作进程：

详情见各变量 man page。要移除 host 选项并终止关联进程，使用 remove_host_options 命令。例如，移除并停止 host1 与 host3：

```
pt_shell> remove_host_options {host1 host3}
```

配置分布式环境 使用 set_distributed_parameters 命令配置分布式环境（远程启动脚本、工作进程启动脚本及集合属性最大层级）。在运行 start_hosts 命令之前设置配置。

启动算力资源 使用 start_hosts 命令启动 set_host_options 指定的主机。该命令持续运行直至以下事件之一发生：

- 所有请求的远程主机进程可用
- 可用远程主机进程数达到 start_hosts 命令中指定的 -min_hosts 值（若指定）
- 秒数达到 start_hosts 命令中指定的 -timeout 值（默认 21600）

可用主机数直接影响 PrimeTime 同时分析场景的能力。可用主机过少时，会发生昂贵的映像交换，直至有足够主机可用。

运行 start_hosts 命令时，先前添加的主机开始经历其生命周期状态转换，如下图所示。

表 4.2: 分布式分析配置变量

变量	说明
<code>distributed_cleanup_variables_for_swap</code>	控制场景交换时清理变量状态
<code>distributed_custom_protocol_error_detection_timeout</code>	分布式管理进程等待自定义协议工作进程上线的最长时间
<code>distributed_farm_check_pending_workers_interval</code>	分布式管理进程等待计算农场启动的工作进程上线的最长时间
<code>distributed_farm_protocol_error_detection_timeout</code>	分布式管理进程等待计算农场启动的工作进程上线的最长时间
<code>distributed_heartbeat_period</code>	分布式工作进程向管理进程发送心跳的间隔
<code>distributed_heartbeat_timeout</code>	心跳超时（以错过的心跳次数计）。若管理进程连续错过这么多工作进程心跳，则假定工作进程失败
<code>distributed_logging</code>	指定分布式分析期间日志详细程度
<code>distributed_sh_protocol_error_detection_timeout</code>	分布式管理进程等待 sh、rsh 与 ssh 协议工作进程上线的最长时间
<code>distributed_worker_exit_action</code>	控制分布式工作进程上 <code>exit</code> 命令的行为

（原书 *Figure 14: Life cycle states*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.5: 生命周期状态

DMSA 虚拟工作进程 当场景数超过可用主机数时，至少两个场景须分配到同一主机运行。若在这些场景中执行多个命令，工具须执行保存与恢复操作以在内存中换入换出设计以执行各命令，这可能消耗大量运行时与网络资源。

此时可在 `set_host_options` 命令中设置“load factor”（负载因子）以避免额外运行时与延迟：

```
pt_shell> set_host_options -load_factor 2 ...
```

默认负载因子为 1，禁用此功能。设为 2 以减少保存与恢复操作，代价是更多内存。

此功能通过在内存中创建可各处理一个场景的虚拟工作进程实现。设为 2 将工作进程数加倍，为每个真实工作进程创建一个虚拟工作进程。若真实与虚拟工作进程可同时接受所有场景，则无需保存与恢复操作。

示例：

- 2 个场景，每场景多条命令：-num_processes 2，进程数等于场景数，提高负载因子无收益。
- 4 个场景，每场景多条命令：-num_processes 2 -load_factor 2，工作进程从 2 加倍到 4，内存分配加倍，4 个工作进程可同时接受 4 个场景，无需保存与恢复。
- 6 个场景，每场景多条命令：-num_processes 2 -load_factor 2，工作进程加倍到 4，减少但无法完全消除保存与恢复，因 4 个工作进程无法同时接受全部 6 个场景。

为优化时间收益，使工作进程总数等于场景数（可能使用多个 `set_host_options` 命令），并在主机上为虚拟工作进程分配足够内存。

创建场景

Creating Scenarios 场景是给定配置的特定工作条件与工作模式组合。使用 `create_scenario` 命令创建场景，例如：

```
pt_shell> create_scenario -name scen1 \
    -common_data {common_s.pt} \
    -specific_data {specific1.pt}
```

与场景关联的脚本有两类：

- 处理多个场景共享的公共数据、并包含在基线映像生成过程中的脚本（见「基线映像生成与存储」）
- 仅应用于指定场景的特定数据脚本

管理进程在适当情况下在场景组间共享公共数据，从而减少所有场景的总运行时间。

（原书 *Figure 15: Multi-Scenario Setup Example*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.6: 多场景设置示例

该设计须在称为 `slow` 与 `fast` 的两个工艺变异极端，以及称为 `read` 与 `write` 的两种工作模式下分析。因此四个场景为：`slow-read`、`slow-write`、`fast-read`、`fast-write`。`common_all.pt` 脚本读入设计并应用所有场景共用的约束。`common_s.pt` 在部分（非全部）场景间共享。`specific1.pt` 直接贡献于单个场景条件的指定。

使用 `create_scenario` 命令的 `-image` 选项可从先前保存的场景映像创建场景。该映像可从标准 PrimeTime 会话、从 DMSA 管理进程保存的会话或从 DMSA 会话期间生成的当前映像生成。

若需创建电压缩放组，须在用于设置场景的脚本之一中使用 `link_design` 命令之前创建。

移除场景 使用 `remove_scenario` 命令移除先前用 `create_scenario` 创建的场景：

```
pt_shell> remove_scenario s1
```

指定当前会话与命令焦点

Specifying the Current Session and Command Focus 使用 `current_session` 命令指定要分析的场景集：

```
pt_shell> current_session {s2 s3 s4}
```

该命令列出要成为当前会话一部分的场景。默认情况下，列出的所有场景均在命令焦点中——即当前受管理进程分析命令影响的场景。要进一步缩小命令焦点，使用 `current_scenario` 命令：

```
pt_shell> current_scenario {s3}
```

此命令在交互分析中很有用。例如，可能想修改特定网线上的负载，然后仅在特定场景或当前会话全部场景的子集中重新检查路径时序。

可按以下方式将命令焦点恢复为会话中所有场景：

```
pt_shell> current_scenario -all
```

在开始 DMSA 之前检查分布式处理设置，使用 `report_multi_scenario_design` 命令，该命令生成有关用户定义多场景对象与属性的详细报告。

远程执行命令

Executing Commands Remotely 要显式在工作进程上下文中运行命令，使用 `remote_execute` 命令。将命令列表作为 Tcl 字符串括起。用分号分隔命令使其依次执行。

要在远程求值子表达式与变量，使用花括号生成命令字符串，例如：

```
remote_execute {
  # variable all_in evaluated at worker
  report_timing -from $all_in -to [all_outputs]
}
```

此例中，`report_timing` 在工作进程求值，`all_in` 变量与 `all_outputs` 表达式在工作进程上下文中求值。

要在管理进程本地求值表达式与变量，用引号括起命令字符串。所有管理进程求值必须返回字符串，例如：

```
remote_execute "
  # variable all_out evaluated at manager
  report_timing -to $all_out
"
```

发出 `remote_execute` 命令后，管理进程为命令焦点中所有场景的执行生成任务。示例：

```
remote_execute {set x 10}
# Send tasks for execution in all scenarios in command focus
set x 20
remote_execute {report_timing -nworst $x}
# Leads to report_timing -nworst 10 getting executed at the workers

remote_execute "report_timing -nworst $x"
# Leads to report_timing -nworst 20 getting executed at the workers
```

使用 `-pre_commands` 选项时，`remote_execute` 在远程执行命令之前执行指定命令；使用 `-post_commands` 选项时，在远程执行命令之后执行所列命令：

```
remote_execute -pre_commands {cmd1; cmd2; cmd3} "report_timing"
# On the worker host, execute cmd1, cmd2, and cmd3 before
# executing report_timing on the manager

remote_execute -post_commands {cmd1; cmd2; cmd3} "report_timing"
# On the worker host, execute cmd1, cmd2, and cmd3 after
# executing report_timing on the manager
```

可使用 `-verbose` 选项的 `remote_execute` 命令将所有工作进程数据返回到管理进程终端屏幕，而非管道到 DMSA 文件系统层次工作目录中的 `out.log` 文件。

远程执行网表编辑命令的能力扩展到所有 PrimeTime 命令，但以下命令除外：显式 `save_session`（仅 worker）、显式 `restore_session`、`remove_design`。

4.4.4 DMSA 批处理模式脚本示例

DMSA Batch Mode Script Example

批处理模式下，PrimeTime 命令包含在工具启动时调用的脚本中，例如：

```
% pt_shell -multi_scenario -file script.tcl
```

以下是典型的多场景管理脚本：

```
set multi_scenario_working_directory "./ms_session_1"
set multi_scenario_merged_error_log "merged_errors.log"

# Start of specifying the set-up
set search_path "./scripts"

set_host_options -32bit -num_processes 1 platinum1
set_host_options -32bit -num_processes 1 platinum2
```

```

report_host_usage
start_hosts

create_scenario -name s1 \
  -common_data common_data_scenarios_s1_s3.tcl \
  -specific_data specific_data_scenario_s1.tcl
create_scenario -name s2 \
  -common_data common_data_scenarios_s2_s4.tcl \
  -specific_data specific_data_scenario_s2.tcl
create_scenario -name s3 \
  -common_data common_data_scenarios_s1_s3.tcl \
  -specific_data specific_data_scenario_s3.tcl
create_scenario -name s4 \
  -common_data common_data_scenarios_s2_s4.tcl \
  -specific_data specific_data_scenario_s4.tcl

current_session {s1 s2}

remote_execute -pre_commands {source constraint_group1.tcl \
  source constraint_group2.tcl} {report_timing}

quit

```

任务是工作进程执行以完成某功能的自包含工作块。工作进程在获得适合该任务的许可证后立即开始执行指定场景的任务。

该脚本功能：设置管理进程搜索路径；`set_host_options` 指定算力资源 `host` 选项；`report_host_usage` 生成 `host` 选项详细报告；`start_hosts` 启动主机进程；`create_scenario` 创建 `s1-s4` 四个场景；`current_session` 选择 `s1` 与 `s2`；`remote_execute -pre_commands` 在 `s1` 与 `s2` 中开始任务生成与执行；`quit` 有序终止工作进程与管理进程并释放许可证。

脚本将报告与日志写入以下路径（假设管理 `pt_shell` 从 `/mypath` 启动）：

- 管理命令日志： `/mypath/pt_shell_command.log`
- 工作进程错误合并报告： `/mypath/ms_session_1/merged_errors.log`
- 会话工作输出日志： `/mypath/ms_session_1/default_session/out.log`
- `s1`、`s2` 工作输出日志： `/mypath/ms_session_1/s1/out.log`、`s2/out.log`
- 工作命令日志：`/mypath/ms_session_1/pt_shell_command_log/platinum1_pt_shell_command.log` 等

4.4.5 基线映像生成与存储

Baseline Image Generation and Storage

管理进程生成并委派给工作进程的第一个任务是基线映像生成。基线映像由网表表示与该场景的公共数据文件组成。许多情况下，同一映像可由多个场景共享。

在其他类型任务进行之前，基线映像生成必须成功完成。映像生成失败也会导致依赖该映像的场景的多场景分析失败。

对于命令焦点中提供基线映像的每个场景,该映像写入多场景工作目录下 `scenario_name/baseline_image` 目录。

基线映像生成后,管理进程生成并委派工作进程执行命令焦点中场景的用户指定命令。

当场景数多于执行场景任务可用进程数时,工作进程为一个场景保存当前映像,同时继续为另一场景执行任务。工作进程将映像保存在多场景工作目录下 `scenario_name/current_image` 目录。

4.4.6 主机资源亲和性

Host Resource Affinity

工作进程自动重新配置以在不同场景执行不同任务可能是昂贵操作。多场景流程尝试最小化场景换出,但受命令焦点中场景数与已添加工作进程数限制。场景与工作进程的严重不平衡会降低系统性能。

若每个命令焦点场景有一个工作进程、每个工作进程有可用 CPU、且每个工作进程有所需功能的许可证,则可从 DMSA 获得最佳性能,使所有工作进程并发执行以获得最大吞吐量。

可选项为场景指定在指定主机上执行的“亲和性”(affinity),以更高效分配有限计算资源。

(原书 *Figure 16: Random Scenario-to-Host Distribution (Default)*)

请参阅原版 PDF 插图;可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.7: 随机场景到主机分布(默认)

(原书 *Figure 17: Directed Scenario-to-Host Distribution Using Affinity*)

请参阅原版 PDF 插图;可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.8: 使用亲和性的定向场景到主机分布

指定场景 host 资源亲和性示例:

```
set_host_options -name SmallHosts -max_cores 8 -num_processes 2 \
  -submit_command {bsub -n 8 -R "rusage[mem=4000] span[ptile=1]"}

set_host_options -name BigHosts -max_cores 16 -num_processes 2 \
  -submit_command {bsub -n 16 -R "rusage[mem=8000] span[ptile=1]"}
...
create_scenario -name S1 -affinity SmallHosts ...
create_scenario -name S2 -affinity SmallHosts ...
create_scenario -name S3 -affinity BigHosts ...
create_scenario -name S4 -affinity BigHosts ...
```

报告场景亲和性设置，使用 `report_multi_scenario_design -scenario` 命令。

4.4.7 场景变量与属性

Scenario Variables and Attributes

可在 DMSA 期间在管理进程或其任一工作进程中设置变量：

- 管理进程上下文变量
- 工作进程上下文变量与表达式
- 设置分布式变量
- 获取分布式变量值
- 合并分布式变量值
- 跨场景同步对象属性

管理进程上下文变量 管理进程上下文变量示例：

```
set x {"ffa/CP ffb/CP ffc/CP"}
report_timing -from $x
```

注意 `x` 被强制为字符串。

工作进程上下文变量与表达式 管理进程不知道工作进程上的变量，但可将变量或表达式字符串传递给工作进程远程求值。要将 `token` 传递给工作进程远程求值，使用花括号。例如场景 1 中 `x` 为 `ff1/CP`，场景 2 中为 `ff2/CP`，则 `report_timing -from {$x}` 在场景 1 上下文中报告从 `ff1/CP` 开始的所有最差路径，在场景 2 上下文中报告从 `ff2/CP` 开始的所有路径。

设置分布式变量 要从管理进程在工作进程级别设置变量，使用 `set_distributed_variables` 命令。变量可以是 Tcl 变量、集合、数组或列表。示例：

```
pt_shell> array set x {s1 10 s2 0}
pt_shell> array set y {s1 0 s2 15}
pt_shell> set_distributed_variables {x y}
```

这会把管理进程中数组 `x` 与 `y` 指定的值发送到相应场景的工作进程；若工作进程尚无这些变量，则创建变量并赋予指定值。

创建并设置单个变量时无需创建数组：

```
pt_shell> set my_var 2.5
pt_shell> set_distributed_variables my_var
```

获取分布式变量值 使用 `get_distributed_variables` 命令获取命令焦点中场景内设置的变量值。检索的值返回到管理进程并存储在以场景名索引的 Tcl 数组中。示例：

```
current_session {scen1 scen2}
remote_execute {
    set pin_slack1 [get_attribute -class pin U1/Z slack];
```

```
set pin_slack2 [get_attribute -class pin U2/Z slack]
}
get_distributed_variables {pin_slack1 pin_slack2}
```

该命令返回以场景名为索引的 `pin_slack1` 与 `pin_slack2` 两个数组；例如，`pin_slack1(scen1)` 是场景 `scen1` 中 `U1/Z` 的 `slack` 标量值。

检索集合时需指定要从集合检索的属性：

```
current_session {scen1 scen2}
remote_execute {set mypaths [get_timing_paths -nworst 100]}
get_distributed_variables mypaths -attributes (slack)
```

合并分布式变量值 默认情况下，`get_distributed_variables` 将场景变量作为数组带回。使用 `-merge_type` 选项可将值合并回变量，例如保留所有场景中 `worst_slack` 的最小（最负）值：

```
pt_shell> get_distributed_variables {worst_slack} -merge_type min
pt_shell> echo $worst_slack
-0.7081
```

`-merge_type` 支持的合并类型：`min`、`max`、`average`、`sum`、`list`、`unique_list`、`none`。可使用 `-null_merge_method` 覆盖选项指定合并空（empty）条目时的行为。空数据值不同于未定义值；数组条目未定义时，`-null_merge_method` 不适用，合并过程会对其余已有数据值正常执行。

跨场景同步对象属性 例如，在当前命令焦点的所有场景间同步单元 `U72` 的 `dont_touch` 属性：

```
pt_shell> synchronize_attribute -class cell {U72} dont_touch
```

此命令在 `DMSA` 管理进程中工作，可同步用户定义属性或 `dont_touch` 应用属性，但不能用于其他应用属性。`-merge_type` 可设为 `min` 或 `max`，默认 `max`：数值属性取各场景最大值，字符串属性取字母数字排序中的最后一个值，布尔属性取 `true`。

4.4.8 合并报告

Merged Reporting

在多场景分析中运行报告命令可能从各场景产生大量数据，难以识别关键信息。为帮助管理这些数据集，工具支持合并报告（merged reporting）。

合并报告自动消除冗余数据并按数值排序，使您可将所有场景视为单个虚拟 PrimeTime 实例。此功能适用于：

```
get_timing_paths    report_analysis_coverage    report_clock_timing    report_constraint
report_min_pulse_width    report_si_bottleneck    report_timing
```

要在管理进程获得合并时序报告，像普通单场景会话一样在管理进程发出 `report_timing` 命令（在管理进程上下文中执行）。管理进程与工作进程协作在命令焦点中所有场景执行 `report_timing`，在管理进程控制台产生最终合并报告。每份报告显示其来源的场景名。

可选择在合并报告命令之前或之后在工作进程上下文中运行一个或多个命令：

```
pt_shell> remote_execute -pre_commands {source my_constraints.tcl} \
-post_commands {source my_data_proc.tcl} {report_timing}
```


当场景数多于工作进程数时，使用这些选项可减少主机上换入换出场景的次数，因为所有场景特定任务（前置命令、报告命令与后置命令）在映像单次加载到主机时一起执行。

get_timing_paths

get_timing_paths 从管理进程使用 **get_timing_paths** 命令根据命令中设置的参数从各场景返回时序路径对象集合。时序路径集合按指定要求（如 **-nworst** 或 **-max_paths**）压缩，返回按报告标准跨所有场景的最差时序路径的合并输出集合。要查找特定时序路径对象的场景名，查询时序路径对象的 **scenario_name** 属性。使用 **-attributes** 选项指定时序路径集合中包含的属性。默认仅包含 **full_name**、**scenario_name** 与 **object_class** 属性。

report_analysis_coverage

report_analysis_coverage **report_analysis_coverage** 命令报告当前会话中所有活动场景的时序检查覆盖率。时序检查由受约束引脚、相关引脚和约束类型共同定义。时序检查状态报告为：**Violated**（到达时间不满足时序要求）、**Met**（满足时序要求）、**Untested**（时序检查被跳过）。仅当时序检查在所有场景中均未测试时才归入 **Untested** 类别。若至少一个场景中测试了时序检查，则报告为 **Violated** 或 **Met**，即使在某些场景中未测试。这允许在功能模式与测试模式等行为显著不同时进行有意义的报告。

report_clock_timing

report_clock_timing 管理进程执行的 **report_clock_timing** 返回显示设计中时钟网络时序属性的合并报告。合并报告在 **Scen** 列下显示场景列表。

report_constraint

report_constraint 管理进程执行的 **report_constraint** 返回显示命令焦点中所有场景基于约束信息的合并报告，报告最差违例大小及导致违例的设计对象。DMSA 的 **report_constraint** 合并过程将命令焦点中所有场景视为单个虚拟 PrimeTime 实例。当对象在多个场景中被约束时，这些约束视为重复。PrimeTime 报告每个相关对象的最严重违例实例。指定 **-verbose** 或 **-all_violators** 选项时，PrimeTime 报告约束违例者的场景名。

在摘要模式下为 DMSA 指定 **report_constraint** 处理 **setup** 检查时，报告显示每组所有场景的最差 **setup** 约束。**hold** 信息显示各时钟组跨所有场景的最差总端点代价之和。

report_min_pulse_width

report_min_pulse_width 使用 **report_min_pulse_width** 命令定位具有最严重最小脉冲宽度违例的引脚。工具修剪、排序并报告焦点中所有场景最差脉冲宽度违例的引脚。

report_si_bottleneck

report_si_bottleneck 使用 **report_si_bottleneck** 命令定位串扰延迟中最关键的网线。以最小网线修复工作量识别并修复大多数问题。跨所有场景执行排序与打印，修剪重复项以留下跨场景的唯一网线集。

report_timing

report_timing 管理进程执行的 **report_timing** 返回消除跨场景冗余数据并按 slack 排序的合并报告,有效地将命令焦点中的场景视为单一分析。合并过程报告所有场景的最差路径,同时保持 **-nworst**、**-max_paths** 及其他 **report_timing** 选项施加的限制。

当同一路径从多个场景报告时, PrimeTime 仅在合并报告中保留该路径最严重实例并显示该实例最严重的场景。使用 **-dont_merge_duplicates** 选项可防止路径合并。

PrimeTime 认为来自两个场景的两条路径是同一路径的实例,当且仅当满足以下全部条件: 路径组、数据部分路径上引脚序列、数据部分每个引脚的转换、发射时钟、捕获时钟、约束类型。

标准选项处理 **report_timing** 的所有选项均可用于合并报告,少数例外见「DMSA 限制」。只要未向选项传递集合,即可像在 PrimeTime 单场景会话中一样在管理进程指定,例如:

```
pt_shell> report_timing -nworst 10
pt_shell> report_timing -delay_type max_rise -path_type full_clock -nosplit
pt_shell> report_timing -from {all_in} -to {[all_outputs]}
```

使用 **-pre_commands** 选项使集合在与 **report_timing** 相同的任务中生成,管理进程的合并 **report_timing** 然后使用您指定的名称引用显式集合。

复杂选项处理 以下选项在多场景管理进程中已更改,以允许求值管理进程与工作进程变量与表达式: **-from**、**-rise_from**、**-fall_from**、**-through**、**-rise_through**、**-fall_through**、**-to**、**-rise_to**、**-fall_to**、**-exclude**、**-rise_exclude**、**-fall_exclude**。

在 PrimeTime 单场景会话中,这些变量接受列表参数;但在多场景合并报告中,这些变量接受字符串参数。

4.4.9 将场景设计数据加载到管理进程

Loading Scenario Design Data Into the Manager

默认情况下, DMSA 分析中的管理进程不加载设计数据。这降低内存与运行时需求,对大多数合并报告与 ECO 任务已足够。

对于更复杂任务,可使用 **load_distributed_design** 与 **cache_distributed_attribute_data** 命令在管理进程加载设计与库数据。

加载网表与库数据 **load_distributed_design** 命令加载跨所有场景相同的网表与库数据,如引脚方向与时序弧。从 **current_scenarios** 列表中第一个场景加载网表与库。不保证跨场景相同的数据(如时序或 slack 数据)不会加载。

加载网表与库数据后:

- 可用 **get_cells**、**get_pins**、**get_timing_arcs** 等命令获取网表对象
- 可用 **get_lib_cells**、**get_lib_pins**、**get_lib_timing_arcs** 获取库对象
- **get_timing_paths** 返回真正的 **timing_path** 集合对象
- 可用 **all_fanin**、**all_fanout** 探索网表拓扑
- 在 PrimeTime GUI 中可使用 ECO 与 Path Analyzer 功能

(原书 *Figure 18: Netlist and Path Query After Loading Scenario Data Into Distributed Manager*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.9: 将场景数据加载到分布式管理进程后的网表与路径查询

所有场景网表必须相同（约束与工作条件可不同）。若有任何网表差异，或管理进程无当前场景，工具报错：

```
Error: Restore session at manager failed due to: Scenario has a
different signature compared to known signatures. (MS-037)
```

在管理进程加载网表会为 DMSA 管理进程保留 PrimeTime 许可证，意味着该许可证（或在基于核的许可情况下的一部分）不与运行 DMSA 场景的任何工作进程共享。若不加载网表，管理进程启动时检出的 PrimeTime 许可证与工作进程共享。使用 `remove_distributed_design` 命令移除已加载的分布式数据。

加载场景特定设计数据 `cache_distributed_attribute_data` 命令将当前活动场景的场景特定设计数据加载到管理进程。此命令要求已使用 `load_distributed_design` 加载设计数据。为节省内存，命令仅将您指定的属性加载到管理进程。支持的 `-class` 值为 `port`、`cell`、`pin`、`net`。多次调用 `cache_distributed_attribute_data` 是累积的。使用 `purge_distributed_attribute_data` 命令从管理进程内存移除部分或全部属性数据。

4.4.10 保存与恢复会话

Saving and Restoring Your Session

可在 `$sh_launch_dir/images/scen1` 与 `scen2` 目录中找到管理进程发出的 `save_session` 命令生成的映像，然后可将这些映像重新加载到单场景 PrimeTime 会话。

保存与恢复功能允许显式将保存的映像恢复到 DMSA 场景，通过 `create_scenario -image` 选项实现。使用 `-image` 选项可将 `save_session` 命令产生的任何单核分析 PrimeTime 映像（在 DMSA 内外）直接作为场景加载到 DMSA，然后在 DMSA 上下文中正常操作。

4.4.11 许可证资源管理

License Resource Management

在 DMSA 分析中，管理进程执行所有许可证管理。从中央许可证服务器获取工作进程所需的任何许可证，然后动态分配给工作进程。

DMSA 分析中可使用以下许可功能：

- 分布式基于核的许可（Distributed Core-Based Licensing）——每份 PrimeTime-ADV 或 PrimeTime-APX 许可证分别启用跨场景 32 或 64 核，无论核如何分布，常降低整体许可需求

- 增量许可处理（Incremental License Handling）——降低许可上限；超出上限的许可归还许可证服务器池
- 许可池化（License Pooling）——从不同许可证服务器检出不同功能的许可证
- 许可自动缩减（License Autoreduction）——指示管理进程在不需要时自动将已检出功能的许可证数减至最小
- 许可排队（License Queuing）——若所有许可证当前在使用中，等待许可证可用

限制 **DMSA** 分析中的许可使用 在管理进程使用 `set_license_limit` 命令限制管理进程可为工作进程获取的最大许可证数：

```
pt_shell> set_license_limit PrimeTime -quantity 5
```

默认情况下，仅在工作进程开始处理任务时检出许可证。要立即检出，在管理进程使用 `get_license` 命令。为获得最佳 **DMSA** 吞吐量，使许可限制与工作进程数匹配，确保所有工作进程可并发执行。

4.4.12 工作进程故障处理

Worker Process Fault Handling

当工作进程发生导致异常终止的故障时，工具根据 `multi_scenario_fault_handling` 变量（可设为 `ignore` 或 `exit`）采取行动。

设为 `ignore`（默认）且工作进程发生严重故障时，异常终止的场景及依赖这些场景的场景从当前会话的进一步分析中排除，显示警告消息说明哪些场景异常终止，会话继续分析命令焦点中剩余场景。

- 若当前会话命令焦点中的全部场景均异常终止，之后在管理进程发出的分布式命令会失败；必须在当前会话中更改命令焦点后再分析所选场景。
- 若当前会话中的全部场景均异常终止，当前会话会被移除；之后的分布式命令会因不存在当前会话而报错。
- 严重故障会使资源离线；若没有任何资源保持在线，会话必须终止。

设为 `exit` 且发生严重故障时，活动命令完成任务后管理进程终止整个分析。

合并报告与工作进程故障处理 若一个或多个工作进程异常终止，报告命令的相关信息不可用。场景异常终止时收到警告消息，**PrimeTime** 从当前会话移除此场景，然后自动向命令焦点中剩余场景重新发出报告命令。

网表编辑命令与工作进程故障处理 在管理进程将网表更改提交到任何场景之前，必须能提交到所有场景。**PrimeTime** 先检查更改可在所有场景中成功，再提交更改。检查或提交期间场景异常终止时，从命令焦点移除该场景并自动向剩余场景重新发出网表编辑命令。

remote_execute 与工作进程故障处理 `remote_execute` 支持在命令焦点中所有场景执行任意数量命令。若遇到异常终止的场景，发出警告、从当前会话移除场景，后续调用仅应用于剩余场景。

其他命令与工作进程故障处理 `get_distributed_variables`、`get_timing_paths`、`save_session`、`set_distributed_variables` 等命令在异常终止时类似处理：警告、从会话移除场景，并对剩余场景重新发出或正常执行命令。

4.4.13 消息与日志文件

Messages and Log Files

多场景分析可能产生大量数据。为避免在管理进程用数据过载用户，所有数据写入一组日志文件。管理进程仅显示有限的交互消息。使用 `-verbose` 选项时，`remote_execute` 将所有数据发送到管理终端。

关于 DMSA 中可用的消息与日志文件类型，请参阅：

- 交互消息
- 进度消息
- 任务执行状态消息的用户控制
- 错误消息
- 警告消息
- 日志文件
- 命令输出重定向

交互消息

Interactive Messages 工作进程执行任务时，向管理进程发送消息以指示其进度。

进度消息

Progress Messages 工作进程在各分析阶段推进时，会发送置信度消息（confidence messages），向您更新分析进度。例如：

```
pt_shell> current_session {scen1 scen2}
pt_shell> remote_execute {cmd1}

Start of Manager/Worker Task Processing
-----
Started   : Netlist image generation for session 'session'
Succeeded : Netlist image generation for session 'session'
Started   : Baseline image generation for scenario 'scen1'
Succeeded : Baseline image generation for scenario 'scen1'
```

任务执行状态消息的用户控制

User Control of Task Execution Status Messages 可控制 DMSA 任务执行状态消息的详细程度。将 `multi_scenario_message_verbosity_level` 变量设为 `default` 或 `low`。设为 `default` 时显示所有任务执行状态消息，例如：

```

Start of Manager/Worker Task Processing
-----
Started      : Command execution in scenario 'wc'
Started      : Command execution in scenario 'bc'
Succeeded    : Command execution in scenario 'bc'
Succeeded    : Command execution in scenario 'wc'
-----
End of Manager/Worker Task Processing

```

设为 `low` 时，仅显示 `error`、`fatal` 与 `failure` 消息；这对需要更多输出控制的 Tcl 脚本可能有用。若 Tcl 脚本更改了该变量，宜在脚本或过程完成后将其改回原值，以免造成混淆。

错误消息

Error Messages 工作进程上给定场景发生错误时，在管理进程完整报告该错误及该场景名。例如：

```

pt_shell> current_session {scen1 scen2}
pt_shell> remote_execute {cmd1}

Start of Manager/Worker Task Processing
-----
Started    : Netlist image generation for session 'session'
Error      : Problem in read_verilog. No designs were read
(DBR-011) (default_session)

```

警告消息

Warning Messages 工作进程上给定场景出现警告时，仅当该警告在该场景中首次出现时，才在管理进程报告。例如：

```

pt_shell> current_session {scen1 scen2}
pt_shell> remote_execute {cmd1}

Start of Manager/Worker Task Processing
-----
Started    : Netlist image generation for session 'session'
Succeeded  : Netlist image generation for session 'session'
Started    : Baseline image generation for scenario 'scen1'
Warning    : RC-009 has occurred on scenario 'scen1'

```

日志文件

Log Files 可在 `log`、`output log`、`command log` 或 `merged error log` 文件中查看工作进程生成的全部信息。多场景分析全部数据（含日志文件）的根工作目录由 `multi_scenario_working_directory` 变量决定。若未显式设置，则设为启动 PrimeTime 管理进程时的当前目录。您必须对该工作目录具有写权限，且该目录须对管理进程及所有工作进程可访问。

- 输出日志 (**Output Log**): 每个场景有自己的输出日志，位于场景工作目录中。例如，名为 `s1` 的场景，日志为 `multi_scenario_working_directory/s1/out.log`。不能设置或更改输出日志名。某

一场景的输出日志包含所有 error、warning 与 information 消息，即您在传统单次分析 PrimeTime 运行的终端窗口中通常看到的全部信息。

- **命令日志 (Command Log)**: 每个远程进程有自己的命令日志，位于工作目录下的 `command_log` 目录。例如，在名为 `srv1` 的主机上启动且为第 6 个启动的进程，命令日志为 `multi_scenario_working_directory/command_log/srv1_6_pt_shell_command.log`。不能设置或更改远程进程的命令日志名。某一进程的命令日志包含在该进程上执行的全部命令。管理进程命令日志位于 `$sh_launch_dir/pt_shell_command.log`。当 `sh_source_logging` 设为 `true` 时，`source` 脚本中的各条命令会写入命令日志。
- **合并错误日志 (Merged Error Log)**: 多场景分析中可能产生大量 error、warning 与 information 消息。为便于调试，可在管理进程设置合并错误日志变量。例如，下列命令在 `multi_scenario_working_directory/error.log` 中创建合并错误文件：

```
pt_shell> set_app_var multi_scenario_merged_error_log "error.log"
```

设置该变量后，从特定任务开始到结束的所有错误消息合并在一起；若同一错误消息出现在多个场景中，则仅向合并错误文件打印一次，并列出发生该消息的场景。

注意

须在发出 `start_hosts` 命令之前设置 `multi_scenario_working_directory` 与 `multi_scenario_merged_error_log` 变量。

命令输出重定向

Command Output Redirection 多场景分析支持命令输出重定向。须为每个进程重定向到不同目标文件，否则两个不同进程同时试图控制同一文件时会出错。

注意

使用输出重定向时务必使用相对路径。不要使用绝对路径；否则所有工作进程尝试写入同一文件，可能导致读/写冲突并使工作进程失败。

`multi_scenario_working_directory` 变量值与场景名一起确定输出目标文件名。交互重定向示例：先执行 `set x myfile`，再执行 `remote_execute {report_timing} > $x.out`，输出会定向到当前工作目录的 `myfile.out`，其中 `$x` 仅在管理进程求值。脚本重定向示例：`report_timing > rt.log` 在 `/mypath/s3/rt.log` 与 `/mypath/s4/rt.log` 中分别生成输出。

4.4.14 DMSA 变量与命令

DMSA Variables and Commands

DMSA 变量

DMSA 命令

工作进程不允许的命令 不能在 worker 进程上运行：`exit`、`quit`、`remove_design`、`remove_lib`、`rename_design`、`restore_session`。

表 4.3: DMSA 变量

变量	说明
<code>multi_scenario_merged_error_limit</code>	每任务写入命令日志的特定类型错误最大数量。默认 100
<code>multi_scenario_merged_error_log</code>	指定存储工作进程产生数据的 <code>error</code> 、 <code>warning</code> 、 <code>information</code> 消息的文件位置。默认 “”。须在运行 <code>start_hosts</code> 之前设置
<code>multi_scenario_working_directory</code>	指定管理进程与工作进程可写的工作目录。默认当前工作目录。须在运行 <code>start_hosts</code> 之前设置
<code>sh_host_mode</code>	指定工作模式: <code>scalar</code> 、 <code>manager</code> 或 <code>worker</code> 。只读。可用于脚本判断是否在正常 PrimeTime 会话或管理/工作进程上操作

表 4.4: DMSA 命令

类别	命令	说明
场景设置	<code>create_scenario</code>	创建单个场景（必需）
	<code>remove_scenario</code>	移除一个或多个指定场景
	<code>remove_multi-scenario_design</code>	移除所有多场景数据，含会话、所有场景与配置
	<code>report_multi-scenario_design</code>	报告当前用户定义的多场景分析
Host 与许可配置	<code>set_host_options</code>	定义算力资源（必需）
	<code>remove_host_options</code>	移除已添加到农场池的所有分布式主机
	<code>set_license_limit</code>	限制管理进程为工作进程使用获取的许可证数
	<code>start_hosts</code>	在 <code>set_host_options</code> 指定的主机上启动工作进程（必需）
分析焦点	<code>report_host_usage</code>	报告主机与资源使用
	<code>current_scenario</code>	将命令焦点更改为当前会话中选定的场景
	<code>current_session</code>	将特定场景组置于分析焦点（必需）
工作进程上下文	<code>remove_current-session</code>	移除先前设置的会话；取消所有场景焦点
	<code>remote_execute</code>	从管理进程创建在工作进程上下文中执行的命令缓冲区

4.4.15 DMSA 限制

Limitations of DMSA

DMSA 功能有以下限制：

- 管理进程可控制的最大工作进程数为 256 个并发主机
- 不支持使用 GUI

4.5 HyperGrid 分布式分析

HyperGrid Distributed Analysis

超大设计的时序分析具有挑战性。若设计包含物理设计块，可使用层次化时序分析先单独验证各块，再用块模型验证顶层设计。

HyperGrid 分布式分析可在单次运行中通过在不同主机上运行的多个工作进程并行分析来分析超大设计。可用常规 PrimeTime 分析中相同的命令探索并报告整个设计。

HyperGrid 可用于不适合层次化时序分析流程的设计，也可作为层次化流程中的额外最终验证。此功能需要 PrimeTime-ADV-PLUS 许可证。

4.5.1 HyperGrid 流程概述

Overview of the HyperGrid Flow

HyperGrid 将超大设计的分析分布到多台机器，使用相同分析脚本并生成与全 Flat 分析相同的报告。

划分设计

Partitioning the Design HyperGrid 分布式分析中，拆分设计的过程称为划分（partitioning）。HyperGrid 划分不受逻辑设计层次限制，而是按到时钟与数据端点的扇入逻辑锥（fanin logic cones）划分。扇入锥可穿过层次块。不同划分可包含同一块的不同部分。

（原书 *Figure 19: HyperGrid Partitioning, Using Fanin Logic Cones to Endpoints*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 4.10: HyperGrid 划分：使用到端点的扇入逻辑锥

每个划分包含到一组时钟/数据端点的扇入逻辑锥。划分过程使用网表、详细寄生参数与约束数据以确保考虑设计逻辑的完整时序上下文。此外，扇入逻辑锥包括：

- 侧负载级（side load stages）——建模发散到当前划分外端点的路径的接收端负载

- 攻击者级（aggressor stages）——建模划分间耦合时序与噪声相互作用；HyperGrid 按需同步划分间信号完整性效应

不同划分中的不同端点可共享公共起点，此时起点扇出的部分包含在两个划分中。时钟扇入锥往往比数据扇入锥起点更少、侧负载引脚更多。

管理进程与工作进程

The Manager Process and the Worker Processes HyperGrid 分布式分析使用单个管理进程调用并控制多个工作进程。管理进程：确定如何划分设计；调用工作进程（每个划分一个）；通过查询工作进程并合并信息生成全设计报告。工作进程：读取特定划分的设计数据；执行时序与噪声更新；收集并返回管理进程查询的结果。

（原书 *Figure 20: Analysis Script Execution in a HyperGrid Distributed Analysis*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 4.11: HyperGrid 分布式分析中的分析脚本执行

HyperGrid 分布式分析始终使用基于核的许可。详见「分布式基于核的许可」。

分布式分析工作目录

The Distributed Analysis Working Directory HyperGrid 分布式分析中，分布式分析工作目录包含工具生成的脚本与各划分的日志文件。目录结构：`WORK/partition0/`、`partition1/`、...、`partitionN/`，各含 `RUN.ptsh`、`parasitics.log`、`out.log`。

设计链接后，各划分目录中的 `out.log` 显示划分特征（叶引脚数、层次引脚数、叶单元数、划分编号、各类型引脚/网线/单元占比等）。

4.5.2 准备分布式分析

Preparing for Distributed Analysis

HyperGrid 分布式分析需要包装脚本（wrapper script），该脚本启用 HyperGrid、配置并启动工作进程，然后在分布式模式下运行实际分析脚本。示例：

```
# enable Hypergrid analysis
set_app_var distributed_enable_analysis true
set_app_var distributed_working_directory ./MY_WORK_DIR

# start worker processes
set_host_options \
  -num_process 4 \
  -max_cores 4 \
  -submit {...compute farm submission command...}
start_hosts
```

```
# run the timing analysis script, distributed across the workers
start_dsta \
    -script run_top_flat.tcl
```

关键设置如下：**distributed_enable_analysis** 变量启用 HyperGrid；**distributed_working_directory** 变量设置工作目录（默认 **dsta_working_dir**）；**set_host_options** 与 **start_hosts** 命令配置并启动远程工作进程；**start_dsta** 命令开始分布式分析（在管理进程划分设计、在工作进程加载划分数据、在管理进程运行指定分析脚本并与工作进程通信）。包装脚本中 **start_dsta** 之后的命令在指定分析脚本完成后作为分布式报告命令执行。

4.5.3 运行分布式分析

Running the Distributed Analysis

1. 按「准备分布式分析」创建包装脚本
2. 在新 PrimeTime 会话中运行包装脚本：`% pt_shell -f wrapper.tcl`

若分析脚本（由包装脚本调用）在 **update_timing** 之前将命令输出重定向到文件，目标文件名必须为相对路径，否则多个工作进程在读入与约束设计时尝试写入同一文件会发生冲突。

若分析脚本运行 **exit** 或 **quit** 命令，分布式分析会话结束。

4.5.4 在 HyperGrid 分析中查询属性

Querying Attributes in a HyperGrid Analysis

分布式分析中，管理进程维护全设计的逻辑网表表示，但不计算任何时序信息。在管理进程查询属性时，工具行为取决于属性数据存储位置：若可从管理进程逻辑网表获得（如对象名或引脚方向），查询立即返回；若仅从划分级分析获得（如时序或噪声信息），须向工作进程请求数据。

为提高划分源属性的性能，HyperGrid 提供属性缓存功能以最小化从工作进程传输的数据量。

foreach_in_collection 中的自动属性缓存 在管理进程的 **foreach_in_collection** 循环内执行属性查询时，HyperGrid 检测查询的属性并从各划分缓存其值以加速后续循环迭代。属性数据仅为循环集合对象缓存，且仅在循环持续期间缓存。

显式属性缓存 脚本可使用 **cache_distributed_attribute_data** 命令在管理进程显式缓存属性数据。显式缓存的属性可供后续所有命令使用，直至 **purge_distributed_attribute_data** 显式释放。

4.5.5 保存分布式会话

Saving a Distributed Session

可使用 **save_session** 命令将 HyperGrid 分布式分析会话保存到磁盘。结果目录包含管理进程与所有工作进程的数据。由于分布式设计数据开销，分布式分析会话目录大于等效全 Flat 会话目录。恢复保存的分布式会话时，工具以相同划分重新创建原始分布式分析，使用与原始分布式分析相同的 **set_host_options** 规范启动工作进程。

恢复已保存的分布式会话时，可先按需设置分布式分析工作目录，再按通常方式恢复：

```
pt_shell> set_app_var distributed_working_directory ./MY_WORK_DIR
pt_shell> restore_session my_distributed_session
```

4.5.6 支持 HyperGrid 分布式分析的命令

Commands With HyperGrid Distributed Analysis Support

支持 HyperGrid 分布式分析的工具命令包括：

```
check_noise check_timing get_attribute get_clock_relationship get_timing_paths
report_analysis_coverage report_annotated_parasitics -list_annotated report_attribute
report_clock_gating_check report_constraint report_crpr report_delay_calculation report_global_slack
report_global_timing report_min_period report_min_pulse_width report_net report_noise
report_noise_calculation report_qor report_timing restore_session save_session write_rh_file
```

以下命令支持在管理进程查询与探索多电压信息：add_power_state、add_pst_state、create_pst、create_supply_set、get_supply_groups、get_supply_nets、get_supply_ports、get_supply_sets、report_power_domain、report_power_pin_info、report_power_switch、report_pst、report_supply_group、report_supply_net、report_supply_set、report_timing -supply_net_group、set_retention、set_retention_control、set_retention_elements。

4.5.7 HyperGrid 分布式分析的限制

Limitations of HyperGrid Distributed Analysis

HyperGrid 分布式分析不支持以下 PrimeTime 流程与功能：

- AOCV 深度调整（支持距离调整）
- ECO 修复（自动或手动）、网表编辑命令
- ETM 模型
- HyperScale 层次化分析
- HyperTrace 加速 PBA
- SMVA/DVFS 同时多电压分析

4.6 报告 CPU 时间与内存使用

Reporting the CPU Time and Memory Usage

PrimeTime 执行静态时序分析所使用的内存与 CPU 资源取决于设计规模、分析模式与报告要求等多种因素。PrimeTime 为分布式进程提供性能与容量信息。无论使用单核分析、多核分析还是 DMSA，均可在 PrimeTime 中监控内存与 CPU 使用。

使用 cputime 命令报告与当前 pt_shell 进程关联的总处理器时间（秒）。使用 mem 命令报告当前 pt_shell 进程分配的总内存（KB）。为后续 PrimeTime 会话指定内存需求时，使用此报告内存值并额外增加 20% 以保障操作系统高效运行。

在 DMSA 流程中使用 `report_host_usage` 命令报告所有已设置的 `host` 选项。此命令还报告本地进程的峰值内存、CPU 使用与经过时间；在 DMSA 中还报告所有分布式进程的峰值内存、CPU 使用与经过时间。报告显示指定的 `host` 选项、分布式进程状态、各进程使用的 CPU 核数及分布式主机使用的许可证。

对于配置为在同一主机上使用多个 CPU 核的进程（如单核或多线程多核分析），`report_host_usage` 提供当前 PrimeTime 会话的资源使用。单核或多线程多核分析流程的报告输出示例如下：

```
pt_shell> report_host_usage
*****
Report : host_usage
...
*****
Usage limits (cores)

Options Name          #      Effective
-----
(local process)      -      1
-----
Total                  1

Memory usage
Options Name      #      Peak Memory (MB)
-----
(local process) -      35.88

Performance
Options Name      #      CPU Time (s)      Elapsed Time (s)
-----
(local process) -      4              33
```

在 DMSA 流程中，资源可分布在多台主机上，因此为每个分布式进程提供资源使用数据。DMSA 流程的 `report_host_usage` 输出示例如下：

```
pt_shell> report_host_usage
*****
Report : host_usage
...
*****

Options Name      Host Name      32Bit      Num Processes
-----
my_opts1          >>farm<<      N          2

Options Name      #      Host Name      Job ID      Process ID      Status
-----
my_opts1          1      ptopt018      136604      1394           ONLINE
                  2      ptopt018      136605      1393           ONLINE

Usage limits (cores)
```

Options Name	#	Effective

(local process)	-	1
my_opts1	1	4
	2	4

Total		9

退出 PrimeTime 时，工具报告会话的峰值内存、CPU 使用与经过时间：

```
Maximum memory usage for this session: 31.08 MB
CPU usage for this session: 23 seconds
Elapsed time for this session: 211 seconds
```

退出 DMSA 会话时，工具报告分布式进程信息（各 host 选项的峰值内存、CPU 时间与经过时间，以及会话总计）。

4.7 流程摘要报告

Flow Summary Reports

现代分析流程复杂。完整分析流程分散在多个脚本中：工具配置、设计读入与链接、设计约束与报告。脚本常嵌套并调用更多脚本。确定流程主要步骤消耗的运行时间很困难；追踪意外隐式更新的发生位置更难。

PrimeTime 提供流程摘要（flow summary）功能，可：

- 生成整个分析流程中脚本执行的结构化摘要
- 突出显著工具事件（设计链接、工具更新等）
- 报告脚本及其中显著命令使用的运行时间
- 将消耗非平凡运行时间的重复命令作为一组突出显示
- 性能开销很小（小于 5%）

4.7.1 流程摘要报告结构

The Flow Summary Report Structure

流程摘要是脚本执行的文本表示，并为显著与耗时事件添加注释。

流程摘要上下文层级 开头的 **START** 标记表示首次启用流程摘要功能的位置，后续缩进层级对应嵌套脚本执行：

```
START {
| source read_design.pt {
| | <<< Event: Invoking link_design; File: read_design.pt; Line: 3; "link"
| } 0.6s
| source constraints.pt {
| | source multicycles.pt {
| | | <<< Event: Invoking logical_update_timing; ...
```

```
| | } 0.1s
| } 0.5s
| <<< Event: Invoking update_timing; File: run.pt; Line: 8; "update_timing"
| update_timing: 14.9s
| uncaptured: 0.2s
} 16.3s
```

可使用支持查找匹配开/闭花括号的文本编辑器轻松在摘要中导航上下文层级。

事件标记 “<<< Event” 标记指示显著工具事件及引起该事件的命令（含脚本与行号）。可能的事件类型包括: Invoking link_design、Invoking update_timing、Halting background parasitics read、Invalidating update (logical/full)、Invoking logical update_timing、Invoking update_noise、Forcing full update、helper_command > 30s。

显著命令标记 消耗非平凡运行时间（10 秒或更多）的命令显式注明其运行时间。

未捕获命令标记 快速命令通常不显式列出，其总运行时间由“uncaptured”运行时间标记汇总。

命令捆绑 若多个快速命令的累积运行时间变得显著（大于 10 秒），它们被分组为命令捆绑（command bundles）。命令不在显著命令条目间捆绑，以保持报告与实际命令执行顺序一致。

辅助命令 辅助命令（helper command）通常用于为其他命令提供信息，包括: get_pins、get_nets、get_ports、get_cells、all_fanin、all_fanout。通常辅助命令本身不引人注意，除非意外贡献运行时间。因此仅 30 秒或以上的辅助命令视为显著，使用特殊“显著辅助命令”事件标记报告。运行不足 30 秒的辅助命令会组成辅助命令捆绑；仅当捆绑总运行时间达到 30 秒或以上时，才视为显著并显式报告。

运行时间 流程摘要显示命令（显著与未捕获）与闭合上下文层级的运行时间信息。所有运行时间值为经过的挂钟时间（wall clock time）测量。

4.7.2 生成流程摘要报告

Generating a Flow Summary Report

要开始新的流程摘要报告，将 sh_flow_summary_file 变量设为应写入摘要的文件名：

```
pt_shell> file delete ./summary.txt
pt_shell> set_app_var sh_flow_summary_file {./summary.txt}
Information: Writing flow summary to summary.txt.
```

默认此变量为空字符串，禁用该功能。若指定文件已存在，后续流程摘要信息追加到末尾，允许在流程中开关该功能以仅包含感兴趣区域。因此须删除任何先前的摘要文件以开始新文件。

流程摘要要在工具执行期间增量写出，允许检查进行中分析的流程摘要。要停止写入，将 sh_flow_summary_file 设为空字符串。停止时，工具在输出文件添加流程摘要功能本身总开销的最终行：

```
Runtime overhead of flow summary utility is: 3.1 s
```

4.7.3 定义自定义流程段

Defining Custom Flow Segments

自定义流程段（custom flow segments）允许将脚本部分分组为额外的命名上下文层级结构，便于分离长脚本部分或洞察未捕获运行时间发生位置。使用 `define_custom_flow_segment` 命令在脚本中定义命名起点与终点。流程段应在同一脚本文件上下文中开始与结束；嵌套流程段须按顺序关闭（低层级先于高层级）。

```
define_custom_flow_segment -start -name drc_max_tran
# commands in this custom segment
define_custom_flow_segment -end -name drc_max_tran
```

4.7.4 写入多个流程摘要报告

Writing to Multiple Flow Summary Reports

要将当前流程摘要报告停止并开始写入新报告，将 `sh_flow_summary_file` 设为新文件名。若有进行中的现有流程摘要报告，会自动关闭。从顶层以下上下文层级停止与启动流程摘要报告时需注意：若在较低上下文层级停止，待处理层级关闭并写入报告；若从较低上下文层级启动，当前待处理上下文层级不作为报告中的开层级写入，当前上下文视为顶层，“<end of source preceding flow summary>”消息指示现有上下文退出的位置。

4.7.5 限制

Limitations

- 时间值四舍五入到最近的 0.1 秒打印，个别值之和可能与总值不完全一致
- Tcl 过程不在流程摘要中引入显式层级，但其内部活动正常汇总
- `parallel_execute` 与 `parallel_foreach_in_collection` 命令作为单条命令列出，无工作线程详情

4.8 分析 Tcl 脚本性能

Profiling the Performance of Tcl Scripts

要识别 Tcl 脚本中的运行时与内存瓶颈，可在时序运行期间执行 Tcl 分析（profiling）。此能力可帮助修复性能与容量问题。

执行 Tcl 分析的步骤：

1. 运行 `start_profile` 命令开始收集分析数据。默认 `start_profile` 跟踪运行超过 0.1 秒或使用超过 100 KB 内存的命令。使用 `-verbose` 选项时跟踪所有命令。
2. 运行要分析的 Tcl 脚本。
3. 使用 `stop_profile`、`exit` 或 `quit` 命令停止收集分析数据。

4. 评估自动生成的 Tcl profile 与 stopwatch 报告。默认情况下，工具将这些文件写入当前工作目录下 profile 子目录。

表 4.5: Tcl profile 与 stopwatch 报告文件

文件名	说明
tcl_profile_summary_latest.html	HTML 格式 Tcl profile 报告摘要页，显示分析起止时间并提供按 CPU 时间、调用次数、经过时间或内存使用排序的报告链接
tcl_profile_summary_latest.txt	纯文本摘要页
tcl_profile_sorted_by_cpu-time*.txt	按 CPU 时间排序的 Tcl profile 报告
tcl_profile_sorted_by_delta_mem-*.txt	按内存使用排序
tcl_profile_sorted_by_elapsed-time*.txt	按经过时间排序
tcl_profile_sorted_by_num_calls-*.txt	按调用次数排序
tcl_stopwatch_*.txt	纯文本 stopwatch 报告

Tcl 脚本 profile 报告摘要示例：get_timing_paths 命令从 gtp_only.tcl 脚本调用一次，执行耗时 8 秒（Elapsed 列）。



图 4.12: Stopwatch 报告示例

stopwatch 报告（tcl_stopwatch_*.txt）是显示每条命令的时间、性能与内存指标的 ASCII 文本文件，包含 STOPWATCH[START]、STOPWATCH[start_update_timing]、STOPWATCH[after_update_timing] 等条目，记录命令执行开始与完成时的当前指标及与上一 stopwatch 条目的差值。

Chapter 5

处理设计数据

Working With Design Data

在执行时序分析之前，需要读取并链接设计及其关联的逻辑库。有关处理设计数据的说明，请参阅以下主题：

- 逻辑库（Logic Libraries）
- 读取与链接设计（Reading and Linking the Design）
- 处理设计对象（Working With Design Objects）
- 设置当前设计与当前实例（Setting the Current Design and Current Instance）
- 处理属性（Working With Attributes）
- 保存属性（Saving Attributes）
- 保存与恢复会话（Saving and Restoring Sessions）
- 创建测试用例（Creating Test Cases）

5.1 逻辑库

Logic Libraries

逻辑库使用 Library Compiler 工具描述 ASIC 工艺中宏单元（macro cell）的时序与功能。其他 Synopsys 工具（如 Design Compiler 综合工具与 IC Compiler 布局布线工具）也使用这些逻辑库。

逻辑库包含库单元描述，其中包括：

- 描述各单元与外部连接关系的单元、总线与引脚结构
- 单元输出引脚的逻辑功能
- 时序分析与设计优化信息，例如引脚到引脚的时序关系、延时参数，以及时序单元的时序约束
- 描述面积、功耗与设计规则约束的其他参数

PrimeTime 可读取 `.db` 与 `.lib` 格式的逻辑库。这些库可以具有不同的时间、电容与电压单位。有关逻辑库的更多信息，请参阅 Library Compiler 文档。

5.2 读取与链接设计

Reading and Linking the Design

在执行时序分析之前，需要读取并链接设计与逻辑库。PrimeTime 可读取以下文件格式：

表 5.1: PrimeTime 支持的输入数据格式

输入数据	支持的文件格式
设计数据	二进制数据库 (<code>.db</code>) Milkyway Synopsys 逻辑数据库 (<code>.ddc</code>) Verilog VHDL
逻辑库	二进制数据库 (<code>.db</code>) Synopsys Library Compiler 格式 (<code>.lib</code>)

要读取并链接设计数据，请按以下步骤操作：

1. 指定 PrimeTime 搜索设计、逻辑库及其他设计数据（如时序模型）的目录。为此，设置 `search_path` 变量。例如：

```
pt_shell> set_app_var search_path ". /abc/design /abc/libs"
```

PrimeTime 按您指定的顺序搜索这些目录。

2. 通过设置 `link_path` 变量，指定 PrimeTime 在设计层次结构中查找单元的库。例如：

```
pt_shell> set_app_var link_path "* STDLIB.db"
```

该变量可包含星号（*）、库名与文件名。星号指示 PrimeTime 在内存中搜索设计。PrimeTime 按您指定的顺序搜索库。主库（main library）是链接路径中的第一个库。

3. 将设计读入内存：

```
pt_shell> read_verilog TOP.v
```

若搜索路径包含子设计文件，则只需读取顶层设计。搜索路径条目必须包含子设计文件的完整路径，而不仅仅是包含该文件的目录。

4. 链接设计以解析对库单元与子设计的引用：

```
pt_shell> link_design TOP
```

在设计链接期间，若子设计名称与文件名匹配，工具会自动加载子设计。

5. 使用 `list_designs` 与 `list_libs` 命令验证已加载的设计与库。

5.2.1 识别与验证所需设计文件

Identifying and Verifying the Required Design Files

在运行完整时序分析之前，快速识别脚本所需的全部设计文件并验证其存在，通常很有用。为此，在运行脚本之前设置以下变量：

```
pt_shell> set_app_var sh_program_mode file_identification
```

在此模式下，工具会打开并立即关闭任何被访问的文件，但不读取设计数据。工具命令采用表 5.2 所述的修改后行为，使脚本能够在没有设计数据的情况下运行完成。

表 5.2: 文件识别模式下的修改后命令行为

命令类型	修改后行为
显式或隐式读取库的命令	打开库以确认读访问权限，然后立即关闭。库按常规方式从配置设置（link_path 变量、缩放组等）收集。
读取文件的命令	打开文件以确认读访问权限，然后立即关闭。search_path 变量按常规方式使用。
返回集合（collection）的命令	返回特殊的 =mock_collection= 集合句柄。
get_attribute 命令	对于 =mock_collection= 对象，返回字符串值 =mock_collection=attr=。
sizeof_collection 命令	对于 =mock_collection= 对象，返回值 1。
条件与迭代（循环）命令	若条件或迭代列表包含特殊的 =mock_collection= 集合句柄，则命令体执行一次；否则按常规方式求值循环或条件。
restore_session 命令	打开会话目录中的所有文件，然后立即关闭。不读取或恢复任何数据。
echo 命令	按常规方式打印输出。
所有其他命令	返回值 1。

可使用 pt_shell 调用命令的 -x 选项在脚本运行前设置该变量。用法示例如下：

- 运行本地脚本：

```
% $PT_PATH/pt_shell \  
-x 'set sh_program_mode file_identification' \  
-f my_script.pt
```

- 使用 Synopsys Testcase Packager（STP）实用程序：

```
% stp -r \  
"$PT_PATH/pt_shell \  
-x 'set sh_program_mode file_identification' \  
-f my_script.pt"
```

- 使用 Synopsys Cloud Transfer Packager（SCTP）实用程序：

```
% sctp -r \
"$PT_PATH/pt_shell \
-x 'set sh_program_mode file_identification' \
-f my_script.pt"
```

要识别脚本使用的完整文件列表，可使用 Synopsys Cloud Transfer Packager。该实用程序会生成所需文件列表供审阅；无需打包测试用例即可生成此列表。

请注意以下限制：

- 不支持 DMSA 分析。
- 不支持分布式分析。
- 不支持从设计对象名称派生文件名的脚本（例如根据设计名称创建 SDC 文件名）。

5.2.2 按实例链接库路径

Per-Instance Link Library Paths

`link_path` 变量提供用于整个设计的链接路径规范。

如有需要，可将链接路径规范应用于各个单元实例。这些按实例的链接路径优先于全局 `link_path` 规范。

按实例链接规范可用于建模工艺与版图效应，例如体偏置（body bias）与邻近相互作用（proximity interactions）。也可用于为设计块分配电压相关库（不过对此更推荐使用缩放库组）。

有两种方式指定按实例链接路径：

- `link_path_per_instance` 变量，设置为所有按实例链接路径的列表
- `set_link_lib_map` 命令，指定包含按实例链接路径的文件

两种方法在功能上完全相同。`link_path_per_instance` 变量无需额外文件，但规范存储在内存中，适用于较小的链接路径列表。`set_link_lib_map` 命令需要一个或多个文件，仅在链接时读取，适用于较大的链接路径列表。两者不能同时使用。

`link_path_per_instance`

link_path_per_instance `link_path_per_instance` 变量格式为列表的列表。每个子列表由一对元素组成：一组实例，以及用于这些实例的 `link_path` 规范。该变量必须在设计链接之前设置。例如：

```
set_app_var link_path {* lib1.db}
set_app_var link_path_per_instance [list \
  [list {ucore1 ucore2}          {* lib2.db}] \
  [list {ucore1/usubblk}         {* lib3.db}] \
  [list {ucore1/U12 ucore1/U34} {* lib4.db}]]

read_verilog my_design.v
link_design MY_DESIGN
```

所列实例可以是层次单元（块）或叶单元。所有实例路径均相对于顶层设计。优先级规则如下：

- 规范向下传播到层次结构中。

- 较低层级规范优先于较高层级规范。
- 叶单元规范优先于块规范。

set_link_lib_map

set_link_lib_map `set_link_lib_map` 命令以文件作为输入。文件中每行包含一组实例、两侧带空格的冒号分隔符 (:), 以及用于这些实例的 `link_path` 规范。例如:

```
% cat ./link_paths.txt
ucore1 ucore2      : * lib2.db
ucore1/usubblk     : * lib3.db
ucore1/U12 ucore1/U34 : * lib4.db
```

`set_link_lib_map` 在设计链接之前将该文件作为输入。例如:

```
set_app_var link_path {* lib1.db}
set_link_lib_map ./link_paths.txt

read_verilog my_design.v
link_design MY_DESIGN
```

所列实例可以是层次单元 (块) 或叶单元。默认情况下, 实例路径相对于顶层设计。不过, `set_link_lib_map` 命令提供两个选项, 允许文件相对于单元实例与设计引用应用:

```
set_link_lib_map -instance {ucore1/usubblk1} ./link_paths_subcore.txt
set_link_lib_map -reference {subblk}          ./link_paths_subcore.txt
```

`-instance` 选项将文件相对于指定单元实例应用。`-reference` 选项将文件相对于指定设计名称的所有单元实例应用。这些选项允许在输入文件中缩短较长的实例名称。两个选项均只接受单个名称。

优先级规则如下:

- 规范向下传播到层次结构中。
- 较低层级规范优先于较高层级规范。
- 叶单元规范优先于块规范。
- `-instance` 规范优先于 `-reference` 规范。

5.3 处理设计对象

Working With Design Objects

设计是由单元、端口与网络等对象组成的层次实体。

在 PrimeTime 工具中, 设计包含下表所列对象。

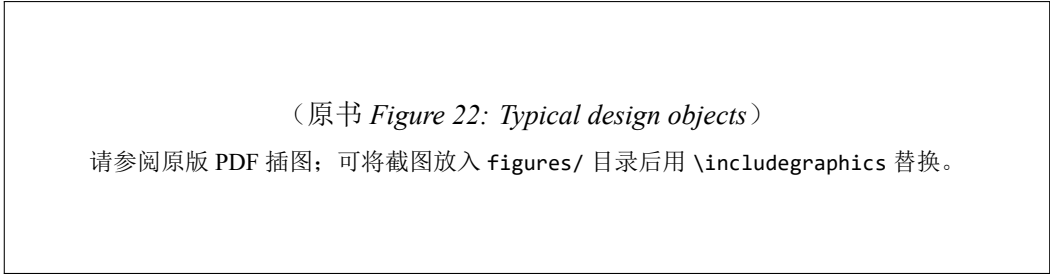


图 5.1: 典型设计对象

表 5.3: PrimeTime 中的设计对象

对象类	说明	创建对象集合的命令
cell	设计中的实例；可以是层次块或原始库单元	get_cells
clock	时钟	get_clocks
design	设计	get_designs
lib	库	get_libs
lib_cell	逻辑库中的单元	get_lib_cells
lib_pin	库单元上的引脚	get_lib_pins
lib_timing_arc	库单元上的时序弧	get_lib_timing_arcs
net	当前设计中的网络	get_nets
path_group	用于代价函数计算与时序报告的路径组	get_path_groups
pin	设计中较低层级单元的引脚；可以是输入、输出或双向 (inout)	get_pins
port	当前设计的端口；可以是输入、输出或双向 (inout)	get_ports
timing_arc	时序弧	get_timing_arcs
timing_path	时序路径	get_timing_paths

要对设计施加约束、执行详细时序分析并定位时序问题来源，需要访问设计对象。可通过相应的“get”命令创建对象集合来实现。例如，get_ports 命令创建端口集合。

您可以将“get”命令的结果嵌套在操作这些对象的另一命令中。例如：

```
pt_shell> set_input_delay 2.3 [get_ports IN*]
```

更多信息请参阅 SolvNetPlus 上的 *Using Tcl With Synopsys Tools* 中「Searching for Design Objects」一节。

5.4 设置当前设计与当前实例

Setting the Current Design and Current Instance

当前设计与当前实例定义了许多 PrimeTime 命令的焦点。要设置或返回当前设计或实例，请使用以下命令。

表 5.4: 当前设计对象

对象	说明	设置或返回当前对象的命令
当前设计 (<code>current design</code>)	当前设计的顶层；大多数对象相对于当前设计引用	<code>current_design</code>
当前实例 (<code>current instance</code>)	作为当前设计范围内当前作用域的实例（层次单元）；通过更改当前实例遍历层次结构	<code>current_instance</code>

5.5 处理属性

Working With Attributes

属性是与对象关联的字符串或值，携带有关该对象的信息。您可以用 Tcl 编写程序，从设计数据库获取属性信息并生成自定义设计报告。

PrimeTime 提供以下命令用于设置、报告、列出与创建属性。

表 5.5: 处理属性的命令

属性命令	说明
<code>define_user_attribute</code>	为一个或多个对象类创建新属性
<code>get_attribute</code>	从单个对象检索任意属性的值
<code>list_attributes</code>	显示每个对象类或指定对象类已定义的属性；可选显示应用属性
<code>remove_user_attribute</code>	从一个或多个对象移除用户定义属性
<code>report_attribute</code>	显示一个或多个对象上所有属性的值；可选显示应用属性
<code>set_user_attribute</code>	在一个或多个对象上设置用户定义属性

有关各对象类预定义应用属性的说明，请参阅 man 页。例如，要查看库单元属性的说明，使用以下命令：

```
pt_shell> man lib_cell_attributes
```


5.6 保存属性

Saving Attributes

`save_session` 命令不保存属性。要保存在会话期间应用的属性，请使用 `write_script` 命令创建 `pt_shell`、`dc_shell` 或 `dctcl` 格式的脚本。恢复 PrimeTime 会话后，可使用该脚本在设计上重新创建这些属性。

5.7 保存与恢复会话

Saving and Restoring Sessions

可使用 `save_session` 与 `restore_session` 命令保存一个会话中的设计数据，然后在后续会话中恢复以供继续使用。

会话是工具创建的目录，包含以下信息：

- 已链接的设计与已加载的库
- 时钟、时序例外与其他约束
- 工作条件
- 反标 SDF 延时与寄生参数
- 变量设置
- 网表编辑（`insert_buffer`、`size_cell`、`swap_cell`）
- 分析数据
- 交叉耦合延时数据与噪声数据

保存与恢复功能仅用作 PrimeTime 会话工具，而非设计数据库归档工具。工具不保存或恢复以下信息：

- 派生数据类型的集合
- Tcl 过程与命令历史
- GUI 状态（时序路径表、原理图、直方图等）
- 快速时序模型生成状态（`create_qtm_model` 与 `save_qtm_model` 命令之间）
- 上下文表征状态（`characterize_context` 与 `write_context` 命令之间）

5.7.1 保存会话

Saving Sessions

要保存会话，使用 `save_session` 命令并指定会话目录名（必需）：

```
pt_shell> save_session my_session
Saving environmental constraints.....
Saving netlist information.....
```

```

Saving miscellaneous application information.....
Saving timing information.....
Saving variable information.....
Information: Executed with 4 workers
Information: At least 6 MB of free disk space in pt_tmp_dir will be
required to restore this session. (SR-044)

```

若目录已存在，工具会发出信息消息并覆盖目录内容：

```

pt_shell> save_session my_session
Information: Cleaning and overwriting all data in the existing directory
'/remote/cae1180/chrispy/hist/my'. (SR-002)
Saving environmental constraints.....
...

```

5.7.2 保存版本兼容会话

Saving Version-Compatible Sessions

默认情况下，`save_session` 命令保存仅可恢复到相同工具版本的版本特定会话。

不过，`-version` 选项允许您保存版本特定（默认）、版本兼容或两者兼有：

- `save_session -version compatible`

保存可恢复到相同或更高版本 PrimeTime 工具的版本兼容会话。恢复会话后始终需要时序更新。

- `save_session -version specific`

保存版本特定会话。恢复会话时不需要时序更新。虽然这是默认行为，但该关键字提供显式指定默认的方式。

- `save_session -version both`

保存组合的版本兼容与版本特定会话。仅当恢复到不同版本时才需要时序更新。

要将已保存会话标识为版本兼容，工具会在会话目录内的 README 文件中包含以下说明：

```

***** This session is saved in version compatible mode and
it can be restored in newer versions of PrimeTime *****

```

此会话格式无法存储时序更新后的内存数据；因此恢复会话后需要时序更新才能执行分析。恢复到较新版本工具时，这可能导致 QoR 出现轻微差异。

同样，用于 ECO 修复的内存物理数据不会存储，将在运行 `check_eco` 命令或 `fix_eco_*` 命令时重新加载。

典型的版本兼容会话恢复方式如下：

```

# restore the session
restore_session session_name

# (optional) apply any new settings not stored in the session
set_app_var ...
set_app_var ...

# recompute timing

```

```
update_timing

# perform any additional analysis operations here (reporting, etc.)
...
```

在 ECO 流程中，上述脚本继续如下：

```
# (optional) apply any new settings not stored in the session
set_eco_options ...

# ensure everything is ready to go
check_eco

# perform ECO fixing
fix_eco_*

# perform any additional ECO operations here
```

5.7.3 在保存的会话中包含 ECO 物理数据

Including ECO Physical Data in Saved Sessions

在物理感知 ECO 流程中保存版本兼容会话时，可使用 `-include` 选项的 `physical_data` 关键字包含物理数据文件的副本：

```
pt_shell> save_session my_session \
          -version compatible \
          -include {physical_data}
```

如「物理感知 ECO」（Physically Aware ECO）所述，物理数据文件由 `set_eco_options` 命令配置。这包括：

- LEF 工艺与单元定义文件
- DEF 设计数据文件
- 物理库约束与规则
- Via ladder 定义与关联

使用此功能在保存的会话文件中包含物理数据时，ECO 物理数据目录会被复制到已保存的会话目录中。（默认情况下，仍存储 ECO 配置数据，但其指向会话目录外的文件。）

注意

通过 `set_eco_option -physical_icc2_lib` 选项指定的 IC Compiler II 参考库不会复制到会话目录。其原始位置存储在 `session_dir/eco_config` 文件中。如有需要，可在恢复会话前编辑这些路径。

此功能仅支持版本兼容会话。

5.7.4 恢复会话

Restoring Sessions

`restore_session` 命令读取由 `save_session` 命令写入的目录，将 PrimeTime 会话恢复到发出 `save_session` 命令时会话的状态。

默认情况下,PrimeTime 要求恢复到与原始会话相同的 PrimeTime 版本。从会话目录中的 README 文件可找到用于保存会话的版本。

若使用「保存版本兼容会话」所述功能，则会话可恢复到相同或更高版本的 PrimeTime。此时，恢复会话时工具会报告原始工具版本：

```
pt_shell> restore_session my_vc_session
...
Information: Restoring the version compatible session saved with version
'Q-2019.12-SP4'. (SR-048)
```

若内存中已有设计或库数据，会在恢复会话之前将其移除。

该命令首先尝试获取 `save_session` 命令发出时存在的所有许可证。若失败，则恢复会话失败。

恢复目录包含名为 `lib_map` 的 ASCII 文件。该文件包含恢复会话所需的每个逻辑库的路径与叶名称。如有必要，可编辑路径名，为 `restore_session` 命令提供所需逻辑库文件的更新位置。这些文件的叶名称不能更改。假定逻辑库与发出 `save_session` 命令时使用的相同。

5.7.5 恢复版本兼容会话

Restoring Version-Compatible Sessions

PrimeTime 在会话中保存两类设置：

- 应用变量（application variables）——由 `report_app_var` 命令报告的默认与用户指定设置
- 程序选项（program options）——影响分析且有时在不同版本间会变化的内部程序设置，但不由 `report_app_var` 命令报告

将版本兼容会话恢复到不同版本时，`restore_session` 命令允许您控制内部程序选项的处理方式：

- `restore_session -program_options save_version`

此设置使用已保存会话中存储的程序选项，有助于新版本中的分析与原始会话匹配。这是默认行为。

- `restore_session -program_options restore_version`

此选项使用当前（恢复）版本的程序选项默认值，允许使用当前版本中的更新程序选项。

5.8 创建测试用例

Creating Test Cases

在某些情况下，您可能希望将部分设计逻辑提取为可独立运行的测试用例。例如，您可能希望试验设置或约束以观察分析如何受影响；或者需要将测试用例发送给 Synopsys 技术支持以作进一步调查。

可使用 `create_testcase` 命令完成此操作。它会创建一个目录，其中包含指定部分的设计、适用于该部分的约束文件、所使用的详细寄生参数，以及测试用例的运行脚本。

`create_testcase` 命令的语法如下：

Usage:

```
create_testcase      # Create a testcase for specified design objects
[-paths list]       (create testcase for the given paths)
[-ports list]        (create testcase for the given ports)
[-endpoints list]    (create testcase for the given endpoints)
[-cells list]        (create testcase for the given cells)
[-directory string] (Output directory name)
[-include_libs]      (Include libraries)
[-mim]              (Include multi-instance modules)
```

以下各节说明这些选项。

5.8.1 控制保留哪些设计逻辑

Controlling What Design Logic to Keep

`create_testcase` 命令仅保留您指定的设计逻辑。表 5.6 列出指定保留内容的不同方式。

表 5.6: 指定 `create_testcase` 保留的设计逻辑

选项名	保留的设计逻辑	是否包含到时序单元的时钟路径？
<code>-paths path_collection</code>	时序路径上的所有单元，包括起点与终点	是
<code>-ports port_list</code> 到输出端口扇出的逻辑，包括起点	从输入端口扇出的逻辑，包括终点 是	
<code>-endpoints pin_port_list</code>	到终点引脚/端口的扇入逻辑，包括起点	是
<code>-cells cell_list</code> 这些单元驱动的网络 连接到被驱动网络的叶单元与端口	指定的叶单元 否	

必须指定且只能指定一个设计逻辑选项（`-paths`、`-ports`、`-endpoints` 或 `-cells`）。不能组合多个选项。

在 PrimeTime SI 分析中，会包含已反标 aggressor 时序窗口的 aggressor stage。

使用 `-ports` 或 `-endpoints` 选项时，设计不必已经完成时序更新。此时，aggressor 使用零输入引脚转换时间和无限时序窗口。这对于复现首次时序更新期间发生的崩溃很有用。

5.8.2 包含逻辑库

Including the Logic Libraries

默认情况下，`create_testcase` 命令创建的测试用例在其当前位置引用所需的逻辑库。这可避免为在当前设计环境中使用的测试用例占用不必要的磁盘空间。

若测试用例必须在另一设计环境中运行，请指定 `-include_libs` 选项以将所需逻辑库复制到测试用例目录：

```
pt_shell> create_testcase ... -include_libs
```

5.8.3 指定输出目录

Specifying the Output Directory

默认情况下，测试用例创建在以当前设计命名的目录中：

```
pt_shell> create_testcase ...
...
Testcase is generated in the TOP directory.
1
```

要指定特定的测试用例输出目录，使用 `-directory` 选项：

```
pt_shell> create_testcase ... -directory ./my_testcase_FLAT
...
Testcase is generated in the my_testcase_FLAT directory.
1
```

5.8.4 创建多实例模块测试用例

Creating a Multiple-Instance Module Test Case

对于具有多实例模块（MIM）的设计，`create_testcase` 命令提供一项功能，可在 MIM 实例之间并行保留相同逻辑。

当保留到 MIM 块内终点的逻辑时，可指定 `-mim` 选项以在该 MIM 块的所有实例中保留相同的终点扇入逻辑：

```
pt_shell> create_testcase \
    -endpoints $endpoint_in_one_MIM \
    -mim
```

此功能要求必须使用 `-endpoints` 选项。

5.8.5 限制

Limitations

`create_testcase` 命令不支持以下工具功能：

- HyperScale 数据（块模型或上下文数据）
- SMVA/DVFS 分析

Chapter 6

约束设计

Constraining the Design

要了解如何约束设计的输入与输出，请参阅：

- 时序约束
- 输入延迟
- 输出延迟
- 输入端口驱动特性
- 端口电容
- 线负载模型
- 转换时间传播
- 设计规则约束
- 理想网络
- 检查约束

6.1 时序约束

Timing Constraints

开始时序分析之前，需要为设计指定时序约束。时序约束（也称 **timing assertion**）限制信号到达器件输入或在器件输出保持有效的允许时间范围。

图中展示了数据到达时间（**data arrival times**）与数据要求时间（**data required times**）的关系。`create_clock` 定义时钟；`set_input_delay` 与 `set_output_delay` 分别约束输入与输出端口相对于时钟边沿的延迟。

(原书 *Figure 23: Design-level timing constraints*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 6.1: 设计级时序约束

6.2 输入延迟

Input Delays

要在设计输入处进行约束检查，工具需要输入信号到达时间的信息。要指定通向输入端口的外部路径时序，使用 `set_input_delay` 命令。工具利用该信息检查输入端口及其传递扇出（transitive fanout）中的时序违例。使用该命令可指定从时钟边沿到指定输入端口信号到达的最小与最大延迟量。

对端口应用 `set_drive` 或 `set_driving_cell` 命令，会使该端口具有 cell delay，其值为外部驱动单元延迟中与负载相关的部分。为避免重复计入该延迟，应估算驱动单元延迟中与负载相关的部分，再从该端口的输入延迟中减去这一数值。

输入延迟应等于从源触发器时钟引脚到驱动单元输出引脚的路径延迟，减去驱动单元延迟中与负载相关的部分。下图展示从 L1 时钟端口到 IN1 端口的外部路径。

(原书 *Figure 24: Two-phase clocking example for setting port delays*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 6.2: 设置端口延迟的双相时钟示例

使用 `set_input_delay` 时，可指定延迟值是否包含 network latency 或 source latency。

示例 1: 若从 L1 时钟端口到 IN1（减去驱动单元 load-dependent 延迟）的延迟为 4.5，则：

```
pt_shell> set_input_delay 4.5 -clock PHI1 {IN1}
```

示例 2: 若多条来自不同时钟或边沿的路径到达同一端口，使用 `-add_delay` 选项分别指定。若省略 `-add_delay`，现有数据将被移除。例如：

```
pt_shell> set_input_delay 2.3 -clock PHI2 -add_delay {IN1}
```

若延迟源为电平敏感锁存器，使用 `-level_sensitive` 选项，使 PrimeTime 能确定从该端口出发路径的正确单周期时序约束。使用 `-clock_fall` 表示负电平敏感锁存器；否则 `-level_sensitive` 隐含正电平敏感锁存器。

查看端口输入延迟，使用 `report_port -input_delay`。移除 `set_input_delay` 设置的端口或引脚输入延迟信息，使用 `remove_input_delay`。默认移除 `port_pin_list` 选项中的所有输入延迟信息。

6.2.1 输入端口同时作为时钟与数据

Using Input Ports Simultaneously for Clock and Data

PrimeTime 允许输入端口同时作为时钟端口与数据端口。可用 `timing_simultaneous_clock_data_port_compatibility` 变量启用或禁用该行为。

当该变量为 `false`（默认）时，同时行为被启用，可用 `set_input_delay` 相对于时钟定义输入端口时序要求。此时：

- 若 `set_input_delay` 相对于同一端口定义的时钟，且该端口有 `data sink`，命令被忽略并发出错误消息——同一端口信号不能同时作为相对于时钟的数据和时钟本身。
- 若 `set_input_delay` 相对于不同端口定义的时钟，且端口有 `data sink`，输入延迟被设置并控制从该端口相对于时钟发射的数据边沿。
- 无论数据端口位置如何，若时钟端口未扇出到 `data sink`，时钟端口上的输入延迟被忽略并发出错误消息。

将 `timing_simultaneous_clock_data_port_compatibility` 设为 `true` 时，同时行为被禁用，`set_input_delay` 定义相对于时钟的到达时间。此时若输入端口上定义了时钟，PrimeTime 将该端口仅视为时钟端口，并对发射的数据边沿施加限制，且阻止相对于另一时钟设置输入延迟。

要控制输入端口上定义时钟的 `clock source latency`，必须使用 `set_clock_latency` 命令。

6.3 输出延迟

Output Delays

要在设计输出处进行约束检查，工具需要输出时序要求的信息。要指定输出端口到寄存器的延迟，使用 `set_output_delay` 命令。

使用该命令可指定输出端口与从该端口捕获数据的外部时序器件之间的最小与最大延迟量，从而确定信号在输出端口必须可用的时间，以满足外部时序元件的 `setup` 与 `hold` 要求：

- `Maximum_output_delay` = 到寄存器 `data pin` 的最长路径长度 + 寄存器 `setup_time`
- `Minimum_output_delay` = 到寄存器 `data pin` 的最短路径长度 - `hold_time`

（原书 *Figure 25: Two-phase clocking example for setting port delays*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 6.3: 设置端口延迟的双相时钟示例（输出）

上例中，以下命令为 OUT1 端口设置相对于 PHI1 时钟上升沿 4.3 的输出延迟：

```
pt_shell> set_output_delay 4.3 -clock PHI1 {OUT1}
```

查看端口关联的输出延迟，使用 `report_port -output_delay`。通过 `set_output_delay` 设置的输出端口或引脚输出延迟，使用 `remove_output_delay` 移除。默认移除列表中每个对象的所有输出延迟。可用 `-clock`、`-clock_fall`、`-min`、`-max`、`-rise` 或 `-fall` 限制移除范围。

6.4 输入端口驱动特性

Drive Characteristics at Input Ports

要准确计时设计，需定义驱动每个输入端口的外部单元驱动能力。PrimeTime 利用该信息计算端口 load-dependent cell delay，并产生准确的 transition time，用于计算后续逻辑级的 cell delay 与 transition time。

(原书 *Figure 26: Driving cells*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 6.4: 驱动单元

`set_driving_cell` 命令可指定驱动单元的库 cell arc，使时序计算在电容变化时仍保持准确。该命令使端口 transition time 按指定库单元驱动该端口的方式计算。

对于精度较低的计算，可使用 `set_drive` 或 `set_input_transition` 命令。最近的 `drive` 命令优先。若先对端口执行 `set_drive`，再对同一端口使用 `set_driving_cell`，`set_drive` 的信息将被移除。

6.4.1 设置端口驱动单元

Setting the Port Driving Cell

`set_driving_cell` 命令指示 PrimeTime 按指定库单元实例计算延迟，端口 delay 仅含 load-dependent 部分。端口 transition time 也按该库单元驱动网络的方式计算。`set_driving_cell` 设置的端口 delay 利用该库单元的实际 delay model（非线性或线性）。对有驱动单元或 drive resistance 的端口，指定输入延迟时不应包含端口 load-dependent delay。

显示端口 transition 或 drive 能力信息，使用 `report_port -drive`。`set_driving_cell` 可用 `-input_transition_rise` 或 `-input_transition_fall` 指定驱动单元输入的上升与下降 transition time。未指定输入 transition 时，默认为 0。

6.4.2 设置端口驱动电阻

Setting the Port Drive Resistance

`set_drive` 命令为当前设计输入与 inout 端口定义外部驱动强度或电阻。存在 wire load model 时，端口报告的 transition time 与 delay 等于 $R_{\text{driver}} \times C_{\text{total}}$ 。PrimeTime 在计算后续逻辑级 delay 时使用该 transition time。

当输入端口驱动能力无法用逻辑库中的单元表征时，可用 `set_drive` 在设计顶层端口设置 `drive resistance`。但该命令对非线性 `delay model` 不如 `set_driving_cell` 准确。`set_drive` 适用于无法指定库单元的情况（例如驱动为未建模为 Synopsys 库单元的自定义模块）。

6.4.3 设置固定端口转换时间

Setting a Fixed Port Transition Time

`set_input_transition` 命令为输入端口定义固定 `transition time`。端口 `cell delay` 为零。PrimeTime 仅在计算端口驱动逻辑 `delay` 时使用指定 `transition time`。

固定 `transition time` 适用于：

- 将设计与仅支持 `input transition time` 的其他工具进行时序对比；
- 定义芯片顶层端口 `transition`——存在大外部驱动与大外部电容时，`transition time` 相对独立于当前设计中的电容。

6.4.4 显示驱动信息

Displaying Drive Information

显示驱动信息，使用 `report_port -drive`。

6.4.5 从端口移除驱动信息

Removing Drive Information From Ports

从端口移除驱动信息，使用以下命令：

表 6.1: 移除驱动信息的命令

命令	说明
<code>remove_driving_cell</code>	从指定端口移除 <code>driving cell</code> 信息
<code>reset_design</code>	移除 <code>drive</code> 数据及所有用户指定数据（如时钟、输入/输出延迟）
<code>set_drive 0.0</code>	移除 <code>drive resistance</code>
<code>set_input_transition 0.0</code>	移除 <code>input transition</code>

6.5 端口电容

Port Capacitance

要准确计时设计，需用 `set_load` 命令描述连到顶层端口网络的 `external load capacitance`，包括 `pin` 与 `wire capacitance`。

示例 1：指定端口 `external pin capacitance`：

```
pt_shell> set_load -pin_load 3.5 {IN1 OUT1 OUT2}
```

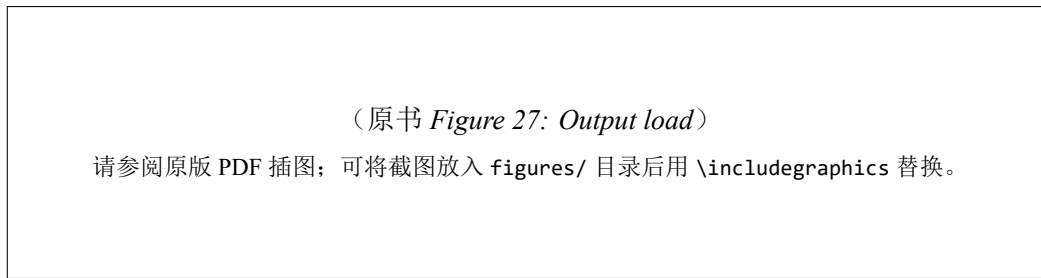


图 6.5: 输出负载

还需考虑端口外部的 wire capacitance。对于 prelayout，指定 external wire load model 与 external fanout 点数（见「手动设置线负载模型」）。

示例 2：对于 postlayout，将 external annotated wire capacitance 指定为端口 wire capacitance：

```
pt_shell> set_load -wire_load 5.0 {OUT3}
```

移除端口电容值，使用 remove_capacitance 命令。

6.6 线负载模型

Wire Load Models

要准确计算 net delay，PrimeTime 需要互连 wire 寄生负载信息。布局布线完成前，PrimeTime 使用逻辑库提供的 wire load model 估算这些负载。逻辑库供应商提供统计 wire load model，根据网络上 fanout pin 数量估算 wire load。可手动或自动设置 wire load model。

6.6.1 手动设置线负载模型

Setting Wire Load Models Manually

手动在设计、实例、cell 列表或端口列表上设置命名 wire load model，使用 set_wire_load_model 命令。

例如，设计层次为：

TOP

```
MID (instance u1)
  BOTTOM (instance u5)
MID (instance u2)
  BOTTOM (instance u5)
```

在 BOTTOM 实例上设置 10x10 model、MID 实例上设置 20x20 model、顶层网络上设置 30x30 model：

```
pt_shell> set_wire_load_mode enclosed
pt_shell> set_wire_load_model -name 10x10 [all_instances BOTTOM]
pt_shell> set_wire_load_model -name 20x20 [all_instances MID]
pt_shell> set_wire_load_model -name 30x30
```

要捕获连到端口网络的外部部分，可设置 `external wire load model` 与 `fanout` 点数。例如对当前设计端口 Z:

```
pt_shell> set_wire_load_model -name 70x70 [get_ports Z]
pt_shell> set_port_fanout_number 3 Z
```

PrimeTime 假设端口 Z fanout 为 3，使用 70x70 wire load model 计算 delay。

查看当前设计或实例的 wire load model 设置，使用 `report_wire_load`。查看端口 wire load 信息，使用 `report_port -wire_load`。移除用户指定的 wire load model 信息，使用 `remove_wire_load_model`。

6.6.2 自动线负载模型选择

Automatic Wire Load Model Selection

更新设计时序信息时，PrimeTime 可自动设置 wire load。若未为设计或模块指定 wire load model，PrimeTime 根据 wire load selection group（若已指定）自动选择 model。

若未应用 wire load model 或 selection group，但库定义了 `default_wire_load` model，PrimeTime 将库定义 model 应用于设计。否则 wire resistance、capacitance、length 与 area 均为 0。

自动 wire load 选择由 selection group 控制，将 cell 模块尺寸映射到 wire load model。若在顶层设计或主逻辑库定义了 `default_wire_load_selection_group`，PrimeTime 自动启用 wire load 选择。

启用 wire load 选择时，根据模块 cell area 自动为大于最小 cell area 的层次模块选择 wire load。设置自动 wire load 选择的最小模块尺寸：

```
pt_shell> set_wire_load_min_block_size size
```

其中 `size` 为库 cell area 单位的最小模块尺寸，须 ≥ 0 。

`auto_wire_load_selection` 环境变量控制自动 wire load 选择，默认为 `true`。禁用：

```
pt_shell> set auto_wire_load_selection false
```

移除 wire load selection group 设置，使用 `remove_wire_load_selection_group`。

6.6.3 设置线负载模式

Setting the Wire Load Mode

当前 wire load mode（由 `set_wire_load_mode` 设置）决定设计层次各层使用的 wire load model。三种模式：`top`、`enclosed`、`segmented`。

若顶层设计 mode 为 `top`，顶层 wire load model 用于计算设计内所有层次所有网络的 wire capacitance。若顶层设计 mode 为 `enclosed` 或 `segmented`，层次 cell 上的 wire load model 用于计算这些模块内网络的 wire capacitance、resistance 与 area。

`enclosed` 模式下，PrimeTime 使用完全包围网络的层次 cell 的 wire load model 确定 net 值。`segmented` 模式下，PrimeTime 分别确定网络各层次段的值，再求和得到总 net 值。

未指定 mode 时，使用主逻辑库的 `default_wire_load_mode`。`enclosed` 模式通常最准确，但 ASIC 供应商可能推荐特定 mode。

将当前设计 wire load mode 设为 `enclosed`：

```
pt_shell> set_wire_load_mode enclosed
```

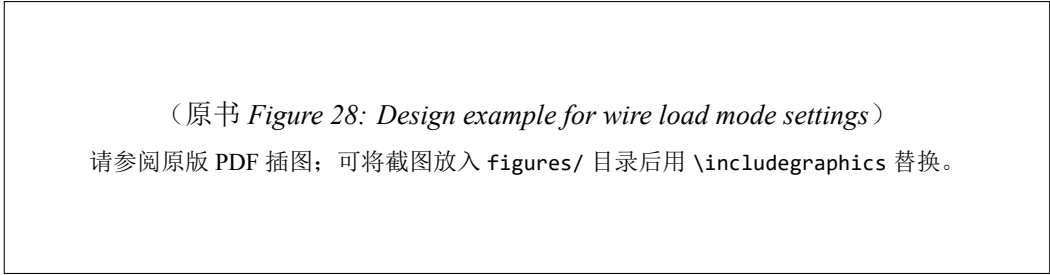


图 6.6: 线负载模式设置的设计示例

表 6.2: 线负载模型应用示例 (Table 11)

Wire load 设置	Wire load model	适用的网络
top	Big	所有网络
enclosed	Big	n3, U1/n2, U2/n4, U2/U1/n5
enclosed	Medium	U1/n1
enclosed	Small	U2/U1/n6
segmented	Big	n3
segmented	Medium	U1/n1, U1/n2, U2/n4
segmented	Small	U2/U1/n5, U2/U1/n6

6.6.4 报告线负载模型

Reporting Wire Load Models

从 PrimeTime 获取 wire load 报告:

- report_wire_load
- report_port -wire_load

6.7 转换时间传播

Slew Propagation

在多条 timing arc 汇合的引脚处, PrimeTime 计算每条驱动 arc 的 slew, 再选择引脚上最差 slew 值沿路径传播。注意所选 slew 可能并非来自对最差路径有贡献的输入, 因此 merge pin 计算的 delay 可能偏悲观。

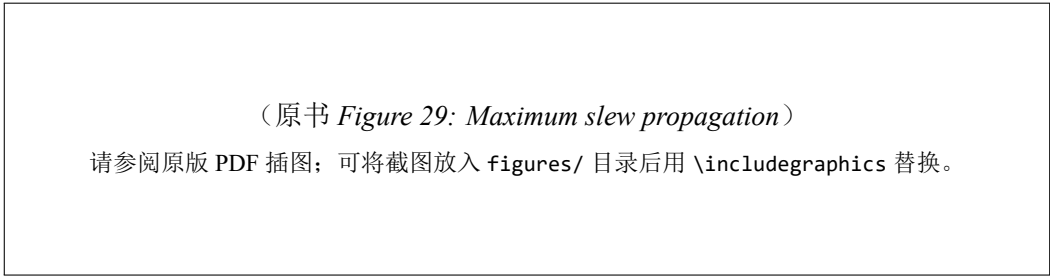


图 6.7: 最大转换时间传播

最小 slew 传播类似最大 slew 传播——PrimeTime 根据 merge 点最佳 delay 的输入选择最小 slew。

6.8 设计规则约束

Design Rule Constraints

PrimeTime 检查库或 PrimeTime 命令定义的设计规则约束违例，包括：

- transition time 最大限制
- capacitance 最大与最小限制
- fanout 最大限制

报告设计规则约束违例，使用 `report_constraint` 命令。

6.8.1 最大转换时间

Maximum Transition Time

PrimeTime 中最大 transition time 的处理方式与 slew 类似。本节先讨论阈值与 derating 语境下的 slew，再扩展到最大 transition time。

可用 Liberty table 中的浮点数表示 SPICE 波形。SPICE 波形以 10–90 测量时，transition time 为 10 ps；线性化波形以 0–100 阈值测量为 12.5 ps；以 30–70 阈值表示为 5 ps。

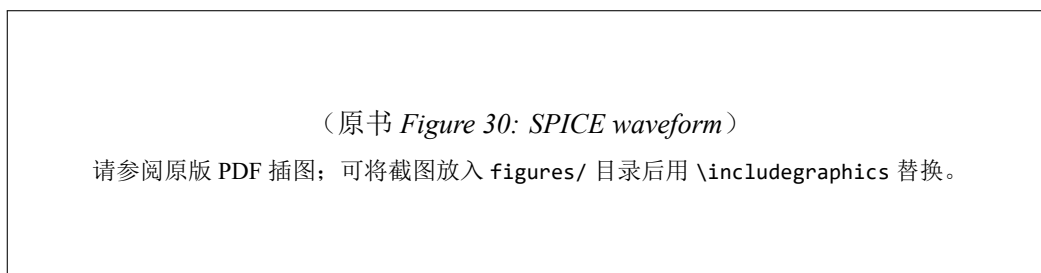


图 6.8: SPICE 波形

SPICE 波形在 Liberty table 或 PrimeTime 报告中可表示为：

- A: 10 ps slew threshold 10–90, slew derating 1.0
- B: 12.5 ps slew threshold 10–90, slew derating 0.8
- C: 5 ps slew threshold 10–90, derating 2.0

注意库中 slew threshold 始终是 SPICE 测量所用阈值。

6.8.2 库中多阈值与 Derating 值

Multiple Threshold and Derating Values in Libraries

Liberty 允许任意 slew threshold 以最小化 slew 线性化误差。当库间（或同库不同 pin 间）slew 信息交互时，需转换到共同基准。

设库 L1 阈值 TL1–TH1、derate SD1；L2 阈值 TL2–TH2、derate SD2。S1 为 L1 本地阈值与 derate 下的 slew；S2 为 L2 本地阈值与 derate 下的 slew。等价 slew S2_1（S2 以 L1 derate/threshold 表示）：

$$S2_1 = S2 \times \frac{SD2}{(TH2 - TL2)} \times \frac{(TH1 - TL1)}{SD1}$$

S1 与 S2_1 可直接比较；S1 与 S2 不可。存在详细 RC 时，slew 在相应 threshold 与 derate 语境下正确计算。

6.8.3 指定最大转换时间约束

Specifying the Maximum Transition Constraint

最大 transition 约束可来自用户输入、库与库 pin。用户指定的最大 transition 约束以 PrimeTime 主库 derate 与 slew threshold 表示。

`set_max_transition` 为指定 pin、port、design 或 clock 设置 transition time 上限。指定于 clock 时，该 clock domain 内 pin 受约束。在 clock domain 内，可进一步限制仅 clock path 或 data path，以及仅上升或下降 transition。

默认情况下，对 pin 或 port 执行约束检查时，采用 design、pin、port、clock（若该 pin 或 port 位于相应 clock domain 中）或 library 上指定的最严格约束。多个 clock 发射同一路径时也是如此。

要改变默认行为，将 `timing_enable_max_slew_precedence` 设为 true。此时约束设置位置决定优先级（与约束值大小无关）：

1. Pin 或 port 级
2. 库派生约束
3. Clock 或 design 级

`set_max_transition` 在指定对象上设置 `max_transition` 属性（设计规则约束）。PrimeTime 以各 pin 或 library 的本地 threshold 与 derate 报告 slew 和最大 transition 约束属性。例如，在 design threshold 和 derate 语境下设置的最大 transition，会缩放到 design pin 的 threshold 和 derate；默认启用这种基于 slew threshold 的缩放。要对 `max_transition` 值应用全局 derating factor，请设置 `timing_max_transition_derate` 变量。

示例 1：对 OUT* 端口设置最大 transition 限制 2.0：

```
pt_shell> set_max_transition 2.0 [get_ports "OUT*"]
```

示例 2：对当前设计设置默认最大 transition 限制 5.0：

```
pt_shell> set_max_transition 5.0 [current_design]
```

示例 3：对 CLK1 clock domain 所有 pin 的 data path 上升 transition 设置最大 transition 限制 4.0：

```
pt_shell> set_max_transition 4.0 [get_clocks CLK1] -data_path -rise
```

示例 4：若 library lib1 的 slew threshold 为 10/90，design 上设置的最大 transition 为 0.3 ns，而 main library 的上升/下降 slew threshold 为 30/70，则该限制换算到 lib1 pin 的 threshold 后为 0.6 ns。此时 `report_constraint -max_transition -all_violators -significant_digits 4` 会相应报告 required transition、actual transition 和 slack。

示例 5：若 library lib1 的 slew derating factor 为 0.5、上升/下降 slew threshold 为 30/70，design 上设置的最大 transition 为 0.3 ns，而 main library 的 slew derate 为 1.0，则换算后的 required transition 同样为 0.6 ns。

6.8.4 评估最大转换时间约束

Evaluating the Maximum Transition Constraint

查看最大 transition 约束评估, 使用 `report_constraint -max_transition`。PrimeTime 以 cell instance pin 的 threshold 与 derate 报告所有约束与 slew, 违例按绝对值排序。

也可使用 `report_constraint` 的 `-object_list` 选项, 仅报告指定 port 或 pin 的最大 capacitance 与最大 transition 约束计算。

查看端口最大 transition 限制: `report_port -design_rule`。查看当前设计默认最大 transition: `report_design`。撤销最大 transition 限制: `remove_max_transition`。

6.8.5 最小电容

Minimum Capacitance

对指定端口或整个设计指定最小 capacitance 限制, 运行 `set_min_capacitance`。端口 capacitance 限制应用于连到该端口的网络; design 级限制为设计中所有网络的默认 capacitance 限制。

`set_min_capacitance` 在指定对象上应用 `min_capacitance` 属性。Capacitance 约束检查仅适用于 output pin。约束检查时采用最严格约束。

查看最小 capacitance 约束评估, 使用 `report_constraint -min_capacitance`; 查看 port capacitance 限制, 使用 `report_port -design_rule`; 查看当前 design 的默认 capacitance 设置, 使用 `report_design`; 移除用户指定的最小 capacitance 限制, 使用 `remove_min_capacitance`。

```
pt_shell> set_min_capacitance 0.2 [get_ports "OUT*"]
pt_shell> set_min_capacitance 0.1 [current_design]
```

6.8.6 最大电容

Maximum Capacitance

指定最大 capacitance 限制, 运行 `set_max_capacitance`。可在 pin 或 clock 上额外指定最大 capacitance 限制。指定于 clock 时, 该 clock domain 内 pin 受约束, 可进一步限制仅 clock path 或 data path, 以及仅上升或下降 capacitance。

`set_max_capacitance` 应用 `max_capacitance` 属性。Capacitance 约束检查仅适用于 output pin, 检查时采用最严格约束。对 `max_capacitance` 值应用全局 derating, 设置 `timing_max_capacitance_derate` 变量。

查看最大 capacitance 约束评估, 使用 `report_constraint -max_capacitance`; 查看 port capacitance 限制, 使用 `report_port -design_rule`; 查看当前 design 的默认 capacitance 设置, 使用 `report_design`; 移除用户指定的最大 capacitance 限制, 使用 `remove_max_capacitance`。

6.8.7 约束上升与下降最大电容

Constraining Rise and Fall Maximum Capacitance

要约束 design 中 clock 的 clock path 和 data path 上 driver pin 输出端的上升与下降最大 capacitance, 请使用 `set_max_capacitance`。工具会将 clock 上的最大 capacitance 约束应用于 clock network 中的

所有 pin (clock path)，以及由该 clock 发射的数据路径中的 pin (该 clock 的 data path)。可进一步用 `-clock_path`、`-data_path`、`-rise` 和 `-fall` 限制约束范围。

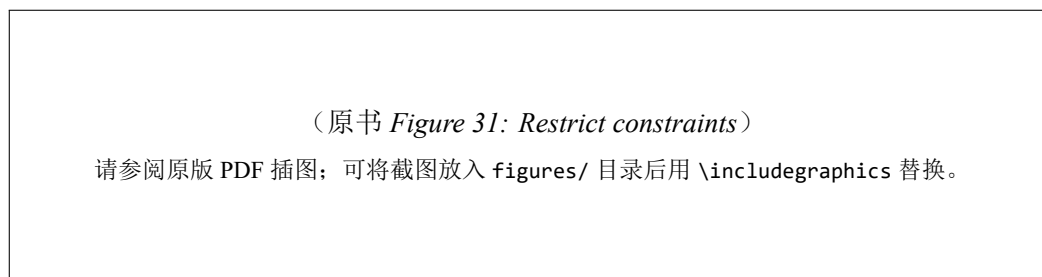


图 6.9: 限制约束范围

若对象为 clock 列表，可用 `-clock_path`、`-data_path`、`-rise` 或 `-fall` 进一步限制约束范围。若 pin (或 port) 受 design、library、clock (以及 port) 上不同最大 capacitance 约束，最严格约束优先。slack 为最严格约束与 pin (或 port) 总 capacitance 之差。该特性适用于 generated clock。使用 `report_constraint` 报告约束违例。

6.8.8 基于频率的最大电容检查

Frequency-Based Maximum Capacitance Checks

库可包含 lookup table，为每个 driver pin 指定基于频率的 `max_capacitance` 值。Pin 频率为到达该 pin 所有时钟的最大频率。

默认情况下，若 lookup table 可用，工具考虑其中的 `max_capacitance` 值。DRC 检查、`report_constraint`、ETM 生成与 ECO 修复使用以下最小值：

- 库 lookup table 中的 `max_capacitance` 值
- 库 cell `max_capacitance` 值
- pin、port、clock 与 design 级所有用户指定的 `max_capacitance` 值

完全忽略 lookup table 中的 `max_capacitance`，将 `timing_enable_library_max_cap_lookup_table` 设为 `false`。

使 lookup table 的 `max_capacitance` 优先于库 cell 值，将 `timing_library_max_cap_from_lookup_table` 设为 `true`。

6.8.9 含 Case Analysis 的最大电容检查

Maximum Capacitance Checking With Case Analysis

默认情况下，不对 constant propagation 路径上的 pin 执行最大 capacitance 检查。要启用，将 `timing_enable_max_capacitance_set_case_analysis` 设为 `true`。

6.8.10 最大扇出负载

Maximum Fanout Load

`set_max_fanout` 为指定 output port 或 design 设置最大 fanout load，在指定对象上设置 `max_fanout` 属性。端口级限制应用于连到该端口的网络；design 级为设计中所有网络的默认最大值。冲

突时采用更严格值。库 cell pin 可有 `max_fanout` 值，PrimeTime 采用用户设置与库指定中更严格的限制。

查看最大 fanout 约束评估，使用 `report_constraint -max_fanout`；查看 port 的最大 fanout 限制，使用 `report_port -design_rule`；查看当前 design 的默认最大 fanout 设置，使用 `report_design`；撤销用户在 port 或 design 上设置的最大 fanout 限制，使用 `remove_max_fanout`。

```
pt_shell> set_max_fanout 2.0 [get_ports "IN*"]
pt_shell> set_max_fanout 5.0 [current_design]
```

6.8.11 输出端口扇出负载值

Fanout Load Values for Output Ports

网络的 fanout load 为连到该网络 input pin 与 output port 的 `fanout_load` 属性之和。Output pin 可有最大 fanout 限制（库中定义或通过 `set_max_fanout` 设置）。默认端口 fanout load 为 0.0。`set_fanout_load` 为当前设计 output port 指定预期 fanout load:

```
pt_shell> set_fanout_load 3.0 "OUT*"
```

参见

更多信息见 *Synopsys Timing Constraints and Optimization User Guide*。

6.9 理想网络

Ideal Networks

PrimeTime 允许创建 ideal network，其上不检查设计规则约束。在 pre-layout 设计阶段，可能希望忽略 fanout 与 capacitance 较大的未优化网络，专注于其他来源的违例。使用 ideal network 可减少运行时间，因为 PrimeTime 使用「理想时序」而非内部计算时序，类似 ideal clock network，但也可应用于 data network。

Ideal network——相连的 port、pin、net 与 cell 集合——免于时序更新与设计规则约束修复，即忽略 `max_capacitance`、`max_fanout` 与 `max_transition` 检查。指定 ideal network 源后，其中包含的 pin、port、net 与 cell 被视为 ideal 对象，需由用户或 ideal propagation 标记。

6.9.1 传播理想网络属性

Propagating Ideal Network Properties

指定 ideal network 源对象（port、leaf-level pin）后，源对象传递扇出中的 net、cell 与 pin 可被视为 ideal。传播规则：

- Pin 标记为 ideal，若其为以下之一：
 - `set_ideal_network object_list` 中指定的 pin
 - Driver pin 及其 cell 为 ideal
 - Load pin 连到 ideal net

- Net 标记为 ideal，若其所有 driving pin 均为 ideal
- 组合逻辑 cell 标记为 ideal，若所有 input pin 均为 ideal，或连到 constant net 且所有其他 input pin 均为 ideal

注意

Ideal network propagation 可穿越组合逻辑 cell，但在时序 cell 处停止。

PrimeTime 在时序更新时传播 ideal network，并在设计变更（如 ECO）后从 ideal 源对象重新传播。

6.9.2 使用理想网络

Using Ideal Networks

用 `set_ideal_network` 指定要设为 ideal 的网络，用 `object_list` 指明 ideal network 源：

```
pt_shell> set_ideal_network P1
```

使用 `-no_propagate` 限制传播范围，仅将与 ideal network 源电气相连的 net 与 pin 标记为 ideal：

```
pt_shell> set_ideal_network -no_propagate net1
```

在此示例中，只有 net1、它的 driver pin 和 load pin 被标记为 ideal，不再继续传播。若不指定 `-no_propagate`，PrimeTime 会在给定的 pin、port 或 net 列表上设置 ideal 属性，并按前述规则继续传播。

移除 ideal network： `remove_ideal_network`。报告 ideal port、pin、net 或 cell： `report_ideal_network`。

6.9.3 使用理想延迟

Using Ideal Latency

默认 ideal network delay 为零。可用 `set_ideal_latency` 在 ideal network 的 pin 与 port 上指定 ideal latency，沿路径累积方式与 delay 值相同。注意 ideal latency 仅在对象为 ideal 时生效；否则 `report_ideal_network` 发出警告。用 `remove_ideal_latency` 移除。

6.9.4 使用理想转换时间

Using Ideal Transition

Ideal network 默认 transition time 为零。可用 `set_ideal_transition` 指定 ideal transition time。在对象上设置 ideal transition 后，该值沿 ideal network 传播到网络边界 pin，或直到遇到另一 ideal transition 值。仅在对象为 ideal 时生效。用 `remove_ideal_transition` 移除。

6.9.5 全时钟展开模式中的理想时钟报告

Reporting of Ideal Clocks in Full Clock Expanded Mode

默认情况下，`report_timing -path_type full_clock_expanded` 不在 ideal clock network 中显示 clock path，而显示 clock arrival 点 ideal clock latency 的单行。

要了解 ideal clock network 的拓扑路径，设置：

```
pt_shell> set_app_var timing_report_trace_ideal_clocks true
```

设置后，报告显示完整 ideal clock path 及沿路径的 incremental delay。在 clock arrival 点，报告施加调整以抵消 propagated delay 计算，得到 ideal clock edge 时间，再应用 ideal clock network delay。

该选项同样控制 `report_clock_timing -verbose` 与 `report_min_pulse_width -path_type full_clock_expanded` 的路径报告。

6.10 检查约束

Checking the Constraints

开始完整分析前，宜检查设计特性与约束。错误约束的路径可能不出现在违例报告中，导致遗漏违例路径。

6.10.1 设计报告

Design Reports

完整时序分析前，可用下表命令检查设计；分析后这些命令也有助于调试违例。

6.10.2 使用 `check_timing` 命令进行约束检查

Constraint Checking With the `check_timing` Command

检查未定义时钟、未定义输入到达时间与未定义输出约束等问题，使用 `check_timing`。该命令还提供与最小时钟间隔（主从时钟）、被忽略时序例外、组合反馈环与锁存器扇出相关的潜在问题信息。可通过添加约束（如 `create_clock`、`set_input_delay`、`set_output_delay`）纠正未约束路径。

典型 `check_timing` 报告示例：

```
pt_shell> check_timing
Information: Checking 'no_clock'.
Warning: There are 4 register clock pins with no clock.
Information: Checking 'no_input_delay'.
Information: Checking 'unconstrained_endpoints'.
...
```

默认执行多种约束检查并发出摘要报告。详细报告使用 `-verbose`。用 `-include` 或 `-exclude` 增减默认检查列表，或设置 `timing_check_defaults` 变量。所有检查类型见 `man` 页。

`check_timing` 报告的警告不一定表示真实设计问题。可用多种 `report` 命令获取设计特性与时序约束信息。

表 6.3: 设计报告命令 (Table 14)

命令	说明
report_bus	报告当前实例或设计中总线 (pin 或 port) 信息
report_cell	列出所用 cell 及库名、输入/输出名、网络连接、面积与属性
report_clock	报告设计中定义的时钟 (名称、周期、上升/下降时间及 latency、uncertainty 等)
report_design	列出设计属性 (工作条件、wire load、设计规则、derating)
report_disable_timing	报告被禁用的 timing arc
report_hierarchy	生成当前设计子模块与 leaf cell 层次列表
report_lib	报告指定库的时间/电容单位、wire load、工作条件、trip-point 阈值与 cell 名
report_net	列出网络及 fanin、fanout、电容、wire resistance、pin 数、属性与连接
report_path_group	报告 path group
report_port	列出端口及方向、pin 电容、wire 电容、输入/输出延迟、相关时钟、设计规则与 wire load
report_reference	列出层次引用及参考名、单元面积、出现次数、总面积与属性
report_transitive_fanin	报告指定 pin、port 或 net 的扇入逻辑
report_transitive_fanout	报告指定 pin、port 或 net 的扇出逻辑
report_units	显示当前设计中电流、电容、电阻、时间与电压的测量单位
report_wire_load	显示当前设计或指定 cell 上设置的 wire load model

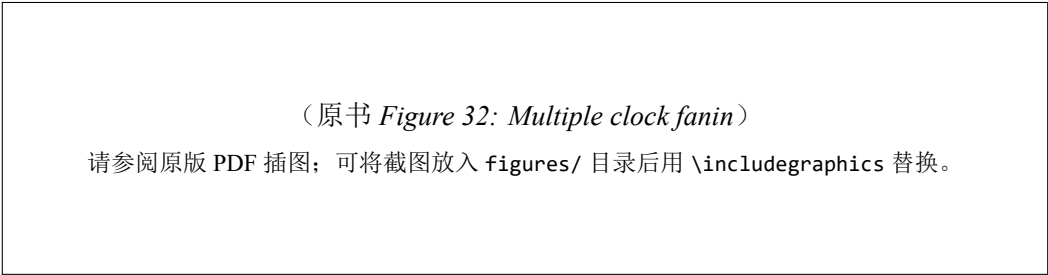


图 6.10: 多时钟扇入

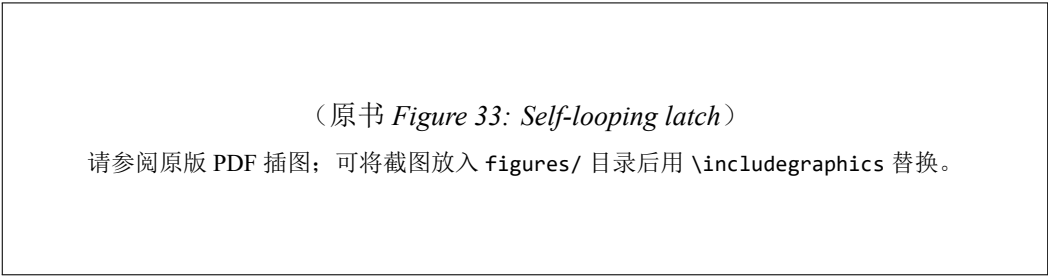


图 6.11: 自环锁存器

（原书 *Figure 34: Latch fanout to another latch*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 6.12: 锁存器扇出到同时钟另一锁存器

注意

若时序路径未约束，`check_timing` 仅报告未约束 endpoint，不报告未约束 startpoint。类似地，对仅由 `set_max_delay`、`set_min_delay` 约束（而非 `set_input_delay` 与 `set_output_delay`）的路径，`check_timing` 仅报告未约束 endpoint。

6.10.3 详细约束检查与调试

Detailed Constraint Checking and Debugging

报告错误约束或其他潜在问题，使用工具的详细约束检查与调试能力。该能力提供覆盖时钟、例外、case 与设计规则问题的预定义规则集，如被阻断的时序路径与缺失时钟定义。

在 PrimeTime shell 中运行约束检查：

- 1.（可选）在 `.synopsys_gca.setup` 文件中指定约束检查设置。
2. 用 `current_design` 指定当前设计：

```
pt_shell> current_design top
```

- 3.（可选）用 `gca_setup_file` 变量指定含用户定义违例豁免与自定义规则的规则文件：

```
pt_shell> set_app_var gca_setup_file ./top_rules.tcl
```

4. 用 `check_constraints` 执行约束分析：

```
pt_shell> check_constraints
```

5. 对另一设计重复步骤 2–4，可指定不同规则文件。

在 GUI 中查看约束检查结果：Constraints > View Constraint Checking Results。

也可在独立约束检查 shell `ptc_shell` 中执行约束分析。详见约束一致性章节。

（原书 *Figure 35: Constraint Checking Results Displayed in the GUI*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 6.13: GUI 中显示的约束检查结果

Chapter 7

时钟

Clocks

时序分析的关键部分是准确指定时钟及时钟效应，如 **latency**（从时钟源的延迟）与 **uncertainty**（时钟边沿到达的 **skew** 或变化量）。要了解如何指定、报告与分析时钟，请参阅：

- 时钟概述
- 指定时钟
- 指定时钟特性
- 使用多个时钟
- 时钟极性（Clock Sense）
- 指定脉冲时钟
- PLL 设计时序分析
- 指定时钟门控 Setup 与 Hold 检查
- 指定内部生成时钟
- 生成时钟边沿特定源延迟传播
- 时钟网格（Clock Mesh）分析

7.1 时钟概述

Clock Overview

PrimeTime 支持以下类型的时钟信息：

多个时钟：可定义具有不同波形与频率的多个时钟。时钟可在设计中有真实源（**port** 与 **pin**），也可为 **virtual**——**virtual clock** 在设计本身中无真实源。

时钟网络延迟与 skew：指定相对于源的时钟网络延迟（**clock latency**）与时钟目的地到达时间变化（**clock skew**）。多时钟设计中可指定 **interclock skew**。可指定 **ideal** 时钟网络延迟（时钟树生成

前分析)，或指定由 PrimeTime 计算延迟（时钟树生成后分析）。PrimeTime 还支持沿时钟网络检查 minimum pulse width。

门控时钟（Gated clocks）：可分析含门控时钟的设计。门控时钟是受门控逻辑控制的时钟信号（非简单缓冲或反相）。PrimeTime 对门控信号执行 setup 与 hold 检查。

生成时钟（Generated clocks）：可分析含生成时钟的设计。生成时钟由设计中另一时钟信号经电路（如时钟分频器）产生。

时钟转换时间：可在寄存器时钟引脚指定时钟信号 transition time——信号从一个逻辑状态变到另一逻辑状态所需时间。

7.2 指定时钟

Specifying Clocks

需指定设计中使用的所有时钟。时钟信息包括：周期与波形、latency（insertion delay）、uncertainty（skew）、分频与内部生成时钟、时钟门控检查、不完整时钟网络的固定 transition time。

PrimeTime 支持分析设计的同步部分，分析寄存器或 port 之间的路径。多交互时钟设计中，PrimeTime 确定 startpoint 与 endpoint 时钟间的相位关系。时钟可为单相、多相或多频时钟。

7.2.1 创建时钟

Creating Clocks

必须用 create_clock 指定设计中所有时钟。该命令在指定源创建时钟，源可定义在设计输入 port 或内部 pin。PrimeTime 自动追踪时钟网络，使时钟到达其源传递扇出中的所有寄存器。

create_clock 创建的时钟具有 ideal 波形，忽略时钟网络延迟效应。创建时钟后须描述时钟网络以进行准确时序分析（见「指定时钟特性」）。

create_clock 创建与时钟同名的 path group，包含所有由该时钟计时的 endpoint 路径。见第 八 章「时序路径组」。

在端口 C1 与 CK2 上创建周期 10、上升沿 2、下降沿 4 的时钟：

```
pt_shell> create_clock -period 10 -waveform {2 4} {C1 CK2}
```

生成的时钟以第一个列出源命名（C1），波形如下。

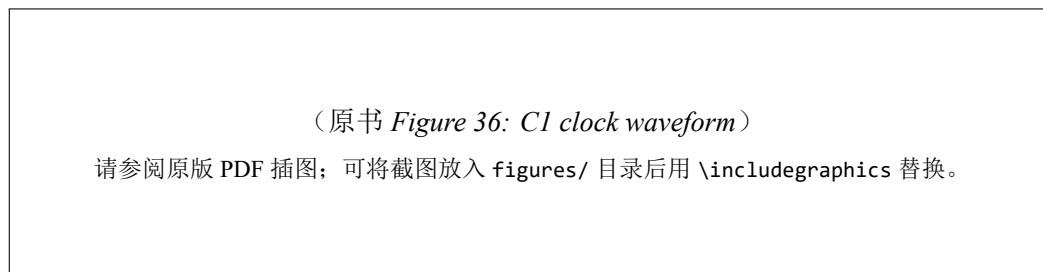


图 7.1: C1 时钟波形

PrimeTime 支持分析传播到单个寄存器的多个时钟。可用 create_clock 的 -add 选项在同一 port 或 pin 上定义多个时钟。

7.2.2 创建虚拟时钟

Creating a Virtual Clock

可用 `create_clock` 定义与外部（片外）时钟器件接口信号的 virtual clock。Virtual clock 在当前设计中无实际源，但可用于设置 input 或 output delay。

创建名为 `vclk` 的 virtual clock：

```
pt_shell> create_clock -period 8 -name vclk -waveform {2 5}
```

（原书 *Figure 37: vclk clock waveform*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.2: vclk 时钟波形

7.2.3 选择时钟对象

Selecting Clock Objects

`get_clocks` 为命令选择时钟对象，例如确保命令作用于 CLK 时钟而非 CLK port。

报告名称以 `PHI1` 开头且周期 ≤ 5.0 的时钟属性：

```
pt_shell> report_clock [get_clocks -filter "period <= 5.0" PHI1*]
```

7.2.4 将命令应用于所有时钟

Applying Commands to All Clocks

`all_clocks` 等价于 `get_clocks *`，返回表示 clock 对象集合的 token，不打印实际时钟名。

禁用所有时钟的 time borrowing：

```
pt_shell> set_max_time_borrow 0 [all_clocks]
```

7.2.5 移除时钟对象

Removing Clock Objects

用 `remove_clock` 移除时钟对象列表。

注意

`reset_design` 除移除时钟外，还移除其他信息。

移除名称以 `CLKB` 开头的有时钟：

```
pt_shell> remove_clock [get_clocks CLKB*]
pt_shell> remove_clock -all
```

7.3 指定时钟特性

Specifying Clock Characteristics

`create_clock` 创建的时钟具有完美波形，忽略时钟网络延迟效应。要准确时序分析，须描述时钟网络。时钟网络主要特性为 `latency` 与 `uncertainty`。

`Latency` 包含 `clock source latency` 与 `clock network latency`。`Clock source latency` 是时钟信号从 `ideal` 波形原点传播到设计中时钟定义点的时间。`Clock network latency` 是时钟信号（上升或下降）从设计中时钟定义点传播到寄存器时钟引脚的时间。

`Clock uncertainty` 是同一 `clock domain` 内寄存器间或不同 `domain` 间时钟信号到达的最大差异，也称 `skew`。因 `clock uncertainty` 可含额外裕量以收紧 `setup` 或 `hold` 检查，可对同一路径的 `setup` 与 `hold` 检查指定不同 `uncertainty`。

7.3.1 设置时钟延迟

Setting Clock Latency

PrimeTime 提供两种表示 `clock latency` 的方法：

- 允许 PrimeTime 沿时钟网络传播 `delay` 计算 `latency`——非常准确，但仅在时钟树综合完成后可用；
- 估算并显式指定各时钟 `latency`，可在单个 `port` 或 `pin` 上指定，影响这些对象传递扇出中的所有寄存器时钟引脚，覆盖 `clock` 对象上的任何值——通常用于时钟树综合前。

设置传播延迟

PrimeTime 可沿时钟网络自动传播 `delay` 确定 `clock latency`，在时钟树综合与布局后（`cell` 与 `net delay` 均已 `back-annotate` 或已计算 `net parasitic`）产生高精度结果。使用 `propagated clock` 的寄存器边沿时间因从时钟源到寄存器时钟引脚的路径 `delay` 而 `skew`。设计有实际时钟树且 `delay` 或 `parasitic` 已标注时适用。

传播时钟网络 `delay` 并自动确定各寄存器时钟引脚 `latency`：

```
pt_shell> set_propagated_clock [get_clocks CLK]
```

可在 `clock`、`port` 或 `pin` 上设置 `propagated clock` 属性。设置在 `port` 或 `pin` 上时，影响对象传递扇出中的所有寄存器时钟引脚。移除 `propagated clock` 规格：`remove_propagated_clock`。

指定时钟源延迟

可为 `ideal` 或 `propagated clock` 指定 `clock source latency`。`Source latency` 是从 `ideal` 波形到源 `pin` 或 `port` 的 `latency`。寄存器时钟引脚 `latency` 为 `clock source latency` 与 `clock network latency` 之和。

对内部生成时钟，若 `generated clock` 的 `master clock` 有 `propagated latency` 且未用户指定 `generated clock source latency`，PrimeTime 可自动计算 `clock source latency`。

Prelayout 设计中 propagated latency 计算通常不准确 (parasitic 未知)。可对各时钟估算 latency 并用 `set_clock_latency` 直接设置——ideal clocking, PrimeTime 中表示 clock latency 的默认方法。

`set_clock_latency` 为一个或多个 clock、port 或 pin 设置 latency。要表示 source latency 的外部不确定性, 即 source latency 的取值范围, 请使用 `-early` 与 `-late` 选项。

(原书 Figure 38: External source latency)

请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 `\includegraphics` 替换。

图 7.3: 外部源延迟

```
pt_shell> create_clock -period 10 [get_ports CLK]
pt_shell> set_clock_latency 1.5 -source -early [get_clocks CLK]
pt_shell> set_clock_latency 2.5 -source -late [get_clocks CLK]
```

PrimeTime 对每个由该时钟计时的 startpoint 与 endpoint 使用更保守的 source latency 值 (early 或 late)。Setup 分析: startpoint 用 late 值, endpoint 用 early 值。Hold 分析: startpoint 用 early 值, endpoint 用 late 值。

(原书 Figure 39: Early and late source latency waveforms)

请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 `\includegraphics` 替换。

图 7.4: 早/晚源延迟波形

设置 CLK 预期上升 latency 1.2、下降 latency 0.9:

```
pt_shell> set_clock_latency -rise 1.2 [get_clocks CLK]
pt_shell> set_clock_latency -fall 0.9 [get_clocks CLK]
```

移除用户指定的 clock network 或 source clock latency: `remove_clock_latency`。

时钟源延迟的动态效应

可用 `set_clock_latency` 的 `-dynamic` 选项建模 clock source latency 的动态效应 (如 PLL clock jitter)。该选项允许指定 clock source latency 的动态分量。Clock reconvergence pessimism removal (CRPR) 以与 PrimeTime SI delta delay 相同方式处理 clock latency 动态分量。

也可用 `set_clock_uncertainty` 建模 clock jitter, 但 clock uncertainty 设置不影响 crosstalk arrival window 计算, CRPR 也不考虑。`set_clock_latency` 允许将 clock jitter 指定为 dynamic source clock latency, 正确影响 arrival window 计算, 静态与动态部分由 CRPR 正确处理。

```
pt_shell> set_clock_latency -early -source 3.0 [get_ports CLK]
pt_shell> set_clock_latency -late -source 5.0 [get_ports CLK]
pt_shell> set_clock_latency -early -source 2.5 -dynamic -0.5 [get_clocks CLK]
pt_shell> set_clock_latency -source -late 5.5 -dynamic 0.5 [get_clocks CLK]
```

前两条命令指定静态 early/late source latency 分别为 3.0 和 5.0；后两条命令在其上叠加 -0.5 和 +0.5 的动态分量，因此总 early/late source latency 分别为 2.5 和 5.5。

若 CLK 使用 propagated latency，则静态与动态 latency 之和用于 delay 计算。不过，对于 launch 与 capture 使用不同时钟边沿的路径，CRPR 在 clock path 公共点之前只使用静态 latency，不使用动态 latency，并从时序结果中移除由此得到的静态悲观量。

report_timing 分别报告 clock latency 的静态与动态分量。report_crpr 报告静态和动态条件下计算的 CRP 时间值，以及实际选用于悲观移除的值。

7.3.2 设置时钟不确定性

Setting Clock Uncertainty

可用 setup 或 hold 及 rise 或 fall uncertainty 值建模 prelayout 设计的预期 uncertainty (skew)。PrimeTime 在 setup 检查（最大路径）时从 data required time 减去 setup uncertainty；在 hold 检查（最小路径）时向 data required time 加上 hold uncertainty。若指定单一 uncertainty 值，setup 与 hold 检查均使用该值。

用 set_clock_uncertainty 指定时钟 uncertainty 或 skew 特性，捕获实际或预测的 clock uncertainty。

可指定 simple clock uncertainty 或 interclock uncertainty。Simple uncertainty 是连续时钟边沿相对于精确名义时间的生成变化，指定一个或多个对象（clock、port 或 pin），uncertainty 值应用于所有由指定 clock 计时的 capturing latch 或其时钟引脚在指定 port/pin 扇出中的 latch。

Interlock uncertainty 更具体灵活，支持不同 clock domain 间不同 uncertainty——不同时钟边沿间 skew 变化。用 -from、-rise_from 或 -fall_from 指定「from」时钟，用 -to、-rise_to 或 -fall_to 指定「to」时钟。Interlock uncertainty 适用于从「from」时钟启动、在「to」时钟捕获的路径。

（原书 *Figure 40: Example of interclock uncertainty*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.5: 时钟间不确定性示例

Setup 或 hold 检查时，PrimeTime 根据最差可能的时钟边沿时间差调整 timing check。Setup 检查示例：从 data required time 减去 uncertainty，要求数据更早到达，以计入 late launch 与 early capture 的最差 clock skew。

若路径同时有 simple clock uncertainty 与 interlock uncertainty，使用 interlock uncertainty 值。

```
pt_shell> set_clock_uncertainty 5 [get_clocks CLK_A]
pt_shell> set_clock_uncertainty 2 -from [get_clocks CLK_B] -to [get_clocks CLK_A]
```

从 CLKA 到 CLKB 与从 CLKB 到 CLKA 的路径须分别指定 `uncertainty`，即使值相同：

```
pt_shell> set_clock_uncertainty 2 -from [get_clocks CLKA] -to [get_clocks CLKB]
pt_shell> set_clock_uncertainty 2 -from [get_clocks CLKB] -to [get_clocks CLKA]
```

对 U1/FF*/CP 计时的 endpoint 设置 simple clock uncertainty 0.45：

```
pt_shell> set_clock_uncertainty 0.45 [get_pins U1/FF*/CP]
```

对 CLK1 设置 setup uncertainty 0.21、hold uncertainty 0.33：

```
pt_shell> set_clock_uncertainty -setup 0.21 [get_clocks CLK1]
pt_shell> set_clock_uncertainty -hold 0.33 [get_clocks CLK1]
```

在 clock port 或 pin 上设置 simple uncertainty 时，uncertainty 应用于时钟引脚在指定对象传递扇出中的 capturing latch。若多个 port/pin 传播不同 uncertainty 值到 clock MUX 输入，最差 uncertainty 设置应用于整个 MUX 扇出，即使 MUX 处关联 clock 被禁用。为确保正确使用 uncertainty，应在 clock 对象而非 clock port/pin 上设置 uncertainty。

移除 clock uncertainty: `remove_clock_uncertainty`。

7.3.3 设置时钟抖动

Setting the Clock Jitter

在 master clock 上设置 clock jitter，运行 `set_clock_jitter`。该命令在指定 master clock 生成的所有 clock 上设置相同 jitter 属性。未指定 master clock 时，在所有 clock 上设置 jitter。

7.3.4 移除时钟抖动

Removing the Clock Jitter

`remove_clock_jitter` 自动从 generated clock 移除 jitter 属性。未指定 master clock 时，从所有 clock 移除 jitter。

7.3.5 报告时钟抖动

Reporting the Clock Jitter

`report_clock_jitter` 报告 cycle jitter、duty-cycle jitter 及 generated clock 使用的带 jitter 的 master clock。

7.3.6 估计时钟引脚转换时间

Estimating Clock Pin Transition Time

Transition time 通常为时钟网络中 port、cell 与 net 计算。若时钟网络不完整，寄存器时钟引脚 transition time 可能不准确（例如单缓冲驱动 10000 个寄存器时钟引脚，时钟树未构建）。

可用 `set_clock_transition` 估计整个时钟网络的 clock transition time：

```
pt_shell> set_clock_transition 0.64 -fall [get_clocks CLK1]
```


Transition time 值应用于直接馈送指定 clock 计时时序元件的所有网络。可用 `-rise`、`-fall`、`-min`、`-max` 分别指定仅上升/下降边沿或最小/最大工作条件下的 transition time。移除估计的 clock pin transition time: `remove_clock_transition`。

7.3.7 检查最小脉冲宽度

Checking the Minimum Pulse Width

Minimum pulse width 检查对确保时序电路正确工作很重要。原始时钟 pulse width 可能因门控逻辑或时钟网络 delay 特性而减小，导致：

- 寄存器时钟引脚处时钟脉冲过小，器件可能无法正确捕获数据；库 cell pin 可有 minimum pulse width 限制，由 `report_constraint` 检查；
- 脉冲宽度在时钟网络某点缩小到无法继续传播的程度；PrimeTime 可检查整个时钟网络，确保脉冲不低于某阈值。

(原书 Figure 41: Clock network pulse width check)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 `\includegraphics` 替换。

图 7.6: 时钟网络脉冲宽度检查

对普通非 pulse generator 的 clock、cell、pin、port 与当前 design，用 `set_min_pulse_width` 指定约束。对 pulse generator 网络，用 `set_pulse_clock_min_width` 与 `set_pulse_clock_max_width`。

注意

`report_constraint` 对寄存器时钟引脚与时钟网络检查 minimum pulse width。也可在库 cell 描述的 clock pin 上指定 minimum pulse width。

移除时钟树或时序器件上时钟信号的 minimum pulse width 检查: `remove_min_pulse_width`。

默认 `set_clock_uncertainty` 指定的 clock uncertainty 应用于所有时序约束类型，包括 pulse width 检查。可选指定是否将 clock uncertainty 应用于 pulse width 检查及仅 setup、仅 hold 或两者：

```
set_app_var timing_include_uncertainty_for_pulse_checks \
  none | setup_only | hold_only | setup_hold
```

默认 `setup_hold`。

脉冲宽度测量阈值

默认在电源电压 50% 处测量脉冲宽度。可对 clock network pin 将测量阈值设为 0.0 到 1.0 之间任意电源电压比例。更保守的 minimum pulse width 检查：低脉冲阈值 < 0.5，高脉冲阈值 > 0.5。

```
set delay_calc_waveform_analysis full_design
set timing_enable_min_pulse_width_waveform_adjustment true
```

```
set timing_min_pulse_width_low_pulse_threshold 0.12
set timing_min_pulse_width_high_pulse_threshold 0.88
report_min_pulse_width -path_type short ...
```

`report_min_pulse_width -path_type` 报告会将非默认阈值引起的路径调整显示为“open edge threshold voltage adjustment”和“close edge threshold voltage adjustment”。

(原书 *Figure 42: Minimum Pulse Width Checking Thresholds*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.7: 最小脉冲宽度检查阈值

电压摆幅检查

可对 clock network pin 执行 voltage swing 检查，确定高脉冲是否足够接近电源轨、低脉冲是否足够接近地轨。

```
set delay_calc_waveform_analysis full_design
set timing_enable_voltage_swing true
set timing_voltage_swing_low_pulse_threshold 0.10
set timing_voltage_swing_high_pulse_threshold 0.90
report_min_pulse_width -voltage_swing ...
```

(原书 *Figure 43: Voltage Swing Checking Thresholds*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.8: 电压摆幅检查阈值

Voltage swing 检查作为 minimum pulse width 检查的特殊情况执行，minimum width 设为 0.0，并在指定的电源电压比例处测量宽度。达到或越过阈值的任意宽度脉冲均通过检查；未越过阈值的脉冲报告为违例。对于通过检查的脉冲，正 slack 为波形越过阈值电压处测得的脉冲宽度。

7.3.8 最小周期检查

Minimum Period Checking

Minimum period 检查查找 cell pin 库中定义的 min_period 约束违例，要求同类型（上升或下降）连续时钟边沿到达间隔至少为指定值。

执行 minimum period 检查：

- `report_min_period [-verbose] ...`
- `report_constraint -min_period [-verbose] ...`
- `report_analysis_coverage -check_type min_period`

报告中「open edge clock latency」为考虑时钟网络 cell 最大 delay 的第一边沿最晚到达时间;「close edge clock latency」为第二边沿最早到达时间, 加回 CRPR, 减去 clock uncertainty。「actual period」为 close edge 减 open edge。

7.4 使用多个时钟

Using Multiple Clocks

设计中定义多个时钟时, clock domain 间关系取决于时钟生成与使用方式。两时钟关系可为同步、异步或互斥。

(原书 *Figure 44: Synchronous, asynchronous, and exclusive clocks*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.9: 同步、异步与互斥时钟

要使 PrimeTime 正确分析不同 clock domain 间路径, 可能需要指定时钟间 false path、排除分析中的时钟, 或指定不同时钟间关系性质。

7.4.1 同步时钟

Synchronous Clocks

两时钟彼此同步, 若它们共享共同源且具有固定相位关系。除非另行指定, PrimeTime 假设若存在由一时钟发射、由另一时钟捕获数据的路径, 则两时钟同步。时钟波形在 `create_clock` 定义的 time zero 处同步。

```
pt_shell> create_clock -period 4 -name CK1 -waveform {0 2}
pt_shell> create_clock -period 4 -name CK2 -waveform {1 3}
pt_shell> create_clock -period 6 -name CK3 -waveform {2 3}
```

设计中可能存在由一时钟发射、由另一时钟捕获的路径。为测试不同时钟边沿间所有可能时序关系, PrimeTime 内部将时钟「展开」到所有同步时钟周期的最小公倍数 (base period), 创建具有多个上升/下降边沿的更长周期时钟。

PrimeTime 检查展开时钟所有边沿间的时序路径。以最严格 setup 检查为例: CK2 下降沿到 CK3 上升沿, 从 time=7 到 time=8。

以与实际工作方式一致的方式定义多个时钟。声明非同步时钟, 可用 `case analysis` 或 `set_clock_groups -logically_exclusive`、`set_false_path` 等命令。

（原书 *Figure 45: Synchronous clock waveforms*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.10: 同步时钟波形

（原书 *Figure 46: Expanded clock waveforms*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.11: 展开时钟波形

7.4.2 异步时钟

Asynchronous Clocks

两时钟异步，若它们在设计中不相互通信。例如片内自由运行振荡器与来自片外的系统时钟异步，两 clock domain 时钟边沿可任意相对发生。

可声明两时钟关系为异步。PrimeTime 不检查由一时钟发射、由另一时钟捕获的时序路径，类似在两时钟间声明 `false path`。若进行 crosstalk 分析，PrimeTime SI 对两 clock domain 间 aggressor-victim 关系的网络赋予无限 arrival window。

声明异步关系：`set_clock_groups -asynchronous`。

7.4.3 互斥时钟

Exclusive Clocks

两时钟互斥，若它们不相互交互。例如电路可能在时钟线上复用两个不同时钟信号——正常运行快时钟与低功耗慢时钟，任一时刻仅一个时钟使能，两时钟无交互。

为防止 PrimeTime 花费时间检查互斥时钟间交互，可声明时钟间 `false path`，或用 `set_clock_groups -logically_exclusive` 声明互斥。否则可用 case analysis 禁用当前分析中不需要的时钟。

```
pt_shell> set_clock_groups -logically_exclusive -group {CK1} -group {CK2}
```

这使 PrimeTime 忽略从 CK1 domain 到 CK2 domain 或从 CK2 到 CK1 的时序路径，类似设置 CK1 到 CK2 与 CK2 到 CK1 的 `false path`，但 `report_exceptions` 不报告该设置。查询已设置的 clock group: `report_clock -groups`。

避免在已声明互斥的 clock domain 间设置 `false path`（冗余）。`set_clock_groups -logically_exclusive` 与 `-asynchronous` 均使互斥 clock domain 间已设置的 `false path` 失效。

可在每组中指定多个时钟：

```
pt_shell> set_clock_groups -logically_exclusive -group {CK1 CK2} -group {CK3 CK4}
pt_shell> set_clock_groups -logically_exclusive -name EX1 -group {CK1 CK2} -group {CK3 CK4}
```

`set_clock_groups -asynchronous` 定义彼此异步的 clock 组，与互斥 clock group 赋值分开（尽管均用 `set_clock_groups` 定义）。Clock group 也可因时钟信号复用或物理分离而物理互斥，此时用 `-physically_exclusive` 而非 `-logically_exclusive`，防止工具尝试对 clock net 间执行 crosstalk 分析。

报告时钟间关系：`get_clock_relationship {CK1 CK2}`。移除 clock grouping 声明：`remove_clock_groups`。

示例：四时钟与一选择信号

（原书 *Figure 47: Four clocks and one selection signal*）
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.12: 四时钟与一选择信号

可用 `false path`、`case analysis`、`set_disable_timing`、`set_clock_groups` 与 `set_active_clocks` 等方法防止检查无关时钟间路径。

复用时钟互斥点

现代芯片设计常用多个时钟在不同时刻施加到给定电路。MUX cell 常用于选择这些时钟。

（原书 *Figure 50: Multiplexed Clocks*）
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.13: 复用时钟

默认 PrimeTime 考虑所有 launch 与 capture clock 组合。但对如图所示复用的时钟，MUX cell 下游任一时刻仅一个时钟可操作——只要 MUX 控制信号在 launch 与 capture 间保持静态，它们为互斥时钟。

一种指定互斥时钟的方法是在设计中仅一个时钟可传播的 pin 处设置「互斥点 (exclusivity points)」。
 手动设置互斥点：

```
pt_shell> set_clock_exclusivity -type mux -output "U15/Z"
pt_shell> set_clock_exclusivity -type mux -output "U23/Z"
```

(原书 *Figure 51: Multiplexed Clocks With Exclusivity Points*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.14: 带互斥点的复用时钟

设置 clock exclusivity point 后，任一时刻仅一个时钟可通过该 pin 传播，在 pin 传递扇出中的不同时钟间强制物理互斥关系。

自动推断互斥点：

```
pt_shell> set_app_var timing_enable_auto_mux_clock_exclusivity true
```

工具自动在 MUX cell 输出放置互斥点。禁用特定 MUX 的自动推断：

```
pt_shell> set_disable_auto_mux_clock_exclusivity MUX32/Z
```

自动检测仪适用于 MUX cell (is_mux_cell 属性)。随机逻辑构建的复用功能不会自动检测，但可手动定义：

```
pt_shell> set_clock_exclusivity -type user_defined \  
-inputs "OR1/A OR1/B" -output OR1/Z
```

报告互斥点：report_clock -exclusivity。查询 pin 互斥性：

```
pt_shell> get_attribute -class pin [get_pins U53/Z] is_clock_exclusivity
```

除互斥点外，还可使用 false path、set_clock_groups、case analysis、set_disable_timing 等方法。

7.4.4 从分析中移除时钟

Removing Clocks From Analysis

默认 PrimeTime 分析设计中指定的所有时钟及不同 clock domain 间所有交互。多时钟设计中，常希望仅用特定时钟进行时序分析。

一种方法是用 case analysis 为控制时钟选择的 port 或 pin 指定逻辑值：

```
pt_shell> set_case_analysis 0 [get_ports SEL]
```

另一种方法是用 set_active_clocks 指定分析中活跃的 clock 列表，未列出的 clock 为 inactive：

```
pt_shell> set_active_clocks {CK1 CK2}
```

若 generated clock 基于 inactive clock，generated clock 也变为 inactive。set_active_clocks 触发完整时序更新。使所有 clock 活跃（默认）：set_active_clocks [all_clocks]。每次使用该命令覆盖先前设置。set_active_clocks 适用于标准时序分析，但不适用于 context characterization、model extraction 或 write_sdc。

7.5 时钟极性

Clock Sense

工具追踪时钟树中的反相器与缓冲器, 识别到达各寄存器时钟引脚的时钟信号正/负极性(sense)。仅含缓冲器与反相器的时钟树无需指定 sense, 到达寄存器时钟引脚的时钟信号称为 unate。

Positive unate: 时钟源上升沿导致寄存器时钟引脚上升沿; 时钟源下降沿导致寄存器时钟引脚下降沿。

Negative unate: 时钟源上升沿导致寄存器时钟引脚下降沿; 时钟源下降沿导致寄存器时钟引脚上升沿。

(原书 *Figure 52: Positive and negative unate clock signals*)
请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.15: 正/负 unate 时钟信号

Non-unate: 因时钟路径中存在 non-unate timing arc 导致 clock sense 不明确。例如通过 XOR 门的时钟 non-unate, 取决于 XOR 另一输入状态, sense 可为正或负。

(原书 *Figure 53: Non-unate clock signals*)
请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.16: 非 unate 时钟信号

对 non-unate 时钟网络中某 pin 指定正或负 clock sense 以消除歧义:

```
set_sense -type clock -positive | -negative object_list
pt_shell> set_sense -type clock -positive [get_pins xor1.z]
```

set_sense -type clock 仅影响时钟网络 non-unate 部分。若在 unate 部分指定且请求 sense 与实际不符, PrimeTime 发出错误并忽略命令。

可限制到特定时钟:

```
pt_shell> set_sense -type clock -positive -clocks [get_clocks CLK] [get_pins mux1.z]
```

从特定 pin 或 cell timing arc 停止 clock 传播:

```
pt_shell> set_sense -type clock -stop_propagation -clocks TESTCLK UMUX2/Z
```

在 clock network 叶 pin 停止传播, 用 -clock_leaf 替代 -stop_propagation:

(原书 *Figure 54: Clock sense examples*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.17: 时钟极性示例

```
pt_shell> set_sense -type clock -clock_leaf -clocks CLK3 UMUX3/Z
```

移除先前指定的 sense: `remove_sense -type clock`。

7.6 指定脉冲时钟

Specifying Pulse Clocks

脉冲时钟 (pulse clock) 由另一时钟同一边沿触发的短脉冲序列组成。脉冲时钟常用于提升性能并降低功耗。

分析含脉冲时钟的电路，PrimeTime 需要时钟时序特性信息。三种提供方式：

- 使用在 .lib 描述中以 pulse generator 属性表征的 pulse generator cell；
- 用 `create_generated_clock` 相对于源时钟描述脉冲时序；
- 用 `set_sense -type clock` 指定相对于源时钟的生成脉冲 sense。

最佳方法是在 .lib 库描述中已表征的 pulse generator cell，PrimeTime 中无需额外操作。若无表征 pulse generator cell，须在每个脉冲生成点用 `create_generated_clock` 或 `set_sense -type clock` 指定特性。`create_generated_clock` 在脉冲生成点创建新 clock domain；`set_sense -type clock` 不创建新 domain，仅指定现有时钟下游某点的 sense。

(原书 *Figure 56: Pulse clock specified as a generated clock*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.18: 指定为生成时钟的脉冲时钟

CLK 每个上升沿在 CLKP 上生成脉冲。指定生成脉冲时钟 CLKP：

```
pt_shell> create_generated_clock -name CLKP -source CLK \
    -edges {1 1 3} [get_pins and2/z]
```

重复边沿数字（而非脉冲边沿时间）确保源时钟与脉冲时钟间 delay 检查正确：

- `-edges {1 1 3}`——源上升沿触发高脉冲

- -edges {2 2 4}——源下降沿触发高脉冲
- -edges {1 3 3}——源上升沿触发低脉冲
- -edges {2 4 4}——源下降沿触发低脉冲

用 `set_sense -type clock` 指定现有时钟 sense（不创建新 clock domain）：

```
pt_shell> set_sense -type clock -pulse rise_triggered_high_pulse [get_pins and2/z]
```

生成脉冲标称宽度为零。实际脉冲宽度由 pulse generator 输出 pin 的不同 rise 与 fall latency 确定：

- 高脉冲宽度 = fall network latency – rise network latency
- 低脉冲宽度 = rise network latency – fall network latency

可用 `set_clock_latency` 显式指定 latency（从而指定脉冲宽度）：

```
pt_shell> set_clock_latency -rise 0.6 and2/z
pt_shell> set_clock_latency -fall 1.1 and2/z
```

7.6.1 约束 Pulse Generator Cell 扇出中的脉冲宽度

用 `set_pulse_clock_min_width -transitive_fanout` 与 `set_pulse_clock_max_width -transitive_fanout` 约束 pulse generator 扇出中的脉冲宽度。若启用 CRPR，CRP 值作为 credit 应用于脉冲宽度。

（原书 *Figure 57: Sense propagation affects pulse width*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 `\includegraphics` 替换。

图 7.19: 极性传播影响高/低脉冲宽度

移除约束: `remove_pulse_clock_min_width`、`remove_pulse_clock_max_width`。报告: `report_pulse_clock_min_width`、`report_constraint -pulse_clock_min_width` 等。

默认在 pulse clock network 应用更具体的 pulse clock 约束，其他地方应用通用 minimum pulse width 约束。将 `timing_enable_pulse_clock_constraints` 设为 false 可忽略更具体的 pulse clock 约束。

7.6.2 约束 Pulse Generator Cell 转换时间

`set_pulse_clock_min_transition` 约束 pulse generator 实例输入的最小 transition。`set_pulse_clock_max_transition -transitive_fanout` 约束 pulse generator 扇出的最大 transition。移除: `remove_pulse_clock_min_transition`、`remove_pulse_clock_max_transition`。

7.7 PLL 设计时序分析

Timing PLL-Based Designs

锁相环 (PLL) 在高速设计中很常见, 能几乎消除大时钟树 delay, 实现更高片间通信速度。OCV 与信号完整性分析等效应需要完整准确的静态时序分析, 包括 PLL。PLL 使 launch 与 capture 触发器处时钟 skew 接近零, 使反馈 pin 处时钟相位与参考时钟相同。

PrimeTime 支持 PLL cell 分析。PLL 库 model 以 cell pin 属性形式包含 reference clock pin、output pin 与 feedback pin 信息, 用于在 PLL 输出定义 generated clock 时执行额外错误检查。时序更新时自动计算反馈路径时序并作为 PLL cell 上的相位校正应用。支持:

- 多个 PLL
- 多输出 PLL
- PLL jitter 与长期漂移
- PLL 路径 CRPR 计算
- PrimeTime SI 分析
- 反馈路径上的时序 cell
- 基于路径分析中的 PLL 调整

7.7.1 PLL 时序用法

用 create_generated_clock 的 -pll_feedback 与 -pll_output 选项创建各 PLL 生成时钟。-pll_feedback 指示用于时钟相位偏移校正的 PLL feedback pin; -pll_output 指定传播到 feedback pin 的 PLL 时钟输出 pin。

单输出 PLL 中 -pll_output pin 与 generated clock 源 pin 相同。多输出 PLL 共享反馈路径时, -pll_output pin 可与源 pin 不同。

定义 PLL 时仅特定选项可用, 且须满足: source_objects 为 PLL 时钟输出 pin 的单个 pin 对象; master_pin 对应 PLL reference clock 输入 pin; feedback_pin 对应 feedback 输入 pin; output_pin 对应反馈路径起始 output pin; 四个 pin 须属于同一 cell。PLL 输出时钟与到达 PLL reference pin 的 reference clock 须设为 propagated clock。

(原书 Figure 58: Circuit example)

请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.20: PLL 电路示例

反馈路径有时序 cell 时, 须在最后时序 cell 输出定义 generated clock。

7.7.2 PLL 漂移与抖动

用 `set_clock_latency` 的 `-pll_shift` 选项为 PLL 输出时钟定义 PLL drift 与 jitter。 `-pll_shift` 表示指定 delay 加到 PLL generated clock 的基本 early/late latency 上，与通常覆盖 clock latency 的行为不同。 Jitter 用 `-pll_shift` 与 `-dynamic` 共同指定，须对连到 PLL feedback pin 的 PLL 输出上定义的 generated clock 指定。

7.7.3 PLL 路径 CRPR 计算

多输出 PLL 中，PrimeTime 将所有输出时钟视为同相位，CRPR 计算中将输出视为彼此不可区分。从 OUTCLK1 发射、OUTCLK2 捕获的路径，PrimeTime 移除直到 PLL 输出的 clock reconvergence pessimism，即使物理最后公共 pin 是 PLL reference pin。 `report_crpr` 演示该行为。

7.7.4 报告与时序检查

`check_timing` 验证 PLL 时钟到达各 feedback pin。 `report_timing` 在 PLL 输出旁包含 PLL adjustment。

7.7.5 PLL 库 Cell 要求

PLL 库 cell 应有从 reference clock pin 到各输出的单一 positive unate timing arc。 Reference pin、output clock pin 与 feedback pin 分别由 `is_pll_reference_pin`、`is_pll_output_pin`、`is_pll_feedback_pin` 属性标识。可用 `is_pll_cell` 标识 PLL cell。

7.8 指定时钟门控 Setup 与 Hold 检查

Specifying Clock-Gating Setup and Hold Checks

门控时钟信号出现在时钟网络包含除反相器或缓冲器外的逻辑时。例如时钟信号作为逻辑 AND 一输入、控制信号作为另一输入，输出为门控时钟。

PrimeTime 自动检查门控输入的 setup 与 hold 违例，确保时钟信号不被门断或削波。该检查仅对组合门执行——一信号为可传播时钟、门控信号非时钟。

Clock-gating setup 检查确保控制数据信号在时钟激活前使能门。检查时钟引脚前沿到达时间与馈送 data pin 的任何数据信号两沿，防止时钟脉冲前沿 glitch 或削波脉冲。

(原书 Figure 59: Clock-gating setup violations)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 `\includegraphics` 替换。

图 7.21: 时钟门控 setup 违例

Clock-gating hold 检查确保控制数据信号在时钟激活期间保持稳定。用 hold 方程检查时钟引脚后沿到达时间与馈送 data pin 的数据信号两沿。Hold 违例导致后沿 glitch 或削波脉冲。Clock-gating 检查要求 gating check 须到达 clock pin 才能成功。

(原书 Figure 60: Clock-gating hold violations)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.22: 时钟门控 hold 违例

默认 PrimeTime 检查门控时钟的 setup 与 hold 时间，setup 与 hold 时间设为 0.0（除非门库 cell 有 gating setup/hold timing arc）。可用 set_clock_gating_check 指定非零 setup 或 hold 值。

set_clock_gating_check 仅影响指定 cell 或 pin 上存在的 clock-gating 检查。对整个 clock domain 网络应用参数，指定 clock 对象：

```
pt_shell> set_clock_gating_check -setup 0.2 -hold 0.4 [get_clocks CLK1]
pt_shell> set_clock_gating_check -setup 0.5 [get_cells and1]
```

report_clock_gating_check 执行 clock-gating setup 与 hold 检查，识别违例。用 -auto_inferred_only 仅报告工具自动推断的门，不报告 Power Compiler 插入或 clock-gating 库 cell 定义的检查。

移除 set_clock_gating_check 设置的检查：remove_clock_gating_check。

7.8.1 禁用或恢复时钟门控检查

set_disable_clock_gating_check objects 禁用 cell 或 pin 上的 clock-gating 检查。remove_disable_clock_gating_check objects 恢复。未指定 objects 时禁用所有检查，等价于 timing_disable_clock_gating_checks = true。

7.9 指定内部生成时钟

Specifying Internally Generated Clocks

设计可能包含时钟分频器或其他从 master source clock 产生新时钟的结构。由片上逻辑从另一时钟产生的时钟称为 generated clock。

(原书 Figure 61: Divide-by-2 clock generator)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.23: 二分频时钟发生器

PrimeTime 不考虑时钟发生器电路逻辑，须将 `generated clock` 行为指定为独立时钟。但可定义 `master clock` 与 `generated clock` 间关系，使 `master clock` 变更时 `generated clock` 特性自动变更。

`create_generated_clock` 指定内部生成时钟特性。默认对现有 `generated clock` 对象使用 `create_generated_clock` 会覆盖该时钟。Generated clock 对象在分析时展开为 `real clock`，在 `report_timing` 内自动发生，或显式用 `update_timing`。

可用 `-divide_by`、`-multiply_by` 或 `-edges` 创建 `generated clock`。用 `-combinational` 修改 `generated clock` 波形时，`master clock` 的 `rise` 与 `fall` 边沿须有有效路径传播到 `generated clock` 源 `pin`。

7.9.1 指定二分频生成时钟

Specifying a Divide-by-2 Generated Clock

用 `-divide_by` 指定分频因子：

```
pt_shell> create_generated_clock -name DIVIDE \
    -source [get_ports SYSCLK] -divide_by 2 [get_pins FF1/Q]
```

`-source` 指定 `master` 源（`port` 或 `pin`，非 `clock` 名）；指定 `pin` 作为新 `generated clock` 创建点。Generated clock 边沿基于 `master clock` 源 `pin` 上升沿出现。若需基于 `master` 源 `pin` 下降沿，用 `-edges` 而非 `-divide_by`。

7.9.2 基于边沿创建生成时钟

用 `create_generated_clock -edges` 以 `master pin` 上 `master clock` 波形边沿指定 `generated clock`。

（原书 *Figure 62: Divide-by-3 clock generator*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.24: 三分频时钟发生器

```
pt_shell> create_generated_clock -edges { 1 5 7 } \
    -name DIV3A -source [get_ports SYSCLK] [get_pins U2/Q]
```

7.9.3 为脉冲发生器创建时钟极性

Pulse generator 可改善性能并节省功耗。对不改变输入时钟频率的 `pulse generator`，可设置 `pulse_clock` 属性为以下 `sense`，无需添加 `generated clock`：

- `rise_triggered_high_pulse`
- `rise_triggered_low_pulse`
- `fall_triggered_high_pulse`
- `fall_triggered_low_pulse`

(原书 *Figure 63: Pulse clock types that do not change frequency*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.25: 不改变频率的脉冲时钟类型

理想时钟宽度为 0。脉冲宽度计算：高脉冲 = $\text{fall_network_latency} - \text{rise_network_latency}$ ；低脉冲 = $\text{rise_network_latency} - \text{fall_network_latency}$ 。

改变频率的 pulse generator（如倍频器）须用 `create_generated_clock`。

(原书 *Figure 64: Pulse clock types that change frequency*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.26: 改变频率的脉冲时钟类型

7.9.4 基于下降沿创建分频时钟

(原书 *Figure 65: Generated divide-by-2 clocks based on different edges*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.27: 基于不同边沿的生成二分频时钟

DIV2A 基于 SYSCLK port 上升沿，可用 `-divide_by` 或 `-edges`：

```
pt_shell> create_generated_clock -name DIV2A \
    -source [get_ports SYSCLK] -divide_by 2 [get_pins FF1/Q]
pt_shell> create_generated_clock -name DIV2A \
    -source [get_ports SYSCLK] -edges { 1 3 5 } [get_pins FF1/Q]
```

DIV2B 基于下降沿，不能用 `-divide_by`，但可用 `-edges`：

```
pt_shell> create_generated_clock -name DIV2B \
    -source [get_ports SYSCLK] -edges { 2 4 6 } [get_pins FF2/Q]
```

7.9.5 偏移生成时钟边沿

可用 `-edge_shift` 将 generated clock 边沿偏移指定时间量（不计为 clock latency）。

（原书 *Figure 66: Generated Divide-by-3 Clock With Shifted Edges*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.28: 边沿偏移的生成三分频时钟

```
pt_shell> create_generated_clock -edges { 3 5 9 } -edge_shift { 2.2 2.2 2.2 } \
    -name DIV3C -source [get_ports SYSCLK] [get_pins U4/QN]
```

注意

`-edge_shift` 仅可与 `-edges` 联用。

7.9.6 源引脚处的多个时钟

`create_generated_clock` 的 `-source` 引脚可能接收多个时钟，可能需用 `-master_clock` 与 `-source` 指定用于创建 generated clock 的时钟。

7.9.7 选择生成时钟对象

`get_clocks` 可按命名约定或过滤选择 generated clock:

```
pt_shell> set_false_path -from [get_clocks CLK_DIV*] -to [get_clocks CLKB]
pt_shell> set_false_path -from [get_clocks CLK* -filter "is_generated==true"] \
    -to [get_clocks CLKB]
```

7.9.8 报告时钟信息

`report_clock` 报告 clock 与 generated clock 信息。`report_clock_timing` 提供详细 clock network 报告。`report_transitive_fanout -clock_tree` 显示设计中 clock network。

7.9.9 移除生成时钟对象

`remove_generated_clock CLK_DIV2` 移除 generated clock 及其对应 clock 对象。

7.10 生成时钟边沿特定源延迟传播

Generated Clock Edge Specific Source Latency Propagation

PrimeTime 计算 generated clock 的 clock source latency 时考虑 master clock 与 generated clock 间边沿关系，寻找传播到 generated clock 指定边沿类型的最差路径。若路径存在但不满足所需边沿关系，PrimeTime 返回 UITE-461 错误（使用零 clock source latency）。

7.11 时钟网格分析

Clock Mesh Analysis

PrimeTime SI 提供用 SPICE 仿真以晶体管级精度分析 clock mesh 网络的简便方法。若时钟网络不变，可在后续 PrimeTime 时序与签核分析中复用仿真结果。

7.11.1 时钟网格分析概述

指定要分析的时钟网络根节点。PrimeTime SI 从该节点追踪整个时钟网络（含 clock mesh 与时序器件负载），调用外部 HSPICE 兼容电路仿真器仿真网络，确定时钟网络中各 cell arc 与 net arc 的精确时序，将测量 delay back-annotate 到设计，并对 clock mesh 结构执行 timing arc 缩减——保留单个 mesh driver，禁用其他 mesh driver 的 timing，被禁用 driver 的 timing arc 替换为从保留 driver 出发的等价 timing arc。

（原书 *Figure 67: Clock mesh analysis and circuit reduction*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.29: 时钟网格分析与电路缩减

电路缩减目的：避免全 mesh 分析中大量 driver-load 组合，同时保持准确 driver-to-load 时序结果。 n 个 driver、 m 个 load 的 mesh 有 $n \times m$ 条 timing arc；缩减为单 driver 后仅 m 条。

为 CRPR，从 mesh driver 到最近 mesh load 的最短 delay 代表所有 driver-to-load timing arc 共享的 delay 量。PrimeTime SI 在 CRPR 公共点之前计入该共享 delay。

（原书 *Figure 68: CRPR From Mesh*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 7.30: 来自 Mesh 的时钟重汇聚悲观移除

7.11.2 执行时钟网格分析

步骤 1: 用 `clock network simulation` 命令生成 `clock mesh model` (一次性)。

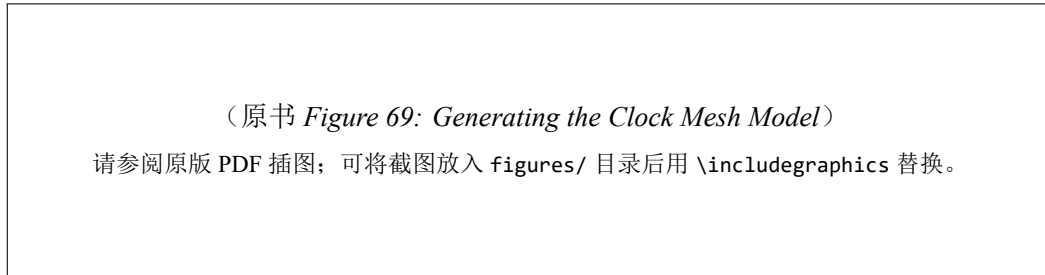


图 7.31: 生成时钟网格模型

步骤 2: 在后续分析中复用生成的 `clock mesh model`。

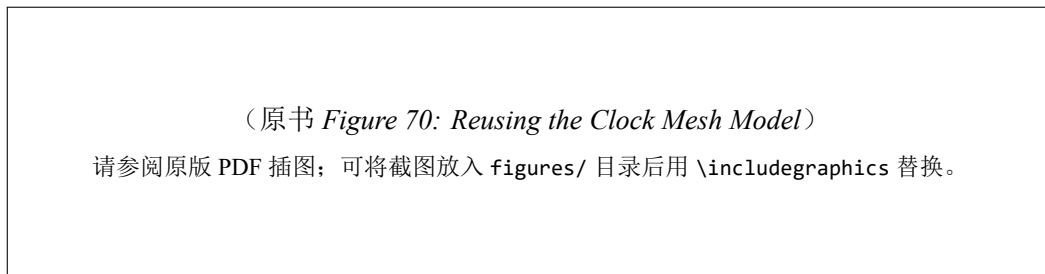


图 7.32: 复用时钟网格模型

用 `sim_setup_simulator` 设置 SPICE 仿真器; `sim_setup_library` 设置逻辑库 cell 关联的 SPICE 电路文件; 可选 `sim_validate_setup` 检查设置; `sim_analyze_clock_network` 执行仿真并将结果 `back-annotate`, 仿真结果存入 Tcl 脚本文件。

参见

时钟网络时序报告, 见相关章节。Clock network simulation 命令详见 PrimeTime SI 文档。

Chapter 8

时序路径与例外

Timing Paths and Exceptions

时序路径是设计中从寄存器时钟引脚或输入 port 出发，经组合逻辑，终止于寄存器 data 输入引脚或输出 port 的点到点序列。

有关路径 startpoint 与 endpoint、delay 计算、setup 与 hold 约束、time borrowing 与时序例外的基础信息，见「静态时序分析概述」。要进一步了解时序路径，请参阅：

- 时序路径组
- 路径时序报告
- 路径时序计算
- 指定时序路径
- 时序例外
- 保存与恢复时序路径集合

8.1 时序路径组

Timing Path Groups

PrimeTime 将路径组织为组。路径分组影响时序分析报告生成。例如 report_timing 与 -group 选项报告所列 path group 中每条组的最差路径。

在 Design Compiler 中，路径分组也影响设计优化。每个 path group 可分配 weight（也称 cost function），weight 越高，Design Compiler 优化该组路径的努力越大。PrimeTime 中也可为 path group 分配 weight，但 PrimeTime 不使用该 weight 信息。

每次用 create_clock 创建新时钟时，PrimeTime 隐式创建 path group，名称与时钟相同。若路径 endpoint 为由该时钟计时的 flip-flop，路径归入该 path group。

PrimeTime 还隐式创建以下 path group：

- **clock_gating_default**——终止于时钟门控用组合元件的路径组
- **async_default**——终止于 flip-flop 异步 preset/clear 输入的路径组

- ****default****——有约束但不属于其他隐式类别的路径组（例如终止于 output port 的路径）

除隐式 path group 外，可用 `group_path` 创建用户定义的 path group，并将特定路径分配到指定 path group。当前 path group 信息：report_path_group。注意 `group_path` 与层次 model extraction 不兼容。

移除 path group: `remove_path_group`，该组路径隐式分配到 default path group。

```
pt_shell> group_path -name out1bus -to [get_ports OUT_1*]
```

未约束路径不属于任何 path group。报告未约束路径，将 `timing_report_unconstrained_paths` 设为 true。report_timing 将未约束路径报告为属于“(none)” path group。

8.2 路径时序报告

Path Timing Reports

路径时序报告用于聚焦特定时序违例并确定违例原因。默认 `report_timing` 报告所有 path group 中 setup slack 最差的路径。

路径时序报告提供特定路径的详细时序信息，可指定输出细节，例如：

- 逻辑路径中的门级
- 各级 incremental delay
- 总路径 delay 之和
- 路径 slack 量
- 源与目的时钟名、latency 与 uncertainty
- 移除 clock reconvergence pessimism 后的 OCV

用 `report_timing` 选项控制报告的信息类型：路径数量、路径类型、详细程度、startpoint、endpoint 及路径中间点。

不带选项的 `report_timing` 等价于：

```
pt_shell> report_timing -path_type full -delay_type max -max_paths 1
```

PrimeTime 生成最大 delay（setup 约束）的完整路径报告，仅显示单条最差路径。报告四条最差 setup 路径：

```
pt_shell> report_timing -max_paths 4
```

`-max_paths` 大于 1 时，仅报告负 slack 路径。要在多路径报告中包含正 slack 路径，用 `-slack_lesser_than`：

```
pt_shell> report_timing -slack_lesser_than 100 -max_paths 40
```

报告每个 path group 中最差 setup 路径：

```
pt_shell> report_timing -group [get_path_groups *]
```

显示 transition time 与 capacitance：

```
pt_shell> report_timing -transition_time -capacitance
```

8.3 路径时序计算

Path Timing Calculation

`report_delay_calculation` 报告从 cell 输入到输出或从 driver 到 net 上 load 的 delay 详细计算。所见信息类型取决于所用 delay model。

许多 ASIC 供应商将 cell delay 计算详细信息视为专有。为保护该信息, `report_delay_calculation` 仅在库供应商通过 Library Compiler 的 `library_features` 属性启用该功能时才显示 cell delay 详情。

8.4 指定时序路径

Specifying Timing Paths

`report_timing`、时序例外命令（如 `set_false_path`）及若干其他命令允许多种方法指定单条或多条路径用于时序报告或应用时序例外。一种方法是在 `report_timing` 中显式指定 `-from` 与 `-to`：

```
pt_shell> report_timing -from PORTA -to PORTZ
pt_shell> set_false_path -from FFB1/CP -to FFB2/D
pt_shell> set_max_delay -from REGA -to REGB 12
pt_shell> set_multicycle_path -setup 2 -from FF4 -to FF5
```

每个指定 startpoint 可为 clock、primary input port、时序 cell、时序 cell 时钟输入 pin、电平敏感 latch data pin 或指定了 input delay 的 pin。若指定 clock，所有与该 clock 相关的寄存器与 primary input 用作路径 startpoint。若指定 cell，命令应用于该 cell 上一个路径 startpoint。

每个指定 endpoint 可为 clock、primary output port、时序 cell、时序 cell data 输入 pin 或指定了 output delay 的 pin。若指定 clock，所有与该 clock 相关的寄存器与 primary output 用作路径 endpoint。

PrimeTime 还支持 startpoint 或 endpoint 变为 `-through` pin、clock 对象变为 `-from` 或 `-to` 对象的特殊形式。可与所有有效 startpoint/endpoint 类型联用。

注意

该方法计算量更大，尤其在大型设计中。尽可能使用 `-from` 与 `-to` 指定路径。

可用 `-rise_from`、`-fall_from`、`-rise_to`、`-fall_through` 等将 `report_timing` 限制为仅上升或下降边沿。

8.4.1 多个 Through 参数

单条命令中可用多个 `-through`、`-rise_through`、`-fall_through` 指定路径组：

```
pt_shell> report_timing -from A1 -through B1 -through C1 -to D1
pt_shell> report_timing -from A1 -through {B1 B2} -through {C1 C2} -to D1
```

后者表示从 A1 出发、依次经过 B1 或 B2、再经过 C1 或 C2、终止于 D1 的任意路径。

8.4.2 Rise/Fall From/To Clock

可用 `-rise_from`、`-fall_to` 等限制命令仅针对上升或下降边沿。当“from”或“to”对象为 clock 时，命令基于源时钟特定边沿在 startpoint 发射数据或在 endpoint 捕获数据指定路径。路径选择考虑

时钟路径上任何逻辑反相。

(原书 *Figure 71: Circuit for path specification examples 1-3*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 8.1: 路径指定示例 1–3 的电路

示例 1: `set_false_path -to [get_clocks CLK]`——Figure 71 中所有由 CLK 在 endpoint 计时的路径（四条路径）均为 false。

示例 2: `set_false_path -rise_from [get_clocks CLK]`——由 CLK 在 startpoint 计时且由源时钟上升沿发射数据的路径为 false（Path A 与 Path D）。

示例 3: `set_false_path -fall_to [get_clocks CLK]`——由 CLK 在 endpoint 计时且由源时钟下降沿捕获数据的路径为 false（Path B 与 Path C）。

示例 4: Figure 72 中两 flip-flop 在时钟正沿捕获数据，电平敏感 latch 在上升沿打开、下降沿关闭。

(原书 *Figure 72: Circuit for path specification example 4*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 8.2: 路径指定示例 4 的电路

`report_timing -rise_from [get_clocks clk]` 报告由时钟上升沿发射数据的路径。`report_timing -fall_to [get_clocks clk]` 报告由时钟下降沿捕获数据的路径（FF0 到 LAT1）。

示例 5: Figure 73 与示例 4 相同，但 latch 在下降沿打开、上升沿关闭。

(原书 *Figure 73: Circuit for path specification example 5*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 8.3: 路径指定示例 5 的电路

示例 6–7: 时钟反相时，`-fall_from` 与 `-rise_to` 的行为相应反转。

示例 8: Figure 76 中多个时钟到达 flip-flop。

仅对 `clk1`（非 `clk2`）在 FF1 D pin 结束设置 false path:

(原书 *Figure 76: Circuit for path specification example 8*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 8.4: 路径指定示例 8 的电路

```
pt_shell> set_false_path -through FF1/D -to [get_clocks clk1]
pt_shell> set_false_path -through FF1/D -rise_to [get_clocks clk1]
```

8.4.3 无效 Startpoint 或 Endpoint 的报告

`timing_report_always_use_valid_start_end_points` 变量指定 PrimeTime 如何报告无效 startpoint 与 endpoint, 影响 `report_timing`、`report_bottleneck` 与 `get_timing_paths` 对 `-from`、`-rise_from`、`-fall_from`、`-to`、`-rise_to`、`-fall_to` 的处理。

默认 `false` 时, `from_list` 解释为所列对象范围内所有 pin。例如 `-from [get_pins FF/*]` 包含 input、output 与异步 pin, PrimeTime 默认将这些无效 startpoint/endpoint 视为 through 点并继续搜索。

设为 `true` 时, 各报告命令忽略无效 startpoint 与 endpoint。无论该变量设置如何, 建议在 `from_list` 中始终指定有效 startpoint (input port 与寄存器时钟 pin), 在 `to_list` 中指定有效 endpoint (output port 与寄存器 data pin)。

8.5 时序例外

Timing Exceptions

默认 PrimeTime 假设在路径 startpoint 发射的数据在路径 endpoint 由下一个时钟边沿捕获。对非如此工作的路径, 须指定时序例外, 否则时序分析不匹配实际电路行为。

进一步了解时序例外, 请参阅:

- 时序例外概述
- 单周期 (默认) 路径延迟约束
- 设置 False Path
- 设置最大与最小路径延迟
- 设置多周期路径
- 指定时序裕量
- 高效指定例外
- 例外优先级顺序
- 报告例外

- 报告时序例外
- 报告例外源文件与行号信息
- 检查被忽略的例外
- 移除例外

8.5.1 时序例外概述

Overview of Timing Exceptions

可设置以下时序例外：

表 8.1: 时序例外类型

时序例外	命令	说明
False path	set_false_path	阻止分析指定路径；移除路径时序约束
最小/最大路径 delay	set_max_delay、set_min_delay	用特定最大/最小时间值覆盖默认 setup/hold 约束
多周期路径	set_multicycle_path	指定数据从路径起点到终点所需时钟周期数

每个时序例外命令可应用于单条路径或相关路径组（如从一个 clock domain 到另一个的所有路径，或经过指定点的所有路径）。见「指定时序路径」。

指定路径的 startpoint 与 endpoint 时，确保指定 PrimeTime 中有效的时序路径。路径 startpoint 须为寄存器时钟 pin 或 input port；路径 endpoint 须为寄存器 data 输入 pin 或 output port。

查看已应用于设计的时序例外列表：report_exceptions。也可保留源文件与行号信息用于约束跟踪与报告。

用 reset_path 恢复路径上的默认时序约束。

8.5.2 单周期（默认）路径延迟约束

Single-Cycle (Default) Path Delay Constraints

确定是否需要设置时序例外，须理解 PrimeTime 如何计算路径最大与最小 delay，以及该计算与被分析设计实际操作的相似或差异。静态时序分析原理入门见「静态时序分析概述」。

单时钟 Flip-Flop 路径延迟

Figure 77 展示 PrimeTime 如何确定 rising-edge-triggered flip-flop 路径的 setup 与 hold 约束。两 flip-flop 由同一时钟触发，时钟周期 10 ns。

PrimeTime 执行 setup 检查，验证 FF1 在 time=0 发射的数据在 time=10 捕获边沿前到达 FF2 D 输入。数据到达过晚则报告 setup 违例。类似地执行 hold 检查，验证 FF1 在 time=0 发射的数据不会过早被 FF2 在 time=0 捕获边沿捕获。

Setup 与 hold 要求考虑组合逻辑 delay、库中 flip-flop 的 setup/hold 要求及 flip-flop 间 net delay。

(原书 *Figure 77: Single-Cycle Setup and Hold for Flip-Flops*)
 请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 8.5: Flip-Flop 单周期 Setup 与 Hold

不同时钟 Flip-Flop 路径延迟

Launch 与 capture flip-flop 属于不同 clock domain 时, 路径 delay 计算算法更复杂。

(原书 *Figure 78: Setup Constraints for Flip-Flops With Different Clocks*)
 请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 8.6: 不同时钟 Flip-Flop 的 Setup 约束

```
pt_shell> create_clock -period 10 -waveform {0 5} CLK1
pt_shell> create_clock -period 25 -waveform {5 12.5} CLK2
```

默认 PrimeTime 假设两时钟彼此同步且具有固定相位关系。若实际电路并非如此 (例如两时钟从不同时活跃或异步操作), 须用多种方法之一声明, 否则 PrimeTime 会花费时间检查实际电路中不存在的约束并报告违例。

Setup 分析: PrimeTime 考察完整重复周期 (两时钟周期最小公倍数) 内活跃时钟边沿间关系。对每个 destination flip-flop 捕获 (latch) 边沿, 假设对应 launch 边沿为捕获边沿之前最近的源时钟边沿。Figure 78 中 Setup 1 (time=0 到 time=5) 更严格, 确定路径最大允许 delay 为 5 ns。

Hold 分析: Hold 关系基于用于确定 setup 关系的时钟边沿的相邻边沿。对每个 setup 关系, PrimeTime 执行两种 hold 检查: setup launch 边沿发射的数据不能被前一捕获边沿捕获; 下一 launch 边沿发射的数据不能被 setup capture 边沿捕获。

(原书 *Figure 79: Hold Constraints for Flip-Flops With Different Clocks*)
 请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 8.7: 不同时钟 Flip-Flop 的 Hold 约束

最严格 hold 检查为捕获边沿相对 launch 边沿最晚者 (Figure 79 中 Hold 2b), 确定最小允许 delay 为 0 ns。

单周期路径分析示例

Figure 80: CLK1 周期 20 ns, CLK2 周期 10 ns。最严格 setup: time=0 到 time=10; 最严格 hold: time=0 到 time=0。

(原书 *Figure 80: Delay Requirements With 20ns/10ns Clocks*)
 请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 8.8: 20 ns/10 ns 时钟的延迟要求

Figure 81: CLK1 周期 4 ns, CLK2 周期 3 ns。最严格 setup: time=3 到 time=4; 最严格 hold: time=7 到 time=7。

(原书 *Figure 81: Delay Requirements With 4ns/3ns Clocks*)
 请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 8.9: 4 ns/3 ns 时钟的延迟要求

8.5.3 设置 False Path

Setting False Paths

False path 是存在但不应进行时序分析的逻辑路径。例如两个复用逻辑块间的路径若从不同时选中, 则该路径对时序分析无效。

```
pt_shell> set_false_path -from [get_pins FFB1/CP] -to [get_pins FFB2/D]
```

声明路径为 false 移除路径上所有时序约束。PrimeTime 仍计算路径 delay, 但无论 delay 多长或多短均不报告为错误。

设置 false path 是点到点时序例外, 不同于 set_disable_timing (禁用指定 pin、cell 或 port 的时序分析)。若通过某 pin 的所有路径均为 false, set_disable_timing [get_pins pin_name] 比 set_false_path -through [get_pins pin_name] 更高效。

另一 false path 示例: 属于彼此异步的两个 clock domain 的 flip-flop 间路径。声明两 clock domain 间所有路径为 false:

```
pt_shell> set_false_path -from [get_clocks ck1] -to [get_clocks ck2]
pt_shell> set_false_path -from [get_clocks ck2] -to [get_clocks ck1]
```

效率提示: 用 clock 名 (get_clocks) 而非 pin 名 (get_pins) 指定各 clock。

也可用 `set_clock_groups` 排除由一时钟发射、由另一时钟捕获的路径。效果与声明 `false path` 相同，但不视为时序例外，`report_exceptions` 不报告。

8.5.4 设置最大与最小路径延迟

Setting Maximum and Minimum Path Delays

默认 PrimeTime 根据时钟边沿时间计算最大与最小路径 delay。用 `set_max_delay` 或 `set_min_delay` 以自定义时间值覆盖默认最大或最小时间。

```
pt_shell> set_max_delay 12.0 -from [get_cells REGA] -to [get_cells REGB]
pt_shell> set_min_delay 2.0 -from [get_cells REGA] -to [get_cells REGB]
```

有时序例外时，PrimeTime 忽略时钟关系。超过 12 时间单位减 endpoint 寄存器 setup 要求的路径 delay 报告为违例；小于 2 时间单位加 hold 要求的路径 delay 报告为违例。可选指定 delay 值仅应用于 endpoint 上升或下降边沿。

确保用 `-from` 选择有效路径 startpoint、用 `-to` 选择有效 endpoint，除非要覆盖 PrimeTime 检查的路径。指定无效 startpoint 或 endpoint 时，PrimeTime 完全按指定执行检查并忽略路径剩余有效部分。在 clock path 上应用此类例外会阻止时钟从应用点向前传播。指定无效 startpoint/endpoint 时 PrimeTime 发出 UITE-217 警告。

用 `-probe` 为最大 delay 约束设置时序 endpoint 而不强制其他时序路径的 endpoint:

```
pt_shell> set_max_delay 4.5 -to I_TOP/MEM/U133/Z -probe
```

`-from` 参数指定时序 cell 时，命令自动扩展应用于 cell 每个时钟输入 pin；`-to` 指定时序 cell 时扩展应用于每个 data 输入 pin。可用 `report_exceptions` 或 `write_sdc` 查看展开的 pin-by-pin 例外。在 IC Compiler II 等工具中例外保持在 cell 级，可能导致不同工具以不同方式解决重叠例外。

8.5.5 设置多周期路径

Setting Multicycle Paths

用 `set_multicycle_path` 指定数据从路径起点传播到终点所需的时钟周期数。PrimeTime 根据指定周期数计算 setup 或 hold 约束。

（原书 *Figure 82: Multicycle Path Example*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 8.10: 多周期路径示例

Figure 82 中 FF4 到 FF5 的路径设计为两个时钟周期而非一个。默认 PrimeTime 假设所有路径为单周期时序，须为该路径指定时序例外。

虽可用 `set_max_delay` 指定显式最大 delay 值，但更好使用 `set_multicycle_path`，在更改时钟周期时自动调整最大 delay 值。

```
pt_shell> set_multicycle_path -setup 2 -from [get_cells FF4] -to [get_cells FF5]
```

该命令指定路径需两个时钟周期而非一个，建立 Figure 83 所示新 setup 关系——launch 边沿后第二个（而非第一个）捕获边沿成为路径终点适用边沿。

（原书 *Figure 83: Multicycle path setup relationship*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 8.11: 多周期路径 setup 关系

改变 setup 关系也隐式改变 hold 关系，因为所有 hold 关系基于有效 setup 关系。PrimeTime 验证 setup launch 边沿发射的数据不被前一捕获边沿捕获。新 hold 关系见 Figure 84。

（原书 *Figure 84: Multicycle path hold relationship*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 8.12: 多周期路径 hold 关系

新 hold 关系可能不是设计所需的关系。若 FF4 无需在第一个时钟边沿之后继续保持数据，须指定另一时序例外。虽可用 `set_min_delay` 指定显式 hold 时间，但更好的做法是再使用一条 `set_multicycle_path`，将 hold 关系的捕获边沿向较早时间回移一个时钟周期：

```
pt_shell> set_multicycle_path -setup 2 -from [get_cells FF4] -to [get_cells FF5]
pt_shell> set_multicycle_path -hold 1 -from [get_cells FF4] -to [get_cells FF5]
```

Figure 85 展示用两条 `set_multicycle_path` 命令正确设置的 setup 与 hold 关系。第二条命令将 hold 关系的捕获边沿向较早时间回移一个时钟周期，即从虚线箭头所示位置移到实线箭头所示位置。

（原书 *Figure 85: Multicycle path hold set correctly*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 8.13: 正确设置的多周期 hold

PrimeTime 对 `set_multicycle_path` 的 `-setup` 与 `-hold` 选项解释不同：

- `-setup` 整数值指定多周期路径的时钟周期数，无时序例外时默认为 1

- **-hold** 整数值指定相对于默认位置（相对于有效 **setup** 关系）将捕获边沿向较早时间回移的时钟周期数，默认为 0

（原书 *Figure 86: Multicycle path taking five clock cycles*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 8.14: 五时钟周期的多周期路径

```
pt_shell> set_multicycle_path -setup 5 -from [get_cells FF1] -to [get_cells FF2]
pt_shell> set_multicycle_path -hold 4 -from [get_cells FF1] -to [get_cells FF2]
```

-setup 5 使 **setup** 关系跨越五个时钟周期（**time=0** 到 **time=50**），并隐式将 **hold** 关系改为位于 **time=40** 的前一捕获边沿。要将 **hold** 捕获边沿移回 **time=0**，须用 **-hold 4** 将捕获边沿向较早时间回移四个时钟周期。

总结：**hold 周期数 = (setup 选项值) - 1 - (hold 选项值)**。默认 **hold 周期 = 1 - 1 - 0 = 0**。Figure 85: **hold 周期 = 2 - 1 - 1 = 0**。Figure 86: **hold 周期 = 5 - 1 - 4 = 0**。

可选指定多周期路径例外仅应用于路径 **endpoint** 上升或下降边沿。若 **startpoint** 与 **endpoint** 由不同时钟计时，可指定调整路径时钟周期数时考虑哪个时钟。

8.5.6 指定时序裕量

Specifying Timing Margins

用 **set_path_margin** 将任何时序检查收紧或放宽指定时间量：

```
pt_shell> set_path_margin -setup 1.2 -to I_BL/reg30a/D
```

使到达指定寄存器 **pin** 的所有 **setup** 检查更严格 1.2 时间单位。报告 **setup slack** 比无 **path margin** 时少 1.2 时间单位（更负）。**report_timing** 报告中 **margin** 显示为“data required time”计算中的“**path margin**”调整。

对所有时序路径，正 **margin** 使检查更严格；负 **margin** 使检查更宽松。**Path margin** 设置是一种时序例外，类似于 **set_max_delay** 或 **set_multicycle_path** 设置的例外；指定的 **margin** 会叠加到同一路径上适用的 **minimum-delay**、**maximum-delay** 和 **multicycle path** 例外之上。

8.5.7 高效指定例外

Specifying Exceptions Efficiently

许多情况下可用多种方式指定例外路径。选择高效方法可减少分析运行时间。

Figure 87 中三个 16 位寄存器由三个不同时钟计时。**REG2** 仅用于测试，正常运行时 **REG2** 到 **REG3** 的所有路径为 **false path**。防止 **REG2** 到 **REG3** 时序分析的方法（效率从高到低）：

- Case analysis: **set_case_analysis 0** 于 **TEST_ENABLE**

(原书 *Figure 87: Multiplexed register paths*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 8.15: 复用寄存器路径

- 声明 CLK1 与 CLK2 clock domain 互斥
- 声明 CLK2 到 CLK3 clock domain 间 false path (高效——PrimeTime 仅跟踪 clock domain)
- 逐条声明 16 条 REG2 到 REG3 路径 (较低效)
- 用 wildcard 声明所有 REG2 到 REG3 路径 (更低效——256 条路径)
- 声明所有从 REG2 出发的路径为 false (类似低效)

总结：分析使例外必要的根本原因，找到控制受影响路径时序分析的最简方法。使用 false path 前，考虑 case analysis、set_clock_groups 或 set_disable_timing。若必须设置 false path，避免用 -through、wildcard 或逐条列出大量路径。

8.5.8 例外优先级顺序

Exception Order of Precedence

若不同时序例外命令对特定路径冲突，PrimeTime 使用的例外取决于例外类型或冲突命令中使用的路径指定方法。规则集建立不同例外设置情况的优先级顺序。

PrimeTime 在每条路径（非每条命令）上独立应用例外优先级规则。例如：

```
pt_shell> set_max_delay -from A 5.1
pt_shell> set_false_path -to B
```

set_false_path 优先于 set_max_delay，从 A 开始且终止于 B 的路径为 false path，但 set_max_delay 仍应用于从 A 开始但不终止于 B 的路径。

例外类型优先级

以下时序例外类型对不视为冲突，两设置对路径均可有效：

- 两个 set_false_path 设置
- set_min_delay 与 set_max_delay 设置
- set_multicycle_path -setup 与 -hold 设置

冲突时，时序例外类型优先级（从高到低）：

1. set_false_path
2. set_max_delay 与 set_min_delay
3. set_multicycle_path

用 `report_exceptions -ignored` 列出被忽略的时序例外。

路径指定优先级

相同类型时序例外用不同路径指定应用时,更具体的命令优先于更一般的命令。更具体对象 (`pin` 或 `port`) 上的例外优先于更一般对象 (`clock`) 上的例外。

`-from/-to` 路径指定方法优先级 (从高到低):

1. `-from pin`、`-rise_from pin`、`-fall_from pin`
2. `-to pin`、`-rise_to pin`、`-fall_to pin`
3. `-through`、`-rise_through`、`-fall_through`
4. `-from clock`、`-rise_from clock`、`-fall_from clock`
5. `-to clock`、`-rise_to clock`、`-fall_to clock`

可能的组合按优先级排序 (最高到最低): `-from pin -to pin`、`-from pin -to clock`、`-from pin -from clock -to pin`、`-to pin`、`-from clock -to clock`、`-from clock -to clock`。

8.5.9 报告例外

Reporting Exceptions

报告已设置的时序例外: `report_exceptions`。可用 `-from`、`-to`、`-through`、`-rise_from`、`-fall_to` 等路径指定参数缩小报告范围,匹配创建原始例外时使用的路径指定符。

`report_exceptions` 导致完整时序更新,须在设置所有时序断言并准备接收报告后使用。

例外设置命令有时因多种原因被部分或完全忽略。部分忽略的命令,例外报告显示被忽略部分的原因。默认完全忽略的命令不被 `report_exceptions` 报告。用 `-ignored` 报告完全忽略的命令。

(原书 *Figure 88: Design to demonstrate ignored exceptions*)

请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 8.16: 演示被忽略例外的设计

8.5.10 报告时序例外

Reporting Timing Exceptions

`report_timing` 的 `-exceptions` 选项允许报告应用于单条路径的时序例外,可报告: 适用的例外、被更高优先级例外覆盖的例外、未约束路径调试 (含未约束 `startpoint/endpoint`)、显示未约束原因的时序路径属性。

用 `report_timing -exceptions all` 或 `get_timing_paths` 报告未约束原因前,须先将 `timing_report_unconstrained_paths` 设为 `true`。

```
pt_shell> report_timing -exceptions dominant
pt_shell> report_timing -exceptions overridden
pt_shell> report_timing -exceptions all
```

例如，下列冲突例外中，maximum-delay 例外的优先级高于 multicycle path 例外：

```
pt_shell> set_multicycle_path -through buf1/Z -setup 2
pt_shell> set_max_delay -through buf1/Z 1
```

report_timing -exceptions dominant 显示 dominant 例外；report_timing -exceptions overridden 显示被覆盖且未影响时序计算的例外；report_timing -exceptions all 显示两者。

get_timing_paths 可创建时序路径集合并传递给 report_timing。时序路径属性（dominant_exception、startpoint_unconstrained_reason、endpoint_unconstrained_reason）显示路径未约束原因，如 false_path、min_delay/max_delay、multicycle_path、no_capture_clock、no_launch_clock、dangling_end_point、dangling_start_point、fanin_of_disabled、fanout_of_disabled 等。

report_timing -exceptions 始终显示三类信息：dominant exception、startpoint unconstrained reason 和 endpoint unconstrained reason；某一类没有信息时，对应部分为空。上述属性仅在相应信息存在时才出现在 timing_path 对象上，因此可用 filter_collection 的 defined(...) 或 undefined(...) 表达式筛选受例外影响的路径。

8.5.11 报告例外源文件与行号信息

Reporting Exceptions Source File and Line Number Information

对部分约束，PrimeTime 可跟踪并报告约束的源文件与行号。源文件通过 source、read_sdc 或 -f 命令行选项读入 PrimeTime shell。

启用当前设计约束的源文件名与行号信息，将 sdc_save_source_file_information 设为 true（默认 false）。

注意

仅可在尚未输入例外时修改该变量值。若尝试设置时至少一条例外命令已成功输入，将收到错误且值保持不变。

适用范围：set_false_path、set_multicycle_path、set_max_delay、set_min_delay。交互输入的命令无源位置信息。控制结构（if、foreach）内输入的命令报告闭合括号行号；过程调用内输入的命令报告调用过程行号。

8.5.12 检查被忽略的例外

Checking Ignored Exceptions

已指定但 PrimeTime 未接受的时序例外称为被忽略例外。忽略原因：

- 指定 startpoint 或 endpoint 无效
- 指定路径未约束

- 路径因 `set_disable_timing`、constant propagation、环断开或 case analysis 无效
- 例外优先级低于应用于同一路径的另一例外

`report_exceptions` 报告完全与部分有效例外；`report_exceptions -ignored` 报告所有完全忽略例外。宜检查这些报告确认例外指定正确。大量被忽略例外可增加内存使用与分析运行时间。

检查两点间是否存在逻辑路径：`all_fanout -from point_a -to point_b`。时序更新后检查路径时序：`report_timing -from point_a -to point_b`。

8.5.13 移除例外

Removing Exceptions

移除先前用 `set_false_path`、`set_max_delay`、`set_min_delay` 或 `set_multicycle_path` 设置的时序例外：`reset_path`。

用 `-from`、`-to`、`-through`、`-rise_from`、`-fall_to` 等控制移除范围。路径指定与对象（pin、port 或 clock）须匹配设置例外时使用的原始指定，否则 `reset_path` 无效。

在应用新例外前重置路径上所有例外，在例外设置命令中使用 `-reset_path`：

```
pt_shell> set_false_path -through [get_pins d/Z] -reset_path
```

移除设计中所有例外：`reset_design`（移除所有用户指定时钟、path group、例外与属性，`set_user_attribute` 定义的除外）。

8.6 保存与恢复时序路径集合

Saving and Restoring Timing Path Collections

可将时序路径集合保存到磁盘目录并在新工具会话中恢复。保存与恢复路径集合比用新 `get_timing_paths` 重新生成快得多。

```
pt_shell> set my_paths [get_timing_paths -nworst 20 -max_paths 500]
pt_shell> save_timing_paths $my_paths -output paths1
pt_shell> set my_paths [restore_timing_paths paths1]
pt_shell> report_timing $my_paths
```

`save_timing_paths` 须指定时序路径集合及存储目录名。`restore_timing_paths` 须指定存储目录名。设计、约束与 PrimeTime 工具版本须与存储时相同。

参见

有关在 GUI 中分析路径集合中的路径，见「分析时序路径集合」。

Chapter 9

工作条件

Operating Conditions

半导体器件参数随工艺、电压与温度（PVT）条件变化。ASIC 厂商通常在多种条件下表征器件参数，并在逻辑库中以不同工作条件集给出参数值。时序分析所用工作条件会影响分析结果。关于为时序分析指定工作条件，见：

- 工作条件（Operating Conditions）
- 工作条件分析模式（Operating Condition Analysis Modes）
- 最小与最大延时计算（Minimum and Maximum Delay Calculations）
- 指定分析模式（Specifying the Analysis Mode）
- 用两个库进行分析（Using Two Libraries for Analysis）
- 延时降额（Derating Timing Delays）
- 时钟再汇聚悲观移除（Clock Reconvergence Pessimism Removal）
- 时钟片上变异悲观缩减（Clock On-Chip Variation Pessimism Reduction）

9.1 工作条件

Operating Conditions

集成电路在不同工作条件下呈现不同性能特征：制造工艺变化、电源电压与温度。逻辑库定义这些参数的标称值，并在该条件下给出延时信息。

一组工作条件包含下列值：

每条时序弧的延时信息在标称工艺、温度与电压下指定。若实际工作条件不同，PrimeTime 应用缩放因子以反映这些变化。许多库对工艺、温度与电压使用线性缩放。

若逻辑库包含缩放单元信息，则可为特定工作条件包含精确的延时表或系数。对于延时不随工作条件线性缩放的库单元，此方法可获得很高的精度。更多信息见 Library Compiler 与 Design Compiler 文档。

表 9.1: 工作条件参数

工作条件	说明
工艺降额因子 (Process derating factor)	该值与制造工艺变化所引起的器件参数缩放有关。工艺数值小于标称值通常会产生较小的延时。
环境温度 (Ambient temperature)	芯片温度影响器件延时，取决于环境温度、功耗、封装类型与冷却方式等。
电源电压 (Supply voltage)	较高电源电压通常导致更小延时。
互连模型类型 (Interconnect model type)	定义 PrimeTime 在布局前分析中估计 net 电容与电阻所用的 RC 树拓扑。

可用单组工作条件进行分析 (setup 与 hold)，或指定最小与最大条件。若未在设计上设置工作条件，PrimeTime 使用主库中的默认工作条件集 (若存在)，或主库的标称值。

9.1.1 互连模型类型

Interconnect Model Types

PrimeTime 在计算布局前设计的 net 延时时使用互连模型信息 (当无反标 net 延时与寄生信息时)。总电阻与电容相同但 RC 树拓扑不同的两条 net，pin 到 pin 延时可能不同。

本主题提供互连模型类型的背景信息。无法在 PrimeTime 中修改这些类型。更多信息见 Library Compiler 文档。

互连模型由每个逻辑库工作条件集中的 **tree_type** 规范定义。**tree_type** 指示连线电阻与电容拓扑类型：**best_case_tree**、**worst_case_tree** 或 **balanced_tree**。例如：

```
operating_conditions(BEST) {
    process      : 1.1;
    temperature  : 11.0;
    voltage      : 4.6;
    tree_type    : "best_case_tree";
}
operating_conditions(TYPICAL) {
    process      : 1.3;
    temperature  : 31.0;
    voltage      : 4.6;
    tree_type    : "balanced_tree";
}
operating_conditions(WORST) {
    process      : 1.7;
    temperature  : 55.0;
    voltage      : 4.2;
    tree_type    : "worst_case_tree";
}
```

若逻辑库未定义树类型，PrimeTime 使用 `balanced_tree` 模型。下图示出树类型模型网络。

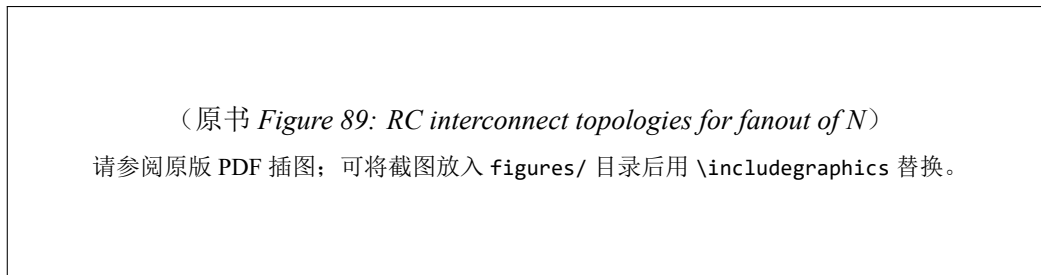


图 9.1: 扇出为 N 的 RC 互连拓扑

9.1.2 设置工作条件

Setting Operating Conditions

要为时序分析指定工艺、温度与电压条件，使用 `set_operating_conditions` 命令。

所指定工作条件必须在指定库或 `link path` 中的库内定义。要为库创建自定义工作条件，使用 `create_operating_conditions`。使用 `report_lib` 获取逻辑库中可用工作条件列表后再使用 `set_operating_conditions`。

将 `tech_lib` 库中的 `WCCOM` 设为单一工作条件：

```
pt_shell> set_operating_conditions WCCOM -library tech_lib
```

将 `WCCOM` 设为最大条件、`BCCOM` 设为最小条件以进行片上变异分析：

```
pt_shell> set_operating_conditions -analysis_type on_chip_variation \
    -min BCCOM -max WCCOM
```

未指定库时，PrimeTime 搜索 `link path` 中所有库。设置工作条件后，可报告或移除工作条件。

9.1.3 创建工作条件

Creating Operating Conditions

逻辑库包含固定的工作条件集。要在库中创建新工作条件，使用 `create_operating_conditions`。这些自定义工作条件可用于当前会话中的设计分析，但不能写入库 `.db` 文件。

查看库中定义的工作条件用 `report_lib`；在当前设计上设置工作条件用 `set_operating_conditions`。

在 `tech_lib` 库中创建名为 `WC_CUSTOM` 的新工作条件：

```
pt_shell> create_operating_conditions -name WC_CUSTOM \
    -library tech_lib -process 1.2 \
    -temperature 30.0 -voltage 2.8 \
    -tree_type worst_case_tree
```

9.1.4 工作条件信息命令

Operating Condition Information

下列命令用于报告、移除或重置工作条件信息：

表 9.2: 工作条件相关命令

命令	操作
report_design	列出设计的工作条件设置
remove_operating_conditions	从当前设计移除工作条件
reset_design	将工作条件重置为默认并移除所有用户指定数据（如时钟、输入/输出延时）

9.2 工作条件分析模式

Operating Condition Analysis Modes

半导体器件参数可随制造工艺、工作温度与电源电压等条件变化。PrimeTime 中 `set_operating_conditions` 指定分析工作条件，使 PrimeTime 使用逻辑库中相应的参数值集。

PrimeTime 提供下列设置时序分析工作条件的方法：

- 单一工作条件模式：基于一组工艺、温度与电压条件，对整个设计使用单组延时参数。
- 片上变异（OCV）模式：执行保守分析，允许最小与最大延时同时应用于不同路径。Setup 检查对 launch 时钟路径与数据路径用最大延时，对 capture 时钟路径用最小延时。Hold 检查相反。
- 高级片上变异（AOCV）模式：根据路径逻辑深度与物理距离等指标确定降额因子。
- 参数化片上变异（POCV）模式：将延时、要求时间与 slack 计算为统计分布。

下表示出各工作条件分析模式下 setup 与 hold 检查所用的时钟到达时间、延时、工作条件与延时降额。

表 9.3: 表 16: Setup 与 Hold 检查所用的时序参数（续表见下文）

分析模式	时序检查	Launch 时钟路径	数据路径	Capture 时钟路径
单一工作条件	Setup	晚时钟，时钟路径采用最大延时，单一工作条件（无降额）	采用最大延时，单一工作条件（无降额）	早时钟，时钟路径采用最小延时，单一工作条件（无降额）
单一工作条件	Hold	早时钟，时钟路径采用最小延时，单一工作条件（无降额）	采用最小延时，单一工作条件（无降额）	晚时钟，时钟路径采用最大延时，单一工作条件（无降额）

表 9.4: 表 16（续）：OCV 模式

分析模式	时序检查	Launch 时钟路径	数据路径	Capture 时钟路径
OCV	Setup	晚时钟，时钟路径最大延时，晚降额，最坏工作条件	最大延时，晚降额，最坏工作条件	早时钟，时钟路径最小延时，早降额，最好工作条件
OCV	Hold	早时钟，时钟路径最小延时，早降额，最好工作条件	最小延时，早降额，最好工作条件	晚时钟，时钟路径最大延时，晚降额，最坏工作条件

9.3 最小与最大延时计算

Minimum and Maximum Delay Calculations

`set_operating_conditions` 定义时序分析的工作条件并指定分析类型（单一或片上变异）。工作条件必须在指定库或库对中定义。

默认情况下，PrimeTime 在单一工作条件集下分析（单一工作条件模式）。此模式下需多次分析运行以处理多种工作条件。通常至少需分析两种工作条件以确保无时序违例：最好情况（最小路径报告）用于 hold 检查，最坏情况（最大路径报告）用于 setup 检查。

在片上变异（OCV）模式下，PrimeTime 执行保守分析，允许最小与最大延时同时应用于不同路径。Setup 检查对 launch 时钟路径与数据路径用最大延时，对 capture 时钟路径用最小延时；Hold 检查相反。

OCV 模式下，当路径段被 launch 与 capture 时钟路径共享时，该段可能同时被视为具有两种不同延时。未考虑共享路径段会导致悲观分析。可通过修正获得更准确分析，见「时钟再汇聚悲观移除」。

最小-最大分析考虑下列设计参数的最小与最大值：

- 输入与输出外部延时
- 自 SDF 反标的延时
- 端口连线负载模型
- 端口扇出数
- Net 电容与电阻
- Net 连线负载模型
- 时钟 latency 与转换时间
- 输入端口驱动单元

例如计算最大延时，PrimeTime 使用最长路径、最坏工作条件、最晚到达时钟边沿、最大单元延时、最长转换时间等。

可用单库（指定最小与最大工作条件）或双库（一库最好、一库最坏）进行最小-最大分析，见「用两个库进行分析」。

启用最小-最大分析的方法：

- 用 `set_operating_conditions` 设置最小与最大工作条件：

```
pt_shell> set_operating_conditions -analysis_type on_chip_variation \
      -min BCCOM -max WCCOM
```

- 读入 SDF 并读取最小与最大延时值：

```
pt_shell> read_sdf -analysis_type on_chip_variation my_design.sdf
```

9.3.1 最小-最大单元与 Net 延时值

Minimum-Maximum Cell and Net Delay Values

每条时序弧可有最小与最大延时以反映工作条件变化。指定方式：

- 从一个或两个 SDF 文件反标延时
- 由 PrimeTime 计算延时

从 SDF 反标：SDF 三元组的最小/最大、两个 SDF 文件、或上述方式加最大/最小乘数。

由 PrimeTime 计算：单一工作条件加时序降额、两种工作条件（最好/最坏）建模 OCV、上述加最小/最大降额、单元与 net 分别降额、不同时序检查分别降额。

表 9.5: 表 17: 来自 SDF 三元组数据的最小-最大延时

分析模式	基于工作条件的延时	一个 SDF	两个 SDF
单一工作条件 Setup	工作条件下最大数据；工作条件下最小 capture 时钟	a:b:c	—
单一工作条件 Hold	工作条件下最小数据；工作条件下最大 capture 时钟	a:b:c	—
OCV Setup	最坏最大数据；最好最小 capture 时钟	a:b:c	SDF1/SDF2
OCV Hold	最好最小数据；最坏最大 capture 时钟	a:b:c	SDF1/SDF2

PrimeTime 使用表中以粗斜体显示的三元组分量。

Setup 与 Hold 检查

Setup and Hold Checks 单一工作条件与 OCV 下 setup/hold 检查示例见：

- 路径延时追踪
- 最坏条件下 Setup 时序检查
- 最好条件下 Hold 时序检查

路径延时追踪

Path Delay Tracing for Setup and Hold Checks 下图示出 PrimeTime 如何执行 setup 与 hold 检查。

- Setup 检查 (DL2/ck 到 DL2/d)：clock_path1 最大延时；数据路径 (data_path_max) 最大延时；clock_path2 最小延时。
 - Hold 检查：clock_path1 最小延时；数据路径 (data_path_min) 最小延时；clock_path2 最大延时。
- data_path_min 与 data_path_max 可因组合逻辑中多条拓扑路径而不同。

最坏条件下 Setup 时序检查

(原书 *Figure 90: Design Example*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 9.2: 设计示例

(原书 *Figure 91: Setup Check Using Worst-Case Conditions*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 9.3: 用最坏条件的 Setup 检查

Setup Timing Check for Worst-Case Conditions PrimeTime 按下列方式检查 setup 违例:

$$\text{clockpath1} + \text{datapathmax} - \text{clockpath2} + \text{setup} \leq \text{clockperiod}$$

其中 $\text{clockpath1} = 0.8 + 0.6 = 1.4$, $\text{datapathmax} = 3.8$, $\text{clockpath2} = 0.8 + 0.65 = 1.45$, $\text{setup} = 0.2$ 。
时钟周期至少为 $1.4 + 3.8 - 1.45 + 0.2 = 3.95$ 。

最好条件下 **Hold** 时序检查

(原书 *Figure 92: Hold Check Using Best-Case Conditions*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 9.4: 用最好条件的 Hold 检查

Hold Timing Check for Best-Case Conditions PrimeTime 按下式检查 hold 违例:

$$\text{clockpath1} + \text{datapathmin} - \text{clockpath2} - \text{hold} \geq 0$$

其中 $\text{clockpath1} = 0.5 + 0.3 = 0.8$, $\text{datapathmin} = 1.6$, $\text{clockpath2} = 0.5 + 0.35 = 0.85$, $\text{hold} = 0.1$ 。
由于 $0.8 + 1.6 - 0.85 - 0.1 = 1.45 > 0$, 因此不存在 hold 违例。

存在延时变异时的路径追踪

(原书 *Figure 93: OCV for Worst-Case Conditions*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 9.5: 最坏条件的 OCV

Path Tracing in the Presence of Delay Variation OCV 下每个单元或 net 延时存在不确定性。可指定最坏条件下 OCV 为标称延时的 80%–100%。

此模式下，给定路径的最大延时按最坏情况的 100% 计算，最小延时按最坏情况的 80% 计算。等价的 setup 约束为：

$$\text{clockpath1} + \text{datapathmax} - \text{clockpath2} + \text{setup} \leq \text{clockperiod}$$

时钟周期至少为 $1.40 + 3.80 - 1.16 + 0.2 = 4.24$ 。

注意

片上变异影响时钟 latency；因此仅在使用传播时钟 latency 时需要。若指定理想时钟 latency，可通过 `set_clock_uncertainty` 增大时钟不确定性以考虑 OCV。

9.4 指定分析模式

Specifying the Analysis Mode

不同工作条件下时序分析示例见：

- 单一工作条件分析
- 片上变异分析

9.4.1 单一工作条件分析

Single Operating Condition Analysis

最好条件分析：

```
pt_shell> set_operating_conditions BEST
pt_shell> report_timing -delay_type min
```

最坏条件分析：

```
pt_shell> set_operating_conditions WORST
pt_shell> report_timing -delay_type max
```


(原书 *Figure 94: Timing path for one operating condition – best case*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 9.6: 单一工作条件时序路径 – 最好情况

(原书 *Figure 95: Timing path for one operating condition – worst case*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 9.7: 单一工作条件时序路径 – 最坏情况

9.4.2 片上变异分析

On-Chip Variation Analysis

执行 OCV 分析：

1. 指定工作条件：

```
pt_shell> set_operating_conditions \
          -analysis_type on_chip_variation \
          -min MIN -max MAX
```

2. 报告 setup 与 hold：

```
pt_shell> report_timing -delay_type min
pt_shell> report_timing -delay_type max
```

(原书 *Figure 96: Timing path reported during OCV analysis*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 9.8: OCV 分析报告的时序路径

示例 1：在最坏商业工作条件 WCCOM 的基础上，执行下偏 20% 的 OCV 时序分析：

```
pt_shell> set_operating_conditions -analysis_type \
          on_chip_variation WCCOM
```

(原书 *Figure 97: Early and late arrival time*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 9.9: 早到与晚到时间

```
pt_shell> set_timing_derate -early 0.8
pt_shell> report_timing
```

示例 2: 在预定义工作条件 WCCOM_scaled 与 WCCOM 之间执行 OCV 时序分析:

```
pt_shell> set_operating_conditions -analysis_type \
        on_chip_variation -min WCCOM_scaled -max WCCOM
pt_shell> report_timing
```

示例 3: 单元延时 OCV 在 SDF 值上下 5%/10%; net 延时 2%/4%; 单元时序检查 setup 上 10%、hold 下 20%:

```
pt_shell> set_timing_derate -cell_delay -early 0.90
pt_shell> set_timing_derate -cell_delay -late 1.05
pt_shell> set_timing_derate -net_delay -early 0.96
pt_shell> set_timing_derate -net_delay -late 1.02
pt_shell> set_timing_derate -cell_check -early 0.80
pt_shell> set_timing_derate -cell_check -late 1.10
```

9.5 用两个库进行分析

Using Two Libraries for Analysis

set_min_library 指示 PrimeTime 同时使用两个逻辑库进行最小与最大延时分析, 例如:

- 最好与最坏工作条件
- 乐观与悲观连线负载模型
- 最小与最大时序延时

用 set_min_library 在两库间建立最小/最大关联: 一库用于最大延时分析, 另一用于最小延时分析。仅最大库应在 link path 中。

使用 set_min_library 时, PrimeTime 先查最大库中的库单元, 再在最小库中查找匹配 (同名、同 pin、同时序弧)。若存在则用于最小分析, 否则使用最大库信息。

9.6 延时降额

Derating Timing Delays

可对计算延时降额以建模片上变异效应。降额将计算延时乘以指定因子，改变 `report_timing` 等报告的延时与 `slack`。

`set_timing_derate` 指定早/晚延时调整因子及可选作用范围：

```
pt_shell> set_timing_derate -early 0.9
pt_shell> set_timing_derate -late 1.2
```

第一条将所有早（最短路径）单元与 `net` 延时减少 10%（如 `hold` 数据路径）；第二条将晚延时增加 20%（如 `setup` 数据路径），使分析更保守。

`set_timing_derate` 会隐式切换到 OCV 模式（若尚未处于该模式），等效于：

```
pt_shell> set_operating_conditions -analysis_type on_chip_variation
```

OCV 模式下同时对早、晚应用最坏情况调整。`Setup` 检查对 `launch` 时钟与数据路径用晚降额，对 `capture` 时钟路径用早降额；`Hold` 检查相反。

必须指定 `-early` 或 `-late`，以选择对最短路径延时或最长路径延时应用降额。早、晚降额必须分别用两条命令设置。降额因子为浮点数；要得到考虑 OCV 效应的更保守分析，应使用小于 1.0 的早降额因子或大于 1.0 的晚降额因子。

报告已设降额用 `report_timing_derate`。在 `report_timing` 中用 `-derate` 报告每级增量延时的降额因子。

取消所有降额用 `reset_timing_derate`。

9.6.1 降额选项

Derating Options

默认 `set_timing_derate` 将指定早/晚降额因子应用于全设计所有早/晚时序路径。可选缩小范围：

- `-clock/-data`：仅时钟/数据路径
- `-net_delay/-cell_delay`：仅 `net`/单元延时
- `-rise/-fall`：仅上升/下降边延时
- 对象列表：特定 `net`、单元实例或库单元
- `-net_delay` 与 `-static/-dynamic`：仅静态或动态 `net` 延时分量
- `-cell_check`：单元时序检查约束（`hold/removal` 用早降额；`setup/recovery` 用晚降额）
- `-aocvm_guardband`、`-pocvm_guardband`、`-pocvm_coefficient_scale_factor`：影响 AOCV/POCV 分析

在 `net` 或叶级单元上设置降额时，命令隐式匹配对象类型。层次单元上必须显式指定 `-net_delay`、`-cell_delay` 或 `-cell_check`。

将 `timing_use_constraint_derates_for_pulse_checks` 设为 `true` 可使最小脉宽与最小周期约束参与降额。也可用 `set_timing_derate` 的 `-min_pulse_width`、`-min_period` 选项（此时上述变量须为 `false`，默认）。

```
set_timing_derate -min_pulse_width rise 1.2
set_timing_derate -min_period fall 1.1
report_timing_derate -min_pulse_width
report_timing_derate -min_period
report_min_pulse_width ...
report_min_period ...
reset_timing_derate -min_pulse_width
reset_timing_derate -min_period
```

此功能与 POCV 分析配合;在 `set_timing_derate` 中与 `-pocvm_guardband`、`-pocvm_coefficient_scale_factor` 联用。

9.6.2 冲突的降额设置

Conflicting Derating Settings

新 `set_timing_derate` 覆盖同范围先前设置的降额值。范围重叠时，范围更窄的命令优先：

```
set_timing_derate -late -cell_delay 1.4 [get_cells U2*]
set_timing_derate -late -cell_delay 1.3 [get_lib_cells libAZ/ND*]
set_timing_derate -late -cell_delay 1.2
set_timing_derate -late -1.1
```

9.6.3 负延时降额

Derating Negative Delays

某些情况下延时可为负（如输入转换慢、输出转换快且延时极短时，输出可在输入到达 50% 触发电平前到达）。串扰导致 `net` 延时变化也可能类似。

一般调整公式：

$$\text{delay_new} = \text{old_delay} + ((\text{derating_factor} - 1.0) \times |\text{old_delay}|)$$

正延时简化为 $\text{delay_new} = \text{old_delay} \times \text{derating_factor}$ ；负延时时为 $\text{delay_new} = \text{old_delay} \times (2.0 - \text{derating_factor})$ 。

静态与动态 `net` 延时分量分别降额后再合并，使负 `delta` 延时在合并到正静态延时前正确降额。

9.7 时钟再汇聚悲观移除

Clock Reconvergence Pessimism Removal

时钟再汇聚悲观是指：两条不同时钟路径共享一部分物理路径，而分析却假定该共享段在一条路径上取最小延时、在另一条路径上取最大延时。当 `launch` 与 `capture` 时钟路径采用不同延时值时会出现这种情况，在 OCV 分析中尤为常见。对此悲观量进行自动修正称为时钟再汇聚悲观移除（CRPR）。

CRPR 默认使能。可禁用以节省运行时间与内存（牺牲精度、增加悲观）。CRPR 仅减少悲观，故仅能增加报告 slack。若禁用 CRPR 时设计无违例，使能 CRPR 后仍无违例。

禁用 CRPR：在时序分析前将 `timing_remove_clock_reconvergence_pessimism` 设为 `false`。更改此变量会导致完整时序更新。

9.7.1 片上变异示例

On-Chip Variation Example

```
pt_shell> set_operating_conditions -analysis_type \
    on_chip_variation -min MIN -max MAX
pt_shell> set_timing_derate -net_delay -early 0.80
pt_shell> report_timing -delay_type min
pt_shell> report_timing -delay_type max
```

（原书 *Figure 98: Clock reconvergence pessimism example*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 9.10: 时钟再汇聚悲观示例

各延时有最小与最大值（最好/最坏工作条件）。LD2/CP 的 `setup` 检查将源锁存器时钟路径（CLK 到 LD1/CP）按 100% 最坏，目标锁存器路径（CLK 到 LD2/CP）按 80% 最坏。

虽为有效方法，但因 `clock path1` 与 `path2` 共享至 U1 输出的时钟树，分析悲观：共享段称为公共部分，最后单元输出为公共点（`common point`）。`Setup` 检查假定 U1 同时有 0.64 与 0.80 两种延时，悲观量为 0.16（公共点最晚与最早到达时间之差），即时钟再汇聚悲观。`Hold` 检查类似。

9.7.2 再汇聚逻辑示例

Reconvergent Logic Example

（原书 *Figure 99: Reconvergent logic in a clock network*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 9.11: 时钟网络中的再汇聚逻辑

即使无 OCV，时钟网络中的再汇聚逻辑也可能产生时钟再汇聚：馈入多路选择器的两路时钟路径不能同时有效，但分析可能对同一次 `setup/hold` 同时考虑较短与较长路径。

9.7.3 最小脉宽检查示例

Minimum Pulse Width Checking Example

`report_constraint` 检查时钟网络 (`set_min_pulse_width`) 与单元输入 (逻辑库) 的最小脉宽违例。CRPR 提高此检查精度。

(原书 *Figure 100: Minimum pulse width analysis*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 9.12: 最小脉宽分析

无 CRPR 时最坏脉宽可能极小并违例，因假设上升与下降边同时最坏延时。实际电路中两边延时至少部分相关。使能 CRPR 时，工具将一定 slack 加回最小脉宽计算：

$$crp = \min[(M_r - m_r), (M_f - m_f)]$$

M_r/m_r 为累积最大/最小上升延时， M_f/m_f 为下降。

9.7.4 CRPR 报告

CRPR Reporting

`report_timing`、`report_constraint`、`report_analysis_coverage`、`report_bottleneck` 详细报告会报告 CRPR 调整。示例 `report_timing -delay_type max` 输出中含 `clock reconvergence pessimism` 行。

9.7.5 公共点不同跳变的 CRPR 降额

Derating CRPR for Different Transitions at the Common Point

`timing_clock_reconvergence_pessimism` 指定 `launch` 与 `capture` 时钟路径跳变不匹配时如何找最晚公共点。`normal` (默认) 找最晚拓扑公共点 (跳变可不同)；`same_transition` 还要求跳变类型相同。

(原书 *Figure 101: Common Point for Different timing_clock_reconvergence_pessimism Settings*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 9.13: 不同 CRPR 变量设置下的公共点

normal 适用于上升/下降到达时间高度相关；不高度相关时 **same_transition** 保证保守（可能更悲观）分析。

可用 **normal** 公共点搜索并对公共点跳变不同时的 CRP 降额：

```
set_app_var timing_crpr_different_transition_derate 0.90
```

跳变不同时 CRP 减至原值的 90% 再加回 slack，比默认 100% 更保守。

POCV 分析中可同时降额 CRP 变化：

```
set_app_var timing_crpr_different_transition_derate 0.90
set_app_var timing_crpr_different_transition_variation_derate 0.90
```

公共点跳变相同时无效果；**timing_clock_reconvergence_pessimism** 为 **same_transition** 时也无效。第一变量默认 1.0（标称不降额），第二默认 0.0（完全消除 CRP 变化）。

最小脉宽示例脚本：

```
set_operating_conditions -analysis_type on_chip_variation
create_clock -period 8 CLK
set_propagated_clock [all_clocks]
set_clock_latency -source -early -1.3 CLK
set_clock_latency -source -late 1.4 CLK
set_annotated_delay -cell -from CT1/A -to CT1/Z 1
set_annotated_delay -cell -from CT2/A -to CT2/Z -min -rise 0.8
set_annotated_delay -cell -from CT2/A -to CT2/Z -max -rise 1.8
set_annotated_delay -cell -from CT2/A -to CT2/Z -min -fall 0.85
set_annotated_delay -cell -from CT2/A -to CT2/Z -max -fall 2.03
```

使能 CRPR 时 **report_constraint** 路径报告含 clock reconvergence pessimism 3.70 等。

9.7.6 CRPR 合并阈值

CRPR Merging Threshold

为计算效率，相邻点 CRP 差小于阈值时合并路径点。**timing_crpr_threshold_ps** 以皮秒指定阈值，默认 5 ps。建议在性能与精度间取时钟网络典型门级延时（门延时 + net 延时）的一半；设计阶段可用较大值，签核用较小值。

9.7.7 CRPR 与串扰分析

CRPR and Crosstalk Analysis

用 PrimeTime SI 做串扰分析时，时钟路径公共段上因串扰的延时变化对零周期检查可能悲观（同一时钟边沿驱动 launch 与 capture）。其他路径类型不悲观，因不能假定 launch 与 capture 边沿的串扰变化相同。

因此 CRPR 仅在零周期检查时移除公共段上的串扰诱导延时。适用情形包括：标准 hold 检查、Q-bar 接 D 的寄存器、串扰反馈 hold、多周期路径为零的 hold、涉及透明锁存器的某些 setup 检查等。

9.7.8 CRPR 与动态时钟到达

CRPR With Dynamic Clock Arrivals

动态反标（`set_clock_latency`、`set_voltage` 设置的动态时钟 latency 与 rail 电压）可导致动态时钟到达。CRPR 与处理串扰延时相同：仅对零周期路径用动态时钟到达计算 CRP；其他路径仅用静态分量。

示例：早源 latency 2.5（静态 3.0 + 动态 -0.5）；晚源 latency 5.5（静态 5.0 + 动态 0.5）。静态 CRP = 2（5 减 3）；动态 CRP = 3（5.5 减 2.5）。

9.7.9 透明锁存器边沿考虑

Transparent Latch Edge Considerations

路径终点为透明锁存器时，PrimeTime 计算两个 CRP 值：公共节点上升与下降各一。开/关时钟边沿各平移对应悲观量。

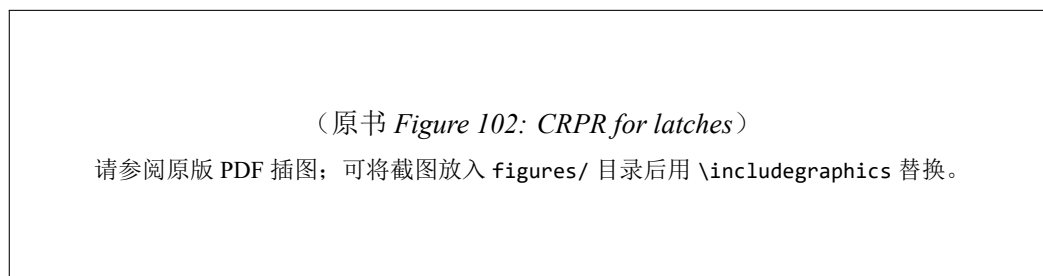


图 9.14: 锁存器的 CRPR

开边沿悲观影响非借用路径 slack 并减少借用路径借用时间；关边沿悲观增加最大借用时间并减少 setup 违例量。用 `report_crpr` 报告锁存器 CRP 计算。

9.7.10 报告 CRPR 计算

Reporting CRPR Calculations

`report_crpr` 报告两个寄存器时钟 pin 或端口之间的 CRP 计算，包括静态和动态条件下计算的时间值，以及其中实际用于悲观移除的值。需指定 launch/capture pin、时钟与检查类型（setup/hold）：

```
pt_shell> report_crpr -from [get_pins ffa/CP] \
    -to [get_pins ffd/CP] \
    -from_clock CLK -setup
```

`report_crpr` 还会报告公共节点位置、launch 与 capture 边沿类型（上升或下降）、公共点处四种到达时间（早/晚、上升/下降）、计算得到的上升与下降 CRP，以及实际用于开边沿和关边沿悲观移除的值。

`report_crpr` 与 `report_timing` 报告的 CRP 可能略有不同，因 `report_timing` 为效率合并相邻点（`timing_crpr_threshold_ps`，默认 5 ps）。不一致时 `report_crpr` 更准确。

9.8 时钟片上变异悲观缩减

Clock On-Chip Variation Pessimism Reduction

`set_timing_derate` 可建模 OCV。launch 与 capture 时钟路径有公共段时，若使能 CRPR，会移除公共段延时降额导致的悲观，但默认仅在路径最终 slack 计算后移除，不在计算串扰到达窗口时移除。若通向 aggressor/victim 的时钟路径共享公共段，到达窗口可能过宽，导致边际重叠的串扰情形在准确 CRPR 下本不存在。

(原书 *Figure 103: Clock reconvergence pessimism in crosstalk arrival windows*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 `\includegraphics` 替换。

图 9.15: 串扰到达窗口中的时钟再汇聚悲观

无降额时到达窗口宽 1.0，不重叠；降额后公共点前长时钟路径早到 6.0、晚到 9.0，窗口扩至 3.0 并重叠，产生实际电路中不存在的串扰。

为在基于路径分析中获得最佳精度，可选在到达窗口重叠分析中应用 CRPR，移除公共点早/晚到达差异导致的悲观。设置：

```
set_app_var pba_enable_xtalk_delay_ocv_pessimism_reduction true
```

默认 `false`。为 `true` 且 CRPR 使能时，PrimeTime SI 在计算 aggressor/victim 到达窗口时应用 CRPR，提高精度，代价是额外运行时间。基于路径分析在使用 `get_timing_paths` 或 `report_timing` 的 `-pba-mode` 时发生。CRPR 在 `timing_remove_clock_reconvergence_pessimism` 为 `true` 时使能。

Chapter 10

延时计算

Delay Calculation

为准确高效地进行延时计算，PrimeTime 可用模型表示 net 上的驱动器、RC 网络与电容负载。理想模型在驱动器输出与各接收器输入产生与 SPICE 仿真相同的延时与 slew。关于延时计算的不同模型与分析模式，见：

- 延时计算概述
- 非线性延时模型
- 复合电流源时序模型
- 跨库电压与温度缩放
- 波形传播
- 表征触发电平
- 快速多驱动延时分析
- 并行驱动器缩减
- 多输入开关分析
- 单位延时分析

10.1 延时计算概述

Overview of Delay Calculation

静态时序分析要求 PrimeTime 准确计算每条时序路径各级的延时与 slew（转换时间）。一级包含驱动单元、单元输出处反标的 RC 网络及网络负载 pin 的电容负载。目标是在给定驱动器输入 slew 或波形下，以最少运行时间计算驱动器输出与网络负载 pin 的响应并得到准确结果。

为准确高效地进行级延时计算，PrimeTime 用模型表示驱动器、RC 网络与电容负载。理想模型在驱动器输出与各接收器输入产生与 SPICE 完全相同的延时与 slew。

(原书 *Figure 104: Models used to calculate stage delays and slews*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.1: 用于计算级延时与 slew 的模型

驱动器模型旨在复现驱动单元底层晶体管电路在任意 RC 网络、给定输入 slew 下的响应。降阶网络模型是完整反标网络的简化表示，响应特性接近原网络；PrimeTime 用 Arnoldi 降阶法创建。接收器模型表示单元输入 pin 的复杂输入电容特性，包括上升/下降跳变、slew、接收器输出负载、单元状态及电压温度条件。

指定 RC 延时计算的驱动器与接收器模型类型，将 `rc_driver_model_mode` 与 `rc_receiver_model_mode` 设为：

- **basic**：使用库中基本非线性延时模型（NLDM）
- **advanced**（默认）：使用更先进的复合电流源（CCS）时序模型（若库中有 CCS 数据）；否则用 NLDM

高级 CCS 时序模型有多项优势，包括解决 RC-009 警告（驱动电阻远小于网络对地阻抗）。CCS 也更善于处理 Miller 效应、动态 IR 压降与多电压分析。

10.2 非线性延时模型

Nonlinear Delay Models

非线性延时模型（NLDM）是表示路径级驱动器与接收器的较早、成熟方法。驱动模型使用串联电阻的线性电压斜坡（Thevenin 模型），电阻平滑斜坡使驱动波形接近实际驱动 RC 网络的曲率。

(原书 *Figure 105: NLDM driver and receiver models*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.2: NLDM 驱动器与接收器模型

驱动模型有三个参数：驱动电阻 R_d 、斜坡起始时间 t_z 、斜坡持续时间 Δt 。PrimeTime 选择参数值，使输出波形尽可能匹配实际波形；并为每条门级时序弧（如 U1/A 到 U1/Z）及每种跳变极性（如上升沿）分别构建简化驱动模型。

当驱动电阻远小于网络对地阻抗时，平滑效果减弱，可能降低 RC 延时计算精度；此时 PrimeTime 调整驱动电阻并发出 RC-009 警告。

NLDM 接收器模型为表示接收器输入负载电容的电容。不同条件（上升/下降、最小/最大分析）可有不同电容值，但单次时序检查仅用一个电容值，不能准确建模 Miller 效应。

注意

Miller 效应是晶体管端子间电压变化导致的有效电容变化。

10.3 复合电流源时序模型

Composite Current Source Timing Models

随着更小纳米工艺，发展了复合电流源（CCS）时序建模以应对深亚微米效应。驱动模型使用时变电电流源，能准确处理高阻抗 net 及其他非单调行为。

（原书 *Figure 106: CCS timing driver and receiver models*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.3: CCS 时序驱动器与接收器模型

CCS 接收器模型用两个电容（C1、C2）而非单一集总电容。第一个电容用作达到输入延时阈值前的负载；波形达到阈值后动态切换到第二个电容。C1C2 模型在 Miller 效应存在时更好地近似负载效应。

不同输入信号可影响接收器输入电容。库中用条件 pin 模型描述。若库中有条件 pin 接收器模型，PrimeTime 考虑所有接收器模型并为分析选择已使能 pin 与弧模型中最坏者。

PrimeTime 中 CCS 时序分析需要详细寄生。使用详细寄生进行延时计算时，库信息优先级（高到低）：

1. CCS 驱动器与接收器模型（若均可用）
2. CCS 驱动器模型 + 接收单元集总 pin 电容
3. NLDM 延时/转换表 + 接收单元 pin 电容（CCS 接收器模型仅与 CCS 驱动器模型联用）

时序更新后，用 `report_delay_calculation` 确认是否使用 CCS（`-from/-to` 或 `-of_objects`）。示例输出显示 Advanced driver-modeling / Advanced receiver-modeling。

10.3.1 Pin 电容报告

Pin Capacitance Reporting

CCS 时序建模含驱动器与接收器模型。输入电容随输入信号跳变变化，CCS 接收器用 C1、C2 建模跳变前后输入电容以保证精度。C1、C2 通常为输入 slew 与输出负载的函数，可能大于由 CCS 接收器模型导出的库 pin 电容。

默认使用 CCS 接收器模型信息计算并报告负载 pin 的电容行为。CCS 接收器模型也用于最大电容违例检查，从而确保 RC-011 报告的所有可能负载外推均能由 report_constraint 报告。实际总电容定义为连线电容与每个 pin 的 $\max(C1, C2)$ 之和。

反映此行为的命令: report_attribute、report_constraint -min_capacitance/-max_capacitance、report_delay_calculation、report_net、report_timing -capacitance。属性: total_ccs_capacitance_max_fall/rise、total_ccs_capacitance_min_fall/rise。

由 report_capacitance_use_ccs_receiver_model 控制（默认 true）。设为 false 则用先前库导出的集总 pin 电容报告方法。

注意

PrimeTime、IC Compiler、Design Compiler 的电容报告可能不一致。PrimePower 中 report_delay_calculation 与 report_power_calculation 显示的电容可能不匹配。

10.3.2 表征设计规则约束的指南

Guidelines for Characterizing Design Rule Constraints

max_transition 通常在库单元输入/输出 pin 上。输入 pin 的值不应超过 NLDM 与 CCS 驱动器及 receiver_capacitance2 表中的最大 slew 索引；取 NLDM 与 CCS 表最大 slew 索引的较低值，参考相关输入 pin 的上升/下降弧（弧级与 pin 级表均考虑）。

max_capacitance 通常在输出 pin 上。值不应超过 NLDM/CCS 驱动器及 receiver_capacitance1/2 表的最大负载索引；取较低值，参考到相关输出 pin 的上升/下降弧。

10.3.3 解决 CCS 外推警告（RC-011）

Resolving the CCS Extrapolation Warning Message (RC-011)

驱动器或接收器负载外推过大可能导致不准确结果，工具在 slew/负载小于库最小索引或大于最大索引时发出 RC-011。

- 小于最小索引：在库中增加小的首个 slew/负载索引。
- 大于最大索引：若库遵循上述指南，通过 report_constraint 修复 max_transition/max_capacitance 违例；将 report_capacitance_use_ccs_receiver_model 设为 true 以确保 RC-011 中所有 max_capacitance 违例均被 report_constraint 报告。

库不合规时，RC-011 很重要，需处理设计规则约束。

默认 CCS 外推在最大索引上方 10%、最小索引 80% 处裁剪。扩大外推范围：

```
pt_shell> set_app_var rc_ccs_extrapolation_range_compatibility false
```

则在最大索引上方 50%、最小索引下方 50% 处裁剪，减少 RC-011。

10.4 跨库电压与温度缩放

Cross-Library Voltage and Temperature Scaling

PrimeTime 通过在不同电压/温度 corner 表征的库之间插值数据进行电压与温度缩放，使分析电压/温度可与 corner 库不同。缩放数据包括 CCS 时序驱动器/接收器模型、CCS 噪声、时序与设计规则约束、PrimePower 功耗数据、NLDM 延时与 slew 等。缩放减少多电压设计所需库数量与表征工作量。

库组（library groups）指定多库间缩放关系：

```
pt_shell> define_scaling_lib_group \
        {lib_0.9V.db lib_1.05V.db lib_1.3V.db}
```

库组中库数量不限；可多次定义多组，但每个库最多属于一组。报告用 `report_lib_groups`；点对点延时计算（含缩放所用库）用 `report_delay_calculation`。

精确匹配流：

```
pt_shell> define_scaling_lib_group -exact_match_only \
        {lib_0.9V.db lib_1.0V.db lib_1.1V.db}
```

工作条件须精确匹配某一库，库间不缩放。

下列命令调用库缩放与精确匹配：`set_operating_conditions`、`set_voltage`、`set_temperature`、`set_rail_voltage`。

10.4.1 多 rail 电平转换器单元的缩放

Scaling for Multirail Level Shifter Cells

PrimeTime 可对多 rail 单元（如电平转换器）缩放，连接不同电源域的驱动与负载 pin。准确多 rail 缩放支持多变量（多 rail 电压与温度）依赖分析。

默认对时序、噪声与功耗分析使能多 rail 缩放；当缩放库组中有足够库支持所需 rail 数时自动应用。

设置步骤：

1. 用 `define_scaling_lib_group` 建立缩放库组（单 rail 与双 rail 电平转换器分别建组）。
2. 将需多维缩放的单元放入不同库并归入不同缩放组（双 rail 电平转换器 on-the-grid 电压与温度缩放至少需 8 个库；单 rail 单元可放一库，4 个库即可做电压与温度缩放）。
3. 用 `set_voltage`、`set_rail_voltage`、`set_operating_conditions`、`set_temperature` 设置实例工作条件。
4. `report_lib_groups -scaling -show {voltage temp process}` 报告库组。

10.4.2 电压缩放流中的时序降额缩放

Scaling Timing Derates in Voltage Scaling Flows

电压缩放流中，对缩放组内库单元应用 `set_timing_derate` 时，工具可按实例电压计算缩放降额：

```
define_scaling_lib_group {lib_1.1V.db lib_1.0V.db lib_0.9V.db}
set_timing_derate -late [get_lib_cell lib_1.1V/INV1] 1.07
set_timing_derate -late [get_lib_cell lib_1.0V/INV1] 1.10
set_timing_derate -late [get_lib_cell lib_0.9V/INV1] 1.15
```

使能电压缩放降额:

```
pt_shell> set_app_var \
    timing_enable_derate_scaling_for_library_cells_compatibility false
pt_shell> set_app_var \
    timing_enable_derate_scaling_interpolation_for_library_cells true
```

效果: 对 `set_timing_derate` 与 `read_ocvm` 表降额均缩放; 电压精确匹配某缩放库时用该库降额; 否则用邻近库完全插值降额; 多 rail 单元用邻近库最坏(非插值)降额。两功能默认均禁用。

若在一缩放库上设置库单元降额, 应在其他缩放库上对同库单元也设置, 否则某些电压条件下可能“缺失”降额, 工具按对象范围优先级列表使用下一降额设置。

10.4.3 从多 rail 缩放库组排除 Rail

Excluding Rails From Multirail Scaling Library Groups

`define_scaling_lib_group` 的 `-excluded_rail_names` 允许忽略指定库电压 rail, 在验证缩放组形成与缩放计算时忽略这些 rail, 将 N 维多 rail 缩放降为更低维。

示例: 两库无法建 2-D 组(至少需三库), 但忽略 VDD2 可建 1-D 组:

```
define_scaling_lib_group { \
    slow_VDD1_1.0_VDD2_1.5.db \
    slow_VDD1_2.0_VDD2_1.6.db \
} \
-excluded_rail_names {VDD2}
```

应慎用此选项, 因为它会禁用通常执行的一致性检查和缩放算法, 并影响组内所有单元。要仅对特定库单元排除 rail, 应使用 `set_disable_pg_pins`。

10.5 波形传播

Waveform Propagation

16 nm 及更小几何中, 长尾效应与接收器 Miller 效应等波形效应显著。忽略这些效应会在设计波形与库表征波形偏差大时导致延时计算不准。

使能高级波形传播, 在延时计算中结合 CCS 时序与 CCS 噪声模型。支持基于图与基于路径的分析; 库需含 NLDM、CCS 时序、CCS 噪声及归一化驱动波形。

使能: `delay_calc_waveform_analysis_mode` 设为 `full_design`。有必需库数据的弧均提高精度; 否则标准延时计算。

`setup/hold` 约束分析中使用波形传播: `delay_calc_waveform_analysis_constraint_arcs_compatibility` 设为 `false`。默认 `true` 时约束分析仅用 NLDM 查表。

10.6 表征触发电平

Characterization Trip Points

设计表征触发电平（波形测量阈值）影响 PrimeTime 计算的延时与转换时间。确定顺序：

1. 库 pin 上定义的阈值（覆盖同名库级阈值）
2. 无 pin 阈值但有库级阈值时用库级
3. 均无时用主库（link path 第一库）阈值
4. 均无则转换时间用 rail 电压 20%/80%，延时用 50%
5. 阈值无效（如 0%/100%）则转换时间用 5%/95%，延时用 50%

图 107、108 示 slew 与延时触发电平。默认 slew 为 rail 电压 20% 到 80%（或反向）的时间；单元延时为输入到输出均达 50% 的时间差。

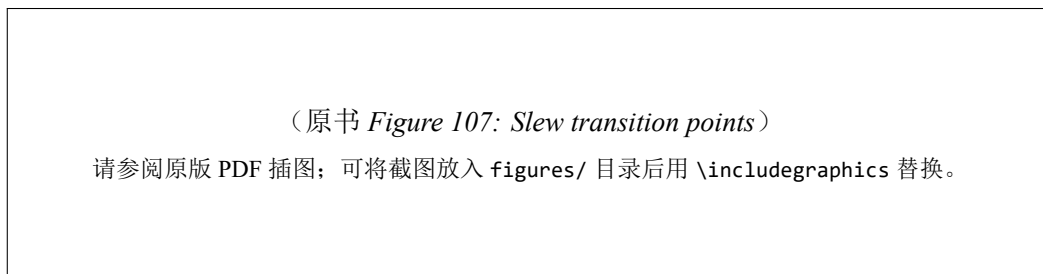


图 10.4: Slew 转换点

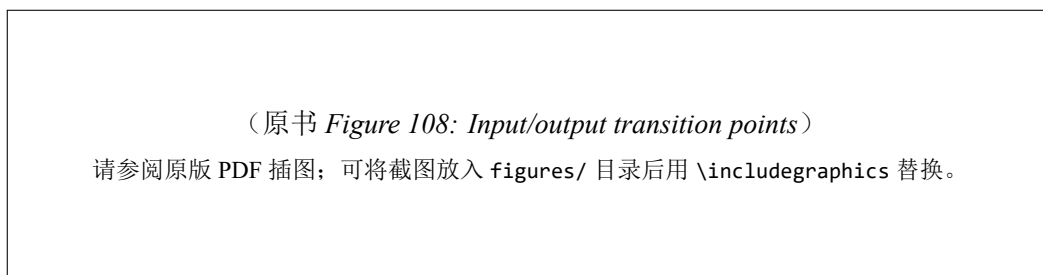


图 10.5: 输入/输出转换点

用 `report_lib` 检查库中阈值；用 `report_delay_calculation -thresholds` 检查特定弧所用阈值。

10.7 快速多驱动延时分析

Fast Multidrive Delay Analysis

大设计中反标寄生后，大规模多驱动网络的 RC 延时计算运行时间可能很长，瓶颈主要在负载 pin 延时与 slew 测量的跳变时间计算。

对同质驱动器（输入 skew、slew、工作条件相似）的大规模多驱动网络，快速多驱动计算模式显著缩短运行时间：将所有驱动器节点用电阻网络短接，在第一个驱动器节点用单一驱动器模型做

所有波形计算，驱动强度放大至等效于全部原驱动器，再将单驱动器结果复制到所有驱动器（延时、slew、驱动器模型参数、有效电容）。

不启用快速多驱动分析的最大驱动器数为 9; 10 个及以上并行驱动器触发该模式，并产生 RC-010 警告（报告驱动器数、负载数、输入 slew/skew 分布、驱动器库弧与工作条件匹配情况）。

精度在驱动器库弧相同、工作条件/输入 slew/skew 相同、网络阻抗相同时最佳（常见于 mesh 网络）。输入 skew 差异应相对网络延时较小。更高精度可用外部工具计算并反标到网络。

另见「并行驱动器缩减」「缩减时钟 Mesh/Spine 网络的 SDF」。

10.8 并行驱动器缩减

Parallel Driver Reduction

时钟通常需驱动大量负载，故常缓冲以降低时钟 skew。大时钟网络可能大量并行驱动器，以 mesh 或 spine 模式分布信号。

若设计有 1000 个并行驱动器驱动 1000 个负载，PrimeTime 需跟踪百万条驱动器-负载时序弧，消耗大量 CPU 与内存。为使能更好性能，PrimeTime 可减少需分析的时序弧数：使能并行驱动器缩减后，在网络中选择一个驱动器仅分析经该驱动器的弧。

（原书 *Figure 109: Parallel driver reduction*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.6: 并行驱动器缩减

10.8.1 调用并行驱动器缩减

Invoking Parallel Driver Reduction

将 `timing_reduce_multi_drive_net_arcs` 设为 `true`（默认禁用）。链接设计时检查多驱动 net；若驱动器 × 负载数超过 `timing_reduce_multi_drive_net_arcs_threshold`（默认 10000），则缩减该 net 驱动器的时序弧。

缩减条件（均满足）：

- 驱动器-负载组合数 > 阈值
- 驱动单元为非时序库单元（非触发器/锁存器/层次）
- 所有驱动器为同一库单元实例
- 所有驱动器连接相同输入与输出 net

要恢复原始网络或以不同阈值缩减，须重新链接设计。

10.8.2 使用缩减后的驱动器

Working With Reduced Drivers

布局完成且有详细寄生数据时，无法将数据反标到缩减网络。要准确结果，用 SPICE 等外部仿真器获取网络详细延时，再将时钟 latency 与转换时间反标到各时序单元时钟 pin，同时仍让 PrimeTime 将缩减网络视为理想零延时。此法准确且高效（时钟网络时序只需分析一次）。

用 read_sdf 反标时，缩减弧上的反标被忽略，产生 PTE-048 消息，可用 suppress_message PTE-048 抑制。

write_sdf 不为缩减网络生成互连延时。并行驱动器缩减仅影响 PrimeTime 分析的时序弧，不影响网表，故不影响 write_changes 输出。大规模多驱动网络的 RC 延时计算见「快速多驱动延时分析」。

10.9 多输入开关分析

Multi-Input Switching Analysis

表征组合逻辑门的输入到输出延时，表征工具通常一次只考虑一个输入发生跳变。多个输入同时跳变时，延时可能不同。多输入同时翻转（MIS）分析可提高这种情况下的精度，需要 PrimeTime-ADV 许可证。

4 输入 NAND 门：单输入变低其余为高（Case A）与四输入同时变低（Case B）均可触发输出低到高；Case B 四管并联上拉，A1 到 Z 延时更短。

（原书 *Figure 110: Multi-Input Switching Example: 4-Input NAND Gate*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.7: 多输入开关示例：4 输入 NAND 门

库中仅含单输入翻转延时。同时翻转可能出现在存储器译码器等电路中；当 NAND 门位于 hold 检查路径上时，同时翻转会使实际延时更短，若仍采用单输入翻转延时，hold 检查会偏乐观，MIS 分析可计入这一效应。输出由高变低时，MIS 会使延时更长并可能影响 setup；但 setup 关键路径通常较长，单个门的延时增量往往可忽略，而 hold 关键路径通常较短。因此，MIS 仅用于最小延时（hold）检查。

10.9.1 多输入开关分析模式

Multi-Input Switching Analysis Modes

三种模式：lib_cell、lib_arc、advanced。

库单元用户定义系数（lib_cell）：用 set_multi_input_switching_coefficient 在库单元上定义延时缩放系数：

```
pt_shell> set_multi_input_switching_coefficient \
    [get_lib_cell lib1/NAND4gx] -rise -delay 0.683
```

用 HSPICE 等仿真确定各组合单元与各输出跳变的正确调整因子。

库时序弧用户定义系数 (**lib_arc**): 在单个库单元时序弧上定义系数, 可在库或脚本中提供, 支持更细粒度 (含开关 pin 组):

```
cell("AND4X1") {
  pin(Z) {
    timing() {
      related_pin : A1;
      when : "A2&!B1&!B2";
      mis_factor_rise : 0.85;
      mis_pin_set_rise : "A1 A2";
      ...
    }
  }
}
```

需要 PrimeTime-ADV-PLUS 许可证。

高级多输入开关分析 (**advanced**): 分析库数据与实际 pin slew、到达、负载、波形自动估计 MIS 效应, 无需用户系数, 但需 CCS 时序、弧级 CCS 噪声模型及全部 when 条件表征。表 18 列出支持的库单元类型 (AND/OR/NAND/NOR/AO*/AOI*/OA*/OAI* 等 2-4 输入组合)。多输出单元若逻辑功能合格且与其他输出无共享输入 pin 则合格。不合格单元可用另两种模式的用户系数。

高级模式需要 PrimeTime-ADV-PLUS 许可证。

10.9.2 定义库时序弧系数

Defining Library Timing Arc Coefficients

系数属性为 mis_factor_rise/fall、mis_pin_set_rise/fall (它们不是标准 Liberty 属性, 须在库顶层声明为 user attribute)。按需在 timing() 组中定义。当实际同时翻转的 pin 少于所定义 pin 集时, 按「到达窗口重叠检查」计算中间系数。

同一弧可有 rise/fall 不同 pin 集与系数。多 pin 集用 “+” 分隔。MIS 分析中 PrimeTime 保守假定 when 条件可表示跳变前后侧 pin 值; 属性须在加载库前用 define_user_attribute -import 声明。库中无属性时可用 Tcl 脚本用 set_user_attribute 与 get_lib_timing_arcs 设置; 复杂单元可用 get_lib_timing_arcs -filter。

10.9.3 配置多输入开关分析

Configuring Multi-Input Switching Analysis

1. si_enable_multi_input_switching_analysis true
2. si_multi_input_switching_analysis_mode {advanced lib_arc lib_cell} 指定模式及优先级
3. lib_cell 模式: set_multi_input_switching_coefficient; 可选 report_multi_input_switching_coefficient
4. lib_arc 模式: 加载库前 define_user_attribute -import 四个 MIS 属性; 或链接后 source lib_arc_coefficients.tcl
5. advanced 模式: 可选 report_multi_input_switching_lib_cells

10.9.4 多输入开关分析示例

Multi-Input Switching Analysis Example

```
set search_path {. my_lib.db}
set link_path {my_lib.db}
read_verilog design.v
link_design
read_parasitics design.spef
source design.sdc
set_app_var si_enable_multi_input_switching_analysis true
set_app_var si_multi_input_switching_analysis_mode {lib_cell}
set_multi_input_switching_coefficient \
  [get_lib_cell lib1/NAND4gx] -rise -delay 0.683
report_multi_input_switching_coefficient
update_timing -full
report_timing -delay_type min -input_pins
report_delay_calculation -min -from Inst1/A -to Inst1/Z
```

工具在 NAND4gx 各实例输出上升且多输入可同时跳变时，将库延时乘以 0.683–1.0 之间的因子，得到更短的最小输入到输出延时。

10.9.5 到达窗口重叠检查

Arrival Window Overlap Checking

默认考虑多输入的信号跳变到达窗口。考虑 A1 到 Z 时，若其他输入到达窗口与 A1 重叠，应用完整 MIS 因子；无重叠则因子 1.0；部分重叠则中间因子（反比例模型）。

（原书 *Figure 111: Arrival Windows for Multi-Input Switching Analysis*）
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 10.8: 多输入开关分析的到达窗口

可禁用到达窗口分析，在同一时钟周期内多输入可跳变时始终应用完整 MIS 因子（更悲观、更快）：

```
set_app_var si_enable_multi_input_switching_analysis true
set_app_var si_enable_multi_input_switching_timing_window_filter false
```

10.9.6 在 MIS 分析中包含或排除特定单元

Including or Excluding Specific Cells in MIS Analysis

`set_multi_input_switching_analysis` 可包含或排除特定单元/库单元: `-include_only`、`-exclude_only`、`-analysis_mode`、`-reset`。对象列表可为叶单元或库单元。

10.9.7 缩放工具计算的高级 MIS 降额

Scaling the Tool-Computed Advanced MIS Derates

`set_advanced_multi_input_switching_factor` 缩放工具计算的高级 MIS 延时系数: `-scale_factor` (0-1)、`-all`、`-inverse`、`-reset`、对象列表。默认因子与加速值相乘以放松调整; 1 保持、0 完全移除、中间值部分放松。带 `-inverse` 时因子直接乘 MIS 调整后单元延时以加强加速。

10.10 单位延时分析

Unit Delay Analysis

流程测试与约束验证时通常关注警告/错误消息而非数值时序结果。PrimeTime 提供单位延时计算模式: 每个单元延时、net 延时、pin 转换、耦合 net delta 延时与转换均计算为 1.0 库单位。

示例 `report_timing` 输出中各级 `Incr` 多为 1.0。此功能可缩短运行时间, 不影响内存占用。

使能:

```
pt_shell> set_app_var timing_enable_unit_delay true
```

若使用 CRPR、SI、AOCVM、POCV、高级锁存器分析, 应禁用:

```
pt_shell> set_app_var timing_remove_clock_reconvergence_pessimism false
pt_shell> set_app_var si_enable_analysis false
pt_shell> set_app_var timing_aocvm_enable_analysis false
pt_shell> set_app_var timing_pocvm_enable_analysis false
pt_shell> set_app_var timing_enable_through_paths false
```

Chapter 11

反标

Back-Annotation

反标（back-annotation）是从外部文件读入延时或寄生电阻电容值供时序分析的过程。通过反标，可在物理设计各阶段后在工具中准确分析电路时序。关于反标，见：

- SDF 反标
- 反标延时、时序检查与转换时间
- 写出 SDF 文件
- 设置集总寄生电阻与电容
- 详细寄生
- 读入寄生文件
- 不完整反标寄生
- 反标优先级
- 报告反标寄生
- 移除反标数据
- GPD 寄生浏览器

11.1 SDF 反标

SDF Back-Annotation

初始静态时序分析中，PrimeTime 基于 wire load 模型估算 net 延时。实际延时取决于单元与 net 的物理布局布线；floorplanner 或 router 可提供更详细准确的延时信息供 PrimeTime 分析，此过程称为延时反标，信息常以 SDF 文件提供。

读入 SDF 反标延时信息的方式：

- 从 SDF 文件读入延时与时序检查

- 不使用 SDF 格式反标延时、时序检查与转换时间

PrimeTime 支持 SDF v1.0 至 v2.1, 以及 v3.0 的一个子集。一般而言, 除以下结构外, 其余 SDF 结构均受支持: PATHPULSE、GLOBALPATHPULSE; NETDELAY、CORRELATION; PATHCONSTRAINT、SUM、DIFF、SKEWCONSTRAINT。此外还支持以下 SDF v3.0 结构: RETAIN、RECREM、REMOVAL、CONDELSE。

若无 SDF 文件, 可在分析脚本中用电容电阻寄生反标命令指定延时。

11.1.1 读入 SDF 文件

Reading SDF Files

`read_sdf` 从 SDF 1.0/1.1/2.0/2.1/3.0 文件中读入与实例相关的 pin-to-pin 单元及 net 时序信息, 并将其反标到当前设计。设计中的实例名必须与时序文件中的实例名匹配 (例如, 两者必须采用一致的 VHDL 命名约定)。读入后, PrimeTime 报告: 读取 SDF 时发现的错误数; 已反标的延时与时序检查数; 不受支持的 SDF 结构及各自出现次数; SDF 中的工艺、温度和电压值; 并可用 `report_annotated_delay` 与 `report_annotated_check` 查看设计的反标状态。

```
pt_shell> read_sdf -load_delay cell adder.sdf
pt_shell> current_design MY_DESIGN
pt_shell> read_sdf -load_delay net -path u1 mult16_u1.sdf
pt_shell> read_sdf -cond_use max boo.sdf
pt_shell> read_sdf -analysis_type on_chip_variation boo.sdf
pt_shell> read_sdf -analysis_type on_chip_variation \
    -min_file boo_bc.sdf -max_file boo_wc.sdf
```

从子设计时序文件反标

Annotating Timing From a Subdesign Timing File 使用 `-path` 时, `read_sdf` 用子设计时序文件反标当前设计; 不能将子设计端口 net 延时用于反标当前设计。

反标负载延时

Annotating Load Delay 负载延时 (extra source gate delay) 是驱动 net 电容负载引起的单元延时部分。部分延时计算器将其计入 net 延时, 部分计入单元延时。默认 `read_sdf` 假定负载延时在单元延时中; 若在 net 延时中, 使用 `-load_delay`。

从 SDF 反标条件延时

Annotating Conditional Delays From SDF SDF 中延时与时序检查可有条件 (通常基于单元输入值的表达式)。反标方式取决于 Synopsys 库是否定义条件弧:

- 库含条件弧: 反标所有 SDF 条件延时; 库中 `sdf_cond` 字符串须与 SDF 完全匹配
 - 库无条件弧: 反标所有条件延时的最大或最小值; 用 `-cond_use max/min` 选择
- 库无条件时, SDF 对所有 A 到 Z 弧使用最坏延时; 库有条件时可映射到对应弧。

注意

IOPATH 语句反标两 pin 间所有弧; COND IOPATH 优先于 IOPATH。

(原书 *Figure 112: Example of state-dependent timing arcs*)
请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.1: 状态相关时序弧示例

(原书 *Figure 113*)
请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.2: 库无条件时的反标延时

(原书 *Figure 114*)
请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.3: 库有条件时的反标延时

选择 B=0 延时: `set_case_analysis 0 [get_pins U1/B]`。常量可通过 `tie-high/low` 或另一 pin 的 `set_case_analysis` 传播到选择条件弧的 pin。

反标时序检查

表 11.1: 时序检查的 SDF 结构 (表 19)

检查类型	SDF 结构
Setup/Hold	SETUP, HOLD, SETUPHOLD
Recovery	RECOVERY
Removal	REMOVAL
最小脉冲宽度	WIDTH
最小周期	PERIOD
最大 skew	SKEW
No change	NOCHANGE

Annotating Timing Checks

报告延时反标状态

Reporting Delay Back-Annotation Status SDF 文件通常很大 (≥100 MB)，建议验证所有 net 已反标延时 (及时序检查)。`report_annotated_delay` 独立报告单元延时与 net 延时；net 延时分三类：主输入端口 net、主输出端口 net、内部 net (SDF 标准仅反标内部 net)。

`report_annotated_check` 报告反标时序检查数量 (setup/hold/recovery/removal/mpw/period/skew/nochange)。

SDF 流中的更快时序更新

Faster Timing Updates in SDF Flows 对路径中的每个点, PrimeTime 通常根据输入 slew 和电容计算 slew, 并从该点继续向前传播。变量 `timing_use_zero_slew_for_annotated_arcs` 允许对已用 SDF 延时反标的弧采用零 slew, 从而缩短几乎所有弧都已反标的设计的分析运行时间。

该变量默认为 `auto`。对于全部或几乎全部时序弧 (例如 95% 以上) 均已用 SDF 反标延时的设计, `auto` 设置会以牺牲 slew 计算精度换取更快运行: 对于由 `read_sdf` 或 `set_annotated_delay` 完全反标的每条弧, PrimeTime 跳过延时与 slew 计算, 并将反标弧负载 pin 上的 slew 设为零。对于未反标的弧, PrimeTime 使用可获得的最佳输入 slew 来估算延时和输出 slew; 对于一段连续的未反标弧, 则在整段中传播最坏 slew。使用此功能时, 建议将 `timing_prelayout_scaling` 设为 `false`, 以禁用布局前 slew 缩放。

11.2 反标延时、时序检查与转换时间

Annotating Delays, Timing Checks, and Transition Times

可用 `set_annotated_delay`、`set_annotated_check`、`set_annotated_transition` 直接反标, 无需 SDF。

`set_annotated_delay` 在 pin 或弧上设置 net 或单元延时；`-cell` 指定单元延时，`-net` 指定 net 延时，`-increment` 增量反标。`set_annotated_check` 反标 setup/hold/recovery/removal 等；`set_annotated_transition` 反标 pin 转换时间。

用 `report_annotated_delay`、`report_annotated_check`、`report_annotated_transition` 验证。`remove_annotated_delay`、`remove_annotated_check`、`remove_annotated_transition` 移除反标。

11.3 写出 SDF 文件

Writing an SDF File

`write_sdf` 将当前设计的延时与时序检查写为 SDF 1.0、2.1 或 3.0 格式；默认输出格式为 2.1。例如：

```
pt_shell> write_sdf -version 2.1 -input_port_nets mydesign.sdf
```

常用选项包括：`-significant_digits`、`-load_delay cell/net`、`-include`、`-exclude`、`-compress gzip`、`-map`（映射文件）、`-mask_violations setup/hold/both`。

并行驱动网络写出 SDF 时，每个驱动器到每个负载的弧均写出；大规模并行缓冲网络 SDF 可能极大。

（原书 *Figure 115: Parallel buffers driving parallel buffers*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.4: 并行缓冲驱动并行缓冲

（原书 *Figure 116: Cell delays in a parallel driver network*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.5: 并行驱动网络中的单元延时

可用 `timing_reduce_multi_drive_net_arcs` 缩减并行驱动器弧后再写 SDF。时钟 mesh/spine 网络可用外部仿真计算延时后反标到时钟 pin。

11.3.1 SDF 映射文件

SDF Mapping Files

默认 `write_sdf` 格式固定；可用映射文件自定义输出格式，并引用库中的 `timing_label` 等属性。映射语法使用 `$SDF_CELL`、`$SDF_CELL_END`、`$1...$N` 占位符，以及 `min_rise_delay(label)`、`pin(name)`、`bus(name[index])` 等函数。

注意

映射文件不支持通配符；不能指定实例级映射；未提供映射的单元使用 `write_sdf` 默认格式。

11.3.2 写出压缩 SDF

Writing Compressed SDF Files

```
pt_shell> write_sdf -compress gzip 1.sdf.gz
```

11.3.3 写出无 Setup/Hold 违例的 SDF

Writing SDF Files Without Setup or Hold Violations

`-mask_violations` 通过调整违例端点最后一条弧的延时掩盖 setup/hold 违例，便于中期门级仿真。可能需要更高 `-significant_digits` 避免舍入导致轻微违例。

11.4 设置集总寄生电阻与电容

Setting Lumped Parasitic Resistance and Capacitance

(原书 *Figure 117: Lumped RC*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.6: 集总 RC

`set_resistance`、`set_load` 在 net 上反标 R/C，临时覆盖 wire load 或详细寄生。`remove_resistance`、`remove_capacitance` 移除后恢复先前寄生形式。可单独设置：读入详细寄生后仅用 `set_load` 覆盖电容，电阻仍用详细寄生计算。

11.4.1 设置 Net 电容

Setting Net Capacitance

`set_load` 设置端口与 net 电容；层次设计须先 `link_design`。默认 net 总电容为 pin、port 与 wire 电容之和；指定值覆盖内部估算。`-wire_load` 将值计为 wire 电容。用 `report_port`、`report_net` 查看。

11.4.2 设置 Net 电阻

Setting Net Resistance

`set_resistance` 覆盖内部估算电阻；`report_net` 查看；`remove_resistance` 移除指定 net；`reset_design` 移除全设计电阻反标。

11.5 详细寄生

Detailed Parasitics

可将详细寄生反标到 PrimeTime，以 R/C 标注布线网表各物理段；比集总寄生更准确但耗时。RC 网络用于计算各子节点有效电容、slew 与延时。PrimeTime 可读 SPEF 等格式。适用于时钟树等关键

（原书 *Figure 118: Detailed RC*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.7: 详细 RC

net；深亚微米设计中 net 延时占比更大时尤其重要。支持 mesh 网络。

（原书 *Figure 119: Meshed RC*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.8: Mesh RC

11.6 读入寄生文件

Reading Parasitic Files

`read_parasitics` 可读以下格式:

- Galaxy Parasitic Database (GPD)
- Standard Parasitic Exchange Format (SPEF)
- Detailed Standard Parasitic Format (DSPF)
- Reduced Standard Parasitic Format (RSPF, IEEE 1481-1999)
- Milkyway (PARA)

SPEF/RSPF 可为 `gzip` 压缩; 格式可自动识别。`net` 与 `pin` 名须与设计匹配。默认 SPEF 电容不含 `pin` 电容, 工具用库 `pin` 电容; SPEF 中 `pin` 电容被忽略。须确保 SPEF 耦合电容对称; 可用 `-syntax_only -keep_capacitive_coupling` 检查不对称耦合。SPEF 中降阶/详细 RC 网络在延时计算中动态计算有效电容; `report_timing`、`report_net` 报告的 `Ctotal` 为集总电容 (含 `pin` 电容)。大文件应放本地磁盘并保证足够内存; `gzip` 可缩短总处理时间。

11.6.1 读入多个寄生文件

Reading Multiple Parasitic Files

```
read_parasitics A.spef -path [all_instances -hierarchy BLKA]
read_parasitics B.spef
read_parasitics chip_file_name
report_annotated_parasitics -check
```

后读入文件在冲突 `net` 上覆盖先读入数据 (若有效)。

(原书 *Figure 120: Second annotation is valid*)

请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 11.9: 第二次反标有效

11.6.2 对寄生数据应用位置变换

Applying Location Transformations to Parasitic Data

`-axis_flip flip_x/flip_y`、`-rotate`、`-transform` 等对寄生坐标变换以匹配设计朝向。

(原书 *Figure 121: Second annotation is invalid*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 11.10: 第二次反标无效

(原书 *Figure 122: Transformations specified by axis_flip options*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 11.11: axis_flip 指定的变换

11.6.3 读入多物理 Pin 的寄生

Reading Parasitics With Multiple Physical Pins

一个逻辑 pin 可对应多个物理 pin（如宽金属连接）；SPEF 中的 *D 定义驱动/负载 pin 映射。read_parasitics 自动处理多个物理 pin 的连接。

11.6.4 从多 corner 寄生数据读入单 corner

Reading a Single Corner From Multicorner Parasitic Data

GPD 等多 corner 数据可用 set_gpd_config 选择 corner，再 read_parasitics -format gpd。

11.6.5 检查反标 Net

Checking the Annotated Nets

report_annotated_parasitics -check 验证 RC 网络完整性；check_parasitics 检查 SPEF 语法与一致性。

11.6.6 缩放寄生值

Scaling Parasitic Values

read_parasitics 的 -scale_capacitance、-scale_resistance 在读入时缩放 R/C；或用 set_parasitic_corner 管理多 corner 缩放。

11.6.7 增量时序分析的寄生读入

Reading Parasitics for Incremental Timing Analysis

ECO 后可用增量寄生更新；读入新 SPEF 覆盖变更 net。

(原书 *Figure 123: Incremental update before and after ECO*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 11.12: ECO 前后增量更新

11.6.8 寄生文件限制

Limitations of Parasitic Files

RSPF/DSPF: SPICE 段实例不校验；电阻不能接地；电容须接地（忽略 node-to-node 耦合并警告）。SPEF: 忽略电感；不支持降阶 net 的极点留数；电阻不能接地；未启用 SI 时交叉耦合电容拆分到地。

11.7 不完整反标寄生

Incomplete Annotated Parasitics

PrimeTime 可补全不完整寄生的 net（须来自 SPEF）。不完整寄生 net 不能完成有效电容计算，延时计算回退到 wire load。仅当缺失段均在两 pin（边界或叶 pin）之间时可补全。

(原书 *Figure 124*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 11.13: 用 fanout 补全缺失段

11.7.1 为不完整 Net 选择 Wire Load 模型

Selecting a Wire Load Model for Incomplete Nets

按顶层 wire load mode (enclosed/top)、包围层次、主库默认等选择 wire load；-complete_with wlm 无模型时用零 R/C。

11.7.2 补全 Net 上缺失段

Completing Missing Segments on the Net

`complete_net_parasitics -complete_with wlm/zero` 或 `read_parasitics -complete_with`。补

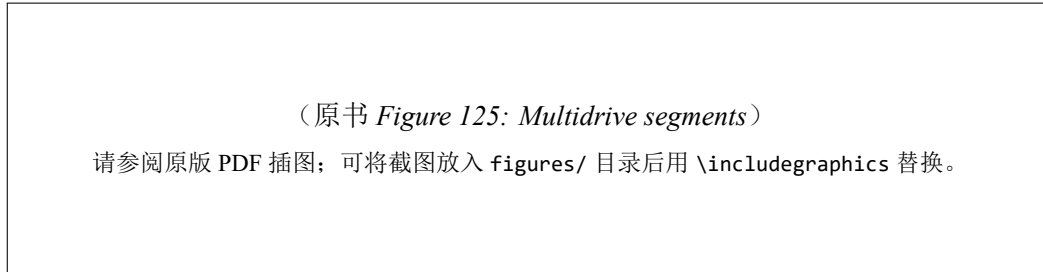


图 11.14: 多驱动段

全后不可撤销；多次 `read_parasitics` 时仅在最后一次使用 `-complete_with`。`remove_annotated_parasitics` 可取消所有寄生读入与补全效果。

11.8 反标优先级

Back-Annotation Order of Precedence

冲突时优先级（高到低）：

1. SDF 或 `set_annotated_delay` 反标的延时
2. `set_resistance/set_load` 集总 R/C
3. `read_parasitics` 详细寄生
4. 库或 `set_wire_load_model` 的 wire load 模型

11.9 报告反标寄生

Reporting Annotated Parasitics

`report_annotated_parasitics` 报告反标统计；`-check` 验证 RC 网络完整。报告按 internal net drive、design input port 等分类显示 Total/RC network/Not Annotated。

11.10 移除反标数据

Removing Annotated Data

`reset_design -keep_parasitics` 移除约束与延时反标但保留寄生。

表 11.2: 移除反标的命令 (表 22)

反标类型	命令
延时	<code>remove_annotated_delay</code>
检查	<code>remove_annotated_check</code>
转换时间	<code>remove_annotated_transition</code>
寄生	<code>remove_annotated_parasitics</code>
全部	<code>reset_design</code>

11.11 GPD 寄生浏览器

GPD Parasitic Explorer

Parasitic Explorer 可查询 GPD 格式反标的寄生 R/C, 支持: `get_resistors`、`get_ground_capacitors`、`get_coupling_capacitors`; 查询电阻、电容、子节点名、层名、层号、物理位置等属性; `report_gpd_properties`、`set_gpd_config`、`report_gpd_config`、`reset_gpd_config`、`get_gpd_corners`、`get_gpd_layers`。

11.11.1 使能寄生浏览器

Enabling the Parasitic Explorer Feature

```
pt_shell> set_app_var parasitic_explorer_enable_analysis true
pt_shell> read_parasitics -format gpd my_design_dir.gpd \
-keep_capacitive_coupling ...
```

读入前设为 true 可读入所有 corner 数据; 读入后启用则仅能查询当前 corner。

11.11.2 寄生电阻与电容集合

Parasitic Resistor and Capacitor Collections

```
get_resistors -of_objects [get_nets {net_rx*}] -parasitic_corners vhi85c
get_ground_capacitors -from_node U235/z -to_node n121:4
get_coupling_capacitors -of_objects n2 -filter "capacitance_max > 0.5e-3"
```

须用 `-of_objects` 或 `-from_node/-to_node` 指定范围; `get_attribute -class resistor ... resistance` 查询属性; `list_attributes -application -class resistor` 列出可用属性。

11.11.3 查询磁盘上的 GPD 数据

Querying GPD Data Stored on Disk

`report_gpd_properties -gpd MyDesignA.gpd` 报告设计名、工具版本、net/cell 数量等; `-layers`、`-parasitic_corners` 报告层与 corner 信息。 `set_gpd_config` 可设耦合电容绝对/相对过滤阈值, 覆盖 StarRC StarXtract 生成的 GPD 配置。

11.11.4 写出 SDF 的常用选项

Common write_sdf Options

write_sdf 支持 -significant_digits、-load_delay cell|net、-include/-exclude 过滤弧类型、-compress gzip、-map 映射文件、-divide_and_conquer 分块写出大设计 SDF。并行驱动网络写出时每个驱动器到每个负载均有弧；可用并行驱动器缩减后再写以减小文件。

11.11.5 SDF 映射函数

SDF Mapping Functions

表 11.3: 常用 SDF 映射函数（摘要）

函数	说明
min_rise_delay(label)	标号弧最小上升延时
max_rise_delay(label)	标号弧最大上升延时
pin(name)	单元 pin 名
bus(name[i])	总线位
min_period(pin)	最小周期检查

库 timing() 组中定义 timing_label 供映射引用；min_pulse_width 可用 timing_label_mpw_low/high 属性。

11.11.6 寄生读入缩放与增量分析

Parasitic Scaling and Incremental Read

read_parasitics 的 -scale_capacitance、-scale_resistance 在读入时缩放；ECO 后增量读入新 SPEF 覆盖变更 net，配合 update_timing -incremental。读入前用本地磁盘存放大 SPEF；gzip 压缩可缩短 I/O 时间。

11.11.7 反标命令示例

Annotation Command Examples

```
set_annotated_delay -cell 0.35 -from U1/A -to U1/Z
set_annotated_check -setup 0.2 -from FF/D -to FF/CP
set_annotated_transition -rise 0.1 -fall 0.12 [get_pins U2/A]
report_annotated_delay -list_not_annotated
report_annotated_parasitics -check -list_not_annotated
```

Chapter 12

Case 与 Mode 分析

Case and Mode Analysis

要限制时序分析的范围，可使用下列技术将设计置于特定工作 mode:

- Case 分析 (Case Analysis)
- Mode 分析 (Mode Analysis)
- 用于场景缩减的 Mode 合并 (Mode Merging for Scenario Reduction)

12.1 Case 分析

Case Analysis

Case 分析利用端口或 pin 上的逻辑常量或逻辑跳变来限制时序分析中信号在设计内的传播。将 case 分析设置为逻辑常量时，该常量会在设计中向前传播，并禁用常量传播所经过的路径。Case 分析不会在设计中向后传播。

将 case 分析设为逻辑跳变（上升或下降）时，通过限制分析中考虑的跳变来消除某些路径。例如，下图所示多路选择器：将 select 信号设为逻辑 0 会阻断 B 到 Z 的数据，从而禁用 B 到 Z 的时序弧。

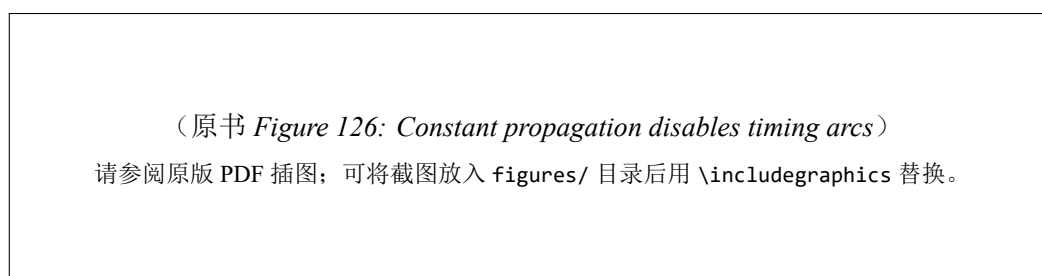


图 12.1: 常量传播禁用时序弧

若 case 分析将常量传播到时序元件的异步 preset 或 clear pin，时序单元输出也会传播常量 1 或 0，如下图所示。

(原书 *Figure 127: Constant propagation through an asynchronous input*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 12.2: 经异步输入的常量传播

关于 case 分析，另见：

- 控制逻辑值的常量传播
- 配置 Case 分析与常量传播
- 设置与移除 Case 分析值
- 使能经时序单元的 Case 分析传播
- 用 Case 分析评估条件弧
- 用 Case 分析禁用扫描链
- 执行 Case 分析
- 报告被禁用弧并追踪常量传播

12.1.1 控制逻辑值的常量传播

Propagation of Constants that Control Logic Values

若单元输入上同时存在网表逻辑常量和 case 分析值，并且该逻辑常量是该单元的控制值 (controlling value)，则从输出传播逻辑常量；否则传播 case 分析值。

示例 1: 输入 A 为常量逻辑 0 的 AND 门

(原书 *Figure (Example 1): AND gate with constant 0 on A*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 12.3: 输入 A 为常量 0 的 AND 门

无论 B 为何值，逻辑 0 均从输出 pin Z 传播。

示例 2: 输入 A 为常量逻辑 1 的 AND 门

pin A 上的逻辑常量 1 不是控制逻辑值，因此逻辑常量不会传播到输出 pin Z。

示例 3: 输入 A 与 B 均为常量逻辑 1 的 AND 门

逻辑常量 1 从输出 pin Z 传播。



图 12.4: 输入 A 为常量 1 的 AND 门

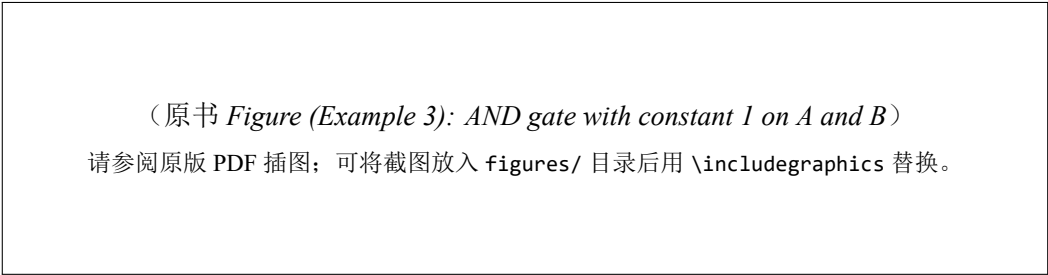


图 12.5: 输入 A、B 均为常量 1 的 AND 门

12.1.2 配置 Case 分析与常量传播

Configuring Case Analysis and Constant Propagation

要配置 case 分析及工具如何传播常量，设置下列变量：

表 12.1: Case 分析与常量传播相关变量

变量	说明
case_analysis_sequential_propagation	控制逻辑常量经时序单元的传播
case_analysis_propagate_through_icg	控制逻辑常量经集成时钟门控单元的传播
disable_case_analysis	禁用用户指定 case 分析逻辑值与网表逻辑常量的传播；此设置覆盖 disable_case_analysis_ti_hi_lo
disable_case_analysis_ti_hi_lo	禁用网表逻辑常量的传播

12.1.3 设置与移除 Case 分析值

Setting and Removing Case Analysis Values

要在指定 pin 或端口上设置 case 分析值，运行 set_case_analysis 命令。设置 case 分析时可指定下列之一：

- 常量逻辑 0、1 或 static

将 test 端口设为常量逻辑 0:

```
pt_shell> set_case_analysis 0 [get_ports test]
```

逻辑值 **static** 表示无任何跳变的常量 0 或 1，因此相连 net 既不能充当串扰 aggressor，也不会作为串扰 victim 而发生延时变化。

- 上升或下降跳变

将 RESET 端口设为上升跳变:

```
pt_shell> set_case_analysis rising [get_ports RESET]
```

(原书 *Figure 128: Setting Case Analysis With a Rising Transition*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 12.6: 用上升跳变设置 Case 分析

注意

PrimePower 计算会忽略上升与下降跳变值。

冲突时适用下列规则:

- set_case_analysis 设置优先于内建常量值 (例如 Verilog 网表中)。
- 同一端口或 pin 上较新的 set_case_analysis 优先于较旧设置。
- 直接在端口或 pin 上设置的 case 分析值优先于传播到该端口或 pin 的冲突 case 分析值。
- 传播得到的 case 分析值冲突时，逻辑 0 优先级最高，其次为 static，再次为逻辑 1。

可将 case 分析与 mode 分析结合使用。例如，因 RAM 处于读 mode 而将某些内部逻辑设为常量:

```
pt_shell> set_case_analysis 1 U1/U2/select_read
```

移除 Case 分析值

运行 remove_case_analysis:

```
pt_shell> remove_case_analysis test
```

要抑制所有逻辑常量的传播 (包括 case 分析设置及 tie-high/tie-low pin)，将 disable_case_analysis 设为 true。可用于编写 Design Compiler 能正确综合的脚本。

12.1.4 使能经时序单元的 Case 分析传播

Enabling Case Analysis Propagation Through Sequential Cells

默认情况下，PrimeTime 不将 case 分析值经时序单元传播。可按下列方式覆盖:

- 要经所有时序单元传播 case 分析值，将 `case_analysis_sequential_propagation` 设为 `always`。
- 要经特定时序单元传播：

1. 确保 `case_analysis_sequential_propagation` 为 `never`（默认）。

2. 运行 `set_case_sequential_propagation`：

```
pt_shell> set_case_sequential_propagation cell_or_libcell_list
```

3. 用下列方法之一验证已使能传播的时序单元：

- 运行 `report_case_analysis` 并带 `-sequential_propagation`。
- 查询 `is_case_sequential_propagation` 属性（对已使能 case 分析传播的时序单元与库单元返回 `true`）。

要禁用特定时序单元的 case 分析传播，运行 `remove_case_sequential_propagation`。

12.1.5 用 Case 分析评估条件弧

Evaluating Conditional Arcs Using Case Analysis

可通过 case 分析使能或禁用 Liberty 模型中的条件弧。Liberty 是一种建模语言，用于描述没有对应门级网表的大型模块的静态时序特性。若某条弧带有关联条件，PrimeTime 会对该条件求值；设计不满足弧条件时，PrimeTime 禁用该条件弧。注意：条件求值不能重新使能已被 case 分析禁用的弧。

下例中，存在从 A 到 X、条件为 `COND(C&&C1)` 的延时弧；当布尔表达式 `(C&&C1)` 为假时禁用。还存在从 CLK 到 B、条件为 `COND(C||C1)` 的 setup 弧；当 `(C||C1)` 为假时禁用。从 CLK 到 Y 的弧为非条件弧。

（原书 *Figure 129: Conditional and nonconditional arcs in a Liberty Model*）
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 12.7: Liberty 模型中的条件与非条件弧

下列命令因 case 分析使条件为假而禁用 A 到 X 的延时弧：

```
pt_shell> set_case_analysis 0 C
pt_shell> set_case_analysis 1 C1
```

要使能 A 到 X 的延时弧及 CLK 到 B 的 setup 弧：

```
pt_shell> set_case_analysis 1 {C C1}
```

关于在 Liberty 中指定条件弧，见 *Library Compiler User Guide*。

12.1.6 用 Case 分析禁用扫描链

Disabling Scan Chains With Case Analysis

通过 case 分析可禁用扫描链，从而将扫描链从时序分析中排除。

(原书 *Figure 130: Scan chain that starts at SCAN_IN and ends at SCAN_OUT*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 12.8: 起始于 SCAN_IN、终止于 SCAN_OUT 的扫描链

要禁用上述扫描链，将 SCAN_MODE 端口设为逻辑 0:

```
pt_shell> set_case_analysis 0 [get_ports "SCAN_MODE"]
```

PrimeTime 将常量传播到各扫描触发器的 TE pin。对 FF1、FF2、FF3、FF4 的时序 case 分析会禁用从 CP 到 TI 的 setup 与 hold 弧。

12.1.7 执行 Case 分析

Performing Case Analysis

可用下列方法执行 case 分析:

- 基本 Case 分析 (Basic Case Analysis)
- 详细 Case 分析 (Detailed Case Analysis)
- 独立 PrimeTime 约束一致性检查 (Standalone PrimeTime Constraint Consistency Checking)

基本 Case 分析

Basic Case Analysis 生成基本 case 分析报告:

```
pt_shell> report_case_analysis
```

示例输出:

```
*****
Report : case_analysis
...
*****

Pin name                Case analysis value
-----
test_port                0
U1/U2/core/WR            1
```


详细 **Case** 分析

Detailed Case Analysis 带 `-from` 或 `-to` 及 pin/端口列表运行 `report_case_analysis`，会调用约束一致性检查（需要 PrimeTime SI 许可证）：

```
pt_shell> report_case_analysis -to y/z
```

也可在 PrimeTime GUI 中选择 **Constraints > View Case Debugging Results** 查看 case 传播详情。

独立 **PrimeTime** 约束一致性检查

Standalone PrimeTime Constraint Consistency Checking 要访问更完整的 case 分析功能,运行 PrimeTime 约束一致性 shell。详见「启动约束一致性会话」。

12.1.8 报告被禁用弧并追踪常量传播

Reporting Disabled Arcs and Tracing Constant Propagation

PrimeTime 执行常量传播时，会在常量传播域的边界处禁用时序弧。要追踪因常量传播而禁用时序弧的原因：

- 1. 将 `case_analysis_log_file` 设为文件名，以生成列出传播常量的 `net` 与 `pin` 的日志。
- 2. 运行 `report_disable_timing` 查看常量传播后被禁用的时序弧。

下例中，因 U5/A 为常量 0 且未向 U5/Z 传播常量，工具禁用 U5/A 到 U5/Z 的弧。

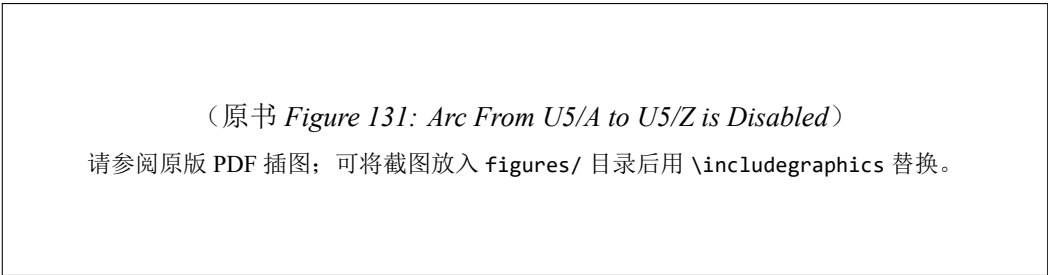


图 12.9: U5/A 到 U5/Z 的弧被禁用

```
pt_shell> set_case_analysis 0 {get_ports "A"}
pt_shell> set_case_analysis 1 {get_ports "B"}
pt_shell> set_app_var case_analysis_log_file my_design_cnst.log
pt_shell> report_disable_timing
```

示例 `report_disable_timing` 输出：

Cell or Port	From	To	Flag	Reason

U5	A	Z	c	A = 0

常量传播日志 `my_design_cnst.log` 会记录各级前向传播详情（`net pin`、单元、逻辑常量等）。

12.2 Mode 分析

Mode Analysis

库单元与时序模型中可以定义工作 **mode**，例如 RAM 单元的读/写 **mode**。每个 **mode** 都关联一组时序弧；该 **mode** 激活时，PrimeTime 分析这些弧，而非激活 **mode** 关联的弧不参与分析。未设置任何 **mode** 时，所有 **mode** 均处于激活状态，所有时序弧都参与分析。

两类 **mode**：

- **单元 mode (Cell modes)**：在时序模型或库单元中定义，如 RAM 的读/写 **mode**。
- **设计 mode (Design modes)**：用户在设计级定义，如正常与测试 **mode**；可将设计 **mode** 映射到单元实例的单元 **mode** 集合或路径集合。设计 **mode** 激活时，映射到该设计 **mode** 的所有单元 **mode** 激活，映射路径使能；设计 **mode** 非激活时，对应单元 **mode** 非激活，映射路径设为 false path。

Mode 组 (Mode Groups)

Mode 按组组织。每个组有两种可能状态：所有 **mode** 均使能（默认），或者只使能一个 **mode** 并禁用其余所有 **mode**。设计中通常只有一个 **mode** 组。

带 **mode** 的库单元可有一个或多个 **mode** 组；每组含若干单元 **mode**，每个单元 **mode** 映射到库单元中的若干时序弧。该库单元的每个实例继承这些 **mode** 组与 **mode**。典型 RAM 可有读、写、锁存、透明 **mode**；可将读/写归为一组，锁存/透明归为另一组。分组的好处是：设置某单元 **mode** 时，PrimeTime 会使对应互斥 **mode** 非激活。例如指定 RAM 为读 **mode** 即隐含写 **mode** 非激活，与透明/锁存 **mode** 设置无关；两组（读/写与透明/锁存）相互独立。

关于 **mode** 操作，见：

- 设置单元 **Mode**
- 报告 **Mode**

12.2.1 设置单元 Mode

Setting Cell Modes

设置单元工作 **mode** 的方法（按优先级从高到低）：

- 直接在单元实例上设置 **Mode**
- 用设计 **Mode** 间接设置单元 **Mode**
- 用 Case 分析将单元置入 **Mode**

直接在单元实例上设置 **Mode**

Setting Modes Directly on Cell Instances 默认每个单元实例中所有单元 **mode** 均使能。可仅使能一个单元 **mode** 并禁用同组其余 **mode**。**mode** 禁用时，映射到该 **mode** 的时序弧均禁用（不参与时序分析）。

```
pt_shell> set_mode -type cell READ U1/U2/core
```

使能读 **mode** 并禁用 U1/U2/core 该 **mode** 组中其余 **mode**。

取消 **mode** 选择、使 U1/U2/core 所有 **mode** 重新激活：

```
pt_shell> reset_mode U1/U2/core
```

用设计 **Mode** 间接设置单元 **Mode**

Setting Cell Modes Indirectly Using Design Modes 设计 mode 是用户在设计级定义的 mode（如芯片的正常/测试 mode）。可在 PrimeTime 中创建设计 mode，并将其映射到单元实例的单元 mode 或设计中的路径。设置设计 mode 后，相关单元实例会进入所映射的单元 mode；或者使能相关路径，同时将其他设计 mode 所关联的路径设为 false path。

步骤：

1. 用 `define_design_mode_group` 为当前设计创建 mode 组；多个组则多次调用。
2. 用 `map_design_mode` 将各设计 mode 映射到关联单元 mode 或路径。
3. 用 `set_mode -type design` 设置当前设计 mode；使该 mode 激活，同组其他设计 mode 非激活。

（原书 *Figure 132: Defining, mapping, and setting design modes*）
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 12.10: 定义、映射与设置设计 mode

映射设计 **Mode**

Mapping Design Modes 用 `define_design_mode_group` 创建设计 mode 后，需指定各设计 mode 的行为（映射）：

- 将设计 Mode 映射到单元 Mode 与实例
- 将设计 Mode 映射到路径集合

映射到单元 **Mode** 与实例

激活该设计 mode 时，对指定实例应用指定单元 mode：

```
map_design_mode design_mode_name \
    cell_mode_list instance_list
```

示例：将 `execute` 设计 mode 映射为在 `U1/U2/core` 上调用 `read_ram` 单元 mode：

```
pt_shell> map_design_mode execute read_ram U1/U2/core
```

映射到路径集合

激活该设计 mode 时使能这些路径；同组其他设计 mode 映射的路径设为 false path 并从时序分析中移除：

```
map_design_mode design_mode_name \
    [-from pin_list] [-through pin_list] [-to pin_list]
```

(原书 *Figure 133: Mapping a design mode to a cell mode and cell instance*)
 请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 12.11: 将设计 mode 映射到单元 mode 与实例

示例:

```
pt_shell> map_design_mode read -from Raddr -through U1/A
pt_shell> map_design_mode write -from Waddr -through U1/B
```

(原书 *Figure 134: Mapping a design mode to a set of paths*)
 请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 12.12: 将设计 mode 映射到路径集合

取消映射

运行 `remove_design_mode` 取消或更改设计 mode 映射, 并撤销因该映射产生的单元 mode 设置或 false path。

用 **Case** 分析将单元置入 **Mode**

Placing Cells Into Modes by Using Case Analysis 部分库单元与时序模型定义条件 mode: 由单元输入逻辑条件触发 mode 选择。例如 RAM 读 mode 可由读/写输入 pin 上的逻辑 0 触发。

在输入 pin 上用 case 分析 (或传播到该 pin 的 case 分析值) 设置控制逻辑值时, 单元隐式进入相应分析 mode。

(原书 *Figure 135: RAM Liberty cell can operate in read and write modes*)
 请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 12.13: 可在读/写 mode 下工作的 RAM Liberty 单元

若 Liberty 中定义:

```
cell(RAM) {
  mode_definition(read_write) {
```

```
mode_value(read) {
  when : "!read_write";
  sdf_cond : "read_write == 0";
}
mode_value(write) {
  when : "read_write";
  sdf_cond : "read_write == 1";
}
...
}
```

使能读 mode:

```
pt_shell> set_case_analysis 0 [get_ports read_write]
```

使能写 mode:

```
pt_shell> set_case_analysis 1 [get_ports read_write]
```

向 read_write pin 设置或传播逻辑值会隐式选择对应 mode 并禁用另一 mode；仅使用激活 mode 关联的时序弧。若 mode 组中无任何 mode 被 case 分析或其他方式选中，则所有 mode 使能；未选中任何 mode 时使能默认 mode。

12.2.2 报告 Mode

Reporting Modes

报告已定义或已设置的单元与设计 mode:

```
pt_shell> report_mode
```

示例输出片段:

Cell	Mode(Group)	Status	Condition	Reason

Core (LIB_CORE)	read(rw)	disabled	(A==1)	cond
	oen-on(rw)	ENABLED	(A==0)	cond
	write(rw)	disabled	(B==1)	cond
...				

12.3 用于场景缩减的 Mode 合并

Mode Merging for Scenario Reduction

复杂设计通常需分析大量场景。可用 PrimeTime 约束一致性 mode 合并能力减少场景数（mode 数 × corner 数）。

在 mode 合并期间，工具会:

- 确定待合并的输入 mode
- 合并多个输入 mode 的时序约束

- 生成一个作为超集的输出 mode

(原书 Figure 136: Mode merging automatically determines which input modes to merge and creates output modes)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 12.14: Mode 合并自动确定待合并输入 mode 并创建输出 mode

示例映射: mode_1/2/3 → merged_1; mode_4/5/6 → merged_2; mode_7 不合并。

12.3.1 运行 Mode 合并

Running Mode Merging

1. 以多场景模式启动 PrimeTime:

```
% pt_shell -multi_scenario
```

2. (可选) 为分布式 mode 合并指定核/进程/主机:

```
set_host_options -max_cores numCores \
  -num_processes numProcesses \
  -submit_command submitCommand [host_name]
```

3. 为指定 mode 与 corner 创建场景:

```
create_scenario -mode mode_name -corner corner_name
```

4. 运行 mode 合并:

```
create_merged_modes
```

5. 在 merged_constraints 目录中检查合并约束与 mode 合并分析报告。

Mode 合并考虑 corner 条件；合并约束仅当对应输入 mode 对该 corner 有效时才对该 corner 有效。有效 corner 由 DMSA 主机变量 snps_corner_name 指定。

12.3.2 调试 Mode 合并冲突

Debugging Mode Merging Conflicts

要在 GUI 中可视化调试 mode 合并冲突，对 create_merged_modes 使用 -interactive。执行仅测试的 mode 合并而不实际合并约束；测试成功后打开 MergeModeConflict Browser。

GUI 显示下列冲突类型的详情、示意图、调试帮助与修复建议：

- 时钟-时钟 (Clock-clock)

(原书 *Figure 137: MergeModeConflict browser*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 12.15: MergeModeConflict 浏览器

- 时钟-数据 (Clock-data)
- 例外 (Exception)
- 组路径 (Group path)
- 再汇聚 (Reconvergence)
- 取值 (Value)

关于 mode 合并，另见 SolvNetPlus 文章 000024378, *PrimeTime Mode Merging Application Note*。

Chapter 13

片上变异

Variation

片上变异（on-chip variation）分析对最小路径与最大路径采用不同的工作条件。为提高精度，可采用以下变异方法：

- 高级片上变异（Advanced On-Chip Variation, AOCV）
- 参数化片上变异（Parametric On-Chip Variation, POCV）

13.1 高级片上变异

Advanced On-Chip Variation

高级片上变异（AOCV）分析根据路径中的门数量与路径跨越的物理距离，对变异较小的路径施加较小的降额（derating），从而降低悲观度。AOCV 比传统 OCV 分析的悲观度更低；传统 OCV 采用固定降额因子，不考虑路径特有的度量。

路径越长、门越多，总变异往往越小，因为门与门之间的随机变异会相互抵消。路径在芯片上跨越的物理距离越大，系统性变异往往越大。AOCV 分析根据路径深度与基于位置的包围盒（bounding box）度量，计算路径特定的 AOCV 降额因子。

AOCV 分析与 PrimeTime 所有其他特性协同工作，并影响所有报告命令。它兼容分布式多场景分析与多核分析。关于 AOCV 的使用，参见：

- AOCV 流程
- 配置高级片上变异分析
- 导入 AOCV 信息
- 查询时序路径上的 AOCV 降额

13.1.1 AOCV 流程

The AOCV Flow

要启用 AOCV，将变量 `timing_aocvm_enable_analysis` 设为 `true`。还需提供适用于不同路径深度与物理路径距离范围的降额因子表，并使用 `read_ocvm` 命令读入 PrimeTime。

下表给出当路径的逻辑深度与物理距离落在指定范围内时，用于对路径中单元 `early` 延时进行降额的示例值：

```
version:          1.0
object_type:      design
rf_type:         rise fall
delay_type:      cell
derate_type:     early
object_spec:     top
voltage:         1.2
depth:          0 1 2 3
distance:        100 200
table:           0.88 0.94 0.96 0.97 \
                 0.84 0.86 0.88 0.91
```

启用 AOCV 并读入降额表后，按常规方式使用 `update_timing` 或 `report_timing` 执行时序分析；分析结果反映基于路径逻辑深度与物理跨度的不同降额因子。

PrimeTime 中的 AOCV 集成于基于图（`graph-based`）与基于路径（`path-based`）的分析能力。签核可使用基于图的 AOCV；基于路径的 AOCV 可作为可选能力，用于细化仍违例路径的分析。

基于图的 AOCV 分析

Graph-Based AOCV Analysis 基于图的 AOCV 分析是在 `update_timing` 期间执行的快速、全设计范围分析。可利用全设计范围内降低的降额悲观度，以节省面积并提升性能。

就构造方式而言，基于图的分析会计算深度与距离度量的保守值。执行基于图的 AOCV 时，选择保守的路径深度与基于位置的包围盒值，以界定穿过某单元的最坏情况路径。例如，某单元的深度计数不超过穿过该单元的最小深度路径的深度；否则对最小深度路径的分析可能过于乐观。

基于图的 AOCV 是进一步用基于路径的 AOCV 分析缩减裕量的前提。对大多数设计，基于图的 AOCV 足以签核。

基于路径的 AOCV 分析

Path-Based AOCV Analysis 若基于图的 AOCV 完成后仍有违例，可调用基于路径的 AOCV 分析以降低悲观度并提高结果精度；该分析将每条路径与其他路径隔离分析。使用 `report_timing` 或 `get_timing_paths` 并将 `-pba_mode` 设为 `path`：

```
pt_shell> report_timing -pba_mode path
```

在基于图的 AOCV 中，PrimeTime 计算穿过每条时序弧的所有路径的深度与距离，并对所有此类路径应用找到的最坏值；而在基于路径模式下，则为每条单独时序路径精确计算深度与距离。

默认情况下，选择带 AOCV 的基于路径分析时，PrimeTime 同时执行常规基于路径分析（不含 AOCV）与 AOCV 基于路径分析。不含 AOCV 的分析执行基于路径的 `slew` 传播重算；AOCV 分析通过对更深路径与跨越更短物理距离的路径减少降额来降低悲观度。

为节省运行时间，可仅执行基于路径分析的 AOCV 部分，并对基于路径分析使用最坏 slew 传播而非重算的 slew 传播。将变量 `pba_derate_only_mode` 设为 `true` 即可。

13.1.2 配置高级片上变异分析

Configuring Advanced On-Chip Variation Analysis

要配置 AOCV 分析，设置下表中的变量。

表 13.1: AOCV 分析配置变量

变量	说明
<code>timing_aocvm_analysis_mode</code>	指定深度度量的计算方式
<code>timing_aocvm_enable_clock_network_only</code>	仅对时钟网络应用 AOCV 分析
<code>timing_aocvm_enable_single_path_metrics</code>	指定 AOCV 是否对网线与单元使用独立的深度与距离值
<code>timing_ocvm_enable_distance_analysis</code>	指定高级或参数化片上变异分析是否执行基于距离的分析
<code>timing_ocvm_precedence_compatibility</code>	控制高级或参数化 OCV 对 OCV 降额的回退

13.1.3 导入 AOCV 信息

Importing AOCV Information

执行 AOCV 分析必须在降额表中指定 AOCV 信息。要建模与工艺变异无关的其他效应，可使用 guard-band 时序降额。

关于如何指定 AOCV 信息，参见：

- 指定 AOCV 降额表
- AOCV 文件格式
- AOCV 表组
- 指定深度系数
- AOCV 中的 Guard-Banding
- 增量时序降额
- 在 AOCV 分析中使用 OCV 降额

指定 AOCV 降额表

Specifying AOCV Derating Tables 使用 `read_ocvm` 从磁盘文件读取 AOCV 降额表。降额表标注到一个或多个设计对象，并直接应用于时序弧延时。允许的设计对象类为层次单元、库单元与设计。

使用 `write_binary_aocvm` 从 ASCII 格式 AOCV 文件创建二进制编码 AOCV 文件，用于保护敏感的工艺相关信息。`read_ocvm` 可读取由 `write_binary_aocvm` 创建的二进制及压缩二进制 AOCV 文件，无需额外参数。

若在 `report_timing` 中指定 `-derate`，内部计算的 AOCV 降额因子显示在 `derate` 列。

使用 `report_aocvm` 显示 AOCV 降额表数据。从二进制或压缩二进制文件导入的 AOCV 降额表在 `report_aocvm` 输出中不可见。可显示标注了 `early`、`late`、`rise`、`fall`、`cell` 或 `net` 降额表的设计对象，也可确定哪些单元与网线已标注或未标注 AOCV 信息。

在基于图的 AOCV 分析中，在 `report_aocvm` 的 `object_list` 中指定时序弧，将显示基于图的深度与距离度量及弧对象上的 AOCV 降额因子：

```
pt_shell> report_aocvm [get_timing_arcs -of_objects [get_cells U1]]
```

若在 `object_list` 中指定时序路径，将显示该路径的路径度量（`distance`、`launch depth`、`capture depth`）：

```
pt_shell> report_aocvm [get_timing_paths -path_type full_clock_expanded \
    -pba_mode path]
```

AOCV 文件格式

File Format for AOCV AOCV 文件格式允许指定多个 AOCV 降额表，支持：

- 一维表（仅 `depth` 或仅 `distance`）
- 二维表（`depth` 与 `distance`）

文件格式定义可将 AOCV 表与对象组关联；PrimeTime 根据以下定义在内部选择对象：

- `object_type design | lib_cell | cell`
- `object_spec [patterns]`

在 `object_spec` 中，`patterns` 可选，指定对象名及基于对象属性求值的表达式。可对 `patterns` 使用正则匹配，与 `get_cells`、`get_lib_cells`、`get_designs` 等命令相同。

注意

除另有说明外，所有描述均为必需项。在文件任意位置添加注释请使用双斜杠 `//`。

当具有相同 `rf_type` 与 `derate_type` 的不同 `object_type` 条目作用于同一单元或网线时，优先级规则与 `set_timing_derate` 一致。

单元弧降额优先级（高到低）：库单元 → 层次单元 → 设计。

网线弧降额优先级（高到低）：层次单元 → 设计。

注意

当多个具有相同 `object_type`、`rf_type`、`derate_type` 的表条目作用于同一对象时，最后一个表条目优先。

若指定 `voltage` 但缺少关联浮点值，或未指定 `voltage`，则降额表适用于所有电压。对多电源轨单元，不要指定电压，而应指定多电源轨所有可能输入-输出电压组合中最保守的降额因子。

以下示例为整个设计设置 `early` AOCV 表，适用于所有 `cell` 与 `net`：

```
version:          1.0
object_type:      design
rf_type:          rise fall
delay_type:       cell net
```

表 13.2: AOCV 文件格式语法（表 23）

说明符	说明
version	AOCV 版本号
group_name	表组名，可用 set_aocvm_table_group 应用
object_type	design lib_cell cell
rf_type	rise fall rise fall
delay_type	cell net cell net
derate_type	early late
path_type	clock data clock data
object_spec	描述受表影响对象的模式
voltage	供电电压（伏），可选；表仅适用于该电压的单元
depth	M 个浮点值，表示路径中连续逻辑单元数；未提供则 $M=0$
distance	N 个浮点值，表示路径物理距离（nm）；未提供则 $N=0$
table	$N \times M$ 个降额因子；PrimeTime 在表内数据点之间线性插值，不超出表中极值进行外推

derate_type:	early
object_spec:	top
depth:	0 1 2 3
distance:	100 200
table:	0.87 0.93 0.95 0.96 \ 0.83 0.85 0.87 0.90

以下示例包含可选 path_type 语句；PrimeTime 仅将表数据应用于指定路径类型。要对时钟路径与数据路径分别降额，timing_aocvm_analysis_mode 须设为 separate_data_and_clock_metrics。省略 path_type 时表同时适用于时钟与数据路径。使用 path_type 时，version 须为 2.0。

AOCV 表组

AOCV Table Groups 可定义多组 AOCV 表，用于层次不同块。为每组命名，并用 set_aocvm_table_group 应用于层次单元。各层可独立应用表，块在更高层次复用时保持 AOCV 表设置。

在 AOCV 表中用 group_name 关键字指定组名。使用 group_name 时 version 须为 3.0。未使用 group_name 的表属于默认 AOCV 表组，应用于未分配命名表组的设计部分。

```
pt_shell> set_aocvm_table_group core_tables [get_cells H1]
```

将 core_tables 应用于层次块实例 H1；该组适用于 H1 内完全封闭的网线及 H1 内所有单元（含更低层次）。core_tables 不适用于部分在 H1 内、部分在外的网线。

report_aocvm 报告各表组及默认表组中已标注 AOCV 的单元数量，并显示各定义表组名与对应层次单元。

用 reset_aocvm_table_group 移除命名表组的应用。注意:在时序更新后使用 set_aocvm_table_group、reset_aocvm_table_group 或 read_ocvm 会触发新的完整时序更新。

指定深度系数

Specifying Depth Coefficients 可为单元指定深度系数，根据单元复杂度修改路径深度计算。复杂单元可能含异常多晶体管，可赋予大于默认 1 的逻辑深度计数。例如缓冲器常由两级反相器串联实现，

可赋予 derate 系数 2.0。

```
pt_shell> set_aocvm_coefficient 2.0 [get_lib_cells lib1/BUF2]
```

设置 AOCV 系数为可选项，默认系数为 1.0。

AOCV 中的 Guard-Banding

Guard-Banding in AOCV Guard-band 时序降额可在 AOCV 流程中建模非工艺相关效应。set_timing_derate、report_timing_derate、reset_timing_derate 提供 -aocvm_guardband 选项。

在 AOCV 上下文中，弧上应用的降额为 guard-band 降额与 AOCV 降额之积；guard-band 在 AOCV 上下文外无效。

report_timing_derate -aocvm_guardband 仅报告 guard-band 降额。-variation 与 -aocvm_guardband 互斥。

增量时序降额

Incremental Timing Derating 增量时序降额可对单元或网线等对象上的降额因子微调。使用 set_timing_derate -increment。-increment 与 -aocvm_guardband 互斥。

增量降额遵循与常规降额相同的优先级与覆盖规则。默认情况下，指定的增量降额替换该对象上先前的增量降额。将 timing_enable_cumulative_incremental_derate 设为 true 可累加而非覆盖。

若无时序降额因子，增量降额使用 0.0，常规降额使用 1.0。增量因子加到常规因子上；若最终因子小于 0.0，则转换为 0.0 以避免负延时。

增量降额计算示例：

- AOCV 分析中，单元 u1/u252 的 regular late 降额 1.082、early 0.924：

```
set_app_var timing_aocvm_enable_analysis true
set_timing_derate -increment -late 0.03 [get_cells u1/u252]
set_timing_derate -increment -early -0.03 [get_cells u1/u252]
```

最终 late = 1.082 + 0.03 = 1.112, early = 0.924 + (-0.03) = 0.894。

- OCV 分析中可类似组合全局与单元级增量降额；reset_timing_derate -increment 仅重置增量部分。

report_timing_derate -increment 报告增量降额因子。

在 AOCV 分析中使用 OCV 降额

Using OCV Derating in AOCV Analysis PrimeTime 提供统一框架，使 AOCV 在特定条件下可使用 OCV 降额。默认同时考虑 OCV 与 AOCV；同级时 AOCV 优先于 OCV。单元弧默认优先级（高到低）：OCV 叶级单元、AOCV 库单元、OCV 库单元、AOCV 层次单元、OCV 层次单元、AOCV 设计、OCV 设计。

将 timing_aocvm_ocv_precedence_compatibility 设为 true 可完全忽略 OCV 降额，此时优先级为：AOCV 库单元、AOCV 层次单元、AOCV 设计。

13.1.4 查询时序路径上的 AOCV 降额

Querying AOCV Derating on Timing Paths

要获取应用于时序路径的 AOCV 降额详细信息，可查询以下 AOCV 属性：aocvm_coefficient、applied_derate、depth、derate_factor_depth_distance、distance、guardband、incremental。

虽然 AOCV 降额基于时序弧，这些属性关联于弧端点的时序点对象，可用于基于图与基于路径的分析。

使用前须：将 timing_aocvm_enable_analysis 设为 true；或使用 get_timing_paths -pba_mode 运行基于路径的 AOCV。

示例脚本遍历时序路径并查询属性：

```
set path_list [get_timing_paths ...]
foreach_in_collection path $path_list {
  foreach_in_collection point [get_attribute $path points] {
    echo [format "Derate value: %f" [get_attr $point applied_derate]]
  }
}
```

13.2 参数化片上变异（POCV）

Parametric On-Chip Variation (POCV)

参数化片上变异（POCV）将实例延时建模为实例特有变量的函数，即以该实例独有的延时变量对实例延时进行参数化。POCV 提供：

- 针对随机变异的统计单参数降额
- AOCV 与 POCV 表数据的统一输入格式与表征来源
- 非统计时序报告
- 时序路径的有限统计报告（均值、sigma）
- 与现有 PrimeTime 功能的兼容性
- 降低基于图与基于路径分析之间的悲观度差距
- 增量时序分析开销更小

使用该特性需要 PrimeTime-ADV 许可证。关于 POCV 分析，参见：

- POCV 变量与命令
- 准备 POCV 输入数据
- 导入带物理位置的 SPEF 文件
- 启用 POCV 分析
- 加载 POCV 输入数据
- 指定 Guard Banding

- 缩放 POCV 系数
- 启用约束与 Slew 变异
- 启用基于矩的建模分析
- 报告 POCV 结果
- 统计图合并悲观度
- 查询 POCV Slack 与到达时间属性

13.2.1 POCV 变量与命令

Variables and Commands for Parametric On-Chip Variation

表 13.3: POCV 分析变量（表 24）

变量	说明
parasitics_enable_tail_annotation	从寄生文件读取 tail 标注数据以进行 via 变异分析
timing_enable_constraint_variation	对 setup/hold 约束启用约束变异
timing_enable_slew_variation	对单元延时启用 slew 变异
timing_enable_via_variation	启用物理 via 变异分析
timing_pocvm_corner_sigma	指定 POCV 延时与约束分析所用的 corner sigma
timing_pocvm_enable_analysis	启用 POCV 分析
timing_pocvm_enable_extended_moments	使用非对称基于矩的建模数据
timing_pocvm_max_transition_sigma	指定 POCV 最大 transition time 分析所用的 corner sigma
timing_pocvm_precedence	文件与库 POCV 系数优先级
timing_pocvm_report_sigma	报告用 sigma
timing_use_slew_variation_in_constraint_arcs	在约束弧变异计算中使用 slew 变异

表 13.4: POCV 分析命令（表 25）

命令	说明
read_ocvm	读取 POCV 表
read_ivm	读取 via 变异表
report_delay_calculation -derate	报告延时计算降额详情
report_ocvm -type pocvm	显示 POCV 系数与基于距离的降额表
report_ivm	显示 via 变异表
report_timing -derate	在时序报告中报告 POCV 信息
report_timing_derate -pocvm_coefficient_scale_factor	报告 POCV 缩放
report_timing_derate -pocvm_guardband	报告 POCV guard banding
reset_timing_derate -pocvm_*	移除 POCV 缩放或 guard banding
set_timing_derate -pocvm_*	指定 POCV 缩放或 guard banding

13.2.2 准备 POCV 输入数据

Preparing Input Data for Parametric On-Chip Variation

使用 POCV 需要以下输入数据之一：

- side file 中的 POCV 单一系数
- Liberty Variation Format (LVF) 中的 POCV slew-load 表

若同时读取两种数据，side file 中的 POCV 单一系数优先级更高，会覆盖库中的 POCV slew-load 表。

Side File 中的 POCV 单一系数

POCV Single Coefficient Specified in a Side File AOCV 表格式 4.0 扩展格式用以下字段指定 POCV 信息：

- `ocvm_type`: `aocvm` | `pocvm` —OCV 方法类型
- `coefficient`: `sigma_value` —随机变异系数 (sigma)

若指定 `ocvm_type`: `pocvm`，不能指定 `depth` 字段；`coefficient` 与 `distance` 互斥。须为 POCV 系数（随机变异）与基于距离的变异分别指定不同表。

表 13.5: 系数与基于距离的 POCV 表（表 26）

变异类型	POCV 应用于	示例
系数	库单元	<code>version: 4.0, ocvm_type: pocvm, coefficient: 0.05</code>
基于距离	设计级	<code>distance: 1 10 50 100 500, table: ...</code>

从 Monte Carlo HSPICE 仿真提取库单元 POCV 系数：

$$\text{POCV coefficient} = \frac{s \text{ (delay variation)}}{m \text{ (nominal delay)}}$$

POCV 数据生成通常比 AOCV 更快：AOCV 需对长单元链进行 Monte Carlo 仿真，而 POCV 仅需单级或少数几级。

LVF 中的 POCV Slew-Load 表

POCV Slew-Load Table in Liberty Variation Format PrimeTime 可读取每条延时时序弧的 POCV slew-load 表 (LVF)。使用 LVF 可根据不同 slew 与负载条件应用不同系数，比固定系数更准确。支持延时弧、setup/hold、recovery/removal 及 clock-gating 检查。

在 link 设计前将带 POCV LVF 的库载入 PrimeTime。POCV LVF 须存在于时序库中，不能从单独文件读取；单位为库的时间单位。PrimeTime 也可读取 LVF 中的基于距离降额表。

参见

示例 20、21 见原书 POCV LVF slew-load 表与基于距离降额语法。

ETM 中 LVF 系数的提取

Extraction of LVF Coefficients in ETMs 使用 `extract_model` 且启用 POCV 时，默认生成 LVF 表建模 POCV 效应，使消费 ETM 的工具可在各角准确分析。可为组合与顺序延时弧生成 delay sigma 表；启用约束变异时也可约束弧生成 constraint sigma 表。

推荐生成 LVF 表以获得顶层最佳精度；若下游工具不完全支持 LVF，可用 `extract_model_-create_variation_tables` 控制 ETM 生成的 LVF 数据类型。

13.2.3 导入带物理位置的 SPEF 文件

Importing a SPEF File With Physical Locations

对 POCV 使用基于距离的降额表时，工具需要坐标计算路径距离（不需要坐标计算路径深度）。从 SPEF 或 GPD 寄生文件读取节点坐标：

1. `set_app_var read_parasitics_load_locations true`
2. `read_parasitics ...`

13.2.4 启用 POCV 分析

Enabling Parametric On-Chip Variation Analysis

```
pt_shell> set_app_var timing_pocvm_enable_analysis true
```

POCV 时序更新作为 `update_timing` 的一部分自动执行。默认在 3 sigma 分析；用 `timing_pocvm_corner_sigma` 修改，例如设为 4。

13.2.5 加载 POCV 输入数据

Loading the Parametric On-Chip Variation Input Data

```
pt_shell> read_ocvm pocv_coefficient_file_name
pt_shell> read_ocvm pocv_distance_based_derating_file_name
```

须在表中使用 Synopsys AOCV 文件格式 4.0 或更高版本指定系数或降额因子。POCV 表可标注库单元、层次单元与设计。单元上优先级（低到高）：设计、层次单元、库单元；网线上：设计、层次单元。

库中的 POCV 数据与 side file 中的单一系数可同时使用；二者作用于同一库单元时，side file 中的单一系数覆盖 LVF 数据。

13.2.6 指定 Guard Banding

Specifying Guard Banding

与 AOCV 类似，POCV 可用 guard banding 建模非工艺效应：

```
pt_shell> set_timing_derate -cell_delay -pocvm_guardband -early 0.95
pt_shell> set_timing_derate -cell_delay -pocvm_guardband -late 1.05
```

POCV guard band 同时作用于标称单元延时（均值）与单元延时变异（sigma）。

`report_timing_derate -pocvm_guardband` 仅报告 guard band；`reset_timing_derate -pocvm_guardband` 重置。

13.2.7 缩放 POCV 系数

Scaling the Parametric On-Chip Variation Coefficient

除 guard banding 外，可仅缩放单元延时变异而不改变 POCV 表中的系数：

```
pt_shell> set_timing_derate -cell_delay \
        -pocvm_coefficient_scale_factor -early 0.97
pt_shell> set_timing_derate -cell_delay \
        -pocvm_coefficient_scale_factor -late 1.03
```

仅将缩放因子应用于单元延时变异（sigma）。

13.2.8 启用约束与 Slew 变异

Enabling Constraint and Slew Variation

可选设置提高 POCV 精度：

- `timing_enable_constraint_variation true` — setup/hold 约束变异
- `timing_use_slew_variation_in_constraint_arcs` — 在约束弧中包含 slew 变异（`setup_hold`、`setup`、`hold` 或 `none`）
- `timing_enable_slew_variation true` — 单元延时 slew 变异

工具从 `lib/.db` 中的 LVF 读取约束与 slew 变异数据。`report_timing -variation` 或 `report_ocvm` 可报告。

13.2.9 启用基于矩的建模分析

Enabling Analysis With Moment-Based Modeling

支持使用含基于矩 LVF 延时与约束数据的单元库，更准确建模先进工艺节点与极低供电电压下的非高斯统计变异特性。

（原书 *Figure 138: Moment-Based Asymmetric Delay Distribution*）
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 13.1: 基于矩的非对称延时分布

统计参数包括均值偏移（mean shift）、标准差（standard deviation）与偏度（skewness），在 LVF 中由 `ocv_std_dev_*`、`ocv_mean_shift_*`、`ocv_skewness_*` 指定。

```
pt_shell> set_app_var timing_pocvm_enable_extended_moments true
```

变异报告中，参数的均值包含标称值与均值偏移调整。该特性需要 PrimeTime-ADV-PLUS 许可证。

13.2.10 报告 POCV 结果

Reporting Parametric On-Chip Variation Results

1. 报告 POCV 系数与降额
2. 报告 POCV 分析结果

`report_ocvm -type pocvm` 显示 POCV 系数与基于距离的降额表；`-list_not_annotated` 报告缺失数据的单元。对 POCV LVF 须在对象列表中指定库时序弧。

POCV 与所有 PrimeTime 报告命令协同。默认在 3-sigma 报告；`timing_pocvm_report_sigma` 修改报告 sigma（不触发 `update_timing`）。设为 0 可查看无 POCV 的 slack。

`report_timing -variation` 显示变异时序报告。Incr 列中的 Mean、Sensit、Corner 分别给出各增量延时分布自身的均值、标准差和 corner 值； $\text{Corner} = \text{Mean} \pm K \times \text{Sensit}$ ，其中 K 为 `timing_pocvm_report_sigma`。Path 列给出截至该引脚的累积到达时间分布信息。

`report_delay_calculation -derate` 详细报告 POCV 分析中所有降额如何得到最终单元延时与 sigma（示例 23、24）。

13.2.11 统计图合并悲观度

Statistical Graph Merging Pessimism

POCV 中延时为统计分布而非固定值。比较多路延时（如多输入门汇聚弧）时，最大延时可大于各单独延时的最大值，称为图合并悲观度（graph merging pessimism）。

POCV 通过在 `report_timing -variation` 中以时序点调整（statistical graph pessimism）移除该悲观度。

注意

同时使用 `-from` 与 `-to` 的 `report_timing` 可能比仅使用 `-to` 时具有更少的图合并悲观度；因此，`report_timing -to xyz` 的结果可作为 `report_timing -from abc -to xyz` 结果的保守边界，但二者不一定相等。

13.2.12 查询 POCV Slack 与到达时间属性

Querying POCV Slack and Arrival Attributes

POCV 在 pin、时序路径与时序点上使用以下 slack/到达属性（表 27）：`max_*_variation_arrival/slack`、`min_*_variation_arrival/slack`、`statistical_adjustment`、`variation_arrival`、`variation_slack`。

这些是统计量，直接查询返回空列表；须查询 `mean` 或 `std_dev` 子属性：

```
pt_shell> get_attribute [get_attribute [get_pins I/ZN] \
    max_fall_variation_slack] mean
0.990180
pt_shell> get_attribute $path variation_slack.std_dev
0.000488
```

Chapter 14

多电压设计流程

Multivoltage Design Flow

PrimeTime 支持使用 IEEE 1801 统一功耗格式 (UPF) 标准指定设计的多电压特性。PrimeTime 可在多电压供电及通过 `set_voltage` 在特定供电网上设置电压值的情况下正确分析设计时序。关于多电压设计流程，参见：

- 多电压分析要求
- 指定供电连接
- UPF 命令
- Golden UPF 流程
- 多供电电压分析

14.1 多电压分析要求

Multivoltage Analysis Requirements

PrimeTime Suite 工具支持多电压分析（不同单元使用不同供电电压）：

- PrimeTime 根据实际电压计算相应的延时与 slew
- PrimeTime SI 根据实际电压计算相应的串扰延时与噪声效应
- PrimePower 根据各单元各供电引脚的实际供电电压计算功耗

执行多电压分析须指定设计的功耗意图（哪些逻辑在何种电压下工作），可采用表 14.1 所示方法。

多电压分析的库要求：

- 库须包含每个单元的电源与地 (PG) 引脚信息。Liberty 语法使用 `voltage_map` 与 `pg_pin`；`voltage_map` 定义电源与默认电压，`pg_pin` 指定各引脚关联的电源及相关参数。详见 *Library Compiler User Guide*。

表 14.1: 为设计提供多电压功耗意图（表 28）

功耗意图来源	Verilog 网表要求	库要求
从 UPF 文件（或等价 Tcl）读取	PG-pin 或非 PG-pin 网表	PG-pin 库
从 PG-pin 网表推断	PG-pin 网表	PG-pin 库

- 若 PG-pin 库中的单元缺少电源引脚或地引脚，PrimeTime 会在 link 时从设计中移除这些单元，也不会报告其时序或功耗信息。
- Liberty 库数据须准确描述特定电压下的时序行为。PrimeTime 通过在分别表征于不同供电电压的库之间插值支持电压缩放；用 `define_scaling_lib_group` 调用电压与温度缩放。注意 PrimeTime 行为与 Design Compiler 不同——DC 在精确角电压下 link 设计。详见交叉库电压与温度缩放。

14.2 指定供电连接

Specifying Power Supply Connectivity

PrimeTime 支持两种方式指定供电连接：

- UPF 指定的供电连接
- 网表推断的供电连接

14.2.1 UPF 指定的供电连接

UPF-Specified Supply Connectivity

默认情况下，对具有 UPF 指定功耗意图的多电压设计，通过加载与设计关联的 UPF 文件获得 PG 连接。例如：

```
set_app_var search_path "/path1/mydata /path2/mydata"
set_app_var link_path "* mylib_1V.db"
read_verilog my_design.v
link_design
load_upf my_design.UPF
read_sdc my_constraints.sdc
set_voltage 0.9 -min 0.6 -object_list [get_supply_nets VDD1]
set_voltage 0.8 -min 0.5 -object_list [get_supply_nets VDD2]
set_voltage 0.0 -min 0.0 -object_list [get_supply_nets VSS]
```

参见

参见「UPF 命令」。

14.2.2 网表推断的供电连接

Netlist-Inferred Supply Connectivity

要从 PG-pin Verilog 网表推导供电连接信息,在读取任何设计信息前将 `link_keep_pg_connectivity` 设为 `true`:

```
set_app_var link_keep_pg_connectivity true
read_verilog my_design.v
link_design
read_sdc my_constraints.sdc # 无需加载 UPF
set_voltage 0.9 -min 0.6 -object_list [get_supply_nets VDD1]
```

设为 `true` 后,工具从 PG 网表数据推导 UPF 功耗意图,并隐式更新设计数据库,等效于运行 `create_power_domain`、`create_supply_port`、`create_supply_net`、`connect_supply_net` 等 UPF 命令。`link` 后可用 `set_voltage` 设置供电电压。

PG 网表流程适用于 UPF-prime 或 golden UPF 流程生成的设计数据,支持时序分析、电压缩放、电源开关与 ECO 等。要求所有网表均含 PG 信息,不接受 PG 与非 PG 网表混合。

在网表推断流程中,可读取 UPF 文件应用边界信息(端口供电与电压),但忽略内部块信息。详见「从 UPF 读取块边界信息」。

工具创建 PG Verilog 文件中指定的 UPF 电源开关对象,可用 `get_power_switches`、`report_power_switch`、`get_supply_nets -of_objects` 查询。

从 UPF 读取块边界信息

Reading Block Boundary Information From UPF 网表推断流程中可读取 UPF 应用边界信息(如端口供电与电压),在使用 `extract_model` 时尤其有用,以确保生成的模型具有正确的端口属性。

表 14.2 列出应用的边界 UPF 命令。

表 14.2: 网表推断供电连接流程中应用的 UPF 命令 (表 29)

UPF 命令	行为
<code>add_pst_state</code>	始终应用
<code>connect_supply_net</code>	默认忽略; <code>pg_allow_pg_pin_reconnection true</code> 时应用
<code>set_port_attributes</code>	始终应用 (仅 ETM 提取使用)
<code>set_related_supply_net</code>	始终应用

`link_keep_pg_connectivity` 为 `true` 时,定义内部块信息的其他命令(如 `create_supply_net`、`set_level_shifter`、`set_isolation`、`create_power_switch`)均被忽略。

14.3 UPF 命令

UPF Commands

不同电压的多个电源可为芯片不同区域的电源域供电。部分电源域可在不活动期间关断以省电。电平转换器 (level shifter) 转换跨域信号; 隔离单元 (isolation cell) 在关断域输出端提供恒定信号电

平。掉电域可含保持寄存器（retention register）在掉电期间保持逻辑值。电源开关单元在电源控制器控制下切换特定域的电源。可在 UPF 语言中指定这些多电压方面。

标准 PrimeTime 许可证包含 UPF 支持，无需额外 UPF 许可证。但功耗分析需要 PrimePower 许可证，IR drop 标注需要 PrimeTime SI 许可证。

参见

Synopsys 多电压流程与 UPF 命令背景见 *Synopsys Multivoltage Flow User Guide*。

PrimeTime 使用 UPF 指定的功耗意图与 `set_voltage` 确定每个单元各电源引脚的电压。基于 UPF 与 `set_voltage` 信息，PrimeTime 构建供电网络虚拟模型，将电压从 UPF 供电网传播到叶实例 PG 引脚。PrimeTime 不直接读取电源状态表，`set_voltage` 须与拟验证的 UPF 电源状态表中特定状态一致。

可在 shell 输入 UPF 命令，或在命令文件或用 `source/load_upf` 加载。重新 link 设计时清除已加载 UPF 信息，与 SDC 类似。PrimeTime 不通过 `write_script` 写出 UPF，也不识别 `save_upf`。

PrimeTime 支持 IEEE 1801（UPF）规范的命令与子集；不支持的命令与 PrimeTime 功能无关或尚未实现。

支持的供电命令类别：

- 电源域: `connect_supply_net`、`create_power_domain`、`create_power_switch`、`create_supply_net`、`create_supply_port`、`create_supply_set`、`set_domain_supply_net`
- 隔离与保持: `set_isolation`、`set_isolation_control`、`set_port_attributes`、`set_design_attributes`、`set_retention`、`set_retention_elements`、`set_retention_control`
- 查找与查询: `find_objects`、`query_cell_instances`、`query_cell_mapped`、`query_net_ports`、`query_port_net`
- 流程: `load_upf`、`set_scope`、`set_design_top`、`upf_version`
- 相关非 UPF 命令: `check_timing -include signal_level/supply_net_voltage/unconnected_pg_pins`、`get_power_domains`、`get_power_switches`、`get_supply_nets`、`report_power_*`、`set_level_shifter_*`、`set_related_supply_net`、`set_temperature`、`set_voltage`

PrimeTime 接受但对分析无用或信息来自其他来源的 UPF 命令（如 `add_port_state`、`create_pst`、`map_*`、`set_level_shifter` 等）作为有效语法但忽略。

PrimeTime 拒绝为未知语法的命令（如 `save_upf`、`merge_power_domains` 等）。

典型 PrimeTime 多电压时序分析命令序列如下（其中 UPF 相关功耗命令在原书中以粗体标出）：

```
read_lib l1.lib
read_verilog d1.v
# Load_upf / create_power_domain / create_supply_set ...
source d1.tcl
define_scaling_lib_group library_list
set_voltage -object_list supply_net_name
set_voltage -cell ... -pg_pin_name ... value
set_temperature -object_list cell_list value
report_timing
```

14.3.1 UPF 供电集 (Supply Sets)

UPF Supply Sets

供电集是供电网的抽象集合, 含 power 与 ground 两个供电功能。供电集与域无关, 其 power/ground 可供创建范围内任意电源域使用, 但各电源域可限制对本域内供电集的使用。

供电集可在 RTL 级定义功耗意图, 在未知实际供电网名之前即可综合。实现 (布局布线) 前须将供电集细化或关联到实际供电网。

14.3.2 UPF 供电集句柄

UPF Supply Set Handles

供电集句柄是为电源域隐式创建的抽象供电集, 可在创建供电集/网/端口之前综合设计。

(原书 Figure 139: Design example using supply set handles)
请参阅原版 PDF 插图; 可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 14.1: 使用供电集句柄的设计示例

示例 25、26 演示供电集句柄与识别 power/ground 供电网。

14.3.3 虚拟供电网络

Virtual Power Network

PrimeTime 构建虚拟供电网络模型传播电压, 不执行物理实现。可用 report_power_network、report_supply_net 等报告供电连接与电压。

14.3.4 设置电压与温度

Setting Voltage and Temperature

set_voltage 在供电网或单元 PG 引脚上设置电压。可指定单一电压, 或同时指定最小延时与最大延时分析所用电压 (-min 与默认的 max 电压)。注意 -min 指定的是最小延时分析使用的电压, 该值通常是较大的电压。-cell 与 -pg_pin_name 用于在单元级标注 IR drop, 须同时使用, 并需要 PrimeTime SI 许可证。

set_temperature 在单元列表上设置温度, 与电压缩放库组配合使用。

14.3.5 多电压分析的库支持

Library Support for Multivoltage Analysis

多电压分析推荐:

- 含 pg_pin 与 voltage_map 的 PG-pin Liberty 库
- 用 define_scaling_lib_group 定义在不同电压/温度表征的库组
- 或用 link_path_per_instance 将不同实例 link 到不同库模型
- CCS 库模型推荐以获得最佳精度

14.3.6 多电压报告与检查

Multivoltage Reporting and Checking

- report_power_pin_info —报告单元电源引脚电压与连接
- report_power_domain、report_supply_set —报告电源域与供电集
- check_timing -include supply_net_voltage —检查未设置电压的供电网
- check_timing -include unconnected_pg_pins —检查未连接 PG 引脚
- check_timing -include signal_level —信号电平不匹配检查

信号电平检查比较驱动器与负载的电压电平，检测电平转换器缺失或电压不匹配。可配置容差与过滤策略；相关 UPF 属性包括 driver_supply、receiver_supply 等。

当驱动器与接收器由同一供电网供电且使用角电压（如 set_voltage 0.8 -min 1.2 VDD）时，PrimeTime 考虑供电相关性——低电压同时作用于两者时不产生不匹配。

（原书 *Figure 140: Signal Level Checking With Supply Correlation*）
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 14.2: 具有供电相关性的信号电平检查

14.4 Golden UPF 流程

Golden UPF Flow

Golden UPF 流程是维护设计 UPF 多电压功耗意图的可选方法。它在综合、物理实现与验证各步骤中使用原始「golden」UPF 文件，以及综合与实现工具生成的补充 UPF 文件。

Golden UPF 流程优势：

- Golden UPF 文件在整个流程中保持不变，保留原始形式、结构、注释与通配命名
- 可使用工具特定条件语句在不同工具中执行不同任务（传统 UPF-prime 流程中会丢失）
- 功耗意图变更易于在补充 UPF 文件中跟踪
- 可选使用 Verilog 网表存储所有 PG 连接，使 UPF 中 connect_supply_net 不必要，显著简化并减小 UPF 文件

(原书 *Figure 141: UPF-Prime and Golden UPF Flows*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 14.3: UPF-Prime 与 Golden UPF 流程对比

在 PrimeTime SI 中使用 golden UPF 须启用：

```
pt_shell> set_app_var enable_golden_upf true
```

启用后，除查询命令外的 UPF 命令须放入脚本并用 `load_upf` 执行，不能在命令行单独执行或用 `source`。

参见

详见 SolvNetPlus 文章 000024504 「Golden UPF Flow Application Note」。

14.5 多供电电压分析

Multiple Supply Voltage Analysis

PrimeTime SI 可分析不同单元使用不同供电电压的设计。参见：

- 多电压分析概述
- 同时多电压分析 (SMVA)
- IR Drop 标注
- 在时序分析中使用 Ansys RedHawk-SC

14.5.1 多电压分析概述

Multivoltage Analysis Overview

PrimeTime SI 能在不同单元供电电压下准确确定延时、slew 与噪声效应，利用逻辑库中指定的单元延时模型。

多电压基础设施支持：

- 考虑精确驱动器与负载电源轨电压，计算非耦合信号网延时、约束与时序违例
- 考虑精确 aggressor 电源轨电压，计算耦合延时、噪声凸起、时序与噪声违例
- 驱动器与负载之间信号电平不匹配检查
- 在电源轨上进行实例级 IR drop 标注及块级/实例级温度分析

准确分析需要：PG 连接；`set_voltage` 设置的 PG 网电压；可选 IR drop (`set_voltage -cell -pg_pin_name`，需 PrimeTime SI)；CCS 库组 (`define_scaling_lib_group`) 或每电压库 (`link_path_per_instance`)；以及定义如何使用库数据构建延时模型的设置。

14.5.2 同时多电压分析 (SMVA)

Simultaneous Multivoltage Analysis (SMVA)

多电压设计中的时序路径可能跨越电源域。当各域可在多种供电电压下工作时，传统分析无法准确反映各种电压组合对每条路径的影响：每个域只使用一个电压无法涵盖所有组合；对每个域使用 min/max (OCV) 电压又会过于悲观，因为同一域内的不同单元不可能同时工作在不同电压下。

基于图的同时多电压分析 (SMVA) 消除该悲观度：域内路径在该域所有电压级分析，跨域路径分析所穿越域的所有电压组合，单次运行完成。详见第 15 章。

14.5.3 IR Drop 标注

IR Drop Annotation

用 `set_voltage` 的 `-cell` 与 `-pg_pin_name` 在单元 PG 引脚标注供电电压，建模 IR drop 对时序与噪声的影响：

```
pt_shell> set_voltage 1.10 -cell [get_cells U5*] -pg_pin_name VDD1
```

两选项须同时使用，需要 PrimeTime SI 许可证。`-object_list` 设置供电网电压不需要 PrimeTime SI 许可证。

带 IR Drop 的信号电平检查

Signal Level Checking With IR Drop `check_timing -include signal_level` 检查不同供电电压单元之间的电平不匹配。

若在供电网上设置角电压又在单元级标注 IR drop：

```
pt_shell> set_voltage 0.8 -min 1.2 VDD
pt_shell> set_voltage 0.7 -min 1.1 -cell U3 -pg_pin_name VDD
```

(原书 *Figure 142: Signal Level Checking With Cell-Level Voltage Annotation*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 14.4: 带单元级电压标注的信号电平检查

此时 PrimeTime SI 假定单元级供电电压与主供电电压不相关，考虑驱动器与接收器之间的全最坏情况差异（如比较 0.8 V 与 1.1 V、1.2 V 与 0.7 V），可能触发两次不匹配违例。

14.5.4 在时序分析中使用 Ansys RedHawk-SC

Using Ansys RedHawk-SC With Timing Analysis

RedHawk-SC 功耗与可靠性分析工具可与 PrimeTime 协同分析 IR drop 与时钟抖动对时序的影响。

写出 Ansys RedHawk-SC STA 文件

Writing Ansys RedHawk-SC STA Files RedHawk-SC 需要 PrimeTime 的 STA 数据来执行 IR drop、时钟抖动和动态电压降分析。

1. 运行 `write_rh_file -output string [-filetype jitter|irdrop]`。需要时执行时序更新,以压缩.gz 格式写出 STA 数据。动态压降分析使用 `-filetype irdrop`。需要 PrimePower 或 PrimeTime-ADV-PLUS 许可证。
2. (可选) 对所需时序路径用 `write_dvd_timing_paths` 写出路径信息供 RedHawk-SC 关键路径场景使用。

读回 RedHawk 分析结果

Reading Back RedHawk Analysis Results 可将 RedHawk 的 IR drop 与抖动结果读回 PrimeTime, 用 `set_voltage -cell -pg_pin_name` 标注单元级电压, 或用抖动数据更新时钟不确定性。

注意

RedHawk 与 PrimeTime 的接口与许可证要求随版本变化, 请参阅当前版本文档与 SolvNetPlus 文章。

Chapter 15

SMVA 基于图的同时多电压分析

SMVA Graph-Based Simultaneous Multivoltage Analysis

基于图的 SMVA（simultaneous multivoltage analysis）在单次分析运行中，同时考虑全设计所有路径的各路径供电电压的所有组合。关于 SMVA，参见：

- SMVA 概述
- 配置 SMVA 分析
- 在 SMVA 分析中报告时序路径
- DVFS 场景
- 将 SMVA 与其他 PrimeTime 特性配合使用
- 使用示例
- 支持 DVFS 场景的命令与属性
- 特性兼容性

15.1 SMVA 概述

Overview of SMVA

多电压设计中的时序路径可能跨越电源域。当各电源域可在多种供电电压下工作时，传统分析无法准确反映各种电压组合对每条路径的影响：每个域只使用一个电压无法涵盖所有组合；对每个域使用 min/max（OCV）电压又会过于悲观，因为同一域内的不同单元不可能同时工作在不同电压下。

基于图的同时多电压分析（SMVA）消除该悲观度：域内路径在该域所有电压级分析，跨域路径分析所穿越域的所有电压组合，单次运行完成。

SMVA 对采用动态电压与频率缩放（DVFS）技术的设计尤其有用——芯片运行中根据任务需求调整供电电压与频率，高负载时更高功耗，其他时候更低功耗。提供 `scaling` 库组并指定各电源域供电电压即可获得准确时序结果。

15.1.1 SMVA 要求

SMVA Requirements

基于图 SMVA 分析要求：

- 设计须有多电压功耗意图信息（见第 十四 章「多电压分析要求」）
- SMVA 特性由 PrimeTime-ADV-PLUS 许可证启用；所需许可证数量取决于分析中使用的电压级数与 DVFS 场景数（见「SMVA 许可证」）

15.1.2 SMVA 电源域分析

SMVA Power Domain Analysis

启用 SMVA 后，基于图分析在单次运行中同时考虑每条路径的所有供电电压组合（跨域与域内），排除各路径与路径段的不可能组合。

（原书 *Figure 143: Supply Combinations for Cross-Domain and Within-Domain Paths*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 15.1: 跨域与域内路径的供电组合

域 A 与 B 的供电可独立设为低或高电压。从 FF2（蓝）与 FF5（紫）开始的域内路径分别在各自域的低、高供电电压下分析。从 FF1（红）开始的跨域路径在两域供电的四种可能组合下分析。

给定供电条件下的每条路径都作为单独路径进行收集和报告；因此，同一条物理路径可由 `report_timing -nworst` 针对不同供电组合分别报告。数据路径或时钟路径中的任一 `timing arc` 跨越电源域边界时，该路径即为跨域路径。UPF 基础设施确定电源域归属；每个单元的 PG 引脚都应连接到供电网，每个端口也应定义供电网。工具既识别 Verilog PG 网表中的连接，也识别 UPF `connect_supply_net` 指定的连接，其中 Verilog PG 网表是首选方式。若路径中有对象缺少供电网信息，工具会根据设计和库信息推断最可能的供电，并使用该供电的单一电压值；可用 `check_timing -include unconnected_pg_pins` 和 `check_timing -include supply_net_voltage` 报告缺失信息。

15.2 配置 SMVA 分析

Configuring SMVA Analysis

在首次时序更新前：

1. 确保多电压功耗意图与 `scaling` 库组已配置（第 十四 章）
2. 启用 SMVA: `set_app_var timing_enable_cross_voltage_domain_analysis true`
3. 用 `get_supply_groups` 获取供电组，并用 `set_voltage_levels` 定义命名电压参考
4. 用 `set_voltage` 为各供电组的命名电压参考设置电压值

可选地，将 `timing_cross_voltage_domain_analysis_mode` 设为 `full`（默认）、`only_crossing`、`exclude_crossing` 或 `legacy`，以控制计算 slack 以及报告/ECO 命令所考虑的路径范围。

15.2.1 定义命名 SMVA 电压参考

Defining Named SMVA Voltage References

用 `get_supply_groups` 获取具有指定供电连接的供电组集合；用 `set_voltage_levels -reference_names` 为供电组定义命名电压参考；再用 `set_voltage -reference_name` 将具体电压值与各参考名称关联。

例如，顶层含下级块 B1、B2，各块供电组均可在 0.8 V 与 1.0 V 下工作：

```
set_app_var timing_enable_cross_voltage_domain_analysis true
set sg_b1 [get_supply_groups B1/VDD]
set sg_b2 [get_supply_groups B2/VDD]
set_voltage_levels -reference_names {lo hi} $sg_b1
set_voltage_levels -reference_names {lo hi} $sg_b2
set_voltage -reference_name lo -object_list $sg_b1 0.8
set_voltage -reference_name hi -object_list $sg_b1 1.0
set_voltage -reference_name lo -object_list $sg_b2 0.8
set_voltage -reference_name hi -object_list $sg_b2 1.0
update_timing
```

上述 SMVA 专用命令与选项仅在启用 `timing_enable_cross_voltage_domain_analysis` 后可用。

注意

SMVA 配置须在首次 `update_timing` 之前完成。

15.3 在 SMVA 分析中报告时序路径

Reporting Timing Paths in an SMVA Analysis

SMVA 分析中，`report_timing` 在每条路径报告中增加一行，显示该路径的供电条件（各供电网组上的命名电压参考）。

（原书 *Figure 144: Reference Voltage Conditions*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 15.2: 参考电压条件

可用 `-supply_net_group` 查看路径上各点的供电组信息。`get_timing_paths` 返回的 `timing_path` 集合对象含 `dvfs_scenario` 属性，其 `description` 子属性描述路径的 DVFS 场景。

如「SMVA 电源域分析」所述，给定电压条件下的每条路径作为单独路径。默认 SMVA 同时考虑域内与跨域路径；`report_timing` 和 `get_timing_paths` 的 `-domain_crossing` 可取 `all`（默认）、`only_crossing` 或 `exclude_crossing`，分别选择所有路径、仅跨域路径或仅域内路径。

15.4 DVFS 场景

DVFS Scenarios

对使用 DVFS 的设计，可用 DVFS 场景功能在 SMVA 之上进一步指定频率缩放与其他工作条件。

注意

DVFS 场景是对 SMVA 的扩展；单独 SMVA 仅变化供电电压。

15.4.1 DVFS 场景概念

DVFS Scenario Concepts

SMVA 允许为各供电网组定义多种供电电压。DVFS 是调整供电电压与频率以降低平均功耗的设计技术。SMVA 的 DVFS 场景特性允许在电压组合之外指定频率与其他条件，以正确分析这些设计。

15.4.2 DVFS 场景集合对象

DVFS Scenario Collection Objects

DVFS 场景表示为 `dvfs_scenario` 集合对象，描述各供电网组上的命名电压参考组合。例如 `{VDD1:lo VDDT:lo}` 表示 VDD1 与 VDDT 均在低电压参考。

`get_dvfs_scenarios` 通过一组或多组 `-supply_group/-reference_names` 选项对构造 `dvfs_scenario` 集合对象。供电组与命名电压参考的顺序不重要；对象按需构造，工具不会在内存中预先枚举所有局部和完整条件组合。

注意

`dvfs_scenario` 集合对象在查询时构造与使用；文档中简称为「DVFS 场景」。

15.4.3 传播的 DVFS 场景

Propagated DVFS Scenarios

每条时序路径都有一组完整描述其可能电压条件的 DVFS 场景；频率等条件则通过应用于这些场景的时序约束体现。

获取设计中每条路径的完全描述场景即得到传播 DVFS 场景集合。例如三供电网组（VDDT、VDD1、VDD2）各两电压级，传播场景为所有合法组合。

(原书 *Figure 145: Fully Describing DVFS Scenarios for Individual Timing Paths*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 15.3: 完全描述单条时序路径的 DVFS 场景

(原书 *Figure 146: The Propagated DVFS Scenarios for a Design*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 15.4: 设计的传播 DVFS 场景

15.4.4 用 DVFS 场景控制命令作用域

Using DVFS Scenarios to Control Command Scope

对约束或报告命令使用 DVFS 场景时，命令仅作用于与该场景兼容的路径。

(原书 *Figure 147: Applying a DVFS Scenario to a Command*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 15.5: 将 DVFS 场景应用于命令

示例：对 `report_timing` 指定 `{VDD1:lo VDDT:lo}` 会排除 VDD1 或 VDDT 使用其他命名电压参考的不兼容路径；但完全位于 VDD2 内的路径不会被排除，因为该命令场景未对 VDD2 施加条件。也就是说，命令会排除不兼容场景，并非只纳入字面列出的供电组。

15.4.5 查询 DVFS 场景

Querying DVFS Scenarios

DVFS 场景以集合对象引用。

为供电网组条件构造 DVFS 场景

Constructing DVFS Scenarios for Supply Group Conditions 指定每个供电网组上的一个命名电压参考，返回具有指定条件的 DVFS 场景集合，用于传递给约束或报告命令。

获取所有传播的 DVFS 场景

Obtaining All Propagated DVFS Scenarios `get_dvfs_scenarios -propagated` 返回当前设计的完整传播 DVFS 场景集合，可用于设计探索；若设计尚未进行逻辑更新，该命令会触发逻辑设计更新。并非所有支持 `-dvfs_scenarios` 的命令都接受包含多个场景对象的集合。

15.4.6 将 DVFS 场景应用于命令与属性

Applying DVFS Scenarios to Commands and Attributes

默认情况下，支持 DVFS 场景的命令与属性考虑所有传播 DVFS 场景。

应用于单个命令

Applying DVFS Scenarios to Individual Commands 带 `-dvfs_scenarios` 的命令允许显式指定 DVFS 场景；显式指定优先于脚本或上下文级设置。

应用于属性查询

Applying DVFS Scenarios to Attribute Queries 查询特定 DVFS 场景下的属性值时，应在属性名后的圆括号中放入 `dvfs_scenario` 集合对象，例如 `get_attribute $pin pin_capacitance_max($low_low)`。若集合含多个场景，工具按属性类型返回最坏情况值；若字符串等值无法合并，则发出 ATTR-11 警告。不支持 DVFS 场景的属性查询会报 ATTR-3 错误。属性下标优先于脚本或上下文级 DVFS 场景设置。

应用于脚本

Applying DVFS Scenarios to Scripts 用 `read_sdc -dvfs_scenarios` 或 `source -dvfs_scenarios` 对整个 SDC 或脚本应用 DVFS 场景，等效于对每个命令使用 `-dvfs_scenarios`。

注意

并非所有 SDC 命令都支持 `-dvfs_scenarios`。为避免含义不明确，带 DVFS 场景的约束脚本中应只使用明确支持该选项的命令。

为命令执行设置 DVFS 场景上下文

Setting a DVFS Scenario Context for Command Execution `push_dvfs_scenario -dvfs_scenario` 为后续支持 DVFS 的命令和属性查询建立场景上下文，`pop_dvfs_scenario` 移除最近压入的上下文。上下文可以嵌套，但不支持在 DVFS 上下文中执行时序更新。

15.5 将 SMVA 与其他 PrimeTime 特性配合使用

Using SMVA Analysis With Other PrimeTime Features

15.5.1 与 OCV/AOCV/POCV 配合

OCV, AOCV, and POCV

SMVA 处理供电电压组合，而 OCV、AOCV、POCV 处理片上变异，两者描述的是不同分析维度，可同时启用。

15.5.2 SMVA 流程中的电压缩放时序降额

Voltage-Scaled Timing Derates in SMVA Flows

`set_timing_derate` 可在不同对象作用域上指定 `early/late` 时序调整。在 SMVA 流程中，可应用与电压相关的时序降额，并由工具在分析中按需动态缩放。

要启用该特性，设置：

```
pt_shell> set_app_var \
    timing_enable_derate_scaling_for_library_cells_compatibility false
pt_shell> set_app_var \
    timing_enable_derate_scaling_interpolation_for_library_cells true
```

启用后，若在 `scaling` 库组的库单元上设置时序降额，工具会按各 DVFS 场景中计算得到的实例电压缩放降额。例如：

```
define_scaling_lib_group {lib_1.1V.db lib_1.0V.db lib_0.9V.db}
set_timing_derate -late [get_lib_cell lib_1.1V/INV1] 1.07
set_timing_derate -late [get_lib_cell lib_1.0V/INV1] 1.10
set_timing_derate -late [get_lib_cell lib_0.9V/INV1] 1.15
set_voltage_levels sg1 -reference_names {hi lo}
set_voltage -object_list sg1 -reference_name lo 0.90 ;# exact derate
set_voltage -object_list sg1 -reference_name hi 1.05 ;# scaled derate
```

在该示例中，对 INV1 单元：

- `sg1:lo` — 0.90 V 与 0.9 V 库精确匹配，应用该库上的降额 1.15
- `sg1:hi` — 1.05 V 与任一库均非精确匹配，取最邻近两库（1.0 V 与 1.1 V）上降额 1.07 与 1.10 中的更差者 1.10

`set_timing_derate` 还提供 `-dvfs_scenarios`，可直接指定降额所适用的 DVFS 场景；仅支持叶级单元实例，且场景只能引用该单元相关供电。与常规降额相同，作用于单元实例的设置会覆盖对应库单元上的更一般设置。

15.5.3 与多场景分析（DMSA）配合

Distributed Multi-Scenario Analysis

DVFS 场景与 DMSA 场景不是同一概念：DVFS 电压/频率场景在一次 SMVA 运行中同时分析，而 DMSA 的工作条件与约束场景在分布式运行中分析。

15.5.4 与电平转换器、隔离与保持

Level Shifters, Isolation, and Retention

SMVA 尊重 UPF 指定的电平转换器、隔离单元与保持寄存器约束。跨域路径经过电平转换器时，在相应电压组合下分析转换器延时。

15.5.5 与 IR Drop 标注配合

IR Drop Annotation

SMVA 流程可以使用实例级 `set_voltage -cell -pg_pin_name` 电压标注。每个 DVFS 场景按其供电条件和实例级标注计算，不应把实例级标注简单理解为对命名参考电压作数值相加。

15.5.6 与信号完整性分析配合

Signal Integrity Analysis

PrimeTime SI 可在 SMVA 电压条件下考虑 aggressor 与 victim 的相应电源轨电压。噪声分析有明确限制：`update_noise -dvfs_scenarios` 一次只接受一个完整描述设计供电条件的 DVFS 场景，`report_noise` 报告最近一次更新的该场景结果；分析多个场景时须逐个执行。

15.5.7 与路径分析配合

Path-Based Analysis

`report_timing` 与 `get_timing_paths` 可用 `-dvfs_scenarios` 将报告范围限制到兼容的 DVFS 场景，并可结合 `-domain_crossing` 选择跨域或域内路径。

15.6 使用示例

Usage Examples

15.6.1 两域双电压 SMVA

Two Domains, Two Voltages Each

```
# Define supply groups and voltage references
set_app_var timing_enable_cross_voltage_domain_analysis true
set sg_core [get_supply_groups VDD_CORE]
set sg_io [get_supply_groups VDD_IO]
set_voltage_levels -reference_names {v0p9 v1p0} $sg_core
set_voltage_levels -reference_names {v0p9 v1p0} $sg_io
set_voltage -reference_name v0p9 -object_list $sg_core 0.9
set_voltage -reference_name v1p0 -object_list $sg_core 1.0
set_voltage -reference_name v0p9 -object_list $sg_io 0.9
set_voltage -reference_name v1p0 -object_list $sg_io 1.0
update_timing
report_timing -max_paths 10
```

15.6.2 带 DVFS 场景的报告

Reporting With DVFS Scenarios

```
set scen [get_dvfs_scenarios \  
    -supply_group $sg_core -reference_names v0p9 \  
    -supply_group $sg_io -reference_names v1p0]  
report_timing -dvfs_scenarios $scen -max_paths 5  
get_attribute [get_pins U1/ZN] pin_capacitance_max($scen)
```

15.6.3 跨域路径过滤

Cross-Domain Path Filtering

```
report_timing -domain_crossing only_crossing -max_paths 20  
report_timing -domain_crossing exclude_crossing -max_paths 20
```

15.7 支持 DVFS 场景的命令与属性

Commands and Attributes With DVFS Scenario Support

支持 -dvfs_scenarios 或 DVFS 场景下标的命令包括（非穷尽）:

- report_timing、report_constraint、get_timing_paths
- set_false_path、set_multicycle_path、set_max_delay、set_min_delay
- set_clock_uncertainty、set_input_delay、set_output_delay
- 多种 pin、port、net、clock 与 pg_pin_info 属性

表 15.1: 支持 DVFS 场景的属性（节选）

属性名	类	说明
dvfs_scenario	timing_path	路径的 DVFS 场景
pin_capacitance_max, pin_capacitance_min	pin, port, net	可按 DVFS 场景查询
power_rail_voltage_max, power_rail_voltage_min	pin, port	可按 DVFS 场景查询

表 15.2: SMVA 特性限制

特性	限制
噪声分析	不支持一次分析多个 DVFS 场景，必须逐场景更新
ETM 生成	不支持生成单个汇总所有 SMVA 条件的 ETM；工具按相关活动场景分别生成模型
Reduced-resource ECO	不兼容
SDF 回标	不支持为不同 DVFS 场景使用不同的 SDF 延时数据
PLL generated clocks	不支持 create_generated_clock 的 -pll_feedback 与 -pll_output
高级属性用法	某些用法不兼容，须以属性是否明确支持 DVFS 场景为准

15.8 特性兼容性

Feature Compatibility

注意

SMVA 会增加分析状态空间以及运行时间和内存消耗。电压级、供电组与 DVFS 场景较多时，应评估状态组合及许可证需求。

参见

第 14 章多电压流程；第 13 章片上变异；PrimeTime 许可证章节中的 SMVA 许可证说明。

Chapter 16

信号完整性分析

Signal Integrity Analysis

PrimeTime SI 是为 PrimeTime 静态时序分析器增加串扰分析能力的选项。PrimeTime SI 计算网间交叉耦合电容的时序效应，并将由此产生的延时变化纳入 PrimeTime 分析报告；同时计算串扰噪声的逻辑效应，报告可能导致功能失效的条件。运行 PrimeTime SI 需要 PrimeTime SI 许可证。

关于串扰分析与 PrimeTime SI，参见：

- 信号完整性与串扰概述
- Aggressor 与 Victim 网
- 串扰延时分析
- 静态噪声分析

16.1 信号完整性与串扰概述

Overview of Signal Integrity and Crosstalk

信号完整性（signal integrity）是电信号可靠承载信息并抵抗邻近信号高频电磁干扰影响的能力。串扰（crosstalk）是两个或多个物理相邻网之间因电容交叉耦合产生的不良电气相互作用。随着集成电路工艺向更小几何尺寸发展，相对于单元延时和网线延时，串扰效应日益重要。

工艺尺寸缩小时，互连线间距更近、高度增加，网间交叉耦合电容增大；同时互连线变窄使对衬底寄生电容减小，晶体管变小使单元延时降低。

PrimeTime SI 可分析并报告两类主要串扰效应：延时与静态噪声。可选择仅计算串扰延时、仅计算串扰噪声，或两者皆算。

16.1.1 串扰延时效应

Crosstalk Delay Effects

串扰通过改变信号跳变发生时刻影响信号延时。图 16.1 示出交叉耦合网 A、B、C 上的信号波形。

(原书 *Figure 148: Transition slowdown or speedup caused by crosstalk*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.1: 串扰引起的跳变减慢或加快

由于电容交叉耦合，网 A 与网 C 上的跳变可影响网 B 上跳变发生的时刻。图中所示时刻网 A 的上升沿可使网 B 上的跳变推迟，可能加剧含 B 的路径的 setup 违例；网 C 的下降沿可使网 B 上的跳变提前，可能加剧 hold 违例。

PrimeTime SI 确定最坏情况的延时变化量，用该信息计算并报告总 slack；同时报告串扰延时位置与大小，以便修改设计或版图以降低关键点的串扰。

16.1.2 Delta 延时与扇出级效应

Delta Delay and Fanout Stage Effect

串扰效应使开关波形畸变，从而给扇出级的传播波形增加延时（图 16.2）。

(原书 *Figure 149: Delta delay includes the fanout stage effect*)
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.2: Delta 延时包含扇出级效应

PrimeTime SI 计算 delta 延时（串扰在开关网上引起的额外延时）。扇出级效应（fanout stage effect）表示传播到扇出级的延时；PrimeTime SI 将其作为 delta 延时的一部分。

16.1.3 串扰噪声效应

Crosstalk Noise Effects

PrimeTime SI 还确定稳态网上串扰噪声的逻辑效应。图 16.3 示出交叉耦合网 A 与 B 上的噪声凸起示例。

网 B 应恒为逻辑 0，但网 A 的上升沿在 B 上引起噪声凸起或毛刺（glitch）；若该凸起具有足够的高度和宽度，错误逻辑值可能传播到路径中的下一级门。

PrimeTime SI 考虑这些效应，确定串扰噪声凸起在何处可能导致逻辑失效，并报告噪声 slack 与违例。

（原书 *Figure 150: Noise bump due to crosstalk*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.3: 串扰引起的噪声凸起

16.2 Aggressor 与 Victim 网

Aggressor and Victim Nets

在串扰分析中，**victim**（受扰网）是受到邻近网不良交叉耦合影响的网，**aggressor**（侵扰网）是对 victim 造成该影响的网。同一网相对于不同耦合关系可以既是 aggressor，又是 victim。

aggressor 对 victim 的时序效应取决于多种因素：

- 交叉耦合电容大小
- aggressor 与 victim 信号跳变的相对时刻及 slew
- 跳变方向（rise 或 fall）
- 多个 aggressor 对同一 victim 的组合效应

PrimeTime SI 计算串扰效应时考虑所有这些因素。

（原书 *Figure 151: Effects of crosstalk at different arrival times*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.4: 不同到达时刻的串扰效应

图 16.4 说明时序考虑对串扰计算的重要性。aggressor 信号 A 有从早到晚的可能到达时间范围。

若 A 的跳变与 B 的跳变大致同时，串扰可能使 B 的跳变推迟并加剧 setup 违例，也可能使其提前并加剧 hold 违例。若 A 跳变较早，在 B 跳变之前感应出向上毛刺，通常不影响 B 的跳变时刻；但足够大的毛刺可能使传输晶体管正向偏置。若 A 跳变较晚，在 B 跳变之后感应出凸起，同样不影响 B 的跳变时刻；但足够大的凸起可能改变逻辑值并沿时序路径传播。

16.2.1 时序窗口与串扰延时分析

Timing Windows and Crosstalk Delay Analysis

PrimeTime 提供 single 与 on-chip variation 两种工作条件分析模式。使用 on-chip variation 模式时，PrimeTime SI 为 aggressor 与 victim 找到最早与最晚到达时间，形成到达窗口（arrival window）。

默认 PrimeTime SI 用两次迭代执行串扰分析。第一次迭代不考虑时序窗口（保守假设所有 aggressor 在最坏时刻最坏方向跳变）；第二次及以后迭代根据 aggressor 与 victim 跳变的时间分离与方向剔除不可能发生的串扰延时。

16.2.2 交叉耦合模型

Cross-Coupling Models

（原书 *Figure 152: Detailed model of cross-coupled nets*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.5: 交叉耦合网的详细模型

详细 SPICE 级模型可非常准确预测仿真中的串扰，但对实际集成电路元素过多。给定外部工具提供的足够准确且足够简单的交叉耦合电容网络，PrimeTime SI 可在合理时间内获得准确串扰分析结果。

16.3 串扰延时分析

Crosstalk Delay Analysis

使用 PrimeTime SI 进行串扰延时分析的基础，参见：

- 执行串扰延时分析
- PrimeTime SI 如何工作
- 使用指南
- 时序报告
- 报告串扰设置
- 双开关检测

16.3.1 执行串扰延时分析

Performing Crosstalk Delay Analysis

PrimeTime SI 串扰延时分析使用与普通 PrimeTime 相同的命令集、库与 Tcl 脚本。额外步骤：

1. 启用 PrimeTime SI: `set_app_var si_enable_analysis true`
2. 用 SPEF 或 GPD 回标交叉耦合电容: `read_parasitics -keep_capacitive_coupling file_name.spf`, 或对 GPD 使用 `read_parasitics -keep_capacitive_coupling -format GPD file_name.gpd`

3. (可选) 设置控制串扰分析速度与精度的变量 (见「PrimeTime SI 变量」)
4. 若时序报告显示明显串扰效应, 调试或修正; GUI 可生成串扰延时与感应电压直方图

示例如下 (串扰相关项在原书中以粗体标出):

```
set_operating_conditions -analysis_type on_chip_variation
set_app_var si_enable_analysis true
read_verilog ./test1.v
current_design test1
link_design
read_parasitics -keep_capacitive_coupling SPEF.spf
create_clock -period 5.0 clock
check_timing -include { no_driving_cell ideal_clocks \
    partial_input_delay unexpandable_clocks }
report_timing
report_si_bottleneck
report_delay_calculation -crosstalk -from pin -to pin
```

`set_operating_conditions` 将分析类型设为 `on_chip_variation`, 使 PrimeTime SI 正确处理 min/max 时序窗口关系; 未显式指定时 PrimeTime SI 自动切换。

读取 SPEF 时须确保耦合电容对称。先读入设计, 再用 `read_parasitics -syntax_only -keep_capacitive_coupling`; 若发现非对称耦合, PrimeTime 发出警告。

16.3.2 PrimeTime SI 如何工作

How PrimeTime SI Operates

启用串扰分析后, `update_timing` 或 `report_timing` 时 PrimeTime SI 执行图 16.6 所示步骤。

(原书 *Figure 153: PrimeTime SI Crosstalk Analysis Flow*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.6: PrimeTime SI 串扰分析流程

电气滤波 (electrical filtering): 根据 victim 网上计算的凸起电压大小, 移除影响过小的 aggressor。可指定阈值。

初始网选择: 从未被滤除的网中选择初始串扰分析网集; 可指定包含或排除特定网。

延时计算: 对所选网执行含串扰考虑的延时计算。

迭代: 串扰分析为迭代过程。初始迭代不考虑时序窗口 (保守最坏假设); 后续迭代考虑 victim 跳变时间与方向, 剔除不可能发生的串扰延时。默认第二次迭代后退出, 通常可在合理时间内获得良好结果。

可按 `Ctrl+C` 中断当前迭代; PrimeTime SI 完成当前迭代后退出循环并报告诊断信息。

16.3.3 PrimeTime SI 变量

PrimeTime SI Variables

表 16.1 列出控制串扰分析的变量。启用串扰分析须 `si_enable_analysis true`。

表 16.1: PrimeTime SI 变量（表 33，节选）

变量	默认设置
<code>delay_calc_waveform_analysis_mode</code>	<code>disabled</code>
<code>si_analysis_logical_correlation_mode</code>	<code>true</code>
<code>si_ccs_aggressor_alignment_mode</code>	<code>lookahead</code>
<code>si_enable_analysis</code>	<code>false</code>
<code>si_filter_accum_aggr_noise_peak_ratio</code>	<code>0.03</code>
<code>si_filter_per_aggr_noise_peak_ratio</code>	<code>0.01</code>
<code>si_noise_composite_aggr_mode</code>	<code>disabled</code>
<code>si_noise_endpoint_height_threshold_ratio</code>	<code>0.75</code>
<code>si_noise_limit_propagation_ratio</code>	<code>0.75</code>
<code>si_noise_slack_skip_disabled_arcs</code>	<code>false</code>
<code>si_xtalk_composite_aggr_mode</code>	<code>disabled</code>
<code>si_xtalk_delay_analysis_mode</code>	<code>all_paths</code>
<code>si_xtalk_double_switching_mode</code>	<code>disabled</code>
<code>si_xtalk_exit_on_max_iteration_count</code>	<code>2</code>
<code>si_xtalk_max_transition_mode</code>	<code>uncoupled</code>

16.3.4 逻辑相关性

Logical Correlation

保守分析假设所有 `aggressor` 可同时同向开关，导致 `victim` 最坏减慢或加快。某些情况下因逻辑关系，`aggressor` 不能同向同时开关。

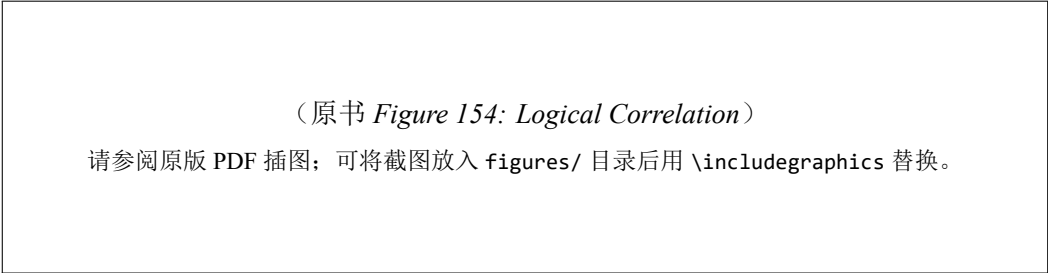


图 16.7: 逻辑相关性

默认 PrimeTime SI 考虑经缓冲器与反相器的多 `aggressor` 逻辑关系，提供更准确（更少悲观）的多 `aggressor` 分析。将 `si_analysis_logical_correlation_mode` 设为 `false` 可禁用，分析更快但更悲观。

16.3.5 电气滤波

Electrical Filtering

为在合理时间内获得准确结果，PrimeTime SI 滤除对 victim 凸起贡献过小的 aggressor。若单个 aggressor 在 victim 上的凸起高度贡献很小（低于每 aggressor 阈值），则滤除。

（原书 *Figure 155: Voltage bumps from multiple aggressors*）
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.8: 多 aggressor 的电压凸起

多个较小 aggressor 的组合若低于另一较大累积阈值，也可能被滤除。可提高阈值以加快运行、降低精度，或降低阈值以提高精度、增加运行时间。

16.3.6 使用指南

Usage Guidelines

多数 PrimeTime SI 变量在分析精度与执行时间之间权衡。

准备运行串扰分析

Preparing to Run Crosstalk Analysis 首先确保设计约束良好且通过常规静态时序检查。

电容耦合数据 用 `read_parasitics -keep_capacitive_coupling` 读入 SPEF/GPD。若提取工具支持基于阈值滤除小电容，注意不要滤得过激导致 PrimeTime SI 遗漏重要耦合。PrimeTime SI 忽略网与自身的交叉耦合。

工作条件 PrimeTime SI 使用 on-chip variation 工作条件寻找 aggressor 与 victim 到达窗口。未显式设置 `on_chip_variation` 时自动切换，影响包括 min/max 条件分离等。

使用 check_timing `check_timing` 可检查多种串扰相关条件：`no_input_delay`、`partial_input_delay`、`unexpandable_clocks`、`no_driving_cell` 等。除 `ideal_clocks` 外，串扰相关检查默认开启。

16.3.7 包含或排除特定网的串扰分析

Including or Excluding Specific Nets From Crosstalk Analysis

- `set_si_delay_analysis` — 在串扰延时分析中包含或排除指定网
- `set_si_noise_analysis` — 在串扰噪声分析中包含或排除指定网
- `set_si_aggressor_exclusion` — 指定不能同时同向跳变的互斥 aggressor 组

- `set_coupling_separation` — 从串扰延时和噪声分析中排除网或网对

`set_coupling_separation` 与 `set_si_delay_analysis` 或 `set_si_noise_analysis` 的设置冲突时, `set_coupling_separation` 优先。

可仅排除 victim 的上升或下降沿 (`set_si_delay_analysis -rise/-fall`), 或排除 setup/hold 分析。

排除时钟网

Excluding Clock Nets 若时钟延时受串扰影响不显著, 可用 `set_si_delay_analysis -exclude -victims [get_nets CLK*]` 将时钟网排除为 victim; 这些网仍可作为 aggressor。

排除安静 aggressor

Excluding Quiet Aggressor Nets `set_case_analysis` 可将特定网标记为静态值, 使其作为安静 (不开关) 的 aggressor 从时序和噪声分析中排除。

排除 aggressor-victim 对

Excluding Aggressor-Victim Pairs 用 `set_si_delay_analysis -exclude` (串扰延时) 或 `set_si_noise_analysis -exclude` (串扰噪声) 同时指定 `-victims` 与 `-aggressors`, 即可排除特定 aggressor-victim 对。例如:

```
set_si_delay_analysis -exclude -victims [get_nets CLK_NET_*] \
    -aggressors [get_nets SCN_CLK_*]
```

Aggressor 互斥组

(原书 *Figure 156: One-Hot Decoder Multiple Aggressor Example*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.9: One-hot 译码器多 aggressor 示例

(原书 *Figure 157: Multiple aggressor arrival windows at decoder outputs*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 16.10: 译码器输出处多 aggressor 到达窗口

Exclusive Groups of Aggressors 默认多个 aggressor 到达窗口与 victim 跳变重叠时，工具考虑多个 aggressor 同时同向跳变的最坏情况。可用 `set_si_aggressor_exclusion` 指定互斥 aggressor 组以降低悲观度；`-rise` 或 `-fall` 指定互斥方向，`-number_of_active_aggressors` 指定可同时活跃的最大数量，省略该选项时默认为 1：

表 16.2: 互斥组对 aggressor 活跃性的影响

	Agg1	Agg2	Agg3
无互斥组	Quiet	Active	Active
互斥组 {agg1 agg2 agg3}	Quiet	Active	Quiet

移除相应设置时，使用 `remove_si_delay_analysis`、`remove_si_noise_analysis`、`remove_si_aggressor_exclusion` 或 `remove_coupling_separation`。

16.3.8 Victim-Aggressor 对齐与窗口模式

Victim-Aggressor Switching Time Alignment

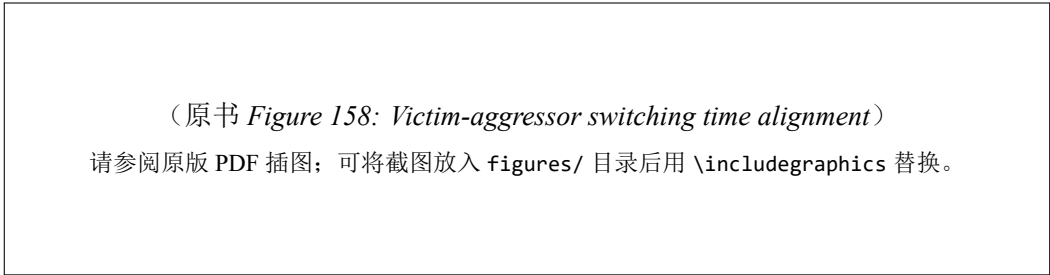


图 16.11: Victim-aggressor 开关时间对齐

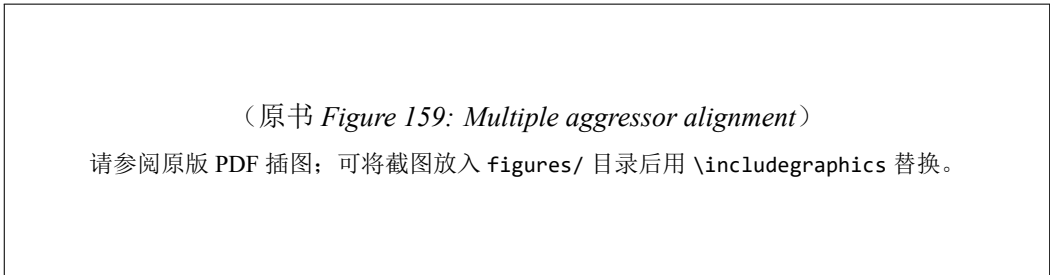


图 16.12: 多 aggressor 对齐

PrimeTime SI 将 aggressor 到达窗口与 victim 跳变时间对齐以计算 delta 延时。异步时钟之间 aggressor 视为相对 victim 的无限窗口。

16.3.9 「All Paths」与「All Path Edges」重叠模式

“All Paths” Window Versus “All Path Edges” Overlap Modes

默认考虑 victim 网上所有路径的最早到最晚到达时间范围 (`all_paths`)。任一 aggressor 窗口与「所有路径」victim 窗口重叠即触发 delta 延时计算。

该模式简单快速但保守——实际 victim 跳变可能不与 aggressor 窗口重叠。对关键路径可：

(原书 *Figure 160: “All Paths” (Default) Window Overlap Mode*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.13: 「所有路径」(默认) 窗口重叠模式

- 基于路径分析: `report_timing -pba_mode path` 或 `exhaustive`
- 「所有路径边」模式: `set_app_var si_xtalk_delay_analysis_mode all_path_edges`

(原书 *Figure 161: “All Path Edges” Overlap Mode*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.14: 「所有路径边」重叠模式

(原书 *Figure 162: “All Path Edges”, Aggressor Between Edges*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.15: 「所有路径边」模式，aggressor 窗口位于边之间

`all_path_edges` 考虑实际早/晚到达边时刻，不合并为连续窗口；对最大延时分析，若晚边不与 aggressor 重叠则不施加 delta 延时。

16.3.10 时钟组

Clock Groups

`set_clock_groups` 指定时钟间关系，使 PrimeTime SI 正确分析时钟间串扰。

时钟可定义为逻辑互斥 (`-logically_exclusive`)、物理互斥 (`-physically_exclusive`) 或异步 (`-asynchronous`)。逻辑互斥抑制逻辑时序检查但 SI 仍检查串扰；物理互斥假定无串扰交互；异步时钟使用无限到达窗口。

最准确做法是对每种时钟组合分别分析，或用 DMSA。时钟数量极大时，可用 `set_clock_groups` 在单次运行中指定关系，或用 `case analysis` 分多次分析。

(原书 *Figure 163: Circuit with multiplexed clocks*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.16: 含复用时钟的电路

16.3.11 数据网络跨时钟域 Delta 延时

Data-Network Delta Delay Across Clock Domains

时钟网络始终按 victim 网的各时钟域分别计算 delta 延时。数据网络（包括作为数据使用的时钟）则由 `timing_enable_independent_crosstalk_in_data_network` 控制：`none`（默认）在所有时钟域间取最坏 delta 延时；`all_clocks` 为每个时钟域分别计算；`physically_exclusive_clocks` 按显式 `set_clock_groups -physically_exclusive` 定义的可共存时钟组分别计算。后两者可降低悲观度，但可能增加内存和运行时间。

16.3.12 时序报告

Timing Reports

`report_timing` 在启用 SI 时显示 delta 延时列。`report_si_bottleneck` 报告串扰瓶颈网。`report_delay_calculation -crosstalk` 显示单条弧的详细串扰计算，包括各 aggressor 贡献、滤波状态与对齐信息。

串扰相关网属性包括：`delta_delay`、`si_aggressor_nets`、`si_victim_nets`、`si_double_switching_slack` 等。

16.3.13 报告串扰设置

Reporting Crosstalk Settings

`report_si_delay_analysis`、`report_si_noise_analysis` 和 `report_si_aggressor_exclusion` 分别报告串扰延时、串扰噪声和互斥 aggressor 设置。这些命令只报告已有属性，不触发时序或噪声更新。

16.3.14 双开关检测

Double-Switching Detection

双开关（double-switching）指 victim 在预期稳定期间因 aggressor 耦合发生额外跳变，可能导致时钟网络上功能错误。

将 `si_xtalk_double_switching_mode` 设为 `clock_network` 或 `full_design` 启用检测。该检测依赖 CCS noise model。工具用 `si_has_double_switching` 网属性标记检测结果，并将严重程度报告为 double-switching slack；风险越高，slack 越负。slack 存于 `si_double_switching_slack` 网属性；没有 CCS noise model 时，该值为 INFINITY。

报告双开关违例

Reporting Double-Switching Violations `report_si_double_switching` 报告所有检测到双开关的 victim 网。`-clock_network` 仅报告时钟网。`-rise/-fall` 过滤上升/下降 victim。

示例输出：

```
pt_shell> report_si_double_switching -nosplit
Victim Switching Actual Required Double Switching
Net Direction Bump Height Bump Height Slack
I2 max_rise 0.69 0.42 -0.26 (Violating)
I1 max_fall 0.50 0.30 -0.20 (Violating)
```

`report_delay_calculation -crosstalk` 在启用双开关模式时显示 victim 上是否存在双开关违例。

修复双开关违例

Fixing Double-Switching Violations 可通过增大线间距、屏蔽相邻线降低交叉电容，或加大 victim 驱动降低 transition time。时序违例可用 `fix_eco_timing` 修复。

16.4 静态噪声分析

Static Noise Analysis

PrimeTime SI 可分析稳态网上串扰噪声导致的逻辑失效。参见：

- 静态噪声分析概述
- PrimeTime SI 噪声分析流程
- 噪声分析命令
- 使用 CCS 噪声数据的噪声建模
- 使用非线性延时模型的噪声建模

16.4.1 静态噪声分析概述

Static Noise Analysis Overview

静态噪声分析用于找出设计中最坏的噪声效应以便降低或消除，从而最大化成品可靠性。与静态时序分析一样，不依赖测试向量或电路仿真，而考虑 aggressor 与 victim 间交叉耦合电容、aggressor 跳变到达窗口、aggressor 驱动特性、victim 驱动器稳态负载特性及噪声穿过单元的传播。

静态时序分析确定串扰引起的 victim 最坏延时变化；静态噪声分析确定稳态 victim 网上最坏噪声凸起或毛刺（稳态指网恒为逻辑 1 或 0）。稳态网上的噪声凸起可沿时序路径传播，在端点被错误捕获为数据。

主要噪声命令：`check_noise`、`update_noise`、`report_noise`，分别类似于 `check_timing`、`update_timing`、`report_timing`。

`check_noise` 检查驱动器与负载引脚处噪声模型的存在与有效性。`update_noise` 执行噪声分析并用噪声凸起信息更新设计。`report_noise` 报告最坏噪声效应，包括宽度、高度与噪声 slack。

（原书 *Figure 175: Crosstalk effects on timing and steady-state voltage*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.17: 串扰对时序与稳态电压的效应

（原书 *Figure 176: Combined effects of crosstalk and propagated noise*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 16.18: 串扰与传播噪声的组合效应

16.4.2 噪声凸起特性

Noise Bump Characteristics

电子设计中的「噪声」通常指应保持恒定电压的网上任何不良电压偏差。深亚微米 CMOS 中主导效应是物理相邻逻辑网之间的串扰噪声及由此产生的串扰凸起在单元输入到输出的传播。

噪声效应（电压-时间曲线）可有多种形态：凸起、振铃、阻尼振荡等。PrimeTime SI 将噪声凸起参数化为高度（height）、宽度（width）与 time-to-peak（或 time-peak-ratio）。

16.4.3 PrimeTime SI 噪声分析流程

PrimeTime SI Noise Analysis Flow

噪声分析流程：

1. 启用：si_enable_analysis true（与延时分析共用）
2. 读入含耦合电容的寄生参数
3. check_noise 验证噪声模型
4. update_noise 更新噪声分析
5. report_noise 报告违例与 slack

噪声分析可与串扰延时分析迭代协同，或独立运行。

表 16.3: 主要噪声分析命令

命令	说明
<code>check_noise</code>	检查噪声模型与网表完整性
<code>update_noise</code>	执行噪声分析并更新数据库
<code>report_noise</code>	报告最坏噪声凸起与 slack
<code>report_noise_calculation</code>	详细噪声计算报告
<code>set_si_noise_analysis</code>	包含/排除网的噪声分析
<code>remove_si_noise_analysis</code>	移除噪声分析设置

16.4.4 噪声分析命令

Noise Analysis Commands

`report_noise` 常用选项包括 `-above/-below`、`-high/-low`、`-nworst_pins`、`-slack_type`、`-slack_lesser_than`、`-all_violators`、`-verbose` 与 `-nosplit`。

16.4.5 使用 CCS 噪声数据的噪声建模

Noise Modeling With CCS Noise Data

Composite Current Source (CCS) 噪声模型提供更准确的驱动器与接收器噪声特性。库中含 `output_current_*` 与噪声相关表格时，PrimeTime SI 使用 CCS 噪声分析。

CCS 噪声分析考虑：

- 非线性 I-V 特性
- 多 aggressor 叠加
- 噪声传播与衰减
- 噪声免疫曲线 (noise immunity curve)

推荐对先进工艺节点使用含 CCS 噪声数据的库。

16.4.6 使用非线性延时模型 (NLDM) 的噪声建模

Noise Modeling With Nonlinear Delay Models

对仅有 NLDM 时序数据的库，PrimeTime SI 使用简化的噪声分析模型，精度低于 CCS。噪声免疫可用曲线、表或多项式指定。

16.4.7 噪声免疫与 Slack 计算

Noise Immunity and Slack Calculation

噪声免疫 (noise immunity) 定义输入引脚可容忍而不导致逻辑失效的噪声凸起。Liberty 库可使用曲线、表或多项式形式描述噪声免疫，也可指定噪声高度裕量。

`report_noise -slack_type` 选择 slack 报告方法：

- `height` 一给定凸起高度的电压裕量 (库电压单位)

- **area** —电压裕量乘以凸起宽度
- **area_percent** —面积 slack 除以约束面积

(原书 *Figure 197: Area slack calculation*)

请参阅原版 PDF 插图；可将截图放入 **figures/** 目录后用 `\includegraphics` 替换。

图 16.19: 面积 slack 计算

(原书 *Figure 198: Area percent slack calculation*)

请参阅原版 PDF 插图；可将截图放入 **figures/** 目录后用 `\includegraphics` 替换。

图 16.20: 面积百分比 slack 计算

对低于失效阈值的凸起 slack 为正，高于则为负。

16.4.8 传播噪声特性

Propagated Noise Characteristics

足够大的输入噪声凸起会在单元输出引起输出噪声凸起，称为噪声传播 (noise propagation)。

(原书 *Figure 199: Noise propagation characterization*)

请参阅原版 PDF 插图；可将截图放入 **figures/** 目录后用 `\includegraphics` 替换。

图 16.21: 噪声传播表征

噪声传播通过在输入施加不同高度与宽度的凸起并测量输出凸起来表征。输出特性取决于输入凸起宽高、输出负载及 **time-peak-ratio**。

库语法支持两种指定传播效应的方式：

- **多项式**：输出凸起高度、宽度、**time-peak-ratio** 为输入凸起参数与负载的函数；每时序弧 12 个多项式（四种输出凸起类型 × 三种函数）
- **查找表**：输出高度与宽度为输入高度、输入宽度与负载的函数；每弧八个表（四种类型 × 高度/宽度）

time-peak-ratio 为 time-to-peak 除以凸起宽度，取值 0.0–1.0；对称凸起为 0.5。

表 16.4: 库中传播噪声的指定方法（表 40）

指定方法	库类型	库中如何指定
查找表	NLDM	每时序弧四对查找表（八种），按输入凸起高度、宽度与输出负载给出输出高度与宽度；分别对应 above low、below low、above high、below high
多项式	CCS/NLDM	输出高度、宽度、time-peak-ratio 的多项式函数

16.4.9 多 Aggressor 噪声合成

Composite Aggressor Noise

si_noise_composite_aggr_mode 控制多 aggressor 噪声合成方式。disabled（默认）不使用 composite aggressor 分析；设为 statistical 时，将多个 aggressor 的效应聚合为一个虚拟 aggressor，并用统计分析降低其对 victim 的总体效应。

si_xtalk_composite_aggr_mode 类似地控制串扰延时分析中的多 aggressor 合成。

16.4.10 噪声端点过滤与传播限制

Noise Endpoint Filtering and Propagation Limits

si_noise_endpoint_height_threshold_ratio 设置端点高度阈值比例，低于阈值的传播噪声可能被忽略。si_noise_limit_propagation_ratio 限制噪声沿路径传播的程度。

si_noise_slack_skip_disabled_arcs 控制是否跳过 disabled 弧上的噪声 slack 计算。

16.4.11 报告与调试噪声

Reporting and Debugging Noise

report_noise -violators 仅报告违例。report_noise_calculation -from pin -to pin 显示详细计算，包括各 aggressor 贡献、免疫曲线交点与传播阶段。

噪声相关属性：noise_slack、noise_peak、actual_noise_height、allowed_noise_height 等。

16.4.12 修复噪声违例

Fixing Noise Violations

常见修复方法：

- 增大 aggressor 与 victim 间距或插入屏蔽线
- 加大 victim 驱动或 victim 网负载电容（降低噪声敏感度）
- 插入缓冲器隔离长 victim 网
- 调整布线层或 via 策略
- 使用 fix_eco_timing 或物理 ECO 工具

参见

Library Compiler 文档中的噪声库语法；SolvNetPlus 上 PrimeTime SI 应用笔记；第 十四 章多电压分析（aggressor/victim 电源轨电压）；第 十三 章片上变异。

注意

PrimeTime SI 需要含耦合电容的 SPEF/GPD 寄生文件及支持噪声分析的库（推荐 CCS 噪声模型）。无 CCS 噪声模型时，部分分析（如双开关 slack）可能不受约束（INFINITY）。

Chapter 17

高级分析技术

Advanced Analysis Techniques

要了解 PrimeTime 高级时序分析技术，请参阅：

- 并行弧路径追踪（Parallel Arc Path Tracing）
- 对 Retain 弧的支持（Support for Retain Arcs）
- 异步逻辑分析（Asynchronous Logic Analysis）
- 三态总线分析（Three-State Bus Analysis）
- 数据到数据检查（Data-to-Data Checking）
- 基于锁存器设计中的时间借用（Time Borrowing in Latch-Based Designs）
- 高级锁存器分析（Advanced Latch Analysis）

17.1 并行弧路径追踪

Parallel Arc Path Tracing

为确保 `report_timing` 命令在路径追踪时，对于每一组具有相同时序 `sense` 的并行单元弧只使用其中的最差弧，将变量 `timing_report_use_worst_parallel_cell_arc` 设为 `true`。路径追踪行为如下所示。

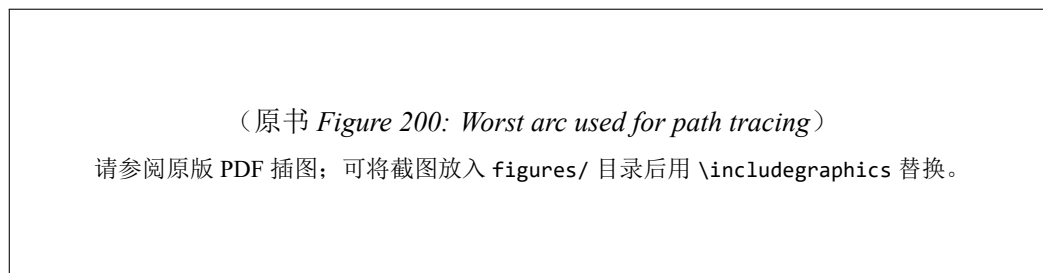


图 17.1: 路径追踪使用的最差弧

仅返回具有给定上升/下降边沿序列，并在逻辑传播过程中使用最差并行弧的路径。最差弧在最小延迟追踪中是最快弧，在最大延迟追踪中是最慢弧。边沿方向序列相同、但使用其他时序弧组合的次关键路径不会返回。不过，不同时序 *sense* 的弧仍视为彼此不同，因此，对于非单调门，具有不同上升/下降边沿序列的路径仍会返回。

当 *-nworst* 值较大时，这可大幅减少返回路径数量，提高分析效率。若未达到 *-nworst* 限制（例如报告到单一端点的路径），返回的时序路径数量可能较少。若 *-nworst* 已达上限（例如跨整个路径组报告），返回的路径（直至上限）可能对逻辑有更丰富的拓扑探索，有助于提高报告脚本、瓶颈分析及由 *timing_path* 集合驱动的 ECO 脚本的效率。

该特性在指定 *-nworst* 大于 1 时同时影响 *report_timing* 与 *get_timing_paths*。它只影响具有相同时序 *sense* 的并行弧。若并行弧具有不同 *sense*（例如 XOR 门上的 *positive_unate* 与 *negative_unate*），它们仍被视为不同的弧。该特性只改变报告阶段的路径追踪行为，不影响 *update_timing*；因此可随时更改变量值，无需付出重新更新时序的开销。

17.2 对 Retain 弧的支持

Support for Retain Arcs

PrimeTime 可从库文件加载 *retain* 弧、从 SDF 输入标注 *retain* 弧并报告这些弧。*Retain* 弧类似于 *hold* 检查弧，通常用于建模随机存取存储器（RAM）。它们定义在时钟引脚与 RAM 数据输出之间，且始终与父弧（同一对引脚之间的普通/默认延迟弧）并行定义。*Retain* 弧在时序分析期间不产生实际时序检查，而是作为与父弧并联的另一条延迟弧处理。

时钟到输出的 *retain* 弧保证 RAM 输出在时钟边沿后特定时间间隔内不改变。当 *retain* 弧延迟小于父弧时，*retain* 弧仅出现在最小延迟路径的时序报告中。当 *retain* 弧延迟大于父弧时，*retain* 弧也可出现在最大延迟路径报告中，且无错误或警告。

要报告逻辑库中所有时序弧（含 *retain* 弧），对 *report_lib* 使用 *-timing_arcs* 选项。使用 *check_timing retain* 检查 *retain* 弧延迟是否大于其父弧。

read_sdf 支持 *retain* 弧，但 *write_sdf* 的默认行为不支持。要写出 *retain* 弧，使用 SDF 3.0 格式（非默认 2.1），语法如下：

```
pt_shell> write_sdf -version 3.0 file
```

可使用以下函数访问映射弧的 *retain* 延迟信息：

- *min_rise_retain_delay*
- *min_fall_retain_delay*
- *max_rise_retain_delay*
- *max_fall_retain_delay*

17.3 异步逻辑分析

Asynchronous Logic Analysis

为简化含异步逻辑设计的时序验证，将异步逻辑隔离到独立模块，然后禁用这些模块的时序分析。

PrimeTime 不支持不使用全局时钟的自定时异步逻辑。在层次中隔离此类逻辑，并使用全时序门级仿真验证时序与功能。

PrimeTime 可分析满足以下条件的设计：

- 无组合反馈环；含触发器或锁存器的环可接受（组合反馈环自动断开）
- 不含无时钟存储元件（如 RS 锁存器）
- 每个寄存器时钟引脚有单一或多时钟扇入
- 每条路径起止寄存器时钟之间有已知且固定的相位关系（交互时钟须具有所有时钟波形重复的最小公倍周期）

（原书 *Figure 201: Base period of clocks*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.2: 时钟基周期

组合反馈环断开

Combinational Feedback Loop Breaking

组合反馈环是可经组合逻辑回溯到起点的路径。

（原书 *Figure 202: Combinational feedback loop*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.3: 组合反馈环

分析此类路径时，PrimeTime 须在环内某点断开环（停止追踪）。用 `check_timing -include loops` 检查设计中是否存在组合反馈环。

默认情况下，PrimeTime 识别每个反馈环并禁用环中一条时序弧（例如环内 NAND 门从某输入到某输出的弧）。某些情况下，被禁用弧会打断真实路径导致部分路径无法报告。

以下示例中，无法断开组合环中任一时序弧而不同时打断设计中的有效路径。

弧 #1 与 #4 不能断开，否则会打断有效路径 `ff1-u2-u1-ff3`。弧 #2 与 #3 不能断开，否则会打断有效路径 `ff2-u1-u2-ff4`。

反馈环中可在多处断开。若要在与默认不同的时序弧处断开，运行 `set_disable_timing` 在指定点断开。

（原书 *Figure 203: Loop breaking example*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.4: 环断开示例

Design Compiler 也使用环断开来分析时序。两工具断开位置可能不同，导致时序结果差异。若要保证一致，在 PrimeTime 中将 `timing_keep_loop_breaking_disabled_arcs` 设为 `true`，则 PrimeTime 继承 Design Compiler 生成的 `.ddc` 或 `.db` 中的环断开选择。默认该变量为 `false`；更改后需时序更新才能看到效果。

`report_disable_timing` 生成的报告区分 PrimeTime 禁用的弧与从 `.ddc/.db` 继承的弧。

无关时钟

Unrelated Clocks

有时设计存在无关时钟之间的路径。无关时钟频率不同且无合理基周期。PrimeTime 仍会尝试找基周期与相位关系，通常无实用价值。

（原书 *Figure 204: Unrelated clock waveforms*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.5: 无关时钟波形

时钟无法展开到合理基周期，得到的建立时间要求往往过严。此时可用动态仿真而非 PrimeTime 验证时序。

要从静态时序分析中排除异步路径以改善运行时间并避免虚假违例：

```
pt_shell> set_false_path -from [get_clocks clk40] \
          -to [get_clocks clk41]
```

若已知无关时钟间寄存器组合逻辑所需的最小/最大路径延迟，可对该路径用 `set_max_delay` 与 `set_min_delay` 指定约束。

17.4 三态总线分析

Three-State Bus Analysis

默认情况下，PrimeTime 检查三态总线设计中的建立与保持违例，检查到寄存器的最差延迟路径，确保锁存信号稳定且非未知（X）值，从而正确检查瞬态总线争用与瞬态浮空总线条件。

PrimeTime 认为三态单元上的禁用转换可在输出引起逻辑 1 或 0 延迟。默认考虑库中定义的 `three_state_disable` 与 `three_state_enable` 弧进行路径追踪。虽不同于传播 X，对静态时序的效果与传播 X 相同。

检查的限制

Limitations of the Checks

三态总线检查存在以下限制：

- PrimeTime 检查因总线争用或浮空引起的建立/保持错误可能性，不检查争用或浮空的逻辑与功耗违例
- 分析偏悲观；部分报告违例可能因状态依赖永不发生
- 部分仿真器中 Z 在电荷衰减时间后才传播为 X；PrimeTime 不建模该效应，用门延迟方程传播 Z 与 X，可能比某些仿真器更悲观

禁用检查

Disabling the Checks

若确认设计中不会发生总线争用或浮空，将以下变量设为 `true` 以禁用检查：

`timing_disable_bus_contention_check` 设为 `true` 时，禁用沿 `three_state_disable` 弧的最大延迟传播及沿 `three_state_enable` 弧的最小延迟传播。这些检查仅在瞬态总线争用期间有效。默认 `false`。

`timing_disable_floating_bus_check` 设为 `true` 时，禁用沿 `three_state_disable` 弧的最小延迟传播及沿 `three_state_enable` 弧的最大延迟传播。这些检查仅在浮空总线条件下有效。默认 `false`。

总线争用

Bus Contention

部分设计依赖多个三态驱动器控制的总线。多数设计中，稳态下（三态使能在各时钟周期达到稳态值时）不会有两个输出逻辑不同的驱动器同时使能。多驱动器驱动同一总线称为总线争用。

虽可设计电路使稳态无争用，仍可能存在瞬态争用：总线控制从一驱动器切换到另一驱动器的短暂时段内，若争用驱动器施加冲突逻辑值，总线逻辑值为未知（X）。静态时序分析中，建立检查确保该 X 值（在瞬态争用期间假定）不被锁存；建立检查从总线稳定时刻量起。

浮空总线

Floating Buses

三态总线配置可能出现浮空：无驱动器使能时总线立即为未知（X）。静态时序分析中，保持检查确保切换到浮空模式时假定的 X 不被锁存；保持检查量至总线开始浮空的时刻。

三态缓冲器

Three-State Buffers

库中用两种时序弧类型描述三态缓冲器行为：

- **three_state_disable**: 三态引脚从高/低态到高阻态所需时间
- **three_state_enable**: 三态引脚从高阻态到高/低态所需时间

执行瞬态总线争用检查

Performing Transient Bus Contention Checks

(原书 *Figure 205: Circuit example*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.6: 电路示例

(原书 *Figure 206: Transient bus contention check*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.7: 瞬态总线争用检查

图 205 中，若 EN1 在 EN2 之后关断，可能存在短暂争用。信号在争用结束、新总线值沿路径传播前为 X。若 I1=0、I2=1，波形如图 206。

FF4/CP 的建立违例仅当考虑 `three_state_disable` 弧延迟时出现。到 Z 态的转换须作为可能到逻辑 0 或 1 的转换传播。保持违例仅当考虑 `three_state_enable` 弧延迟时出现。

争用区域由任一侧总线驱动器最小 `three_state_enable` 弧延迟与另一侧最大 `three_state_disable` 弧延迟界定。若确认此类区域不会发生，将 `timing_disable_bus_contention_check` 设为 `true`，PrimeTime 在保持检查中忽略 `three_state_enable` 弧延迟，在建立检查中忽略 `three_state_disable` 弧延迟。即使设为 `true`，浮空总线区域仍考虑 `three_state_enable` 用于建立、`three_state_disable` 用于保持（见“执行浮空总线检查”）。

执行浮空总线检查

Performing Floating Bus Checks

浮空总线指总线处于有效逻辑值后开始浮空。时序分析确保浮空不会将 X 传播到寄存器数据引脚，直至保持检查之后。PrimeTime 对建立/保持关系与其他数据信号相同假定。

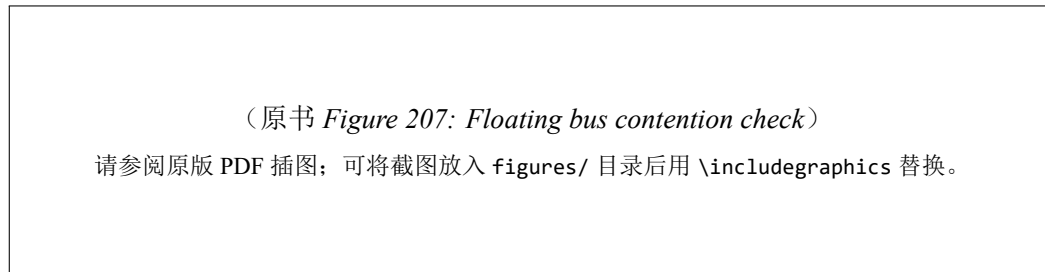


图 17.8: 浮空总线检查

图 207 中，FF4/CP 的保持违例仅当考虑 `three_state_disable` 弧延迟时出现；建立违例仅当考虑 `three_state_enable` 弧延迟时出现。

默认情况下 PrimeTime 检查浮空区域引起的所有建立/保持违例。浮空区域由任一侧最小 `three_state_disable` 与另一侧最大 `three_state_enable` 界定。PrimeTime 忽略电荷衰减（假定总线浮空时逻辑值立即变为 X）。

若确认浮空区域不会发生，将 `timing_disable_floating_bus_check` 设为 `true`。即使设为 `true`，争用区域仍考虑 `three_state_enable` 用于建立、`three_state_disable` 用于保持。

17.5 数据到数据检查

Data-to-Data Checking

PrimeTime 可在任意两个引脚之间对两条均非时钟的数据信号执行建立与保持检查。适用于：

- 握手接口逻辑约束
- 异步或自定时电路接口约束
- 难以用 `create_clock` 指定的非常规时钟波形信号约束
- 总线间 skew 约束
- 异步 `preset/clear` 输入引脚间的 `recovery/removal` 约束

两条数据（非时钟）信号之间的时序约束称为非顺序约束。可用 `set_data_check` 定义，或在 Library Compiler 中为单元设置非顺序约束。仅在上述情形使用数据检查；不可替代标准顺序检查。

数据检查示例

Data Check Examples

单元有两个数据输入 D1、D2。D2 上升沿可能是锁存 D1 的有效边沿。D1 为被约束引脚，D2 为相关引脚。顺序检查中 D2 视为时钟引脚；但有时宜将 D2 视为数据信号而非时钟。

(原书 *Figure 208: Simple data check example*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.9: 简单数据检查示例

D1 须在有效边沿前稳定一定建立时间，有效边沿后稳定一定保持时间。若库单元未定义非顺序约束，可在 PrimeTime 中定义：

```
pt_shell> set_data_check -rise_from D2 -to D1 -setup 3.5
pt_shell> set_data_check -rise_from D2 -to D1 -hold 6.0
```

“from” 为相关引脚，“to” 为被约束引脚。若对相关引脚上升沿与下降沿均适用，用 -from 代替 -rise_from/-fall_from：

```
pt_shell> set_data_check -from D2 -to D1 -setup 3.5
pt_shell> set_data_check -from D2 -to D1 -hold 6.0
```

(原书 *Figure 209: Data checks on rising and falling edges*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.10: 上升/下降沿数据检查

通过仅指定上升沿建立检查与下降沿保持检查可定义 no-change 数据检查：

```
pt_shell> set_data_check -rise_from D2 -to D1 -setup 3.5
pt_shell> set_data_check -fall_from D2 -to D1 -hold 3.0
```

PrimeTime 将其解释为正向脉冲上的 no-change 检查。

(原书 *Figure 210: No-change data check*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.11: No-change 数据检查

数据检查为非顺序，不会打断时序路径。图 211 中 D1 与 D2 之间的数据检查不中断虚线箭头所示路径。若将 D2 定义为时钟，检查变为顺序，路径在 D1 处终止。

(原书 *Figure 211: Timing paths not broken by data checks*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.12: 数据检查不中断时序路径

可将数据检查引脚指定为 `report_timing` 的路径端点；PrimeTime 报告适用于该引脚的数据检查。图 211 电路中, `report_timing -to dchk/D1` 生成数据检查报告, `report_timing -through dchk/D1` 报告经该引脚的标准路径时序。

用 `remove_data_check` 移除 `set_data_check` 设置的数据检查。

数据检查与时钟域

Data Checks and Clock Domains

被约束引脚或相关引脚上的信号可来自不同时钟域。PrimeTime 分别检查信号路径并归入不同时钟组，与标准顺序检查相同。

若相关引脚有多时钟域信号，可用 `set_data_check` 的 `-clock clock_name` 指定分析所用时钟域，或禁用除目标时钟外的所有时钟。

基于库的数据检查

Library-Based Data Checks

若库单元定义了非顺序时序约束且相关引脚信号在 PrimeTime 中未定义为时钟，PrimeTime 执行数据检查。若定义为时钟，非顺序检查转为顺序检查，该时钟信号可继续传播。若存在从相关引脚出发的组合弧（如集成时钟门控单元），时钟可沿该弧继续传播。

在 Library Compiler 中通过指定相关引脚并为被约束引脚分配以下 `timing_type` 属性定义非顺序约束：

- `non_seq_setup_rising`、`non_seq_setup_falling`
- `non_seq_hold_rising`、`non_seq_hold_falling`

库中定义非顺序约束比 `set_data_check` 更准确，因建立/保持时间可对被约束引脚与相关引脚 slew 敏感；`set_data_check` 对 slew 不敏感。

对库中定义的数据检查, 用 `set_data_check ... -clock clock_name` 指定相关引脚时钟域。`remove_data_check` 不移除库中定义的数据检查。

17.6 基于锁存器设计中的时间借用

Time Borrowing in Latch-Based Designs

电平敏感锁存器给静态时序分析带来特殊挑战。时间借用（亦称 **cycle stealing**）使锁存器设计相对触发器设计有优势：电平敏感锁存器在门控输入有效期间透明，可放宽同步设计边到边的时序要求，但因多相时钟与缺乏“硬”时钟边沿而更难控制时序。

从逻辑级借用时间

Borrowing Time From Logic Stages

在电平敏感锁存器设计中，路径可利用锁存器在门控有效期间的透明性从下一级逻辑“借”时间。图 212 为简单两相时钟方案下的锁存器级。

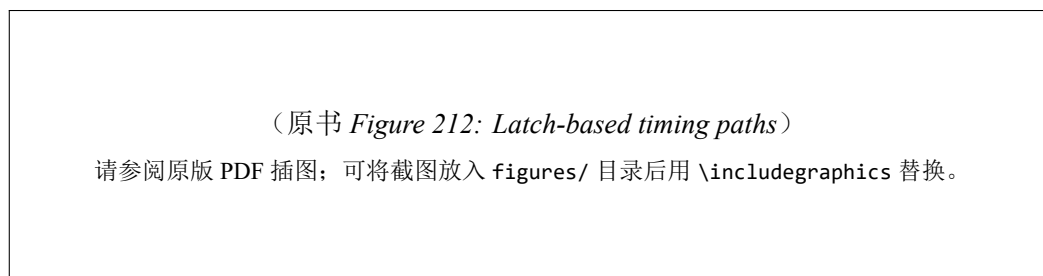


图 17.13: 基于锁存器的时序路径

电平敏感锁存器设计允许组合路径延迟超过可用周期时间，若后续锁存器到锁存器级有更短路径补偿。两相设计中，锁存器到锁存器路径可用时间为半个时钟周期。

图 212 中 U1、U2、U3 为正电平敏感锁存器（G=1 有效），P1、P2 为组合逻辑。暂设库建立时间为 0、透明模式 D 到 Q 延迟为 0。对正电平敏感锁存器，PrimeTime 以上升沿（opening edge）为参考边沿。

P1=8.92、P2=0.77。U2 处数据在 phi2 上升沿后到达，看似违例；但 U2 透明 5 ns 且 P2 更短，P1 可从 U2-U3 路径借 slack 3.92 ns。故 $P1+P2=9.69 < U3$ 要求时间 10.00。

锁存器时序报告

Latch Timing Reports

若数据在端点锁存器 opening 边沿之前到达，PrimeTime 与触发器一样建模：opening 边沿捕获数据，同一边沿在下一路径起点启动数据。

若数据在锁存器透明期间（opening edge 与 closing edge 之间）到达，PrimeTime 将下一级建模为从锁存器数据引脚而非时钟引脚启动；此时没有时序违例，该路径段的 slack 视为零。该级借用的时间成为下一级的启动时间，但仍受“最大借用时间调整”约束。若数据在 closing edge 之后到达，则构成违例。

图 213 说明 PrimeTime 如何对 P1 不同组合延迟计算并报告 slack（未考虑锁存器建立时间、latency、uncertainty 与 CRPR）。

report_timing 报告透明锁存器路径时包含“time borrowed from endpoint”与“time given to start-point”，与图 212 对应。

(原书 *Figure 213: Latch-based timing path slack calculation*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.14: 锁存器路径 slack 计算

最大借用时间调整

Maximum Borrow Time Adjustments

端点锁存器可借用的最大时间基于 `create_clock` 定义的时钟脉冲宽度（opening 到 closing）减去锁存器库建立时间（图 214）。

(原书 *Figure 214–217*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 17.15: 最大借用时间调整（建立、latency、uncertainty、CRPR）

为更准确，PrimeTime 进一步根据时钟 latency、uncertainty 与时钟重汇聚悲观移除（CRPR）调整。Latency 可由 `set_clock_latency` 定义或由传播延迟计算（`set_propagated_clock`）。上升/下降沿 latency 不同会影响脉冲宽度与最大借用时间。

Uncertainty 由 `set_clock_uncertainty` 定义；建立检查考虑捕获时钟边沿最早可能到达。上升/下降 uncertainty 差异影响脉冲宽度与最大借用时间。

将 `timing_remove_clock_reconvergence_pessimism` 设为 true 启用 CRPR。CRPR 像 uncertainty 一样移动时钟边沿时间，但方向相反（使分析更少悲观）。上升/下降 CRPR 差异影响脉冲宽度与最大借用时间。

计算最大允许借用时间时，PrimeTime 从脉冲宽度出发，再按 latency 差、uncertainty 差、CRPR 差与端点库建立时间调整。`report_timing` 在 Time Borrowing Information 段报告各项。

借用时间与给予时间

Time Borrowed and Time Given

PrimeTime 根据 opening 时钟边沿到达时间计算借用量，先按路径相关 uncertainty 与 CRPR 调整 opening 边沿到达时间，再与数据引脚到达时间比较确定端点借用量。

若 opening edge 存在 uncertainty 与 CRPR，端点借用量与给予下一级起点的时间可能不同。确定给予时间时，PrimeTime 会扣除 uncertainty 与 CRPR 调整，使透明模式下的下一级恰在数据到达锁存器数据引脚时启动（图 218）。禁用 `timing_early_launch_at_borrowing_latches` 时，由于工具

有意采用较晚的时钟 **latency** 来启动下一级，数据到达时间与启动时间并不完全相同；启用 **CRPR** 时建议采用这一模式。在此模式下，不对给予下一级起点的时间应用 **CRPR** 调整。

（原书 *Figure 218–219*）

请参阅原版 PDF 插图；可将截图放入 **figures/** 目录后用 **\includegraphics** 替换。

图 17.16: 透明模式借用/给予时间与负借用

在 **D-to-Q** 延迟大于 **clock-to-Q** 且到达时间非常接近时钟边沿时，数据在 **opening** 之前到达仍可能借用（负借用）；下一路径出发时间由到达时间与 **D-to-Q** 决定。借用量为负，报告为 “time given to endpoint” / “time borrowed from startpoint”，且不超过 **D-to-Q** 与 **clock-to-Q** 之差。

限制时间借用

Limiting Time Borrowing

set_max_time_borrow 在设计指定范围内对所有锁存器端点限制借用量。在指定端点，借用限制为指定值与默认调整脉冲宽度中的较小者。

若 **default_max_time_borrow** 为 0，不允许借用，锁存器按触发器分析。若为负，数据须在时钟 **opening** 之前稳定（如门控时钟锁存器的使能输入）。

用 **report_exceptions** 显示 **max_time_borrow** 属性。用 **remove_max_time_borrow** 移除限制。

示例：对 **clk** 时钟的所有锁存器设置最大借用 2.5：

```
pt_shell> create_clock -period 10 -waveform {0 5} clk
pt_shell> set_max_time_borrow 2.5 [get_clocks clk]
pt_shell> report_timing -to latch1/D
```

max_time_borrow 将可用脉冲从 5.0 有效减至 2.5。端点借用量通常等于下一起点给予量（同锁存器 **D** 引脚）；因 **opening** 边沿 **uncertainty/CRPR** 调整可能有微小差异。

17.7 高级锁存器分析

Advanced Latch Analysis

默认通过锁存器间单段路径报告时序违例；对借用（或失败）锁存器引入借用路径，为结束于下一锁存器或触发器 **D** 引脚的单段路径。单段特性可能妨碍锁存器设计的时序修复与功耗优化。

可选地不将路径打断为段而分析经锁存器的路径：透明锁存器同时是 **throughpath** 与端点；每锁存器可有结束于 **D** 的路径及穿过锁存器到另一端点的路径；锁存器时钟引脚也可作为路径起点。高级锁存器分析提供经锁存器时序路径的全局可见性，改善功耗与设计优化。

表 17.1: 高级锁存器分析命令与变量（表 41）

命令或变量	用途
timing_enable_through_paths	启用高级锁存器分析
set_latch_loop_breaker	在指定点断开锁存器环，覆盖工具默认选择
get_latch_loop_groups	返回锁存器环组中 D 引脚集合的列表
report_latch_loop_groups	报告参与透明锁存器环的 D 引脚信息
timing_through_path_max_segments	每条路径分析的最大连续锁存器段数
timing_report_skip_early_paths_at_intermediate_latches	跳过在中间锁存器过早到达的 throughpath 报告
report_timing -trace_latch_borrow/-trace_latch_forward	经环断开锁存器向后/向前追踪

启用高级锁存器分析

Enabling Advanced Latch Analysis

将 timing_enable_through_paths 设为 true（默认 false）。

断开环

Breaking Loops

将 throughpath 视为时序路径时，含环电路的最差路径难以求得。选定锁存器为环断开锁存器，路径不穿过它们。默认由 PrimeTime 根据连通性选择少量环断开锁存器，不考虑到达值。

report_timing 不找经环断开锁存器的路径；每次报告基于不经过任何环断开锁存器的最差路径。

手动指定：

```
set_latch_loop_breaker -pin pin_list
```

适用情形：锁存器不在关键路径；读/写端口不同时使用的寄存器文件等；脉冲时钟或透明窗口很小的锁存器；环中其他锁存器不应作为环断开器。

查询 is_latch_loop_breaker 属性（D 引脚为 true）：

```
get_pins -hierarchical * -filter "@is_latch_loop_breaker"
```

锁存器环组

Latch Loop Groups

单个锁存器可属于多个锁存器环；所有相交环的集合称为锁存器环组。组内任意两锁存器至少共处于一个环。

get_latch_loop_groups [-of_objects list] [-loop_breakers_only] 返回 Tcl 列表，每项为一环组的 D 引脚集合。report_latch_loop_groups 列出 D 引脚、环组编号及属性（b 环断开、p 长路径断开但不在环中、u 用户指定断开、a 用户要求避免断开）。不在环中的 D 引脚显示“NA”。

默认每条路径最多分析五个连续锁存器；无环处也可能引入额外环断开器限制路径长度。timing_through_path_max_segments 可覆盖；设为 0 表示无限制（可能运行时间极长）。

(原书 *Figure 220: Single latch loop group with three latches*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.17: 三锁存器单环组

应用于锁存器路径的时序例外

Timing Exceptions Applied to Latch Paths

启用高级锁存器分析时，时序例外须在一个路径段内满足；所有 `-from/-through/-to` 须在同一段内匹配才生效。

- **False path:** 任一段满足 false path，整条 throughpath 为 false throughpath，不检查路径末端约束（图 221）
- **Multicycle path:** 任一段满足多周期例外，影响下游锁存器透明窗口与路径端点要求时间；不同段可有不同多周期例外（图 222）
- **Max/min delay:** 最大延迟例外在终点为锁存器 D 时仅作用于该段；不满足的 throughpath 不受影响。最小延迟例外仅影响单段保持，throughpath 不做保持检查
- **Clock groups:** 互斥时钟组对 throughpath 生效；两时钟无通信则 throughpath 不受约束，即使仅在中间锁存器依赖互斥时钟（图 223）。不能用 clock-to-clock false path 完全替代 clock groups（false path 须在同一段内满足）；hold 可用 clock-to-clock false path（throughpath 不做 hold）
- **Throughpath 上的例外指定：**多说明符例外须在同一段内满足；跨段指定无效（图 224）。用 `report_exceptions -ignored` 列出无效例外

(原书 *Figure 221–224*)
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 17.18: Throughpath 上的 false/multicycle/时钟组/错误指定例外

报告经锁存器的路径

Reporting Paths Through Latches

传递 slack 估计改善某引脚所需时序余量。找决定某引脚传递 slack 的关键路径：

```
report_timing -through pin_name
```

throughpath 报告描述每个中间透明窗口。将 `timing_report_skip_early_paths_at_intermediate_latches` 设为 `true` 可过滤在中间锁存器过早到达的 throughpath (图 226)。

无 `-to` 时, 工具在计算环断开锁存器要求时间时考虑下游时序。`-trace_latch_forward` 类似 `-trace_latch_borrow` 但向前追踪 (图 227)。

最差负 **slack** 与总负 **slack** 计算

Calculation of the Worst and Total Negative Slack

D 引脚属性 `max_rise_local_slack`、`max_fall_local_slack` 返回锁存器路径局部 **slack** (不考虑透明锁存器引脚下游约束)。`max_fall_slack/max_slack` 考虑下游约束。

WNS 为最差违例路径端点 **slack** (单段或 throughpath); 无违例则为 0。WNS 为所有 `max_rise_local_slack` 与 `max_fall_local_slack` 的最小值。

TNS 计算:

$$\text{TNS} = \sum_{\text{所有时序端点}} \min(0, \min(\text{max_rise_local_slack}, \text{max_fall_local_slack}))$$

归一化 **slack** 分析

Normalized Slack Analysis

除非为恢复路径或含环断开/路径断开锁存器的路径, 可用归一化 **slack** 估算路径最大频率 (图 228)。工具用理想时钟边沿计算允许传播延迟 (忽略建立时间、`uncertainty`、`latency`); 允许延迟可为半周期、全周期或多周期, 跨时钟域时更复杂。

`report_timing -normalized_slack` 按归一化 **slack** 报告与排序。

查找恢复路径

Finding Recovered Paths

当锁存器到达时间晚于透明窗口 closing 边沿减去 `uncertainty` 与建立时间时, 路径被恢复。查询时序路径属性 `is_recovered`; 在中间锁存器被恢复的路径上为 `true`。

参见

路径例外与多周期路径, 见第 八 章; 时钟定义见第 七 章。

Chapter 18

约束一致性

Constraint Consistency

约束一致性功能用于全芯片或模块级时序约束的分析与调试。基于规则检查的设计分析可识别缺失、无效、多余、低效或冲突的多种约束问题。约束一致性提供直观全面的图形界面（GUI），便于进一步调试。可使用大量内置规则运行分析，也可定义自定义一致性检查以扩展检查范围。

要了解约束一致性，请参阅：

- 约束一致性概述
- 设计一致性检查
- 层次一致性检查
- 相关性一致性检查
- 附加分析特性
- 图形用户界面
- 教程

18.1 约束一致性概述

Constraint Consistency Overview

要了解约束一致性功能，请参阅：

- 设计流程中的约束
- 调试特性
- 分析与调试方法学
- 启动约束一致性会话
- 支持的约束
- 报告约束接受情况

- 例外优先级顺序
- 分析违例
- 抑制违例
- `report_constraint_analysis` 命令
- 使用属性
- 自定义规则与报告
- 创建与使用用户定义规则
- 规则相关命令
- 读取不完整或不匹配网表的设计

设计流程中的约束

Constraints in the Design Flow

正确完整的约束对逻辑综合、布局、布线等时序驱动实现步骤至关重要。约束描述工作环境及有效静态时序分析（独立或嵌入优化工具）所需信息。时序约束通常以 Synopsys Design Constraints (SDC) 文件或更一般的 Tcl 脚本指定。错误或不完整约束可导致设计迭代浪费甚至硅片失效。应尽早发现并修复约束问题以降低风险、提高生产率。

约束一致性在门级分析约束，确定时钟、数据信号与常量在设计中的传播位置与方式。约束在各步骤常被修改（例如布局前约束经修改得到布局、CTS、布线、芯片组装约束）。每次创建或修改约束后应分析正确性与一致性。

约束一致性分析设计及其约束的正确性、完整性与一致性。读入并链接 Verilog 格式的顶层或模块设计，用与 Design Compiler 或 PrimeTime 相同的 SDC 命令施加约束。工具应用 SDC 约束，忽略约束脚本中的非 SDC 工具专用命令。读入、链接并约束后，可运行一条或多条约束一致性命令。第一步运行 `analyze_design`，检查设计与约束并生成可在违例浏览器中查看的摘要报告。

从摘要列表可点击违例查看详情；从详细报告可点击链接定位 SDC 脚本中的约束命令、查看原理图相关区域或获取修复建议。检测的违例类型包括：

- 错误电容值、错误指定的时钟/生成时钟
- 错误的时钟 latency 与转换时间
- 错误或未指定的驱动单元
- 错误指定的时序例外、输入/输出延迟
- 网表问题（如冲突驱动器）、冲突 case analysis 值

进一步诊断可用：`analyze_paths`、`analyze_clock_networks`、`analyze_unclocked_pins`、`report_case_details`；`report_analysis_coverage`、`report_exceptions`、`compare_block_to_top` 有助于调试多场景约束、多重时序例外与层次设计中的冲突约束。

调试特性

Debugging Features

重要特性包括：覆盖广泛潜在约束问题的规则集；用户定义规则；单进程多场景分析；数百万实例级设计的高效容量与性能；强大诊断与详细分析命令；丰富的集合命令与对象属性；符合 SDC 2.0 的规则检查器；支持循环/变量/条件的交互式 Tcl shell；含源文件名与行号的 GUI 输出；与 DC/ICC 等实现工具在约束解释、时钟/信号/常量传播及界面方面的高度一致。

分析与调试方法学

Analysis and Debugging Methodology

输入：Verilog 网表、Synopsys .db 库、SDC 或 PrimeTime Tcl 约束、Tcl 控制脚本。

输出：文本报告；以及可用于交互式调试、自定义报告或条件脚本的查询命令与对象属性。

典型流程：

1. 启动：pt_shell -constraints（见“启动会话”）
2. 读设计：set_app_var search_path、link_path、read_verilog
3. 链接：current_design、link_design
4. 各场景施加约束：source（Tcl）或 read_sdc（SDC）
5. 分析：analyze_design
6. 诊断：专用命令（见“附加分析特性”）
7. 按需重复分析与诊断

（原书 *Figure 229: Typical Constraint Consistency Flow*）
请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 18.1: 典型约束一致性流程

启动约束一致性会话

Starting a Constraint Consistency Session

约束一致性为 GUI 驱动，在 Linux 下运行。子主题：启动会话、许可证、Setup 文件、命令日志文件。

启动会话 使用 pt_shell -constraints。自动检出 PrimeTime SI 许可证，启动 GUI，出现 ptc_shell> 提示符。若启动失败，检查：安装正确、PrimeTime 在 PATH 中、许可证服务器运行、许可证文件有效且含 PrimeTime SI 许可证。结束会话：quit 或 exit。

许可证 启动 `ptc_shell` 需 PrimeTime SI 许可证；会话退出自动检入。另需同一服务器上有 PrimeTime 许可证（不检出，但必须存在）。

Setup 文件 每次启动执行 `.synopsys_gca.setup`，按序检查：Synopsys 安装 `admin/setup`、用户主目录、当前工作目录。用 `-no_init` 可禁止执行 `setup` 文件。

命令日志 会话历史写入 `ptc_shell_command.log`（覆盖同名文件）。可在 `setup` 中用 `sh_command_log_file` 指定其他名称（会话中不可改）。

支持的约束

Supported Constraints

支持 SDC 与 Tcl 格式全部约束。Tcl 约束用 `source` 加载；SDC 用 `read_sdc`。支持多场景：先 `create_scenario`，再为各场景加载约束。

示例（Tcl 片段）：

```
foreach_in_collection i [all_clocks] {
    echo [format "Clock : '%s'" [get_attribute $i full_name]]
    set_clock_uncertainty 1.5 $i
}
```

多场景加载示例：

```
set all_scenarios {system myscenario_1 myscenario_2}
foreach scenario $all_scenarios {
    create_scenario ${scenario}
    source ./scripts/common.tcl
    source ./scripts/${scenario}_constraints.tcl
}
```

报告约束接受情况

Reporting Constraint Acceptance

`report_constraint_analysis -include statistics` 报告分析接受的约束，快速确认约束是否正确加载。输出含各约束类型的 Accepted/Rejected/Total 计数。

例外优先级顺序

Exception Order of Precedence

冲突时，约束一致性按路径（非按命令）独立应用优先级规则。例如 `set_max_delay -from A` 与 `set_false_path -to B`：对起自 A 且止于 B 的路径，`set_false_path` 优先；仅起自 A 的路径仍受 `set_max_delay` 约束。

例外类型优先级（高到低）：

1. `set_false_path`
2. `set_max_delay` / `set_min_delay`

3. set_multicycle_path

不视为冲突可同时有效的组合：两个 set_false_path; set_min_delay 与 set_max_delay; set_multicycle_path 的 setup 与 hold。

路径说明优先级（高到低）：

1. -from/-rise_from/-fall_from pin
2. -to/-rise_to/-fall_to pin
3. -through/-rise_through/-fall_through
4. -from/-rise_from/-fall_from clock
5. -to/-rise_to/-fall_to clock

组合示例（高到低）：-from pin -to pin > -from pin -to clock > -from pin > -from clock -to pin > -to pin > -from clock -to clock > -from clock > -to clock。

用 report_exceptions -ignored 列出被忽略的时序例外。

分析违例

Analyzing a Violation

在违例浏览器中选择违例，在信息窗格查看详情。

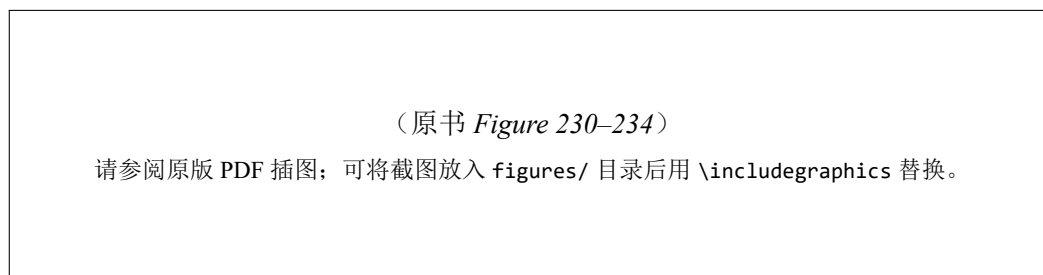


图 18.2: 违例浏览器、信息窗格链接、原理图与引脚标注

可用 Debugging Help、Fix Suggestion 链接或 SDC 源文件链接进一步调试；查阅在线帮助中的规则参考。

抑制违例

Suppressing Violations

可完全禁用规则，或对特定实例豁免（**waiver**）违例。GUI 与 create_waiver、report_waiver、remove_waiver、write_waiver 支持创建、报告、删除与写出豁免。

抑制流程：可在 analyze_design 前后抑制。分析后抑制的违例在浏览器中带特殊图标；分析前完全禁用的规则不出现在报告中；实例豁免仍报告但带图标。豁免后浏览器状态立即更新。

禁用规则：违例浏览器右键 “Disable rule”，或 Design > Waiver Configuration 取消勾选，或 disable_rule。

豁免特定违例：浏览器右键 “Waive violation”，或 Waiver Configuration 中 Create waiver，或 create_waiver。hide_waived_violations false 时 View 菜单可切换隐藏已豁免违例。

(原书 *Figure 235–245*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 18.3: 违例抑制流程、豁免配置、已豁免违例图标

create_waiver 命令 可指定多个 `-condition` 与 `-not_condition`；只有全部条件均满足时才抑制违例。每个选项的实参必须是二元素列表：规则参数名，以及网表对象或约束的集合。

示例 1——豁免 CLK* 时钟上的 CLK_0026:

```
ptc_shell> create_waiver -rule CLK_0026 \
  -condition [list "clock" [get_clocks CLK*]]
```

示例 2——豁免除 CLK3 外所有时钟:

```
ptc_shell> create_waiver -rule CLK_0026 \
  -not_condition [list "clock" [get_clocks CLK3]]
```

示例 3——多参数 (时钟 CLK1/CLK2, 引脚非 U1/A、U1/B):

```
ptc_shell> create_waiver -rule CLK_0003 \
  -condition [list "clock" [get_clocks {CLK1 CLK2}]] \
  -not_condition [list "pin" [get_pins U1/A] [get_pins U1/B]]
```

示例 5——例外参数:

```
ptc_shell> create_waiver -rule EXC_0001 \
  -condition [list "exception" [get_exceptions -from U1/A -to U2/Z]] \
  -condition [list "rise_or_fall" "rise"]
```

用 `report_rule -parameters rule_name` 查询规则参数。B2T 与 S2S 规则豁免需指定 `-design1/-scenario1/-design2/-scenario2` 及 `top/block` 对象上下文。

实例级豁免：层次浏览器右键 Waive Instance，或 Waiver Configuration > Instance Waivers；命令行用 `-cells`：

```
ptc_shell> create_waiver -name my_waiver_1 -cells [get_cell U1/U1]
```

用法：实例级豁免应用于当前场景；`-all_scenarios` 应用于设计所有场景。仅当违例相关对象完全包含在指定实例内才豁免；部分通用规则与用户定义规则不支持实例级豁免。

`write_waiver -output file [-force]` 将豁免写入 Tcl 供后续 `source` 恢复。

report_constraint_analysis 命令

report_constraint_analysis Command

生成汇总设计中违例、用户消息与规则信息的文本报告；按分析类型、设计、场景排序。

- `-include`: violations、statistics、user_messages、rule_info 等

- `-style full|summary`
- `-rules/-rule_types` 过滤
- `-format csv` 输出 CSV

多设计层次比较结果在报告中以 Top/Block 场景分组显示 B2T 违例。

使用属性

Using Attributes

属性是与设计对象关联的字符串或值。命令：`list_attributes`、`get_attribute`、`report_attribute`。`get_attribute` 限于单对象单集合。

属性组（表 43）包括：`cell`、`clock`、`clock_group`、`design`、`exception`、`pin`、`port`、`net`、`rule`、`rule_violation`、`scenario`、`timing_arc`、`lib_*` 等。默认只读。

`get_timing_arcs` 创建时序弧集合供自定义报告；`get_lib_timing_arcs` 创建库时序弧集合。示例——查找 U1 上被禁用的 `positive_unate` 弧：

```
ptc_shell> set arcs [get_timing_arcs -of_objects U1 \
  -filter "sense == positive_unate"]
ptc_shell> foreach_in_collection arc $arcs {
  echo [get_attribute $arc is_disabled]
}
```

自定义规则与报告

Customizing Rules and Reports

可用 `define_rule`、`define_ruleset` 扩展检查范围；`report_rule`、`enable_rule/disable_rule`、`set_rule_property` 管理规则（含容差 `tolerance`）。`analyze_design -rules` 可限定检查的规则子集。

创建与使用用户定义规则

Creating and Using User-Defined Rules

用户定义规则用 Tcl 过程检测自定义条件；可访问设计与约束对象属性。定义后通过 `analyze_design` 或专用规则集运行。详见在线帮助“User-Defined Rules”。

规则相关命令

Rule-Related Commands

- `report_rule [pattern]` 一规则状态与描述
- `enable_rule/disable_rule` 一启用/禁用
- `set_rule_property -tolerance` 一B2T/S2S 比较容差（默认 1e-5%）
- `analyze_design`、`compare_block_to_top`、`compare_constraints` 一各检查不同规则集

读取不完整或不匹配网表的设计

Reading Designs With Incomplete or Mismatched Netlists

`read_verilog` 与 `link_design` 可处理部分不匹配；`report_design_mismatch` 报告主实例与引用不一致（如 DMM-038 引脚不存在、DMM-905 库中找不到单元）。应在分析前尽量解决链接错误。

18.2 设计一致性检查

Design Consistency Checking

主题：规则与违例、内置规则。

规则与违例

Rules and Violations

对已加载、已链接、已约束的设计检查设计与约束相关条件。规则（**rule**）为检查项；违例（**violation**）为未满足条件的一次出现。违例严重级别：Information、Warning、Error。规则代码为三到四字母 + 下划线 + 四位数字（如 `CLK_0003`：生成时钟主源引脚无时钟）。

`analyze_design` 检查标准规则集；`compare_block_to_top` 与 `compare_constraints` 各检查不同规则集。可 `disable_rule` 禁用规则；部分规则默认禁用以节省运行时间。豁免（**waiver**）在特定条件下跳过检查。除内置规则外可定义用户规则。

内置规则

Built-In Rules

启动时自动加载表 44 所示内置规则：

表 18.1: 内置规则分类（表 44）

边界条件	CAP_xxxx 电容；DRV_xxxx 驱动；EXD_xxxx 外部延迟
约束/例外分析	CAS_xxxx case；EXC_xxxx 时序例外
时钟	CGR_xxxx 组；CLK_xxxx 属性；CNL_xxxx 网络 latency； CTR_xxxx 转换；CSL_xxxx 源 latency；UNC_xxxx uncertainty
通用	CMP_xxxx 工具兼容；DES_xxxx 设计约束；HIER_xxxx 层次； LOOP_xxxx 环；PRF_xxxx 性能；NTL_xxxx 网表；UNT_xxxx 库单位

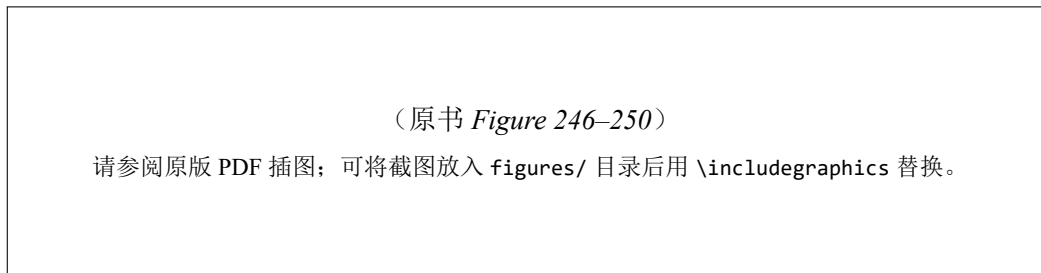
详细规则描述见 `Constraint Consistency` 在线帮助。示例：`CLK_0003` 表示生成时钟因主源引脚无时钟而无法展开。

18.3 层次一致性检查

Hierarchical Consistency Checking

设计团队采用多种策略创建模块级与顶层约束；自底向上流程中模块约束可传播到顶层，迭代中顶层与模块均可能手工插入新约束。`compare_block_to_top` 比较模块级与顶层约束是否匹配。

各模块带有综合/布局所用约束（时钟、例外、边界条件）。层次一致性对同一网表加载两套约束并比较，生成 B2T 规则违例，可在违例浏览器中调试；单会话可保留多块 B2T 结果。



（原书 *Figure 246–250*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 18.4: 多块违例浏览器、B2T 原理图与双 SDC 查看器

比较模块级与顶层约束

Comparing Block- and Top-Level Constraints

复用 IP 或块时须保证并入更大设计后块约束仍有效。`compare_block_to_top` 找层次约束不一致；`analyze_design` 检查单套约束内部问题。

方法学：

- 保持多个链接设计：默认 `link_design` 会移除先前链接设计；B2T 流程需同时保留块设计（块约束）与顶层设计（顶约束）。用 `link_design -add; current_design` 切换。
- 流程脚本示例（Example 56）：

```
set_app_var search_path [list . ${TOP_DIR}/libs ${TOP_DIR}/design]
set_app_var link_path [list * libs.db]
read_verilog ${TOP_DIR}/design/ChipLevel_no2blocks.v
read_verilog ${TOP_DIR}/design/Multiply16x16.v
link_design ChipLevel
link_design -add Multiply16x16
current_design ChipLevel
source ${TOP_DIR}/dc_outdir/ChipLevel_propagate.sdc -echo
current_design Multiply16x16
source ${TOP_DIR}/design/Multiply16x16.sdc -echo
current_design ChipLevel
compare_block_to_top -block_design [get_designs Multiply16x16]
```

GUI 步骤：加载设计 → 顶层与块级分别 `analyze_design` 确保干净 → `compare_block_to_top` → Analysis > B2T Mismatch Browser → 调试（可豁免 B2T 违例）。

支持：多场景比较、同一块的多个实例、创建豁免、文本报告（`report_constraint_analysis`）。

内置层次一致性规则

Built-In Hierarchical Consistency Checking Rules

B2T 规则前缀 B2T_，分类包括：B2T_CLK_XXXX 时钟；B2T_EXC_XXXX 例外；B2T_CAS_XXXX case；B2T_EXD_XXXX 外部延迟；B2T_UNC_XXXX uncertainty；B2T_DIS_XXXX 禁用对象/弧；B2T_OPC_XXXX 工作条件等。比较规则可设容差属性；默认 1e-5%，用 `set_rule_property` 修改。

18.4 相关性一致性检查

Correlation Consistency Checking

设计开发中约束文件可能被修改，导致原版与修改版不一致。相关性一致性用同一网表比较两套约束，或比较两个设计及各自约束集。

比较两套设计约束

Comparing Two Sets of Design Constraints

用 `compare_constraints` 比较。将两套约束建模为场景，在当前设计上创建后比较。

`compare_constraints` 检查：`create_clock/create_generated_clock`；`set_input_delay/set_output_delay`；`set_false_path/set_multicycle_path`；`set_min_delay/set_max_delay`；`set_clock_groups`；`set_case_analysis`；`set_disable_timing`；`set_clock_uncertainty`；`set_clock_transition`；`set_input_transition`；`set_clock_latency`；`set_clock_gating_check`；`set_sense`；`set_load` 等。

方法学：

1. `create_scenario` 建模原版与修改版约束集
2. `compare_constraints -constraints1 ... -constraints2 ...`
3. 用 S2S (SDC-to-SDC) 浏览器识别违例；`report_constraint_analysis` 获取文本报告
4. 按序修复：时钟 → `set_disable_timing` → `set_case_analysis` → 例外
5. 重复比较

(原书 Figure 252–254)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 18.5: S2S 违例浏览器、双原理图与双场景 SDC

违例规则前缀 S2S_。可禁用规则类型；禁用时 GUI 与文本报告均不显示。豁免见“抑制违例”。

比较两个设计及两套约束

Comparing Two Designs With Two Sets of Design Constraints

链接两个功能等价设计并加载两套约束后，须先提供名称映射文件（NMF）再运行 `compare_constraints`。可用 `-use_clock_map` 引用自定义时钟映射。需 PrimeTime-ELT 许可证。

方法学：

1. 链接设计并加载约束（第二设计： `link_design -add -latest design2`）
2. `define_name_maps` 指定 NMF（仅 NMF 格式被识别）
3. `compare_constraints -constraints1 ... -constraints2 ... [-design2 design2]`
4. `report_constraint_analysis` 与 GUI 调试

NMF 语法：

```
define_name_maps -application golden_sdc -design_name design2 \
  -columns {class pattern options names} \
  [list port IN [list] [list IN_2]] \
  [list cell BLK1 [list] [list BLK2]]
```

支持引脚、端口、网线、单元（含层次单元）映射。时钟映射示例：

```
set_clock_map -clocks1 gclk1_hier -clocks2 gclk1_buf \
  -design2 design2 -scenario2 sdc2_model
```

18.5 附加分析特性

Additional Analysis Features

`analyze_paths` 命令

analyze_paths Command

当 PrimeTime/DC/ICC 的 `report_timing` 无法显示某些路径时，用 `analyze_paths` 查明路径传播被阻断的原因及阻断点。可识别特定路径上的例外与约束，统计满足 `from/through/to` 条件的路径数量，并按受影响路径数对引脚排序。

`-path_type full` 且 GUI 打开时生成文本报告与原理图（ ）。`-max_endpoints` 限制原理图显示

（原书 *Figure 255*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 18.6: `analyze_paths` 识别的路径

端点数。`-text_only` 或 GUI 关闭时仅文本。

报告类型:

- 默认: 到/经/自某对象集合的路径数与时钟交互 (Example 64)
- **-path_type full**: 含起点/终点细分、逻辑级、途经点; 若存在例外则显示例外名、脚本名与行号 (Example 65–66)
- **-path_type summary**: 摘要 (Example 67)
- **-traverse_disabled**: 追踪被禁用路径, 显示数据传播阻断原因 (Example 68)

示例:

```
ptc_shell> analyze_paths -to Tranx/pay_count__reg[1]/D
ptc_shell> analyze_paths -to Tranx/pay_count__reg[1]/D -path_type full
ptc_shell> analyze_paths -to Tranx/CRC_tranx/pay_out_reg/D \
-path_type full -traverse_disabled
```

analyze_unlocked_pins 命令

analyze_unlocked_pins Command

分析引脚集合中缺失时钟定义或时钟传播被阻断的情况; 默认分析设计中所有引脚。默认报告: 分析摘要、可能缺失时钟定义的位置、时钟传播被阻断的位置。**-verbose** 列出无时钟的时钟引脚、case 禁用与寄存器禁用引脚。原理图 Pin annotation 菜单可开关时钟与 case 标注。

analyze_clock_networks 命令

analyze_clock_networks Command

分析复杂时钟网络的传播, 并生成报告, 说明时钟网络从指定点出发或到达指定点时的传播情况以及约束所产生的影响。报告可覆盖实际时钟网络、使用 **-traverse_disabled** 检测到的潜在时钟网络, 以及生成时钟的源 latency 网络; 这对于调试时钟定义以及由 **set_disable_timing** 或 **case analysis** 导致的时钟传播阻断尤其有用。

report_case_details 命令

report_case_details Command

报告设计中 case analysis 传播结果: 用户设置值、传播常量、冲突与未解析网络。用于调试 **set_case_analysis** 与 **CAS_xxxx** 违例。

report_clock_crossing 命令

report_clock_crossing Command

报告时钟域交叉路径统计: 发射/捕获时钟对、路径数量、是否被 false path/clock groups 约束。用于验证跨时钟域约束完整性。

report_analysis_coverage 命令

report_analysis_coverage Command

(PrimeTime 侧)报告分析覆盖率;约束一致性环境中用于检查约束导致的未测试路径。与 **check_timing** 配合确保断言有效。

report_exceptions 命令 (冗余约束)

report_exceptions Command (Redundant Constraints)

识别冗余、被覆盖与主导的时序例外:

- **report_exceptions -ignored** 一被其他例外完全覆盖的例外
- **report_exceptions -redundant** 一冗余例外
- **report_exceptions -dominant** 一对至少一条路径起主导作用、移除会改变约束集的例外
- **report_exceptions -ignored -verbose** 一显示主导忽略例外的详细信息

约束一致性不修改约束;编辑 SDC 后重新运行验证。应从 SDC 中删除被忽略的例外以节省后续会话运行时间与内存。

18.6 图形用户界面

Graphical User Interface

约束一致性 GUI 为查看与调试 PrimeTime、Design Compiler、IC Compiler 等工具约束问题的主要环境。

子节: 使用 GUI、约束一致性窗口、用户消息浏览器、违例浏览器、B2T 违例浏览器、相关性一致性违例浏览器、豁免配置对话框、信息窗格、控制台、在线帮助、SDC 视图、层次浏览器与原理图视图、按名选择工具栏、属性对话框、原理图显示选项。

使用 GUI

Using GUI

典型任务: 启动 GUI → 读入/链接/约束/分析 → 深入分析违例路径。默认 **pt_shell** 含 GUI; 可用 **gui_start/gui_stop**。仅 shell: **pt_shell -no_gui**。View > Palette/Toolbars 显示/隐藏界面元素。控制台含 **ptc_shell** 提示符及 Log/History 标签。File > Close GUI 关闭 GUI 保留终端会话; File > Exit 退出。

约束一致性窗口

The Constraint Consistency Window

窗口可平铺 (Window > Tile)、层叠 (Cascade)、移动与调整大小。表 46 窗口类型:

启动时默认停靠: 违例浏览器、信息窗格、控制台 ()。可按需打开层次浏览器 (Window > New Hierarchy Browser)。

表 18.2: 约束一致性窗口类型（表 46）

窗口	描述
Violation Browser	按场景与严重级别列出违例
B2T Mismatch Browser	顶层与块级约束差异
S2S Mismatch Browser	同一设计两套约束差异
User Message Browser	用户消息
Information Pane	当前选择的详细信息
Console	运行 <code>ptc_shell</code> 、查看日志与历史
SDC View	显示 SDC 并标记相关行
Hierarchy/Schematic	浏览层次、选择单元/网线/端口/引脚
Path Schematic	层次单元或路径的原理图
Properties	对象属性列表

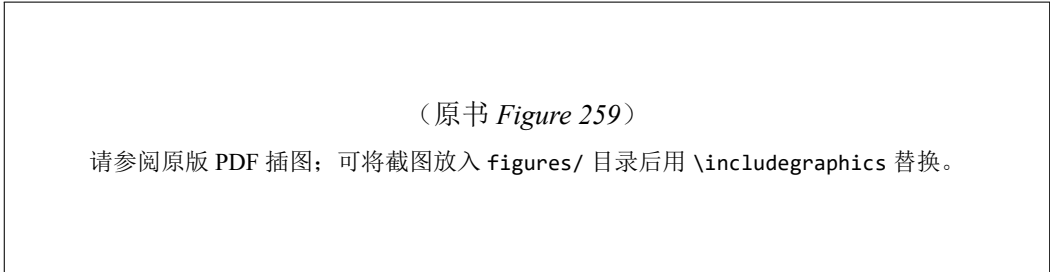


图 18.7: 启动时的约束一致性窗口

工具栏提供常用菜单快捷方式；上下文变化时按钮启用/变灰。菜单栏：File、View、Selection、Highlight、Design、Analysis、Schematic、Window、Help（表 47）。

违例浏览器

Violation Browser

主要调试工具；分层显示设计与约束违例及修复建议。支持网表相关与场景相关违例。显示顺序：Error、Warning、Information。默认每规则最多 1000 个违例标识节点；`display_violations_per_rule_limit` 可改。

多分析运行结果存于多个违例浏览器；Analysis Run Chooser（）选择查看哪次运行。首次打开

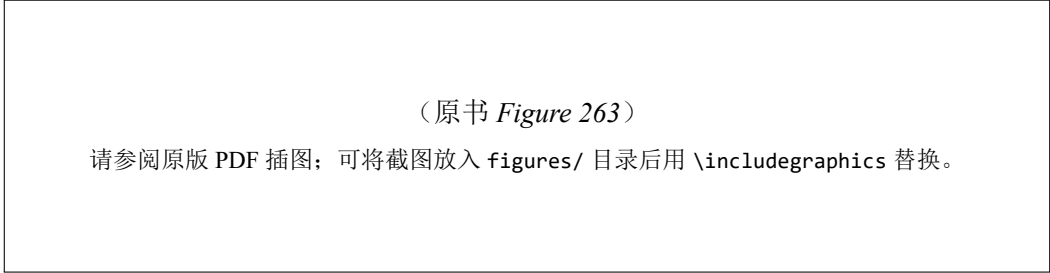


图 18.8: 分析运行选择器

时严重级别节点展开、违例标识节点折叠。

导航指南：单击查看信息窗格；双击展开规则下全部违例；使用过滤与搜索缩小列表；右键可禁用规则、豁免违例、查看 SDC/原理图。

其他 GUI 组件（摘要）

Other GUI Components

用户消息浏览器（Analysis > UserMessage Browser）：按严重级别排序；选择消息后在信息窗格显示 SDC 行号链接。

B2T / S2S 浏览器：与违例浏览器类似，提供并排显示的双原理图和双 SDC 视图，以及修复建议链接。

豁免配置：Design > Waiver Configuration — 禁用规则、创建/编辑/删除豁免、实例豁免。

信息窗格：显示违例详情、超链接（SDC、原理图、Debugging Help、Fix Suggestion）、属性与例外摘要。

SDC 视图：点击链接打开 SDC 并高亮违例行。

原理图：Pin annotation 可显示 case_value、clock_names、exceptions_summary 等；Attribute Group Manager 自定义显示属性组。

在线帮助：Help > Online Help; report_rule CAS_* 等可快速查看规则摘要。

18.7 教程

Tutorial

本教程演示如何在教程设计上设置并运行约束一致性，并调查规则违例。假设读者已熟悉约束一致性关键组件。

子节：约束一致性概述、教程设计、启动、分析 ChipLevel 设计、调试约束问题、结束与重启会话。

约束一致性概述（教程）

Overview of Constraint Consistency

违例浏览器（ ）为关键特性：analyze_design 后按场景汇总 Error/Warning/Info；双击或 “+”

（原书 Figure 275）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 18.9: 约束一致性 GUI

展开查看具体规则违例。信息窗格显示选中项详情与调查链接。控制台可交互输入命令并显示日志；History 标签列出已用命令。点击原理图链接可替换违例浏览器视图，用标签页切回。

`analyze_design` 对照预定义规则检查已加载设计（时钟、case、例外、边界条件等）。表 50 规则类型与表 44 类似，另含 B2T_XXX_XXXX 块到顶规则。`report_rule CAS_*` 列出 case 规则摘要；详细描述见在线帮助。

关于教程设计

About the Tutorial Design

教程设计 **ChipLevel** 实现多种二进制算术功能（加法、乘法等）。主要块：Adder16、CascadeMod、Comparator、Multiply8x8、Multiply16x16、MuxMod、PathSegment。设计处于前期开发阶段，时钟网络与约束仍在构建，适合用约束一致性尽早发现约束问题。

教程文件位于 PrimeTime 安装 `doc/gca/tutorial`：

```
% cp -r $SYNOPSYS/doc/gca/tutorial .
```

目录结构：

```
tutorial/
  run_tutorial.tcl
  design/chiplevel.v, chiplevel.sdc
  libs/lsi_10k.db
```

从 `tutorial` 目录运行 `run_tutorial.tcl`。

启动约束一致性

Starting Constraint Consistency

1. 打开 Linux shell，进入 `user_directory/tutorial`
2. `pt_shell -constraints`

失败时检查安装、PATH、许可证服务器、PrimeTime 与 PrimeTime SI 许可证。

分析 **ChipLevel** 设计

Analyzing the ChipLevel Design

步骤摘要：

1. `set_app_var search_path [list . ./libs ./design]`
2. `set_app_var link_path [list * lsi_10k.db]`
3. `read_verilog chiplevel.v`
4. `current_design ChipLevel`
5. `link_design`
6. `source ./design/chiplevel.sdc`（或 `read_sdc`）
7. `analyze_design`

`analyze_design` 后违例浏览器显示结果。上述命令亦包含在 `run_tutorial.tcl`（Example 84）中。

调试约束问题

Debugging Constraint Problems

在违例浏览器中按严重级别展开 Error，选择具体违例（如 CLK_XXXX、CAS_XXXX）。使用信息窗格中的 SDC 链接、原理图链接、Debugging Help 与 Fix Suggestion。常见修复：补全时钟定义、修正 set_input_delay/set_output_delay、解决 case 冲突、删除无效 set_multicycle_path 对象（如 Q 引脚非有效端点）。

修改 chiplevel.sdc 后重新 source 并 analyze_design 验证。可用 report_constraint_analysis -include violations 生成文本摘要。

结束与重启教程会话

Ending and Restarting the Tutorial Session

exit 结束会话。保留 ptc_shell_command.log 可 source 重复会话。重新启动:pt_shell -constraints 后 source run_tutorial.tcl。

注意

约束一致性（pt_shell -constraints）与标准 PrimeTime STA 使用相同 SDC 语义，但专注于约束质量而非 signoff 时序。修复约束后应在 PrimeTime 中运行完整 STA 验证。

参见

SDC 格式见 Synopsys Design Constraints 应用说明；Tcl 见 *Using Tcl With Synopsys Tools*；PrimeTime 报告与调试见第 十九 章。

Chapter 19

报告与调试分析结果

Reporting and Debugging Analysis Results

加载并约束设计、指定分析条件后，可执行完整静态时序分析并报告时序与设计规则违例。还可生成多种报告以检查与调试违例。常用报告包括：

- 全局时序摘要报告
- 路径时序报告
- 结果质量（QoR）报告
- 约束报告
- 瓶颈报告
- 全局 slack 报告
- 分析覆盖率报告
- 时钟网络时序报告
- 时钟门控与 recovery/removal 检查
- 时序更新效率
- 基于路径的时序分析（PBA）

19.1 全局时序摘要报告

Global Timing Summary Report

`report_global_timing` 报告设计全局时序收敛状态，包括最差负 slack（WNS）、总负 slack（TNS）等端点级信息。与 PBA、DMSA、SMVA/DVFS 等特性配合使用。

特性：

- 仅考虑违例时序路径，不收集/分析通过路径
- 按四种拓扑路径类别组织：in->reg、reg->reg、reg->out、in->out

- 分两阶段：路径收集（gathering）与报告输出（reporting），由不同选项独立配置

控制路径收集

Controlling report_global_timing Path Gathering

默认：每个端点收集单条最差路径（不限拓扑类型或路径组）。表 51 选项：

表 19.1: 控制 report_global_timing 路径收集（表 51）

目的	选项
仅 setup(max) 或 hold(min)	-delay_type min max
每端点每路径组最差路径	-group group_list
每端点每拓扑类别最差路径	-enable_multiple_categories_per_endpoint
每端点每类别 + 路径组 + 发射/捕获时钟组合	-include {per_clock_violations}
PBA 收集路径	-pba_mode none path exhaustive ml_exhaustive
限定 DVFS 场景	-dvfs_scenarios dvfs_scenarios

示例设计含 CLK1/CLK2/CLK3 与输入/输出延迟。默认行为仅每端点一条最差路径；-group {CLK1 CLK2} 为指定组各报告一条；-enable_multiple_categories_per_endpoint 为四拓扑类别各一条；-include {per_clock_violations} 为每组、每发射/捕获时钟组合各一条。

控制报告输出

Controlling report_global_timing Report Output

默认生成摘要表：setup/hold 下四拓扑类别的 WNS、TNS、违例端点数（NUM）。表 52 选项：

表 19.2: 控制 report_global_timing 报告（表 52）

目的	选项
摘要表或逐端点表	-path_type summary end（默认 summary）
非标准路径组单独表	-separate_non_standard_groups
所有路径组单独表	-separate_all_groups
发射/捕获时钟组合单独表	-include {per_clock_violations inter_clock}
包含空类别（零路径）	-include {non_violated}
DMSA 场景详情	-include {scenario_details}
窄/宽表格式	-format narrow wide
CSV 输出	-format csv -output filename
有效数字	-significant_digits digits

非标准路径组为 **default**、**async_default**、**clock_gating_default**。

-path_type end 显示逐端点贡献。-include {per_clock_violations inter_clock} 显示跨时钟（C2C）摘要表（如 C2C * -> CLK1）。-format wide 将 WNS/TNS/NUM 列式排列。

19.2 路径时序报告

Path Timing Report

路径时序报告用于聚焦特定违例并确定原因。默认 `report_timing` 报告最差 setup slack 路径。可显示：逻辑级、各级增量延迟、路径总延迟、slack、源/目的时钟名/latency/uncertainty、去除 CRPR 的 OCV 等。

使用 `report_timing` 命令

Using the report_timing Command

`-significant_digits` 仅控制显示位数，不影响内部分析精度。

典型报告含：起点、终点、路径组、路径类型、各级 Point/Incr/Path、到达/要求时间与 slack。

```
pt_shell> report_timing
pt_shell> report_timing -input_pins -max_paths 4
```

`-max_paths > 1` 时默认仅报告负 slack 路径；要包含正 slack 路径用 `-slack_lesser_than`：

```
pt_shell> report_timing -slack_lesser_than 100 -max_paths 40
```

多个 `-through` 表示按序经过各点：

```
pt_shell> report_timing -from A1 -through B1 -through C1 -to D1
pt_shell> report_timing -from A1 -through {B1 B2} -through {C1 C2} -to D1
pt_shell> report_timing -from A1 -through {B1 C1} -to D1
```

`-start_end_type` 过滤拓扑：reg_to_reg、reg_to_out、in_to_reg、in_to_out（与 `report_global_timing` 四类一致）。此处“reg”含非端口端点（如时钟门控检查端点）。

`-transition_time -capacitance` 显示转换时间与电容。

其他常用选项：`-delay_type min|max`、`-path full|short|full_clock|full_clock_expanded`、`-from/-to/-through`、`-group`、`-nworst`、`-max_paths`、`-sort_by group|slack`、`-nets`、`-crosstalk_delta`、`-variation`、`-attributes {...}`。

`-attributes` 在时序点行打印指定属性（经 `timing_point` 对象的链式属性访问）。合并 DMSA 报告支持 master 可用属性。`-nosplit` 防止行拆分。`timing_report_fixed_width_columns_on_left true` 将点名移到属性列右侧。限制：`-nets` 时不报告网线对象属性。

报告归一化 slack

Reporting Normalized Slack

路径最大频率取决于传播延迟与所需时钟周期数。归一化 slack：

$$\text{归一化 slack} = \frac{\text{路径 slack}}{\text{路径允许传播延迟}}$$

表 53：路径 B slack 更差但归一化 slack 更小，对频率限制更大。用 `report_timing -normalized_slack` 排序。高级锁存器分析中见第十七章。

19.3 结果质量报告

Quality of Results Report

`report_qor` 提供设计时序概览：最差路径长度（Critical Path Length，即端点到达时间）、TNS、最小延迟/hold、详细设计规则约束摘要。

TNS 两种模式（`timing_report_union_tns`）：

- **Union** 模式（默认 **true**）：各端点取所有路径组/场景中最差 slack 一次计入 TNS
- **Every-group** 模式（**false**）：每个路径组分别计入，同一端点在多组违例则多次计入

DMSA 中每种模式均取各场景最差 slack 计入。示例：两路径组两违例端点，union TNS = $-(6 + 8) = -14$ ，every-group = $-(6 + 5 + 4 + 8) = -23$ 。

`report_qor` 支持 `-pba_mode` 与 DMSA/SMVA 场景选项。

19.4 约束报告

Constraint Reports

`report_constraint` 汇总约束违例：违例/满足量及最差违例对象。可报告最大面积及多种时序约束，验证网表是否满足引脚限制。

时序约束类型

Timing Constraints

最大路径延迟/setup；最小路径延迟/hold；recovery（异步控制撤销到下一有效时钟边沿的最小间隔）；removal（时钟边沿在异步控制有效期间到控制撤销的最小间隔）；时钟门控 setup/hold；最小时钟脉宽高/低；最小时钟周期；单元两时钟引脚间最大 skew。

设计规则约束

Design Rule Constraints

检查 ASIC 厂商库定义的设计规则；亦可用 `set_max_capacitance` 等指定。检查项：网线总电容（min/max）、引脚转换时间（min/max）、扇出负载属性之和（min/max）。通常关注最大电容与最大转换；可对上升/下降、不同时钟域分别设置。

生成默认约束报告

Generating a Default Constraint Report

`report_constraint` 显示各约束在当前设计的最差评估与加权 Cost。示例输出含 `max_delay/setup`、`min_delay/hold` 分组 Cost 与总 Constraint Cost。

报告违例

Reporting Violations

`report_constraint -all_violators` 列出所有违例实例。`-verbose` 显示详细路径报告（含完整点列）。

最大 **skew** 检查

Maximum Skew Checks

多时钟顺序器件可能需要 skew 检查。Liberty 关键字: `skew_rising`、`skew_falling`；`timing_type` 设为其中之一，`related_pin` 指定参考时钟引脚。

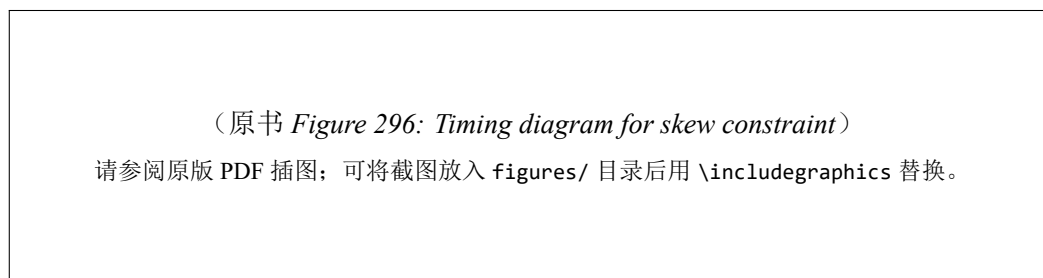


图 19.1: Skew 约束时序图

`report_constraint -max_skew [-all_violators] [-verbose]` 报告最大 skew。实际 skew = 参考与约束时钟边沿延迟差的绝对值；参考引脚用最小 latency、约束引脚用最大 latency；skew 检查不考虑 uncertainty。

No-change 时序检查

No-Change Timing Checks

库中 no-change 检查确保信号在时钟有效间隔内不翻转。等价于对有效边沿做 setup、对无效边沿做 hold。高有效时钟：上升沿 setup、下降沿 hold。`report_timing` 与 `report_constraint` 报告为 library no-change setup/hold time。

其他 `report_constraint` 选项

Other report_constraint Options

`-all_violators`、`-verbose`、`-max_delay`、`-min_delay`、`-recovery`、`-removal`、`-max_skew`、`-clock_gating_setup/hold`、`-min_pulse_width`、`-min_period`、`-significant_digits`、`-pba_mode`。

19.5 瓶颈报告

Bottleneck Report

瓶颈是对多条违例有贡献的公共点。瓶颈分析识别最差瓶颈、评估改善可能性，并指导网表修改（综合、尺寸调整等）。PrimeTime 将瓶颈与叶单元关联。

报告方式:

- GUI: Reports > Histograms > Timing Bottlenecks
- Shell: `report_bottleneck` (使用前将 `timing_save_pin_arrival_and_slack` 设为 `true`)
- Tcl: `install_dir/auxx/pt/examples/tcl/bottleneck_utils.tcl` 中的过程

用 `report_timing` 或 `get_timing_paths` 验证瓶颈分析结果。

`report_bottleneck` 默认基于 `slack < 0` 的路径数报告最差瓶颈单元 (Bottleneck Cost = 经该单元的违例路径数)。Example 87 显示前 20 个单元。

Example 88 Tcl 过程 `verify_bottleneck_cell` 可验证单元瓶颈代价是否与报告一致。

19.6 全局 slack 报告

Global Slack Report

`report_global_slack` 显示指定引脚或端口的 slack。默认报告所有引脚 (层次单元引脚与设计端口除外)。

注意

首次时序更新前将 `timing_save_pin_arrival_and_slack` 设为 `true`; 否则 `report_global_slack` 会将其设为 `true` 并触发另一次时序更新。

`-max/-min` 与 `-rise/-fall` 互斥; `object_list` 指定引脚/端口, 省略则默认所有引脚。

获取 setup 违例端点列表:

```
pt_shell> get_attribute [current_design] violating_endpoints_max
```

hold 违例:

```
pt_shell> get_attribute [current_design] violating_endpoints_min
```

端点按 slack 递增顺序列出。

19.7 分析覆盖率报告

Analysis Coverage Report

`report_analysis_coverage` 报告当前设计/实例的时序检查覆盖情况, 对新设计尤为关键。

推荐流程:

1. `link_design` 并解决链接错误
2. `check_timing` 并解决检查时序错误
3. `report_analysis_coverage` 并解决未测试 (untested) 问题
4. 执行其余分析

汇总检查类型: setup、hold、no-change、min period、recovery、removal、min pulse width、clock separation、clock-gating setup/hold、output setup/hold、max skew。默认按类型汇总 met/violated/untested 数量与百分比; 某类型无检查则不显示。

静态时序理论上检查所有路径, 但若断言不完整或路径被禁用 (false path、禁用弧、case analysis 等), 部分检查为 untested。与 check_timing 配合验证设计与断言。多 case analysis 配置下可查看各配置 untested 检查。

-status_details 显示各项时序检查的状态详情; 对于 untested 检查, 如果工具能够确定原因, 报告中还会给出原因。-exclude_untested 可依据原因对列表进行整理; -check_type、-sort_by slack 等选项可用于过滤和排序。

示例摘要:

Type of Check	Total	Met	Violated	Untested
setup	5	0 (0%)	3 (60%)	2 (40%)
hold	5	3 (60%)	0 (0%)	2 (40%)

-status_details {untested} -check_type {setup} 列出 untested setup 检查及原因 (如 no_-clock)。

19.8 时钟网络时序报告

Clock Network Timing Report

高性能设计中时钟网络时序特性至关重要。report_clock_timing 报告顺序器件指定时钟引脚上的 latency、转换时间与 skew。

指定报告类型 (latency、transition、skew、interclock_skew、summary)、分析范围及过滤/排序选项。PrimeTime 收集信息并按指定顺序报告。

Latency 与转换时间报告

Latency and Transition Time Reporting

基于 PrimeTime 计算的时钟 latency 与转换时间, 针对所有顺序器件 (触发器、锁存器) 时钟引脚维护。某引脚 latency 取决于: 约束类型 (setup/hold)、路径角色 (launch/capture)、转换类型 (rise/fall)。

示例:

```
pt_shell> report_clock_timing -type latency -to U1/CP \
        -hold -capture -rise
```

-to 限制范围; 非顺序器件时钟引脚时, 默认替换为该引脚传递扇出中的时钟引脚集合。-probe 则直接报告指定引脚 latency, 不追溯扇出。

Skew 报告

Skew Reporting

-type skew: 同一时钟下引脚对 skew。-type interclock_skew: 含不同时钟间 skew。

两顺序器件时钟引脚 skew = 两引脚 latency 之差 (图 297)。考虑器件类型、setup/hold、launch/capture 角色。启用 CRPR 时计入 skew。可用 -from/-to 限制; 仅 -from 或仅 -to 时对未指定侧使用

与指定侧通信的所有时钟引脚。-clock 限制时钟；-include_uncertainty_in_skew 将 uncertainty 计入。

仅当两器件在指定方向存在数据路径通信时才报告 skew（不检查路径是否 false）。对电平敏感锁存器，skew 基于“to”器件 opening 边沿计算（即使允许时间借用）。

跨时钟 skew 报告

Interlock Skew Reporting

-type interclock_skew 报告不同时钟间 skew，如 CLK1 与 CLK2 域间最差局部 skew、与某引脚通信的十个最差 skew 等。-from_clock/-to_clock 限制范围；-show_clocks 显示发射/捕获时钟名。分析引脚可能很多，应尽量缩小范围。

时钟时序报告选项

Clock Timing Reporting Options

报告类型层次（图 298）：Summary（每时钟 min/max latency、转换、skew）→ List（skew/interclock_skew/transition 引脚对或单引脚列表，-nworst）→ Verbose（-verbose 完整时钟路径）。

摘要报告右列条件代码（图 299）：- 发射转换、+ 捕获转换、i 相对源反相、p 传播网络、r/f 锁存器时钟引脚上升/下降。

-from/-to 可限制摘要报告范围。

19.9 时钟门控与 recovery/removal 检查

Clock-Gating and Recovery/Removal Checks

report_clock_gating_check 报告时钟门控 setup/hold 违例。report_timing -exception_groups clock_gating_default 可报告相关路径。

Recovery/removal 检查由库或约束定义；report_constraint -recovery -removal -all_violators 汇总违例。report_timing 在异步 preset/clear 路径上显示 recovery/removal 为库检查时间。

异步复位路径调试：确认 set_false_path 未错误覆盖 recovery/removal；检查 set_case_analysis 对复位网络的影响。

19.10 时序更新效率

Timing Update Efficiency

大规模设计中 update_timing 运行时间显著。report_timing 前通常需完整或增量更新。

消息限制：read_parasitics、report_annotated_parasitics -check、read_sdf 及隐式/显式 update_timing 发出的部分消息每次调用有默认上限；超限后日志注明不再输出，会话结束打印抑制摘要。sh_message_limit 控制默认限制；sh_limited_messages 控制受影响消息集，可减小大违例数量时的日志体积。

增量更新：PrimeTime 在约束、寄生、延迟标注等局部变化后执行增量时序更新；避免不必要的全量 update_timing。

保存 `pin arrival/slack:timing_save_pin_arrival_and_slack` 在更新时保存引脚到达时间与 slack, 供 `report_global_slack`、`report_bottleneck` 等使用, 但增加内存与更新开销。

19.11 基于路径的时序分析

Path-Based Timing Analysis

静态时序工具设计为悲观以确保检出所有违例。例如 PrimeTime 对路径同时考虑最差到达时间与最差 slew, 即使二者来自不同侧弧。

基于路径的时序分析 (**PBA**) 通过将关键路径与其他路径隔离来降低悲观性: 沿每条关注路径传播边沿, 忽略该路径沿途侧弧的 slew, 重新计算路径专用的到达时间、slew 和 slack, 并重新计算 CRPR、串扰 delta delay 等路径效应。它适用于仅剩少量违例、需要判断违例是否源于到达时间或 slew 的悲观性的情形。

重要性质: 重算后路径 slack 排序可能改变; 单条最差路径重算不能代表端点最差重算 slack, 可能需要对同一端点多条路径重算。

支持 `-pba_mode` 的命令: `get_timing_paths`、`report_constraint`、`report_global_timing`、`report_qor`、`report_timing`。

PBA 模式

Path-Based Analysis Modes

`-pba_mode` 取值:

- `path` 一不搜索确认是否为真正最差重算路径; 快速估计改善幅度; 结果与 `exhaustive` 相同或略乐观
- `exhaustive` 一穷尽重算路径搜索, 确保返回满足选项的最差重算路径
- `ml_exhaustive` 一机器学习加速的穷尽 PBA; signoff 安全; 早期更激进预测、后期更保守 (见下文)

推荐用法: `exhaustive` 配合 `-slack_lesser_than` 限定搜索; 仅在 `path` 模式显示 slack 接近阈值时使用 `exhaustive`; 报告尽量具体 (路径组、起终点); `timing_report_use_worst_parallel_cell_arc true` 启用最差并行弧报告。

```
pt_shell> report_timing -pba_mode exhaustive -slack_lesser_than 0
```

`-pba_mode path` 时 `-slack_lesser_than` 应用于原始 slack 筛选; 报告的重算 slack 可能改善。`-max_paths/-nworst` 与 `exhaustive` 联用时, 限制为搜索完成后的路径数, 搜索过程中可能分析更多路径。

```
pt_shell> report_timing -pba_mode exhaustive -nworst 1 \
    -slack_lesser_than 0 -max_paths 1000
```

`report_timing -pba_mode exhaustive` 不支持 `-start_end_pair` 与 `-slack_greater_than`。

`get_timing_paths -pba_mode path $coll` 对路径集合重算; `report_timing $recalc_paths` 报告重算值。重算值仅 `report_timing/get_attribute` 可见, 非 `report_constraint` (可用 `report_attribute` 读路径集合内属性)。`is_recalculated` 属性标识重算; 报告头显示 `Path Type: max (recalculated)`。

`pba_recalculate_full_path` (默认 `false`): `false` 时不重算时钟路径与借用路径, 仅数据段; `true` 时始终重算。注意 `-path full_clock_expanded` 仅控制报告是否展开时钟路径, 不控制是否重算。启用全路径重算时, 透明锁存器借用仅当指定 `-trace_latch_borrow` 时才重算。

`timing_report_recalculation_status` 便于调试 `exhaustive` 搜索运行时间。`pba_exhaustive_endpoint_path_limit` 限制 `exhaustive` 搜索考虑的端点数。

机器学习穷尽 PBA

Exhaustive Path-Based Analysis With Machine Learning

ML-PBA (`-pba_mode ml_exhaustive`) 用机器学习在准确性与运行时间间折中; 比 `regular exhaustive` 快且 `signoff` 安全; 可在全流程代替 `path/exhaustive` 切换。`pba_exhaustive_endpoint_path_limit` 须为 `infinity` (默认)。

行为摘要:

- `regular exhaustive` 无违例 $\Rightarrow$ ML-PBA 无违例
- `regular` 单违例 $\Rightarrow$ ML-PBA 单违例 (可能非同一路径)
- `regular` 多端点违例 $\Rightarrow$ ML-PBA 报告其中部分端点违例 (可能非最差路径)

ML-PBA 结果恒为 `regular exhaustive` 结果子集 (在搜索范围内)。ML-PBA 无违例则设计无重算违例路径。

设置重算限制

Setting Recalculation Limits

`-pba_mode path` 的每次重算命令最多可处理 2,000,000 条路径。`exhaustive` 模式下, 每个端点的路径数可由 `pba_exhaustive_endpoint_path_limit` 限制。

达到任一限制时, 由 `pba_path_recalculation_limit_compatibility` (默认 `true`) 控制行为。设为 `true` 时, 不再获取或重算更多路径: 对 `path-specific` 重算, 超出限制的剩余路径被丢弃; 对 `exhaustive` 重算, 达到限制的端点不再纳入搜索, 但该端点已经找到的路径仍保留。设为 `false` 时, 超出限制的结果用原始的基于图分析时序补齐; 这些路径仍保持未重算的基于图时序值。

Exhaustive PBA 设置

Exhaustive Path-Based Analysis Settings

`pba_exhaustive_endpoint_path_limit` 默认为 `infinity`: 算法将每个端点的 `fanin` 锥划分为若干子图, 逐步分析并修改这些子图, 保证覆盖所有路径并返回最差 PBA `slack`。若设为整数 (如 25000), 命令会将每条路径与其他路径隔离后分别分析; 达到限制时发出 UITE-480 警告并停止分析。当基于图分析与 PBA 的 `slack` 已较为接近时, 整数限制模式可能更快。

HyperTrace 加速 PBA

HyperTrace Accelerated Path-Based Analysis

HyperTrace 通过计算精炼的基于图的时序数据加速 `exhaustive PBA` 搜索。需在首次 `exhaustive PBA` 前对关键区域 (`slack` 违例引脚集) 做图精炼 (选择性信号合并)。精炼开销与后续 PBA 加速需

权衡：当 exhaustive PBA 占全流程 >20% 且违例引脚 <10% 时推荐；接近 signoff、PBA 占 >50% 运行时收益最大。需 PrimeTime-ADV-PLUS 许可证。

配置变量包括 `timing_enable_hypertrace` 等；不支持部分流（见原书“Unsupported Flows”）。精炼图仅用于 exhaustive PBA，非 PBA 报告仍用原时序图。

关键区域阈值应设为 exhaustive 搜索使用的最大 `-slack_lesser_than`（显式或默认），确保搜索完全在关键区域内加速。

参见

高级分析（并行弧、时间借用）见第 十七 章；约束一致性调试见第 十八 章；GUI 报告见第 二十 章。

Chapter 20

图形用户界面

Graphical User Interface

PrimeTime 图形用户界面（GUI）允许您可视化设计数据，并使用直方图、原理图、抽象时钟图、波形图与数据表等分析工具查看分析结果。要学习如何使用 PrimeTime GUI，请参阅以下主题：

- 打开与关闭 GUI
- 快速入门指南
- GUI 窗口
- 分析时序路径集合
- 在抽象时钟图中检查时钟路径
- 分析时钟域与时钟间关系
- 在原理图中检查时序路径与设计逻辑
- 查看与修改原理图
- 检查路径元素
- 查看对象属性
- 重新计算时序路径
- 布局视图与 GUI 中的 ECO
- 时序路径的交互式多场景分析
- GUI Tcl 命令

20.1 打开与关闭 GUI

Opening and Closing the GUI

PrimeTime GUI 是在 Linux 上运行于 X 窗口环境中的窗口与菜单驱动界面。您可在启动 PrimeTime 时打开 GUI，也可在 PrimeTime 会话期间打开。

打开 PrimeTime GUI 时，工具会读取 GUI 设置与首选项文件，并打开新的 PrimeTime 主窗口：

- 设置文件执行基本设置任务，例如初始化变量与声明设计库；首选项文件设置原理图与抽象时钟图视图属性以及全局应用首选项。
- 主窗口包含用于执行时序分析与其他分析任务的菜单、工具栏与视图窗口。

您可在会话期间随时打开或关闭 GUI。例如，当需要在 `pt_shell` 中执行耗时任务或批处理时，可关闭 GUI；完成后再重新打开 GUI 进行可视化分析。

参见

- 打开 GUI
- 关闭 GUI

20.1.1 打开 GUI

Opening the GUI

打开 GUI 前，请确认 `DISPLAY` 环境变量已设置为您的 Linux 显示名称。也可在启动会话时同时设置 `DISPLAY` 变量。

要启动 PrimeTime 会话并打开 GUI，输入：

```
% pt_shell -gui
```

若要在启动 PrimeTime 会话时设置 `DISPLAY` 环境变量，可使用 `-display` 选项，其中 `host_name` 为 Linux 显示终端名称。例如：

```
% pt_shell -gui -display 192.180.50.155:0.0
```

要在 PrimeTime 会话期间打开 GUI，在 `pt_shell` 提示符下输入：

```
pt_shell> gui_start
```

20.1.2 关闭 GUI

Closing the GUI

关闭 GUI 时，设计仍保留在内存中，命令行提示符在 `shell` 中保持活动。

要在不退出 `pt_shell` 的情况下关闭 GUI：选择菜单 `File > Close GUI`；或在控制台或 `pt_shell` 命令行输入 `gui_stop`。

要关闭 GUI 并完全退出 PrimeTime 会话，选择 `File > Exit`。

20.2 快速入门指南

Quick Start Guide

以下步骤介绍 GUI 的部分功能。请从已加载、已约束且已完成时序更新的设计开始。在 GUI 中创建要分析的路径集合。例如：

```
pt_shell> set mypaths [get_timing_paths -nworst 20 -max_paths 500 \
-slack_lesser_than 0.2]
```

打开 GUI:

```
pt_shell> gui_start
```

1. 单击 Path Collections 选项卡。
2. 右键单击路径集合名称，选择 Show Paths。将显示路径 slack 直方图与路径集合中的路径列表。
3. 在列表中选择并高亮感兴趣的路径（例如最差 slack 的路径），然后单击 Inspector 按钮打开路径检查器。
4. 在 Path Inspector 中查看 Delay Profiles、Path Summary、Path Elements 与 Timing report 等选项卡。
5. 在 Schematic 视图中查看路径逻辑；使用 View Settings 面板调整显示。
6. 使用 Clock > Clock Analyzer 分析时钟域关系。

（原书 *Figure 302*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 20.1: PrimeTime 主窗口

20.3 GUI 窗口

GUI Windows

PrimeTime GUI 在窗口中显示信息，为执行不同任务提供灵活工作环境。这些窗口在 X 窗口环境中独立运行，但共享内存中的同一设计、当前时序信息以及工具中的全局选择数据。

打开 GUI 时自动显示 PrimeTime 主窗口，提供多种分析视图。每个应用窗口包含标题栏、菜单、工具栏与面板、状态栏，以及显示特定设计信息的视图窗口。视图窗口与面板出现在工具栏与状态栏之间的工作区。

20.3.1 视图窗口

View Windows

视图窗口是 GUI 窗口内显示设计信息的子窗口。打开新视图窗口时，GUI 在工作区底部显示对应选项卡。

在视图窗口内单击可激活该视图（标题栏高亮）。重叠时活动视图位于最前。可通过选项卡或 Window 菜单激活视图；也可使用 Window > Next Window 或 Previous 循环切换。

含多个子视图的窗口为每个视图提供选项卡；打开时显示默认视图，单击其他选项卡可切换。

(原书 *Figure 302*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.2: PrimeTime 主窗口结构

20.3.2 视图设置面板

View Settings Panel

可通过 View Settings 面板查看并修改活动视图的显示设置。显示或隐藏面板：View > Toolbars > View Settings，或按 F8。可用设置取决于窗口类型。

默认情况下，更改选项后选项与 Apply 按钮变蓝；单击 Apply 使更改生效。可通过 Options 菜单或齿轮图标操作面板；可保存/恢复首选项或写入脚本。Auto apply 选项可跳过 Apply 按钮立即应用所有更改。

(原书 *Figure 303*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.3: 视图设置面板

20.3.3 工具栏与面板

Toolbars and Panels

工具栏为视图窗口中最常用命令与交互操作提供快捷访问；面板是增强工具栏，包含设置选项或处理设计数据的工具。

显示或隐藏工具栏/面板：View > Toolbars > toolbar_name。

主窗口工具栏包括：Clock、View Zoom、History of Zoom/Pan、Schematics、View Tools、Highlight、ECO 等。

20.3.4 层次浏览器

Hierarchy Browser

层次浏览器用于遍历设计层次并选择设计部分以进行后续分析。若未打开层次浏览器视图，选择 View > Hierarchy Browser 打开。

左窗格用 [+] 与 [-] 浏览单元层次；选择左窗格单元后，右窗格显示该单元内容。可选择列出对象类型：层次单元、所有单元、pin/port、net 等。

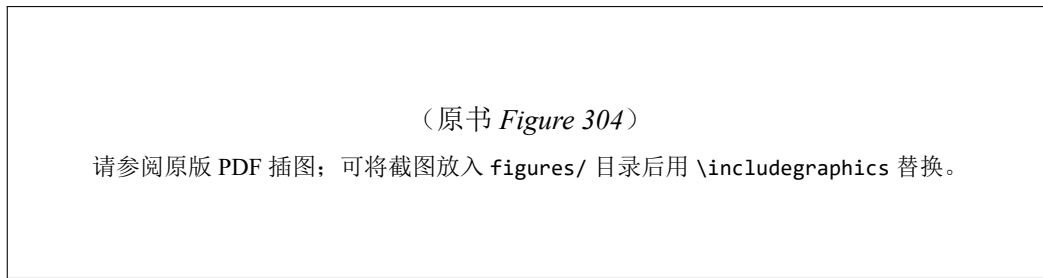


图 20.4: 主窗口工具栏

可用上下箭头键滚动对象列表；列宽不足时悬停指针可显示 InfoTip。单击列标题可排序，可调整列宽。

在层次浏览器中选择对象时，其他视图（如原理图）中也会同步选中。

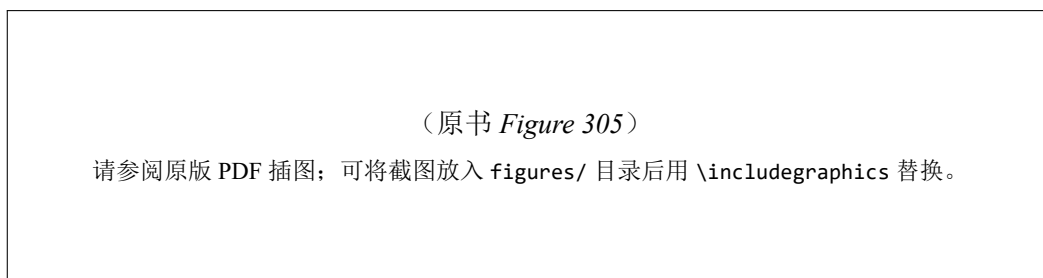


图 20.5: 层次浏览器

20.3.5 控制台

Console

控制台用于执行 `pt_shell` 命令、查看文本格式 PrimeTime 响应以及命令历史。底部 `pt_shell` 按钮右侧的长框中可输入命令。

控制台操作与 `pt_shell` 终端窗口输入输出一致；可在终端或控制台任一处输入命令并查看响应。不需要控制台时可关闭，仅使用终端。

控制台提供终端没有的视图，底部选项卡可选择显示文本类型：

- Log 选项卡：显示工具 `pt_shell` 响应。
- History 选项卡：查看最近执行命令历史。

右下角齿轮图标提供搜索、选择、复制与重用控制台内容的选项。

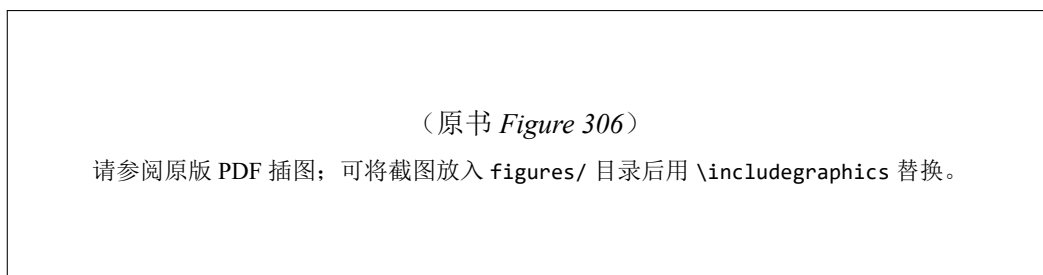


图 20.6: 控制台

20.3.6 对象选择

Object Selection

可在视图窗口中单击对象，或通过 **Select** 菜单命令选择对象以进行后续操作。交互式多选时按住 **Ctrl** 键；否则每次新选择会取消先前选择。

所选对象在所有视图中同步选中，不仅限于当前视图。例如某单元出现在两个原理图窗口与层次浏览器列表中，在任一处选中该单元，三处均会高亮。

20.3.7 选择时序路径

Selecting Timing Paths

可选择特定时序路径，或选择设计中 **slack** 最差的一条或多条路径。例如显示路径 **profile** 前需先选择路径。简便方法是在 **Path Collections** 表中单击感兴趣路径。

创建路径集合示例：

```
set mypath1 [get_timing_paths -from ... -to ...]
```

20.3.8 设置 GUI 首选项

Setting GUI Preferences

打开 GUI 时从首选项文件加载 GUI 首选项。默认系统首选项针对大多数设计已优化。

设置 GUI 首选项：**View > Preferences**。更改后 GUI 自动将新设置保存到主目录下 **.synopsys_pt_prefs.tcl**；下次打开 GUI 时从此文件加载。

20.4 分析时序路径集合

Analyzing Timing Path Collections

可分析时序路径集合以确定设计中时序失败位置。路径分析器（**path analyzer**）基于可用属性规则对路径分类；可选择预定义分类规则或定义自定义规则，也可添加子类别。

路径分析器是高级时序分析工具，支持自定义趋势分析。例如：

- 按 **path group** 分类路径，查看违例是否集中于特定 **path group**。
- 为 **slack** 违例超过一个时钟周期的路径创建自定义规则，判断是否需要 **multicycle path** 约束。
- 按 **start clock** 与 **end clock** 分类，识别跨域路径（两值不同）。
- 为物理分区等块打 **block mark**，再按 **block mark** 属性分类，查看失败是否来自特定块。

打开路径分析器：**Timing > Path Analyzer**。

进一步分析需加载时序路径集合（**Collections** 按钮）并对路径分类（**Create** 按钮）。

20.4.1 加载路径集合

Loading Path Collections

要将时序路径集合加载到路径分析器：

（原书 *Figure 307*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.7: 初始路径分析器

（原书 *Figure 308*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.8: 加载并分类后的路径分析器

1. 在路径分析器中单击 **Collections** 按钮。
2. 在对话框中选择要加载的集合，单击 **OK**。

20.4.2 对时序路径分类

Categorizing the Timing Paths

要对时序路径分类：1. 单击 **Create** 按钮。2. 选择预定义规则（如 **End Point**、**Start Clock**、**End Clock**、**Path Group** 等）或自定义规则。3. 单击 **OK**。

要删除类别或子类别，在 **Categories** 树中右键选择 **Remove**。

20.4.3 将路径类别保存到文件

Saving Path Categories to a File

要将路径类别保存为脚本文件：在路径分析器中选择类别，右键选择 **Save Categories**，指定文件名。

20.4.4 从文件加载路径类别

Loading Path Categories From a File

要从脚本文件加载路径类别：右键选择 **Load Categories**，选择先前保存的文件。

20.4.5 标记块并应用块分类规则

Marking Blocks and Applying Block Category Rules

可为设计中的块（如物理分区）打 block mark，然后使用基于 block mark 属性的分类规则分析路径。

预定义块分类规则示例：

表 20.1: 块的分类规则

规则名	属性	定义
Inside block	block_mark	路径完全位于带 mark 的块内
Crossing block	block_mark	路径穿越块边界
Starting in block	block_mark	路径起点在块内
Ending in block	block_mark	路径终点在块内

20.4.6 创建自定义分类规则

Creating Custom Category Rules

创建自定义分类规则：1. 单击 Create，选择 Custom Rule。2. 输入规则名称。3. 在 Category Attribute Chooser 中选择属性。4. 可选：在 Filter Attribute Chooser 中定义过滤表达式。5. 单击 OK。

使用斜杠 (/) 分隔属性值可创建层次化子类别。

20.5 在抽象时钟图中检查时钟路径

Examining Clock Paths in an Abstract Clock Graph

抽象时钟图 (abstract clock graph) 以图形方式显示时钟树结构与传播。可展开/折叠分支、显示/隐藏时钟元素、查看时钟延迟随时间布局等。

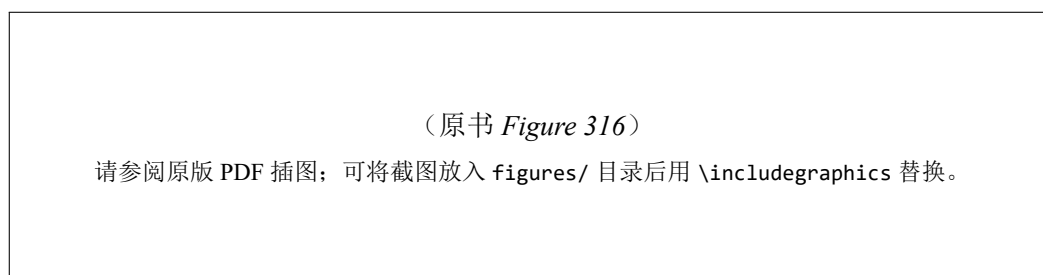


图 20.9: 抽象时钟图窗口

20.5.1 打开抽象时钟图视图

Opening Abstract Clock Graph Views

创建显示所有时钟的抽象时钟图：Clock > Abstract Clock Graph > All Clocks。

创建显示选定时钟的图：先选择时钟对象，再选择 Clock > Abstract Clock Graph > Selected Clocks。

20.5.2 显示与隐藏时钟路径元素

Displaying and Hiding Clock Path Elements

在 View Settings 面板 Elements 选项卡中，可选择显示或隐藏：时钟源、缓冲器、反相器、ICG 单元、寄存器时钟 pin、端口等。

展开弧或 metacell：选中后右键 Expand Selected，或双击。

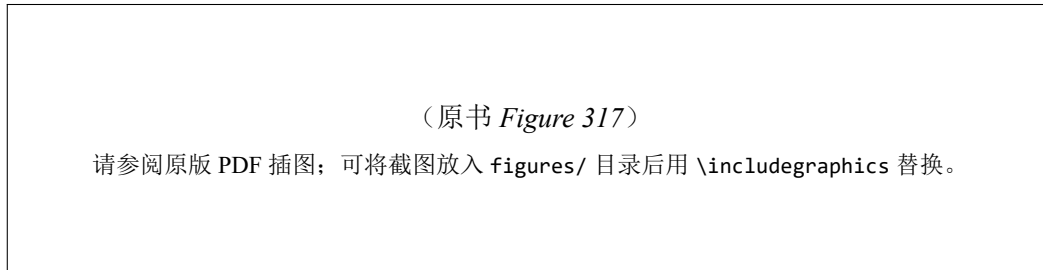


图 20.10: 展开选定弧

20.5.3 撤销与重做更改

Reversing and Reapplying Changes

在抽象时钟图视图中可撤销/重做操作（Edit > Undo / Redo，或工具栏按钮）。

20.5.4 折叠选定侧分支

Collapsing Selected Side Branches

可折叠侧分支为单行以腾出空间：1. 选择要折叠的分支根。2. 右键 Collapse Branches。

展开已折叠侧分支：选择分支根，右键 Expand Branches。

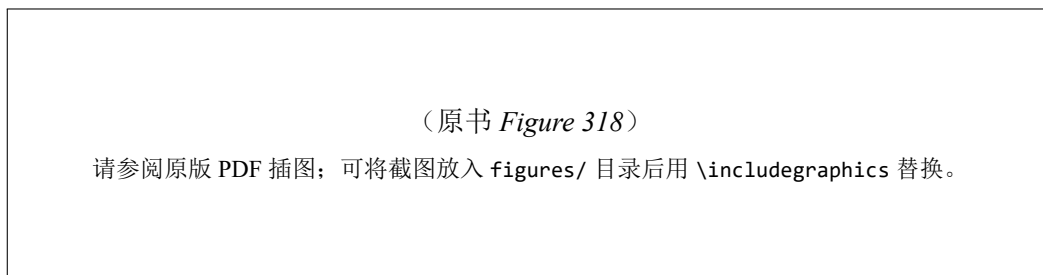


图 20.11: 分支折叠示例

20.5.5 查看时钟延迟随时间变化

Viewing Clock Latency Over Time

可在抽象时钟图视图中按调度关系显示时钟延迟，分析 skew、瓶颈、插入延迟与 ICG 内在路径延迟。

操作步骤：1. 在 View Settings 面板单击 Settings 选项卡。2. 选择 Show latency。3. 单击 Apply。
x 轴表示相对时间的时钟树逻辑；逻辑门按输入时钟 pin 延迟左对齐。

（原书 *Figure 319*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.12: 抽象时钟图中的时钟延迟布局

20.5.6 层次化显示时钟树

Displaying the Clock Trees Hierarchically

默认抽象时钟图以扁平方式显示每条时钟路径。可在 **View Settings > Elements** 中选择 **Hierarchy**，将图折叠到顶层设计，再展开各层次单元。可选择逻辑层次（默认）或物理层次（**Physical hierarchy**）。

20.6 分析时钟域与时钟间关系

Analyzing Clock Domains and Clock-to-Clock Relationships

使用时钟分析器（**clock analyzer**）识别设计中时钟间关系并驱动时钟分析，帮助确定哪些时钟域相互通信并识别时钟域交叉（**CDC**）。

时钟域由主时钟或生成时钟及其派生时钟组成；生成时钟有独立子域。打开时钟分析器：**Clock > Clock Analyzer**。

窗口左侧为层次化时钟树视图，右侧为时钟矩阵视图。

（原书 *Figure 320*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.13: 时钟分析器

时钟树视图显示主时钟名称；选择主时钟即选择整个时钟域（含其下生成时钟域）。时钟矩阵中每行每列代表一个时钟，用编号标注。

矩阵单元表示时钟域或特定 **launch-capture** 时钟对关系；字母与颜色标识路径约束及同步/异步关系。折叠时钟域时单元显示约束摘要（浅棕色单元）。

选中矩阵单元后，可在 **Legend**、**Launch Clock**、**Capture Clock** 框查看信息。右键可对 **launch-capture** 对执行 **Analyze failing paths**、**Report exceptions**、**Report timing**、**Report clock skew** 等操作。

可将时钟树或矩阵数据导出为 CSV 文件。

查询 Launch-Capture 时钟对

Querying Launch-Capture Clock Pairs 使用 Query 工具查询矩阵单元：1. 单击时钟分析器窗口右上角 Query 按钮。2. 单击矩阵单元，Query 面板显示 launch/capture 时钟与 interclock 关系。

选择时钟或时钟域

Selecting Clocks or Clock Domains 可在时钟树或矩阵视图中单击选择时钟；Ctrl+ 单击多选；Shift+ 单击选范围。对给定主时钟深度选择所有生成时钟：选中 launch 时钟，右键 Deep Select all Generated Clocks。

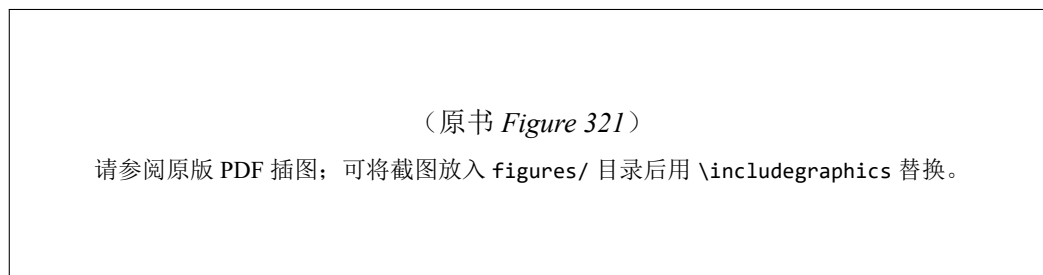
选中时钟后可右键创建 Clock Graph of Selected Clocks、Clock Graph for Clock Domain、Schematic of Selected Clocks、Select Source Pins or Ports 等。

排序时钟树视图

Sorting the Clock Tree View 单击列标题降序排序，再次单击升序；层次结构保持。

按名称查找时钟

Finding Clocks by Name 选中时钟树中时钟，右键 Find Clock(s) 显示 Find 工具栏。可按名称、功能定义（如路径上 pin）等查找。



（原书 *Figure 321*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 20.14: Find 工具栏

过滤时钟域

Filtering Clock Domains 可使用过滤条件缩小显示的时钟集合，Focus 于特定时钟域分析。

保存时钟属性或矩阵约束数据

Saving Clock Attribute or Clock Matrix Constraint Data 可将时钟树属性或 launch-capture 矩阵约束导出为 CSV。

时钟矩阵符号与颜色

Clock Matrix Symbols and Colors 矩阵单元使用字母标识关系类型，例如：

- S — 同步（synchronous）
- A — 异步（asynchronous）
- E — 显式路径（explicit path）

- I—隐式路径（implicit path）
- 等（详见原书颜色图例）

20.7 在原理图中检查时序路径与设计逻辑

Examining Timing Paths and Design Logic in a Schematic

实例原理图（instance schematic）可图形显示时序路径、选定设计逻辑或 fanin/fanout 逻辑，用于可视化分析时序与逻辑并指导其他分析任务。

实例原理图在扁平单页原理图中显示单元实例、pin、net、port、总线、ripper 与层次穿越，可跨多个层次级别。实例可为块（层次单元）或叶单元。

打开实例原理图：1. 选择要在原理图中显示的设计对象或时序路径（层次浏览器、路径表、Select > Paths From/To/Through、Select > Fanin/Fanout 等）。2. 选择 Schematic > Schematic View。

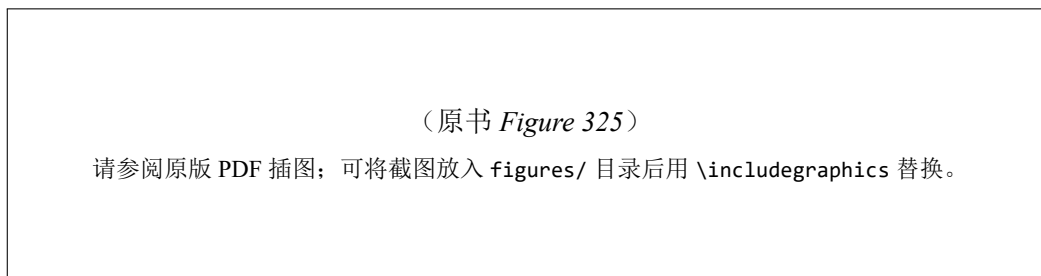


图 20.15: 实例原理图

含多层次时可用彩色边界按层次组织对象，可下移/上移层次查看块内容。可将缓冲器/反相器链或未重要块折叠为抽象 metacell。可在活动原理图视图中增删逻辑（不改变网表，不影响其他视图）。

20.7.1 检查层次单元

Examining Hierarchical Cells

默认以扁平单页显示可跨层次的时序路径与逻辑；层次单元初始显示为折叠 metacell。展开所有层次：Schematic > Expand > All Hierarchy。展开选定层次：选中后 Schematic > Expand > Selected Objects 或双击。折叠所有层次：Schematic > Collapse > All Hierarchy。折叠选定层次：Schematic > Collapse > Selected Hierarchy By Parent。

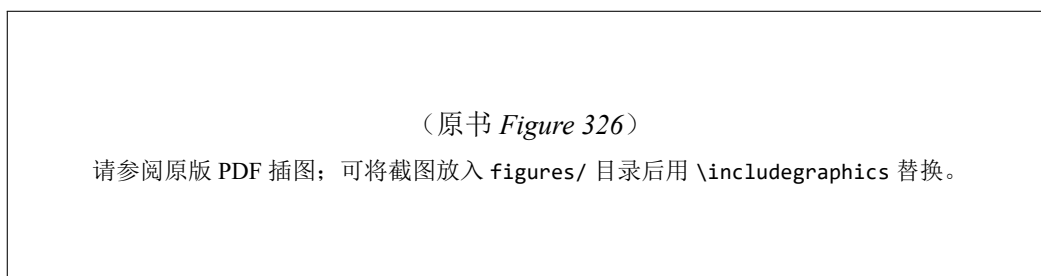


图 20.16: 层次展开与折叠视图

20.7.2 在设计层次中下移或上移

Moving Down or Up the Design Hierarchy

下移：选中层次单元，Schematic > Move Down 或双击。上移：Schematic > Move Up。

注意

下移/上移时 GUI 会更新原理图选项卡上显示的设计名称。

20.7.3 显示或隐藏缓冲器与反相器

Displaying or Hiding Buffers and Inverters

默认扁平显示含层次穿越（菱形）的时序路径。可折叠缓冲器/反相器链或树为 metacell：

- Schematic > Collapse > All Buffers/Inverters/Crossings By Chain
- Schematic > Collapse > All Buffers/Inverters/Crossings By Tree
- 选中对象后 Schematic > Collapse > Selected Buffers/Inverters/Crossings

20.7.4 展开或折叠总线

Expanding or Collapsing Buses

可展开总线 net 与 pin 查看各位，或折叠为总线符号。

20.8 查看与修改原理图

Viewing and Modifying Schematics

原理图视图提供缩放、平移、高亮、标注与查询工具。View Settings 面板可配置显示颜色、线宽、pin 标签、路径高亮样式等。

Select 菜单支持按名称选择/高亮对象；Query 工具可查看 pin/net/单元属性。

可撤销/重做原理图中的展开、折叠、增删对象等操作（Edit > Undo/Redo）。

20.9 检查路径元素

Inspecting Timing Path Elements

路径检查器（Path Inspector）提供时序路径的详细视图，包括：

- Delay Profiles 一路径延迟分布
- Path Summary 一路径摘要（slack、组、时钟等）
- Path Elements 一路径上各 pin 的到达时间、转换时间、增量延迟
- Timing report 一文本格式时序报告

配置路径检查器：在 Path Inspector 窗口 Preferences 中设置显示列与格式。
可从路径分析器或路径表选中路径后单击 Inspector 打开。

（原书 *Figure 330*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 20.17: 路径检查器

20.10 查看对象属性

Viewing Object Attributes

Attribute Browser 与 Query 面板可查看设计对象属性。Attribute Group Manager 可创建、修改、复制与重命名属性组。

20.10.1 查看当前选择

Viewing the Current Selection

使用 `get_selection` 或在 GUI 中查看选择摘要。Select > Selection Summary 可报告当前选中对象数量与类型。

20.10.2 管理属性组

Managing Attribute Groups

打开 Attribute Group Manager: Tools > Attribute Group Manager。可创建自定义属性组、从首选项恢复、修改 Basic/Placement/SchemAnnotAttr/Timing 组。

复制组：选中组后单击 Copy；重命名：Rename 按钮编辑名称。

20.11 重新计算时序路径

Recalculating Timing Paths

可通过重新计算路径表访问并比较正常路径与其重新计算对应项。使用带 `-pba_mode path` 的 `get_timing_paths` 生成重新计算路径集合。例如：

```
set my_recalc_path [get_timing_paths -from ... -to ... -pba_mode path]
```


20.11.1 比较正常与重新计算路径

Comparing Normal and Recalculated Paths

在 GUI 中：从路径分析器选择路径，选择 Timing > Recalculated Path Comparison Table。表格显示 endpoint、path group、正常 slack 与重新计算 slack；违例单元红色，通过为绿色。

(原书 Figure 345)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 20.18: 重新计算路径表

20.11.2 比较正常与重新计算路径 pin

Comparing Normal and Recalculated Path Pins

选择重新计算路径表中的行，选择 Timing > Recalculated Path Pin Comparison Table。并排比较 pin 转换时间、路径延迟、增量延迟等；正负变化分别以绿/红高亮。

仅当正常与重新计算路径具有相同 through pin 数量时启用 pin 比较表。

(原书 Figure 346)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 20.19: 路径 pin 比较表

20.12 布局视图与 GUI 中的 ECO

Layout View and ECOs in the GUI

要在 GUI 中查看芯片布局以准备工程变更指令 (ECO)，在主窗口选择 Window > Layout window。

使用布局视图需用 set_eco_options 指定物理数据 (IC Compiler II 或 LEF/DEF 格式)。用 check_eco 检查物理数据有效性。

可在 GUI 中生成并查看 ECO，包括插入缓冲器、删除缓冲器、调整单元尺寸；需要 PrimeTime-ADV-PLUS 许可证。更改后可轻松撤销。

启用撤销：确保 ECO 菜单中 Enabled 选项有勾选；不使用时可禁用以节省内存。

可高亮时序路径并仅显示路径上实际 net 形状 (含相关 via)，不含无关 fanout。

(原书 *Figure 347*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.20: GUI 中的布局视图

(原书 *Figure 348*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.21: GUI 中的 ECO 菜单

(原书 *Figure 349*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.22: Net 形状显示

(原书 *Figure 350*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.23: Net 形状显示选项

使用 ECO > Insert Buffer 或 ECO > Size Cell (`insert_buffer / size_cell`) 执行手动 ECO 时，工具会自动在内存中执行增量路径时序更新，因此可立即查看更改对时序的影响。

布局期间 on-route 缓冲器引导会遵守 placement blockage。要禁用自动增量更新，单击 GUI 底部 Fast Timing Path update Enabled 旁的 Disable。

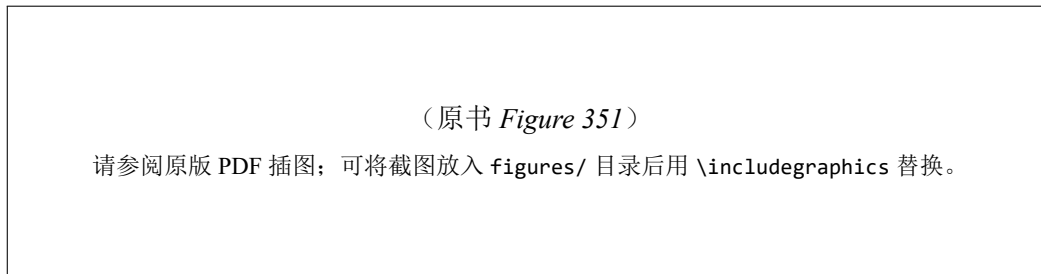


图 20.24: GUI 中的手动 ECO 操作

在布局窗口显示彩色单元密度图与 pin 密度图：View > Map > Cell Density 或 Pin Density（需 PrimeTime-ADV-PLUS）。显示功耗密度图：View > Map > Cell Power Density（需 PrimePower 分析数据）。

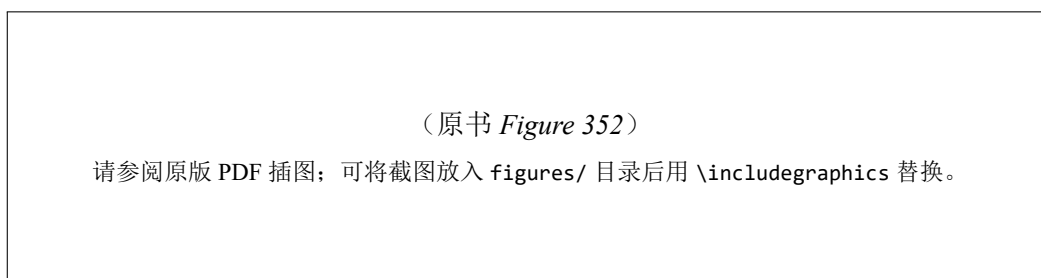


图 20.25: 布局视图中的单元密度图

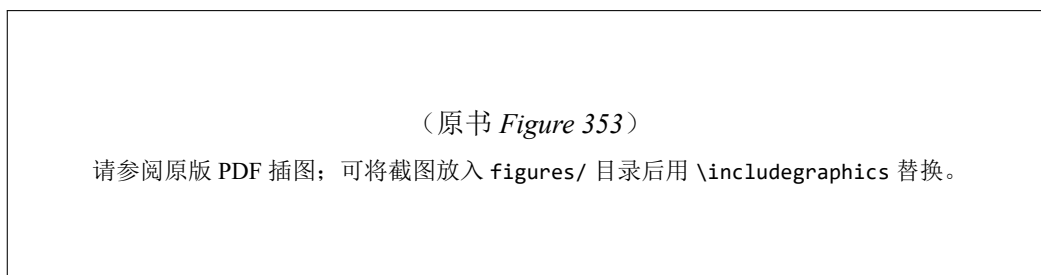


图 20.26: 布局视图中的 pin 密度图

20.13 时序路径的交互式多场景分析

Interactive Multi-Scenario Analysis of Timing Paths

PrimeTime GUI 提供交互式多场景分析（IMSA）模式，可从不同运行或场景恢复时序路径集合并一起分析，作为跨分析运行调试与跟踪违例的快速工具。

示例流程：

(原书 *Figure 354*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.27: 布局视图中的单元功耗密度图

```
# PrimeTime Session 1
...
set my_paths1 [get_timing_paths ...]
save_session -only_timing_paths $my_paths1 /dir/s1

# PrimeTime Session 2
...
set my_paths2 [get_timing_paths ...]
save_session -only_timing_paths $my_paths2 /dir/s2

# PrimeTime multi-scenario analysis session
gui_start
restore_session /dir/s1
restore_session /dir/s2
# Display, categorize, and sort paths from multiple sessions in the GUI
...
```

注意

恢复的仅路径会话须具有相同网表与时钟（约束可不同），且 save/restore 会话的 PrimeTime 版本须匹配。IMSA 会话中命令集受限，仅可运行分析已恢复路径集合的命令。

20.13.1 交互式多场景分析流程

Interactive Multi-Scenario Analysis Flow

IMSA 流程可：加载并调试多组时序路径集合；分类与排序（见路径分析器）；按指定值平移类别 slack 以便组间比较；增删信息列。

使用步骤：1. 用 `get_timing_paths` 创建路径集合。2. 用 `save_session -only_timing_paths` 保存。3. 在新会话中 `gui_start`, `restore_session` 加载各会话路径。4. 在路径分析器 Setup Collections 中选择集合并 Apply。5. 创建分类规则（如 End Point）分析路径。

平移类别的 **Slack**

Shifting the Slack for a Category 在 IMSA 流程中可将类别 slack 平移用户定义值：1. 在路径分析器中选择类别。2. 右键 Shift Category，在 Shift Histogram 对话框设置平移值与父类别。3. 在 Table Histogram 中选择 Slack+Shift 或 Slack 列查看直方图。

（原书 *Figure 355*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.28: Shift Histogram 对话框

（原书 *Figure 356*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.29: Table Histogram 对话框

（原书 *Figure 357*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 20.30: 路径 Slack 与 Slack+Shift

IMSA 保存与恢复的属性

IMSA Attributes Saved and Restored `save_session -only_timing_paths` 保存路径对象及 IMSA 相关属性，包括 session 名称、场景标识、约束差异标记等，便于跨会话比较。

20.14 GUI Tcl 命令

GUI Tcl Commands

PrimeTime 支持一组控制与查询 GUI 窗口的 Tcl 命令，可用于编写脚本控制 GUI 外观与内容。例如：

- 创建菜单命令
- 创建工具栏
- 添加命令按钮
- 向 Favorites 面板添加按钮
- 设置对象颜色
- 在布局窗口中添加颜色高亮与标注

GUI 控制能力类似于 IC Compiler II。列出 GUI 相关命令：

```
pt_shell> help gui_*
gui_change_highlight # 更改全局高亮状态
gui_create_attrgroup # 创建 GUI 属性组
gui_create_pref_category # 创建首选项类别
gui_create_pref_key # 创建首选项键值对
gui_create_vm # 创建新 Visual Mode
...
```

```
pt_shell> man gui_change_highlight
```

`gui_change_highlight` 用于操纵全局高亮对象集。有关具体 GUI Tcl 命令的语法与参数，请查阅相应的 man page。

Chapter 21

ECO 流程

ECO Flow

若设计存在时序、设计规则或噪声违例，或需优化面积/功耗，可使用工程变更指令（ECO）流程修复违例、实现更改并重新运行时序分析。

- ECO 修复概述
- 设置 ECO 选项
- 物理感知 ECO
- ECO 修复方法
- 报告不可修复违例与不可用单元
- HyperTrace ECO 修复
- 写入变更列表
- 在 Synopsys 布局工具中实现 ECO 更改
- 使用 Synopsys 工具的增量 ECO 流程
- 使用缩减资源的 ECO 流程
- Freeze Silicon ECO 流程
- 在较少主机上运行 ECO 场景
- 手动网表编辑

21.1 ECO 修复概述

ECO Fixing Overview

在 PrimeTime 中，工程变更指令（ECO）是对芯片设计的增量更改，用于修复时序违例或设计规则约束（DRC）违例，或降低功耗。PrimeTime 发现这些问题并通过调整单元尺寸、替换单元或插入缓冲器进行纠正，并以脚本格式写出更改，以便在其他工具中实现。

执行 ECO 修复的命令：

- `fix_eco_drc`
- `fix_eco_timing`
- `fix_eco_power`

ECO 修复完成后，使用 `write_changes` 写出更改，在 IC Compiler II 等物理实现工具中运行脚本。实现更改后应重新进行寄生提取并在 PrimeTime 中再次分析。

PrimeTime 可在有或没有物理布局数据的情况下执行 ECO 修复：

- 仅逻辑模式—仅使用设计网表与详细寄生数据，不考虑物理布局。
- 物理感知模式—使用网表、寄生数据与物理布局；仅在有余量处替换单元/插入缓冲器；`write_changes` 写出每个更改的位置以实现快速准确物理实现。

ECO 修复流程兼容单核、多核与分布式多场景分析（DMSA）。为降低内存、运行时间与周转时间，可使用：

- 缩减资源 ECO 流程—在全设计上做时序分析，但在较小“缩减”设计上做 ECO。
- Synopsys 增量 ECO 流程—用 Synopsys 工具实现更改，每步仅写出/分析变更部分。

注意

尝试 ECO 修复前，确保设计已完全布局布线并生成时钟树；使用物理感知 ECO 以获得最佳结果质量。

21.1.1 DRC、串扰与单元电迁移违例修复

DRC, Crosstalk, and Cell Electromigration Violation Fixing

修复 DRC、噪声、串扰延迟或单元电迁移违例，使用 `fix_eco_drc`。它修复以下命令报告的违例：

```
report_constraint -max_capacitance ...
report_constraint -max_transition ...
report_constraint -max_fanout ...
report_noise ...
report_si_bottleneck ...
report_cell_em_violation ...
```

该命令通过调整单元尺寸、插入缓冲器或反相器对来尝试修复违例，并尽量减小面积影响；默认不考虑时序影响。可通过选项指定违例类型和目标 `slack`、修复范围、修复方法、可用库单元、数据路径或时钟网络、`setup/hold` 裕量以及物理放置模式。

串扰 Delta 延迟修复

Crosstalk Delta Delay Fixing `fix_eco_drc` 可修复由串扰引起的 `delta delay` 违例，与 SI 分析结果协同工作。

SI 瓶颈优化与报告

SI Bottleneck Optimization and Reporting 可识别并优化导致串扰问题的瓶颈网络，使用相关报告命令分析修复前后 SI 影响。

21.1.2 时序违例修复

Timing Violation Fixing

使用 `fix_eco_timing` 修复 setup 与 hold 违例。支持按违例类型（setup/hold）、目标（WNS/TNS）、修复方法（insert_buffer、size_cell、remove_buffer 等）配置。

21.1.3 功耗回收修复

Power Recovery Fixing

使用 `fix_eco_power` 在不引入或恶化时序及 DRC 违例的前提下尽量回收功耗和面积。默认方法是在 setup slack 为正的路径中缩小单元；也可按库单元数值属性或字符串优先级替换单元、依据 PrimePower 的 switching/leakage/total power 数据替换单元，或从 hold slack 为正的路径中删除缓冲器。功耗回收后，用 `write_changes` 写出设计更改。

21.1.4 ECO 修复步骤顺序

Order of ECO Fixing Steps

推荐 ECO 修复顺序：

表 21.1: ECO 修复步骤、命令与优先级

步骤	命令	修复机制	优先级规则
Step 1: 功耗回收	<code>fix_eco_power</code>	单元尺寸调整、缓冲器删除	不引入新时序/DRC 违例
Step 2: DRC 修复	<code>fix_eco_drc</code>	缓冲器插入、单元尺寸	会改变 setup/hold slack
Step 3: 时序修复	<code>fix_eco_timing</code>	缓冲器插入、单元尺寸	setup 修复遵守 DRC、必要时可改变 hold slack；hold 修复遵守 setup slack 与 DRC
Step 4: 最终漏电回收	<code>fix_eco_power</code>	阈值电压交换	不引入新时序/DRC 违例，且不改变版图

21.2 设置 ECO 选项

Setting the ECO Options

使用 `set_eco_options` 配置 ECO 行为，包括物理数据路径、可用单元库、修复策略、并行度等。使用 `report_eco_options` 查看当前设置；`reset_eco_options` 恢复默认。

21.2.1 多库配置

Configuring the ECO for Multiple Libraries

若工具须区分多个逻辑等效库，可指定库优先级与排除规则。用 `set_eco_options` 的库相关选项配置。用 `-exclude` 相关变量从设计特定部分排除库单元。

21.2.2 电源管理单元尺寸调整

Resizing Power Management Cells

默认 ECO 可能不调整电源管理单元；可通过 `set_eco_options` 允许在功耗回收期间考虑这些单元。

表 21.2: 常用 ECO 相关命令

命令	说明
<code>set_eco_options</code>	设置 ECO 选项（物理路径、单元列表等）
<code>report_eco_options</code>	报告当前 ECO 选项
<code>check_eco</code>	检查物理数据是否可用于 ECO
<code>fix_eco_timing</code>	修复时序违例
<code>fix_eco_drc</code>	修复 DRC/SI/EM 违例
<code>fix_eco_power</code>	功耗回收
<code>write_changes</code>	写出 ECO 变更列表
<code>estimate_eco</code>	估算 ECO 更改的时序影响

21.3 物理感知 ECO

Physically Aware ECO

物理感知 ECO 流程整合静态时序、SI 分析、ECO 变更列表、寄生数据与 StarRC 寄生提取，在考虑物理布局约束的情况下生成可实现的 ECO。

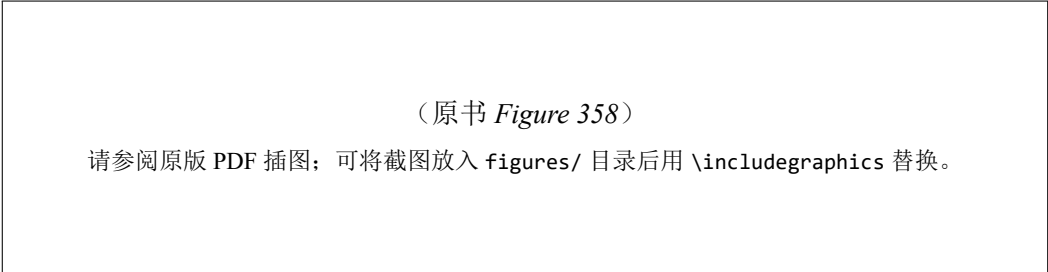


图 21.1: Synopsys 物理感知 ECO 流程

21.3.1 执行物理感知 ECO

Performing Physically Aware ECO

物理 ECO 所需设计数据文件：

数据类型	说明
SPEF/GPD 寄生	须含位置信息；可用 StarRC 从 ICC II 生成
块级 LEF 库	来自 ICC II 数据库或 Milkyway
DEF / ICC II 数据库	布局与布线信息
物理约束文件	含高级间距标签与规则
StarRC 寄生文件	与物理布局一致的寄生

配置示例：

```
set_eco_options -physical_design_path design.def \  
-physical_tech_lib_path tech.lef \  
-physical_lib_path stdcells.lef
```

21.3.2 站点感知物理 ECO 修复

Site-Aware Physical ECO Fixing

PrimeTime 物理感知 ECO 可识别单高度与双高度 site row，在合法 site 上放置 ECO 单元。

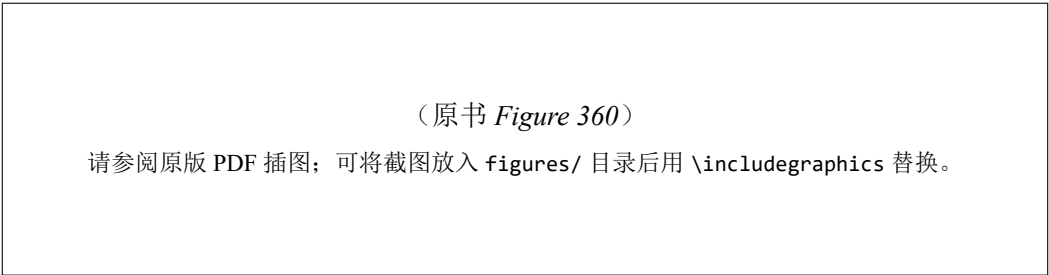


图 21.2: 常规单高度与双高度 site row



图 21.3: 偏移单高度与双高度 site row

21.3.3 多电压物理约束

Multivoltage Physical Constraints

物理 ECO 可处理多电压域的电压岛、电平转换器与隔离单元约束。

(原书 *Figure 359*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 21.4: 多电压物理约束示例

21.4 ECO 修复方法

ECO Fixing Methods

21.4.1 负载缓冲与负载屏蔽

Load Buffering and Load Shielding

负载缓冲 (load buffering) 在关键负载前插入缓冲器以加速网络；负载屏蔽在受害网络附近插入缓冲器隔离串扰 aggressor。

(原书 *Figure 362*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 21.5: 负载缓冲示例

(原书 *Figure 363*)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 21.6: 负载屏蔽示例

21.4.2 时钟网络 ECO 修复

Clock Network ECO Fixing

在时钟网络中插入缓冲器修复 hold 违例时，`fix_eco_timing` 支持 `-cell_type clock_network` 与 `-methods insert_buffer`。

需注意 setup/hold 权衡：时钟网络缓冲可能改善部分路径 hold 但恶化其他路径 setup。

21.4.3 TNS 驱动的时钟网络时序 ECO

TNS-Driven Clock Network Timing ECO

使用 `-target_violation_type tns` 以 TNS 为目标修复；`-wns_limit` 限制 WNS 恶化程度。

图 365–367 示例说明：默认不允许因修复而恶化其他路径 hold；TNS 模式可在 WNS 限制内权衡多条路径。

（原书 *Figure 365*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 21.7: 负 hold slack 示例

（原书 *Figure 366*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 21.8: TNS 驱动 hold 修复（默认 WNS 限制）

（原书 *Figure 367*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

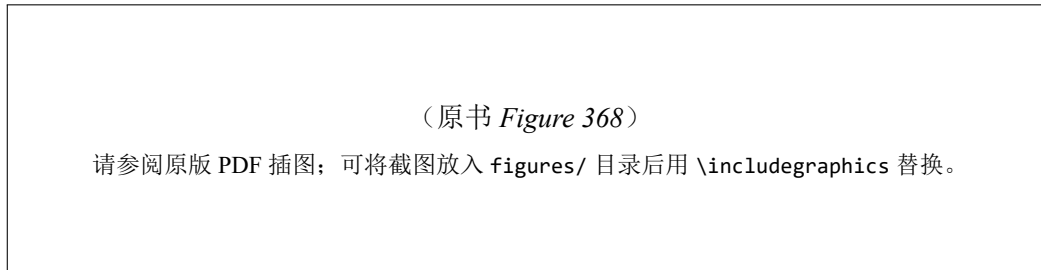
图 21.9: TNS 驱动 hold 修复（显式 WNS 限制）

21.4.4 使用负载电容单元进行 Hold 修复

ECO Hold Fixing Using Load Capacitance Cells

`fix_eco_timing -type hold` 可在数据路径插入负载电容单元。对小 hold 违例（约 5 ps 以内）比最小缓冲器更精确，避免 over-fix。

专用负载单元属性：`function_id=unknown`、`is_black_box=true`、`number_of_pins=1`。



（原书 *Figure 368*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 21.10: 缓冲器插入与负载单元插入进行 hold 修复

21.4.5 基于库单元名的功耗回收

Power Recovery Based on Library Cell Names

可指定高漏电与低漏电单元对进行阈值电压或尺寸交换。

21.4.6 基于库单元字符串属性的功耗回收

Power Recovery Based on a Library Cell String Attribute

通过用户定义属性标记可交换单元组，`fix_eco_power` 按属性匹配进行功耗优化。

21.4.7 用户脚本 ECO 机器学习

User-Scripted ECO Machine Learning

支持基于机器学习的功耗回收加速，训练单元替换模型以在保持时序前提下降低漏电。

21.4.8 MIM 的 ECO

ECOs With Multiply Instantiated Modules (MIMs)

对多重实例化模块，ECO 更改须在所有实例间一致应用；使用 HyperScale MIM 流程与相应 `write_changes` 选项。

21.5 报告不可修复违例与不可用单元

Reporting Unfixable Violations and Unusable Cells

执行时序或 DRC ECO 时，可要求 `fix_eco_timing` 或 `fix_eco_drc` 报告无法修复违例的原因。先将 `eco_report_unfixed_reason_max_endpoints` 设为要报告的端点数，再使用 `-verbose` 在命令输

出中显示原因, 或用 `-unfixable_reasons_prefix` (可配合 `-unfixable_reasons_format text|csv`) 写入日志。仅对 `fix_eco_timing`, 还可用 `-estimate_unfixable_reasons` 在不修改设计的情况下预估原因。

- A —面积限制之外存在可用库单元; B —延时改善不足; C —违例位于时钟网络
- I —给定库单元无法通过插入缓冲器修复; S —备选库单元无法通过调整尺寸修复
- T —时序裕量过紧; U —UPF 限制修复; V —net/cell 无效或带有 `dont_touch`
- W —修复可能恶化 DRC 违例

对功耗回收, 先设置 `eco_power_report_max_unusable_cells`, 再对 `fix_eco_power` 使用 `-verbose` 和/或 `-unusable_reasons_prefix`。其原因代码包括 B (功耗收益过小)、L (物理约束限制尺寸调整)、S (没有功耗更优的备选库单元)、T (可能恶化时序)、U (UPF 限制)、V (cell 无效或带有 `dont_touch`)、W (可能恶化 DRC)、X (cell 不可用于 ECO) 以及 Z (cell 已调整尺寸)。

21.6 HyperTrace ECO 修复

HyperTrace ECO Fixing

HyperTrace 可在 `fix_eco_timing` 或 `fix_eco_power` 使用 `-pba_mode` 时加速 ECO 修复; 该功能需要 PrimeECO 许可证。设置 `eco_enable_graph_based_refinement true` 可启用 HyperTrace ECO 修复, 此设置独立于用于 HyperTrace 报告的 `timing_enable_graph_based_refinement`。

`fix_eco_timing` 支持 `-pba_mode exhaustive` 与 `-pba_mode ml_exhaustive`, 并使用 `timing_refinement_max_slack_threshold`、`timing_refinement_min_slack_threshold` 和 `timing_refinement_maximum_critical_pin_percentage.slack threshold` 必须覆盖 `fix_eco_timing` 隐式或显式的 `-slack_lesser_than` 值。

`fix_eco_power` 支持 `-pba_mode path`、`exhaustive` 与 `ml_exhaustive`; 其算法直接使用 HyperTrace 数据, 不需要上述 `refinement` 变量。HyperTrace 模式不支持 `-start_end_type reg_to_reg` 和 `-pba_path_selection_options`。

21.7 写入变更列表

Writing Change Lists

使用 `write_changes` 将 ECO 更改写出为脚本, 供 ICC II 或其他 P&R 工具读入。

可用 `-format` 选择 `ptsh` (默认, PrimeTime Tcl)、`text`、`dctcl`、`icctcl`、`eco` (供 `read_eco_changes` 读取的二进制格式) 或 `aprtcl` (第三方布局布线工具使用的伪 Tcl) 格式。

在 PrimeTime 中回放 ECO 变更列表

Replaying an ECO Change List in PrimeTime 建议在原 PrimeTime 会话中执行 `write_changes -format eco -output change_list_file`; 在新建或恢复的会话中设置 ECO 选项并运行 `check_eco`, 再用 `read_eco_changes` 回放变更。验证结果后, 可继续执行 ECO, 最后写出供实现工具使用的 `icctcl` 变更列表。

在顶层运行中回放块级 ECO 更改

Replaying Block-Level ECO Changes in a Top-Level Run HyperScale 流程中，块级 ECO 可在顶层会话中回放并验证接口时序。

为第三方 P&R 工具写入 ECO 变更列表

Writing ECO Change Lists for Third-Party Place-and-Route Tools 使用 `write_changes -format aprtcl` 可生成与具体工具无关的 pseudo-Tcl, 描述 `insert_buffer`、`size_cell`、`create_cell`、`connect_net` 和 `remove_buffer` 等操作。

变更列表文件头含版本字符串（当前为 1.0），解析脚本应检查版本。

```
insert_buffer -on_route BUFFD4 \
  -new_net_names {net1} -new_cell_names {BUF1} \
  -location {x1 y1} -route_cut_location {x1' y1'} \
  {sink1/I sink2/I sink3/I sink4/I}
```

21.8 在 Synopsys 布局工具中实现 ECO 更改

Implementing ECO Changes in Synopsys Layout Tools

注意

若有 PrimeECO 许可证，可在单 shell 环境中完成 ECO 物理实现、寄生提取与 signoff 时序分析。详见 PrimeECO User Guide。

使用独立 Synopsys 工具的步骤：1. PrimeTime:

```
write_changes -format icctcl new_change_list_file
```

2. IC Compiler II: source 变更列表, `place_eco_cells`、`create_stdcell_fillers`、`route_eco`。
3. StarRC: 寄生提取。4. PrimeTime: 读入更新网表与寄生，重新分析。5. 验证 QoR，按需迭代。

21.9 使用 Synopsys 工具的增量 ECO 流程

Incremental ECO Flow Using Synopsys Tools

与全网表/全寄生 ECO 流程相比，增量 ECO 流程中：

- IC Compiler II 实现 ECO 并仅写出增量更改。
- StarRC 仅提取增量物理更改并写出增量寄生文件。
- PrimeTime 将增量寄生应用于恢复的会话并验证修改后设计时序。

21.9.1 初始化增量 ECO 流程

Initialize the Incremental ECO Flow

1. ICC II: 打开库、复制块、初始化 ECO 数据库：

(原书 *Figure 372*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 21.11: Synopsys 增量 ECO 流程

(原书 *Figure 373*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 21.12: Synopsys 增量 ECO 流程步骤

```
icc2_shell> open_lib Design.nlib
icc2_shell> copy_block -from_block Block -to_block Block_pre_eco
icc2_shell> open_block Block_pre_eco
icc2_shell> record_signoff_eco_changes -init -def
```

2. StarRC:ECO 模式与增量网表模式寄生提取。3. PrimeTime: 全设计基线时序更新、`save_session`、执行初始 ECO。

21.9.2 增量 ECO 迭代

Incremental ECO Iteration

初始化后每轮迭代：ICC II 实现 PrimeTime 写出的更改并跟踪增量；StarRC 增量提取；PrimeTime `restore_session` 并应用增量寄生。

21.9.3 层次化增量 ECO 流程

Hierarchical Incremental ECO Flow

HyperScale 块级与顶层可分别进行增量 ECO，通过 `record_signoff_eco_changes` 协调跨层次更改。

21.10 使用缩减资源的 ECO 流程

ECO Flow Using Reduced Resources

在全设计时序分析后，创建仅含违例相关逻辑的缩减设计进行 ECO，降低内存与运行时间。支持 DMSA 与非 DMSA 模式。

21.10.1 缩减资源非 DMSA ECO 流程

Reduced Resource Non-DMSA ECO Flow

使用 `create_eco_scenario` 与相关命令创建缩减场景，在缩减设计上执行 `fix_eco_*` 命令。

21.11 Freeze Silicon ECO 流程

Freeze Silicon ECO Flow

Freeze silicon ECO 在已流片硅片上通过备用单元（`spare cell`）实现逻辑更改。需特殊物理规则、导出规则与技术约束。

21.11.1 准备 Freeze Silicon ECO 流程

Preparing for the Freeze Silicon ECO Flow

ICC II 中准备 `spare cell`、填充单元与 ECO 合法化规则；PrimeTime 中配置 freeze silicon 相关 ECO 选项。

21.11.2 PrimeTime 中的 Freeze Silicon ECO

Freeze Silicon ECO in PrimeTime

使用专用 `fix_eco_*` 选项与 `write_changes` 格式，将逻辑映射到 `spare cell` 并实现金属层 ECO。

21.12 在较少主机上运行 ECO 场景

Running ECO Scenarios on Fewer Hosts

优化 DMSA ECO 脚本以在有限主机上更快完成：合并场景、调整并行度、使用场景优先级。

21.12.1 优化 DMSA ECO 脚本以加快周转

Optimizing DMSA ECO Scripts for Faster Turnaround

减少场景镜像交换、批量 `fix_eco` 命令、复用物理数据路径等最佳实践。

21.13 手动网表编辑

Manual Netlist Editing

除自动 ECO 外，可手动编辑网表：

表 21.4: 手动网表编辑任务与命令

对象	任务 / 命令
单元	尺寸调整: <code>size_cell</code> ; 创建: <code>create_cell</code>
网络	连接: <code>connect_pin</code> / <code>disconnect_pin</code>
缓冲器	插入: <code>insert_buffer</code> ; 删除: <code>remove_buffer</code>
负载	插入负载电容单元: <code>create_cell</code> + <code>connect_net</code>

21.13.1 GUI 中的单元尺寸调整与缓冲器插入

Sizing Cells and Inserting Buffers in the GUI

在布局视图中可通过 ECO 菜单交互式 `size cell` 与 `insert buffer`，支持 on-route 放置引导与即时增量时序更新（见第 二十 章）。

21.13.2 `fix_eco_timing` 选项详解

`fix_eco_timing` Option Details

`fix_eco_timing` 主要选项:

选项	说明
<code>-type setup hold</code>	修复类型
<code>-methods {insert_buffer size_cell remove_buffer}</code>	修复方法
<code>-cell_type data_path clock_network</code>	目标单元类型
<code>-buffer_list {...}</code>	可用缓冲器列表
<code>-target_violation_type wns tns</code>	优化目标
<code>-wns_limit value</code>	WNS 恶化限制
<code>-max_cells_per_path N</code>	每条路径最大 ECO 单元数
<code>-effort low medium high</code>	修复力度

21.13.3 `fix_eco_drc` 选项详解

`fix_eco_drc` Option Details

修复 `max_transition`、`max_capacitance`、`max_fanout`、噪声与电迁移违例。支持 `load shielding`、`buffer insertion`、`cell sizing` 等方法。

21.13.4 fix_eco_power 选项详解

fix_eco_power Option Details

功耗回收在不恶化时序/DRC 前提下降低漏电。支持按库单元名、用户属性、机器学习模型选择替换单元。

21.13.5 set_eco_options 物理选项

set_eco_options Physical Options

```
set_eco_options \  
-physical_design_path design.def \  
-physical_tech_lib_path tech.lef \  
-physical_lib_path stdcells.lef \  
-physical_constraint_file phys_constraints.tcl \  
-target_directories {./eco_cells}
```

21.13.6 物理 ECO 条件变量

Physical ECO Condition Variables

表 21.6: 物理 ECO 相关条件与变量

条件	相关变量
单元可用性	eco_allow_footprint_change
缓冲器选择	eco_buffer_list_for_reference
最大扇出	eco_max_fanout
物理合法化	eco_physical_mode
Site 规则	eco_respect_sites

21.13.7 DEF 到 LEF Site 名称转换

DEF to LEF Site Name Conversion

当 DEF 中 SITE 名称与 LEF 不一致时，PrimeTime 提供映射规则（如 SITE CORE2T 映射到 LEF SITE）。

```
SITE CORE2T  
END CORE2T
```

21.13.8 层次块缺失 LEF 文件

Missing LEF Files for Hierarchical Blocks

即使层次块 DEF 组件缺少完整 LEF，仍可读入 LEF/DEF 格式物理数据；工具对黑盒块使用占位物理信息。

21.13.9 高级间距标签与规则

Advanced Spacing Labels and Rules

物理感知 ECO 可考虑先进间距规则 (advanced spacing rules)，通过物理约束文件指定。

21.13.10 StarRC 寄生数据文件

Parasitic Data Files From StarRC

物理 ECO 需要带位置信息的 SPEF/GPD 寄生；StarRC 从 ICC II 数据库提取时须启用相应选项以保留位置。

21.13.11 单场景与多场景功耗回收示例

Single and Multi-Scenario Power Recovery Examples

单场景：fix_eco_power 后直接 write_changes。多场景 DMSA：对各场景分别 fix_eco_power 或合并违例后统一修复。

21.13.12 功耗回收操作统计

Power Recovery Operation Statistics

fix_eco_power 的最终 ECO 摘要会给出已执行的尺寸调整或缓冲器删除操作数，以及面积变化等统计；使用 -verbose 时还会列出不可用单元及其原因汇总。

21.13.13 estimate_eco 命令

The estimate_eco Command

estimate_eco 在不实际修改网表的情况下估算 ECO 更改的时序影响，用于 GUI 手动 ECO 或脚本预评估。

21.13.14 write_changes 格式选项

write_changes Format Options

- -format icctcl —IC Compiler II Tcl 脚本
- -format ptsh —PrimeTime Tcl 脚本
- -format text —说明性文本变更列表
- -format dctcl —Design Compiler Tcl 脚本
- -format eco —供 read_eco_changes 使用的二进制变更列表
- -format aprtcl —第三方 P&R 工具使用的伪 Tcl

21.13.15 ECO 变更列表版本与解析

Change List Version and Parsing

注意

变更列表头版本字符串当前为 1.0；解析脚本应检查版本以避免未来格式升级导致意外行为。

21.13.16 层次化增量 ECO

Hierarchical Incremental ECO Flow Details

HyperScale 块级与顶层分别 `record_signoff_eco_changes`；增量寄生与增量网表在各自层次应用后验证接口时序。

21.13.17 缩减资源 DMSA ECO

Reduced Resource DMSA ECO

DMSA 模式下可为各场景创建缩减 ECO 设计，在场景子集上并行 `fix_eco_*` 以降低总内存。

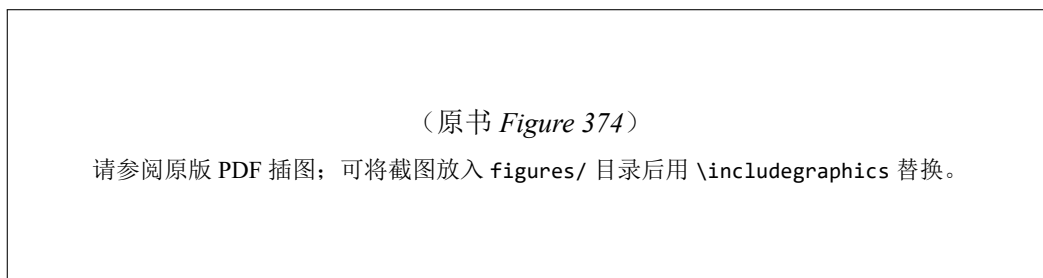


图 21.13: 缩减资源 ECO 流程

21.13.18 Freeze Silicon 技术规则

Freeze Silicon Technology Rules

Freeze silicon 须遵守金属层 ECO 设计规则、spare cell 映射规则与导出规则 (`export rules`)。

21.13.19 ICC II Freeze Silicon 准备

IC Compiler II ECO Preparation for Freeze Silicon

在 ICC II 中标记 spare cell、定义 filler 与 ECO 合法化约束，导出供 PrimeTime freeze silicon ECO 使用。

21.13.20 手动插入缓冲器示例

Manual Buffer Insertion Example

```
insert_buffer -new_cell_names {eco_buf1} -new_net_names {eco_net1} \  
  {driver_pin load_pin1 load_pin2}  
size_cell cell_instance new_lib_cell  
remove_buffer buffer_instance
```

21.13.21 ECO 与 UPF

ECO and UPF

多电压设计中 ECO 须遵守电源域与电平转换规则；不可修复原因 U 表示 UPF 限制阻止修复。

参见

- 第二十章 Layout View 与 GUI ECO
- PrimeECO User Guide

Chapter 22

层次化分析

Hierarchical Analysis

对于大型芯片设计，从顶层层次进行扁平时序分析可能消耗大量内存或运行时间。可采用层次化分析，使每次分析运行仅针对整片芯片的一部分。要学习层次化分析，请参阅：

- 层次化分析概述
- HyperScale 分析
- Context 特征化
- Context 预算（Budgeting）
- 提取时序模型（ETM）
- 快速时序模型（QTM）

22.1 层次化分析概述

Overview of Hierarchical Analysis

大型设计的分析可能消耗大量内存或运行时间。层次化分析使每次运行仅针对芯片一部分。

PrimeTime 提供以下层次化分析特性：

- HyperScale 分析—先进技术，分别分析块级与顶层部分，准确处理顶层与下级块之间的时序接口；在块级分析的缩减运行时间与内存占用下获得接近全扁平分析的精度。
- Context 特征化—写出块在顶层内的时序 context，使块可脱离顶层单独分析并仍获得在顶层 context 下工作的准确时序。
- 提取时序模型（ETM）—直接从下级块网表提取的块时序模型，由时钟、输入与输出 pin 之间的时序弧组成；可移植并导出到其他工具。
- 快速时序模型（QTM）—用一组 PrimeTime 命令指定输入、输出与约束以创建简化时序模型，用于早期层次化分析。



图 22.1: 层次化分析流程

22.1.1 何时使用扁平与层次化分析

When to Use Flat vs Hierarchical Analysis

表 22.1: 扁平全芯片分析与 HyperScale 层次化分析对比

扁平全芯片分析	HyperScale 层次化分析
最终 signoff、最佳顶层性能	大型设计、内存/运行时间受限
整片同时分析	块级与顶层分别运行
无需块模型	使用 HyperScale 块模型与 context
替代旧 ILM 流程	支持自底向上、自顶向下与混合流程

典型层次化流程包括：生成块级约束 → 块级分析 → 顶层 HyperScale 分析 → 时序收敛迭代。

22.2 HyperScale 分析

HyperScale Analysis

HyperScale 技术分别对块级与顶层部分进行分析，准确处理块接口时序，在层次化运行时间与内存下接近扁平分析精度。

PrimeTime HyperScale 层次化分析方法的主要优势：

- 运行时间与容量—相比扁平全芯片分析，HyperScale 将分析拆分为多次较小运行，显著降低内存与运行时间。
- 精度—与扁平分析执行相同延时计算、POCV/AOCV 与耦合/非耦合 SI 分析，在块接口保持完整精度。
- 高效顶层分析—自动记录并缩减顶层需分析的块内部逻辑，仅保留接口时序信息。
- 自动 context 更新—顶层与块级迭代时自动更新并传递块边界条件。
- ECO 指导—HyperScale 信息指导时序 ECO，支持顶层与块级协同修复。
- 约束管理—在块级收敛时序，顶层使用 HyperScale 块模型；支持约束提取保证块级与顶层一致。

大型芯片可跨多个层次级。HyperScale 可在每一级分别分析下级块与当前级，“顶层”可指包含一个或多个下级块的任意分析级。

（原书 *Figure 382*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.2: 扁平分析与 HyperScale 层次化分析对比

（原书 *Figure 383*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.3: 跨多层次的层次化分析

22.2.1 HyperScale 关键技术特性

Key HyperScale Technology Features

- HyperScale 建模一块以 HyperScale 模型表示，保留接口路径、约束与 CRPR 信息。
- HyperScale 块 Context 一顶层向块传递边界条件（到达/要求时间、转换时间、case 值、串扰数据等）。
- 多重实例化模型（MIM）一对多重实例化模块合并 context。
- 上下文覆盖（Context Override）一块级运行中根据顶层 context 调整边界。

HyperScale 块模型包含以下逻辑类型：输入到寄存器、寄存器到输出、输入到输出、时钟路径、接口锁存器与时间借用、最小脉冲宽度/周期检查、噪声与 SI 相关逻辑。模型包含准确时序分析所需的全部接口逻辑。

```
set_app_var hier_enable_analysis true
read_verilog block.v
link_design block
read_parasitics block.spef
source block.sdc
update_timing -full
write_hier_data $blkDir
```

22.2.2 HyperScale 使用流程

HyperScale Usage Flows

支持自顶向下、自底向上与混合流程：

(原书 *Figure 384*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.4: HyperScale 块模型

(原书 *Figure 385*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.5: HyperScale 块 Context

(原书 *Figure 386*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.6: 多重实例化模块的 Context 合并

(原书 *Figure 387*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.7: HyperScale 分析流程

(原书 *Figure 389*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.8: HyperScale 自顶向下与自底向上流程

（原书 *Figure 390*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.9: HyperScale 自顶向下流程

（原书 *Figure 391*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.10: HyperScale 自底向上流程

无 **Context** 的块模型自底向上分析

Bottom-Up Analysis Using Block Models Without Context 自底向上无 context 建模中，用预算约束分析块时序，生成 HyperScale 块模型供顶层使用。比 ETM 更准确（保留块内部网表与接口逻辑），顶层性能与容量优于带 context 的完整流程，但精度低于自顶向下带 context 流程。

从 **ETM** 层次化流程迁移

Moving From an ETM Hierarchical Flow 已有 ETM 层次化流程可少量修改脚本转为 HyperScale 以获得更高精度：块级用 `write_hier_data` 替代 `extract_model`；顶层用 HyperScale 块模型路径替代 ETM `.lib/.db` 路径；可用 `read_context` 读入已有顶层 context（若可用）。

使用块模型与 **Context** 的自顶向下分析

Top-Down Analysis Using Block Models and Context 顶层先运行 HyperScale 分析并写出块 context；块级读入 context 进行独立分析，结果回注顶层。

（原书 *Figure 388*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.11: 使用顶层与块级分析运行的 HyperScale 分析

含 **MIM** 的自顶向下分析

Top-Down Analysis With Multiply Instantiated Modules (MIMs) 对同一块的多实例，HyperScale 合并各实例 context 并处理 MIM 悲观度 (`mim_pessimism`)。

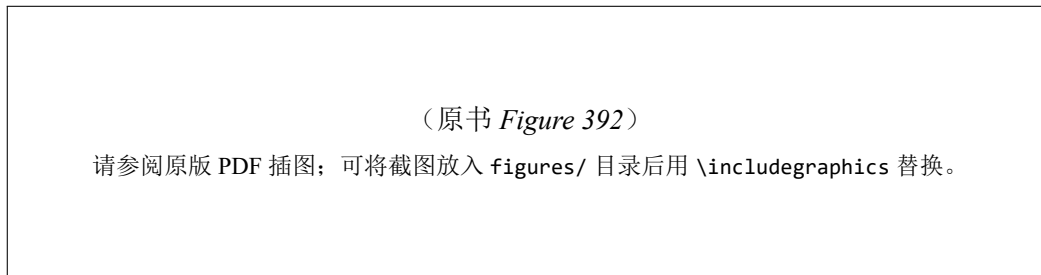


图 22.12: 自底向上流程中的时钟匹配问题

顶层与块之间的时钟映射

Clock Mapping Between Top and Block 块级与顶层须建立时钟映射；`write_context` 写出 `clock_map.pt`，块级读入后映射顶层时钟名。自底向上流程中顶层与块时钟名/周期不匹配会导致接口时序错误。可用 `create_clock -name CLKZ` 在块级创建虚拟时钟调试映射问题。

HyperScale 按优先级映射：共享物理时钟源网络；周期与波形精确匹配；时钟名前缀匹配。失败时发出 HS-004 错误但 `update_timing` 仍完成。用 `report_clock -map` 检查映射；N/A 表示未找到对应时钟。

SMVA/DVFS 设计中，`create_clock -dvfs_scenario` 定义的时钟在映射时选择周期匹配的顶层时钟；未映射时钟端口传播无时钟到达窗口以确保块级分析边界顶层分析。块 context 须用 `write_context -format gbc` 二进制格式以包含 DVFS 信息。

`get_hier_clocks` 返回 `hier_clock` 集合，可在块级查询顶层时钟定义，在顶层查询块级时钟定义；用 `report_attributes -application [get_hier_clocks]` 查看属性。

22.2.3 选择自顶向下或自底向上流程

Choosing a Top-Down or Bottom-Up Flow

自顶向下为首选：时序收敛更快，块约束与顶层约束从一开始保证一致。若块可分析而顶层约束尚未就绪，可用自底向上流程从预算约束开始，顶层分析后须检查顶层与块级约束一致性。

22.2.4 自顶向下 HyperScale 流程

Top-Down HyperScale Flow

自顶向下流程四步（各在独立 PrimeTime 会话）：

1. 全芯片提取块约束 (`hier_characterize_context_mode constraints_only`，最小 `update`，不用寄生)
2. 块级分析（读入提取约束，`write_hier_data` 写初始块数据）
3. 顶层 HyperScale 分析 (`set_hier_config`，`write_hier_data` 写块 context)

(原书 *Figure 390*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.13: HyperScale 自顶向下层次化分析流程

4. 块级分析 (`read_context` 读详细 context, 迭代收敛)

步骤 3-4 可重复直至时序收敛。顶层可用 `report_timing -path_type full_clock_expanded -from IN1 -to BLKA/reg2` 报告跨层次完整路径。块级报告中顶层施加的约束以 “@” 标记 (如 `input external delay 0.34 @`)。

22.2.5 自底向上 HyperScale 流程

Bottom-Up HyperScale Flow

(原书 *Figure 391*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.14: HyperScale 自底向上层次化分析流程

自底向上三步:

1. 用预算约束做块级分析, `write_hier_data` 生成 HyperScale 块模型
2. 顶层 HyperScale 分析 (可用 `set_clock_map` 映射时钟, `report_clock -cells/-map` 检查)
3. 块级用准确 context 最终分析 (`read_context`)

可用约束一致性检查器比较预算块约束与顶层扁平约束: `set_app_var sh_enable_constraint_consistency_checker true, report_constraint_analysis` 报告不一致。

22.2.6 HyperScale 脚本与块 Context 自动生成

Automatic Generation of HyperScale Scripts and Block Context

若有全芯片扁平分析脚本, 可启用 HyperScale 模板脚本生成: `set_app_var hier_enable_analysis true, set_app_var hier_create_script_mode true`。 `characterize_context -block ...` 后为各块创建 `partition0/..partitionN/` 目录及 `RUN.ptsh` 脚本。

22.2.7 HyperScale 配置

HyperScale Configuration

使用 `set_hier_config` (或 `set_hyperscale_configuration`) 配置块数据路径、时间戳与 HyperScale 目录结构。

`-block` 指定块名, `-path` 指定块会话数据目录, `-instances` 指定实例列表, `-name` 关联配置名。顶层分析在配置目录下执行; 各块数据目录建议以 `HS_` 为前缀。

顶层 HyperScale 配置示例

Example of Top-Level HyperScale Configuration

```
set_hyperscale_configuration -block_config_path ./hs_data/BLKA
read_verilog top.v
link_design top
read_parasitics top.spef
source top.sdc
update_timing -full
```

22.2.8 HyperScale 会话数据

HyperScale Session Data

HyperScale 流程须用 `write_hier_data` 保存、`read_context` 恢复会话数据。数据目录含网表快照、约束、context、时钟映射、时间戳与变量设置。顶层 `link_design` 在配置路径下查找块级会话数据; 指定 `-instances` 时按实例路径查找。

22.2.9 HyperScale 使用细节

HyperScale Usage Details

使用 HyperScale 的基本步骤:

1. 配置 HyperScale (`set_hier_config`)
2. 读入并链接设计
3. 读入寄生与约束
4. `update_timing` (及 `update_noise` 若适用)
5. `write_hier_data` 写出块数据或 context

用 `report_hier_analysis` 确认配置; `set_app_var hier_enable_analysis true` 使能层次化分析。

22.2.10 模型与 Context 不匹配处理

Model-Context Mismatch Handling

块网表与 HyperScale 模型/context 不一致时, 工具检测端口 `net` 变更并尝试 context 覆盖。无法解析时钟时应用无时钟 context。`report_context -list_context_override` 列出覆盖信息; 属性

`is_netlist_dont_override`、`is_clock_dont_override`、`has_model_mismatch_clock` 标识问题端口。
`report_constraint -boundary_check` 报告块边界约束违例 (`clock_mapping`、`clock_attributes`、`clock_uncertainty` 等)，许多可通过修正块级约束或继续迭代 HyperScale 流程解决。

22.2.11 块级分析

Block-Level Analysis

块级运行读入网表、寄生、块约束与顶层写出的 context:

```
read_verilog block.v
link_design block
read_parasitics block.spef
read_context -path ./hs_data/BLKA
source block.sdc
update_timing -full
```

带调整后 **Context** 的块级分析

Block-Level Analysis With Adjusted Context 顶层 context 在块边界自动调整到达/要求时间与转换时间，反映顶层实际时序环境。

22.2.12 链接设计

Linking the Design

层次化分析中 `link_design` 须正确解析块引用、HyperScale 模型与黑盒实例。

22.2.13 控制裕量应用方式

Controlling How Margins Are Applied

HyperScale 在块边界应用 OCV/AOCV/POCV 裕量；`report_hyperscale_constraints` 报告边界检查与裕量传递。

22.2.14 层次化约束提取

Hierarchical Constraint Extraction

可用 HyperScale 约束提取器从全芯片扁平约束中提取一致的块级时钟、case 值与例外，用于 context 准确的块级分析。

配置与运行约束提取

Configuring and Running Constraint Extraction

```
set_app_var hier_characterize_context_mode constraints_only
source full_chip_analysis_script.tcl
characterize_context -block BlockA
update_timing
write_context -format ptsh -output $outDir
```


输出目录包含 clock_map.pt、constraints.pt、variables.pt。

22.2.15 Context 覆盖

Context Override

HyperScale 提供 context 覆盖特性：

- 检测端口 net 上的网表更改
- 对每个差异尽力应用 context 覆盖
- 无法解析时钟时应用无时钟 context 数据

查询覆盖信息：report_context -list_context_override; 属性 is_netlist_dont_override、is_clock_dont_override 标识问题端口。

侧输入与 Stub Pin 的时钟映射

Clock Mapping for Side-Input and Stub Pins 在侧输入或 stub pin 上标注到达/要求时间时，工具检测时钟映射错误；has_model_mismatch_clock 属性标识未映射时钟。

22.2.16 控制块边界调整

Controlling the Block Boundary Adjustment

使用 set_dont_override 忽略顶层 context 数据，回退到预算约束。-include 可选择性忽略 arc_delay、case_value、pin_transition 等类型。

表 22.2: set_dont_override -include 关键字

-include	忽略的顶层 context 数据
arc_delay	输入端口到叶负载 pin 的 net 延迟；叶驱动到输出端口的延迟
case_value	输入端口的 case analysis
pin_transition	输入端口的 context 转换时间

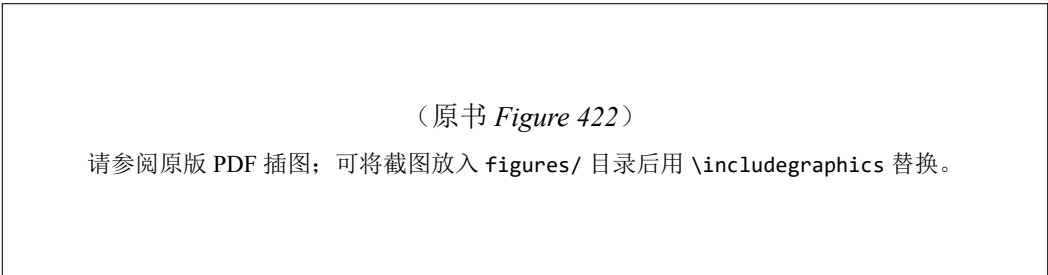


图 22.15: HyperScale 块级分析（抑制不完整块 context）

22.2.17 HyperScale 属性

HyperScale Attributes

可用 `get_attribute` 查询 HyperScale 相关属性，包括：

- 单元对象： `block_config_name`、`block_config_path`、`block_timestamp`、`critical_path_max` 等
- 设计对象： `is_hyperscale_block`、`hyperscale_mode` 等
- pin 对象： `context_arrival`、`context_required`、`has_model_mismatch_clock` 等
- port 对象： `context_capacitance`、`is_user_dont_override` 等

22.2.18 HyperScale 数据目录结构

HyperScale Data Directory Structure

HyperScale 块数据目录包含网表、约束、context、时钟映射与时间戳文件，供顶层与块级运行共享。

22.2.19 为 HyperScale 块写出网表与约束

Writing Netlists and Constraints for a HyperScale Block

使用 `write_context`、`write_script` 等命令保存块数据。

22.2.20 HyperScale ECO 收敛方法

HyperScale ECO Closure Methods

PrimeTime ECO 流程与 HyperScale 集成：

- 顶层 ECO 修复—在顶层运行 `fix_eco_*`，写出变更列表。
- 块级 ECO 修复—在块级运行，使用更新后的块 context。
- 使用更新块 context 进行块级 ECO。
- MIM 的 ECO—跨实例一致应用更改。
- 在 ECO 流程中解析块违例。
- 并行运行 ECO 以改善运行时间与容量。

HyperScale ECO 与 IC Compiler、StarRC 的接口支持 signoff 增量 ECO 流程。

22.2.21 HyperScale 中的时序分析

Timing Analysis in HyperScale

HyperScale 顶层分析使用块 HyperScale 模型替代内部网表，在接口保留完整时序信息。支持 CRPR、SI、POCV 等分析模式。

路径类型标记：I（ideal network）、H（HyperScale context override）等。

22.2.22 路径特定例外建模

Path-Specific Exception Modeling

HyperScale 在块边界建模路径特定例外（false path、multicycle path 等），确保顶层与块级约束一致。

（原书 *Figure 393*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.16: 两级层次设计示例

22.3 Context 特征化

Context Characterization

Context 特征化支持块级分析：写出块在顶层内的时序 context，块可单独分析并获得在顶层环境下工作的准确时序。

（原书 *Figure 424*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.17: Context 特征化示例

22.3.1 将 Context 用于块级时序分析

Using Context for Block-Level Timing Analysis

使用 `characterize_context` 标记待特征化块，`update_timing` 后执行特征化，`write_context` 写出 context 文件。

```
characterize_context -block block1 -instances BLK1
update_timing
write_context -format ptsh -output BLK1_context [get_cells BLK1]
```

读入块级分析：

```
read_context -path BLK1_context
```

context 包含输入端口到达时间、输出端口要求时间、转换时间、case 值、串扰信息与时钟映射。

22.3.2 Context 特征化命令与选项

Context Characterization Commands and Options

- `characterize_context` —指定块与实例
- `write_context` —写出 context (ptsh/sdc 格式)
- `read_context` —读入 context
- `report_context` —报告 context 信息
- `remove_context` —移除已加载 context

22.4 Context 预算

Context Budgeting

Context Budgeting (PrimeTime-APX) 在层次化流程中为块接口分配时序预算，从顶层 slack 按比例分配到各块段，生成块级 I/O 约束。

(原书 *Figure 425*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.18: Context Budgeting 概念

22.4.1 Budgeting 期间如何调整 Context

How Contexts are Adjusted During Budgeting

使用 `set_timing_budget` 配置预算模式，`update_budget` 在时序更新后计算预算，调整各块边界的 input/output delay。

```
characterize_context -block block1 -instances BLK1
characterize_context -block block2 -instances BLK2
update_timing
set_timing_budget -mode pin_slack
update_budget
report_budget -through BLK1/Z
```

22.4.2 报告 Context 预算

Reporting the Context Budget

`report_budget -through pin` 报告通过指定层次 pin 的关键路径各段延迟、权重与分配预算。若 pin 有 context 调整，还显示调整计算细节。

表 22.3: report_budget 输出列示例

Point	Mode	Delay	Weight	Budget
BLK1/Z	pin_slack	4.00	0.33	3.33
MID/A	pin_slack	1.00	0.08	0.83
MID/Z	pin_slack	2.00	0.17	1.67
BLK2/A	pin_slack	1.00	0.08	0.83
BLK2/FF/D	pin_slack	4.00	0.33	3.33

22.4.3 自定义 Context 预算

Customizing the Context Budget

默认按路径段延迟比例分配 slack，可自定义：

- 应用 Slack 裕量（-slack_margin、-positive_slack_margin）
- 设置块段目标 slack
- 设置块段精确延迟
- 设置块段时钟周期百分比
- 设置块段最小延迟
- 预算规范的优先级

应用 Slack 裕量

Applying Slack Margins `set_timing_budget -slack_margin` 影响违例路径（从负 slack 减去裕量）；`-positive_slack_margin` 影响通过路径（最多减至零 slack）。

22.4.4 写出预算后的 Context

Writing Out Budgeted Context

预算计算后使用 `write_context` 写出含预算约束的块 context，供块级独立分析使用。

22.4.5 在 HyperScale 流程中使用 Context 预算

Using Context Budgeting in a HyperScale Flow

HyperScale 顶层运行中使能 context budgeting 时：`set_hier_config -block` 自动选择块进行 context 特征化（非 HyperScale 全网表块仍须 `characterize_context`）；`write_hier_data` 自动包含预算调整（仍可用 `write_context` 做独立测试）。块级默认使用预算 context；用 `read_context -exclude-budget` 读入非预算实际 context。

注意

Context Budgeting 限制：`set_timing_budget -target_slack` 仅用于块 feedthrough 段；`update_budget` 只能运行一次。

22.4.6 预算规范优先级

Budget Specification Precedence

同一路径多段应用多个规范时顺序：

1. 应用 `-slack_margin/-positive_slack_margin` 更新路径 slack
2. 应用 `timing_budget_segment_delay_threshold` 放大小段延迟
3. 应用 `-target_slack/-delay_value/-percent_value`
4. 其余段按延迟比例分配剩余 slack

22.5 提取时序模型（ETM）

Extracted Timing Model (ETM)

ETM 是由 `extract_model` 生成的块抽象时序模型，可在更高层次替代块网表进行分析。优势：

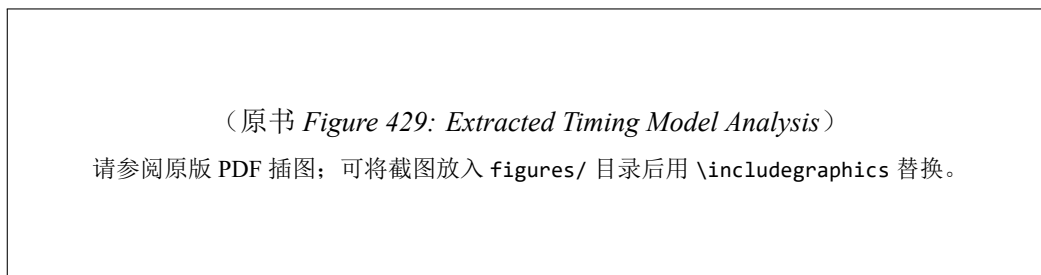


图 22.19: ETM 分析

降低 PrimeTime/DC/ICC 全芯片分析运行时间与内存；保护 IP 网表。模型用组合与时序弧抽象接口行为，延时以 NLDM/CCS 查找表表示，随输入转换与输出负载变化，属 context-independent 模型。

22.5.1 ETM 使用流程

Usage Flow of ETM

块级：读入网表/寄生/约束，`update_timing`，设置 `hier_modeling_version 2.0`，`extract_model ... -validate timing` 自动验证网表与包装器中模型实例的 slack 匹配。顶层：用 `extract_model` 生成的 `.db/.lib` 替换块实例，读入顶层网表与约束分析。

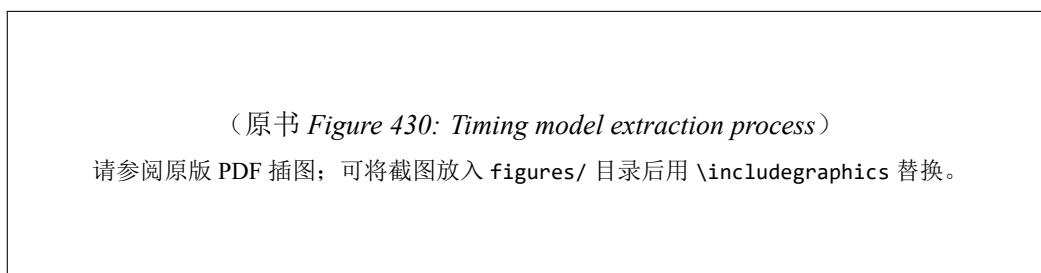


图 22.20: 时序模型提取过程

(原书 *Figure 431–432: Gate-Level Netlist and Extracted Model*)
请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.21: 门级网表与提取模型

22.5.2 ETM 流程概述

ETM Flow Overview

典型 ETM 流程：1. 准备块设计（读入网表、寄生、约束）。2. `check_timing` 检查潜在问题。3. `report_timing` 验证时序。4. 设置模型提取变量。5. `extract_model` 生成.db/.lib 模型。6. 在顶层实例化模型并分析。

22.5.3 生成模型

Generating the Model

```
pt_shell> extract_model -output example_model -format {db}
```

生成.lib 模型时可能含 `min_delay_flag`、`original_pin` 等用户定义属性。

22.5.4 时序模型提取细节

Timing Model Extraction Details

`extract_model` 从原始设计时序路径提取简单时序弧，涵盖：

- 边界 net 与内部 net
- 输入到寄存器路径
- 输入到输出路径
- 寄存器到输出路径
- 寄存器到寄存器路径（不提取，仅遍历接口逻辑）
- 时钟路径
- 透明锁存器与时间借用
- 最小脉冲宽度与最小周期
- False path
- 时钟门控检查
- 反标延迟
- 噪声特性

边界 net 与内部 net

Boundary Nets and Internal Nets 提取模型不保留原始边界 net，而将边界 net 延迟分解到模型中。内部 net 无详细寄生时 PrimeTime 计算电容/电阻；有 SPEF/RSPF 反标时使用详细寄生值。

输入到寄存器路径

Paths From Inputs to Registers 提取为输入 pin 与寄存器时钟之间的 setup 弧与 hold 弧；setup 弧捕获最长路径延迟加寄存器 setup 时间，hold 弧捕获最短路径延迟加 hold 时间。

输入到输出与寄存器到输出路径

Paths From Inputs/Registers to Outputs 各提取两条延迟弧（最长与最短路径）。延迟为输入转换时间与输出负载电容的函数；弧与使用环境无关，在不同环境中延迟可能不同。

时钟路径

Clock Paths 时钟网络延迟与转换时间反映在模型中，但不含时钟 latency；使用模型时须指定 source latency 与 network latency。

透明锁存器与时间借用

Transparent Latches and Time Borrowing extract_model 可提取接口逻辑中简单锁存器结构，在指定条件下保留最多两级借用行为。-latch_levels (0-2，默认 2) 控制考虑的透明锁存器级数。

write_interface_timing 与 compare_interface_timing 用于验证模型与网表时序，对含锁存器接口尤为重要。

最小脉冲宽度与最小周期

(原书 Figure 433)

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 22.22: 最小脉冲宽度与最小周期提取

Minimum Pulse Width and Minimum Period

22.5.5 控制提取表大小

Controlling the Extracted Table Sizes

应用变量控制查找表索引数量与范围，影响模型精度与文件大小。set_extract_model_indexes 显式指定电容/转换索引；reset_extract_model_indexes 移除用户指定索引。set_extract_model_margin 在提取时为延时/约束弧加固定裕量。

22.5.6 指定模型索引与裕量

Specifying Model Indexes and Margins

```
set_extract_model_indexes -type capacitance \
  {0.11 0.12 0.15 0.18 0.20} -ports [get_ports out[1]]
set_extract_model_margin -max 10 -ports [get_ports *]
```

22.5.7 抽头输出端口

Tapped Output Ports

抽头输出（同时连驱动与接收）是不良设计实践，工具发 MEXT-75 警告。NLDM 默认可建 3-D 弧（related_output_load）；设 extract_model_with_3d_arcs false 得 2-D 弧。CCS 提取恒为 2-D。稳健层次建模应缓冲所有端口。

22.5.8 时序例外与生成时钟

Timing Exceptions and Generated Clocks

接口 false path 影响提取弧；multicycle path 调整 setup/hold/延时。建议在接口避免复杂例外，提取后在顶层对 ETM 实例施加端口/时钟特定约束。生成时钟命名：instance_name/generated_clock_name；ETM 保留与接口寄存器相关的生成时钟及 feedthrough 时钟路径。级联生成时钟应逐级引用 master（div-by-2 再 div-by-4），勿均指原始 master。

22.5.9 提取内部 Pin 与检查 Pin

Extracting Internal Pins and Check Pins

内部 pin 时钟信息存为模型内部 pin；时钟网络含组合路径时创建 check pin 以允许时钟追踪继续到时钟输出，否则 setup/hold 弧会阻止时钟路径遍历。

（原书 *Figure 441–442: Check pins in ETM*）

请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 22.23: ETM 中的检查 pin

22.5.10 跨块内电源域合并模型

Merging Models Across Block-Internal Power Supplies

多电源域块可合并为单一 ETM 或按域分别提取，取决于分析需求。

22.5.11 保守块时序分析与模型提取

Conservative Block Timing Analysis and Model Extraction

块分析（`update_timing`）质量直接决定 `extract_model` 模型质量。遵循保守块收敛方法以产生高质量 ETM。

块 **Context** 的保守建模

Conservative Modeling of the Block Context 建议采用 STA 最坏情况方法：

- 穷尽保守评估块使用场景与 `context`（PVT、电压、温度、裕量、驱动/接收类型等）
- 以值或条件范围指定 `context`，覆盖所有实例化场景
- 对时序分析与模型提取应用相同约束

块输出约束

Constraints for Block Outputs

```
set_load -pin_load -min $min_cap [all_outputs]
set_load -pin_load -max $max_cap [all_outputs]
```

通常 $min_{cap} = 0.0 \square max_{cap}$ 保守估计为强驱动器输入电容加典型线电容。

块输入约束

Constraints for Block Inputs

```
set_driving_cell -min -input_transition_rise/fall $min_slew \
-lib_cell $strong_driver ...
set_driving_cell -max -input_transition_rise/fall $max_slew \
-lib_cell $weak_driver ...
```

对长顶层连线可在输入端口加集总电容建模；IP 块周边缓冲环是层次 `signoff` 的最安全做法。

（原书 *Figure 448*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.24: 在输入端口设置 driving cell

转换时间传播效应

Slew Propagation Effects ETM .lib 文件用 NLDM 查找表表示延迟、转换与约束；顶层到达 ETM 的慢转换可能触发 `max_transition` DRC。

22.5.12 ETM 的 SI 与串扰建模

SI and Crosstalk Modeling in ETM

提取时使用当前分析的边界串扰延迟；`si_filter_keep_all_port_aggressors` 在 `extract_model` 时强制为 `true`，包含所有端口级 aggressor 以实现保守 SI 建模。

22.5.13 模型验证

Model Validation

验证流程：1. 对块网表 `write_interface_timing` 作为参考。2. `extract_model -test_design` 生成测试设计。3. 对测试设计应用约束并 `update_timing`。4. `compare_interface_timing` 比较 slack（典型绝对容差 20 ps）。

排查模型验证不匹配

Troubleshooting Model Validation Mismatches

- 工作条件与分析模式须与块分析一致
- 接口路径上不推荐 `multicycle path`；块内部 `multicycle` 不影响 ETM
- 不支持 `set_max_delay/set_min_delay` 异步约束提取
- 包装（wrapper）设计中须对测试网表 `net` 反标零电容/电阻
- 输入 `driving cell` 与集总电容差异可导致验证误差
- ETM 中已含 `derating` 时验证约束避免双重 `derating`

（原书 *Figure 449*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.25: 含 ETM 实例的测试设计

（原书 *Figure 450*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 22.26: 反标边界 `net`

22.5.14 ETM 的噪声特性

Noise Characteristics in ETM

`extract_model` 可将噪声裕量信息纳入模型，供顶层 SI 分析使用。`extract_model -noise` 在模型中包含噪声数据；提取时 `si_filter_keep_all_port_aggressors` 强制为 `true` 以保守包含所有端口 aggressor。

22.5.15 块内部路径与接口路径

Block-Internal and Interface Paths

ETM 替换块后内部寄存器到寄存器路径不可见，须在提取前闭合块内部时序。提取扫描接口路径转换与负载索引，用当前分析边界串扰延时；对输入可用 `set_si_delay_analysis -ignore_arrival` 设无限到达窗口以保守建模未知到达对齐。

22.6 快速时序模型（QTM）

Quick Timing Model (QTM)

QTM 通过 PrimeTime 命令指定输入、输出与约束创建简化时序模型，用于设计早期快速层次化分析，精度低于 ETM 但创建更快。

22.6.1 创建 QTM

Creating a Quick Timing Model

QTM 通过 PrimeTime 命令手动定义端口、弧与约束，无需完整块网表。常用命令：`create_qtm_model`、`create_qtm_port`、`create_qtm_delay_arc`、`create_qtm_constraint_arc`、`create_qtm_drive_type`、`set_qtm_port_drive`、`save_qtm_model`、`report_qtm_model`、`remove_qtm_model`。端口类型：`input`、`output`、`clock`、`bidirectional`。弧可指定 `max/min`（`setup/hold/recovery/removal`）。

表 22.4: 常用 QTM 命令

命令	功能
<code>create_qtm_model</code>	创建 QTM 模型
<code>create_qtm_port</code>	定义端口
<code>create_qtm_delay_arc</code>	创建延迟弧
<code>create_qtm_constraint_arc</code>	创建约束弧
<code>create_qtm_drive_type</code>	定义驱动类型
<code>save_qtm_model</code>	保存 QTM 模型
<code>report_qtm_model</code>	报告 QTM 信息

22.6.2 创建 QTM 的脚本示例

Script Example to Create a Quick Timing Model

```
create_qtm_model -name my_block_qtm
create_qtm_port -type input I1
create_qtm_port -type output O1
create_qtm_port -type clock CLK
create_qtm_drive_type -type input_drive input_drv \
    -lib_cell BUF1
set_qtm_port_drive -type input_drv I1
create_qtm_delay_arc -from I1 -to O1 -path_type max \
    -value 0.5
create_qtm_constraint_arc -from I1 -to CLK \
    -path_type setup -value 0.3
save_qtm_model -format lib -output my_block_qtm.lib
```

22.6.3 在顶层使用 QTM

Using a QTM at the Top Level

顶层读入 QTM .lib 文件，将块实例链接为 QTM 模型进行早期时序评估。随着设计成熟，应迁移到 ETM 或 HyperScale 流程以获得 signoff 精度。

22.6.4 QTM 与 ETM 对比

QTM vs ETM

表 22.5: QTM 与 ETM 对比

特性	QTM	ETM
创建方式	手动指定弧与约束	从网表自动提取
精度	低（早期估算）	高（基于实际实现）
创建速度	快	较慢（需完整块分析）
适用阶段	RTL/早期 netlist	布局后/ signoff 前
SI/串扰	不支持	支持保守 SI 建模
HyperScale 集成	有限	完整支持

参见

- 第二十一章 ECO 流程（HyperScale ECO）
- 第二十二章 HyperScale 与 Context 特征化

22.6.5 从 ETM 层次化流程迁移

Moving From an ETM Hierarchical Flow

若当前使用传统 ETM/ILM 自底向上流程，迁移到 HyperScale 可获得更接近扁平分析的精度，同时保留层次化运行时间与内存优势。迁移步骤概览：

1. 在块级完成布局布线并关闭块内部时序。
2. 运行顶层 HyperScale 分析，characterize_context 并 write_context。
3. 块级读入 context 进行 signoff 级块分析。
4. 迭代顶层与块级直至收敛。
5. 逐步用 HyperScale 块模型替代 ETM 用于顶层 signoff。

MIM 悲观度

MIM Pessimism 多重实例化模块（MIM）中，各实例的时钟到达时间可能不同。HyperScale 在顶层分析时对 MIM 输出路径施加 mim_pessimism 修正，将实际时钟到达与最差到达实例之间的差值加回，消除不必要悲观。

可查询时序路径对象的 mim_pessimism 属性。

```
pt_shell> report_timing -path_type full_clock -delay_type max \
-input_pins -max_paths 1 -sort_by slack
# ... report shows clock reconvergence pessimism and MIM pessimism ...
```

时序降额（Derating）

Timing Derating 层次化分析中 OCV/AOCV/POCV 降额在块边界传递须保持一致。HyperScale 在 write_context/read_context 时保留降额信息；块级与顶层须使用兼容的降额策略。

CRPR 与块边界

CRPR at Block Boundaries 时钟重汇悲观度移除（CRPR）在 HyperScale 顶层分析中跨块边界计算。块模型保留接口 CRPR 信息，使顶层 CRPR 信用与扁平分析一致。

边界检查与违例报告

Boundary Checks and Violation Reporting report_hyperscale_constraints 报告块边界上的约束检查，包括 setup/hold/DRC 在块与顶层的对比、降额窗口类型、时钟映射引用等。用于调试块/顶层不一致。

约束提取器限制

Constraint Extractor Limitations 约束提取器不支持：UPF 命令、set_ocvm_table_group -type pocvm、set_si_delay_disable_statistical；SDC 格式不支持 set_clock_sense（.pt 格式写为 set_sense -type clock）。

常见错误：HCEXT-004（块内生成时钟定义问题）、UITE-461（生成时钟不可满足）、PTE-004（生成时钟 pin 在环中或在两个时钟源 fanout 中）。

表 22.6: HyperScale 边界检查违例类型

违例类型	边界检查名称
Setup	boundary_setup_check
Hold	boundary_hold_check
Max transition	boundary_transition_check
Max capacitance	boundary_capacitance_check

反相时钟提取

Inverted Clock Extraction 约束提取器识别反相时钟关系并在块级约束中正确写出，确保块级时钟定义与顶层一致。

生成块约束与块边界 Context 数据

Generating Block Constraint and Block Boundary Context Data characterize_context 与 write_context 组合可一次性生成块级 SDC 约束与边界 context，供 HyperScale 块级运行使用。

在较少主机上并行运行 HyperScale

Running HyperScale on Fewer Hosts 大型设计的顶层与多块分析可分布式运行；通过合理调度块级作业与共享 HyperScale 数据目录降低硬件需求。

22.6.6 Context 特征化 workflow 详解

Context Characterization Workflow Details

特征化在 update_timing 期间执行，将顶层时序环境映射到块边界：

- 输入端口：到达时间、转换时间、case 值、串扰 delta
- 输出端口：要求时间、转换时间、外部负载
- 时钟端口：时钟定义映射、source latency、网络延迟
- 侧输入/stub pin：关键路径到达/要求时间

22.6.7 Context 文件格式

Context File Formats

write_context 支持 ptsh 与 sdc 格式。ptsh 格式保留完整 PrimeTime 语义；sdc 格式便于第三方工具读入。

22.6.8 多块 Context 特征化

Characterizing Multiple Blocks

一次顶层运行可对多个块 characterize_context；update_timing 后分别 write_context 到各块目录。

22.6.9 Budgeting 流程示例

Context Budgeting Flow Example

典型三块设计（BLK1—MID—BLK2）中，顶层负 slack 通过 `update_budget` 按段延迟比例分配到各块接口，生成块级 input/output delay 预算。

设置块段目标 Slack

Setting the Target Slack of a Block Segment `set_timing_budget` 支持为特定块段指定目标 slack，覆盖默认比例分配。

设置块段精确延迟

Setting the Exact Delay of a Block Segment 可为块接口指定固定预算延迟，用于已知延迟的互连或硬约束通道。

设置时钟周期百分比

Setting the Clock Period Percentage of a Block Segment 按时钟周期百分比分配段预算，适用于均匀分配时钟周期预算的场景。

设置最小块段延迟

Setting a Minimum Block Segment Delay 保证每段获得最小延迟预算，避免小段被分配过少预算。

预算规范优先级

Precedence of Context Budget Specifications 显式段规范优先于全局 `slack_margin`; `pin_slack` 模式为默认。多种规范同时存在时按文档优先级应用。

22.6.10 Budgeting 与层次化块模型

Hierarchical Analysis Using Block Models With Budgeting

预算后的 context 可与 ETM/QTM/HyperScale 块模型配合：先 `update_budget`，再 `write_context`，块级读入后获得带预算的 I/O 约束。

22.6.11 ETM 分析流程详解

ETM Analysis Flow in Detail

False Path 与 ETM

False Paths and ETM 接口逻辑上的 false path 在提取时被尊重；被例外命令标记的路径不生成对应弧。块内部 register-to-register false path 不影响 ETM（ETM 仅覆盖接口）。

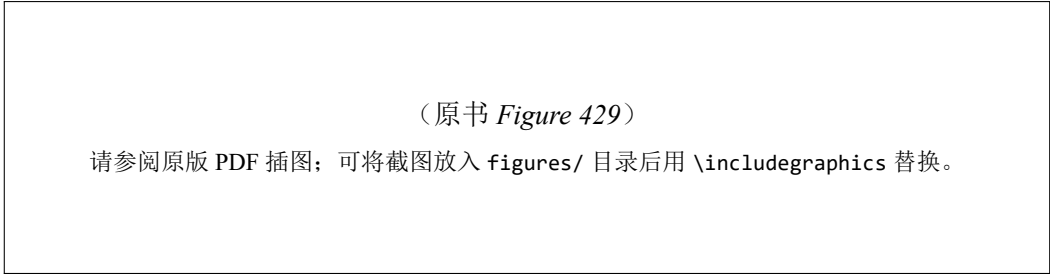


图 22.27: 提取时序模型分析

时钟门控检查

Clock-Gating Checks `extract_model` 将时钟门控 setup/hold 检查传播到提取模型的时钟 pin 属性上，供顶层分析使用。

反标延迟

Back-Annotated Delays 若内部 net 反标了延迟或电容，提取使用反标值而非计算值。这影响弧延迟精度，须确保块分析时寄生数据完整。

噪声特性提取

Noise Characteristics Extraction ETM 可包含噪声裕量表，描述块输出在不同负载与转换时间下的噪声容限。

22.6.12 ETM 表与变量控制

ETM Table and Variable Control

表 22.7: 控制 ETM 提取表大小的变量

变量	说明
<code>extract_model_capacitance_limit</code>	输出负载电容扫描上限
<code>extract_model_transition_limit</code>	输入转换时间扫描上限
<code>extract_model_number_of_capacitance_points</code>	电容索引点数
<code>extract_model_number_of_transition_points</code>	转换时间索引点数

22.6.13 ETM 裕量与角

Margin for Block Analysis and Model

可在提取前对块分析应用裕量，使 ETM 内置保守度。不同工艺角与模式应分别分析与提取模型。

22.6.14 工作条件与分析模式

Operating Conditions and Analysis Modes

注意

每个工艺角与模式须单独执行 `update_timing` 与 `extract_model`；顶层须读入对应角的 ETM 库。

22.6.15 包装设计与验证

Wrapper Design Effects

`extract_model -test_design` 创建的包装设计中，须在全部 `net` 上反标零电阻与零电容以消除线负载延迟：

```
set_resistance 0 [get_nets *]
set_load 0 [get_nets *]
```

22.6.16 ETM 的 Derating 与 AOCV

Derating and AOCV in ETM

若网表分析中应用了降额并纳入 ETM 时序弧，验证时包装设计与 ETM 不得再次应用相同降额，避免双重降额。

启用 AOCV 时，验证流程须与提取时 AOCV 设置一致。

22.6.17 层次化 Signoff 使用 ETM 的最佳实践

Best Practices for Hierarchical Signoff With ETMs

- 块内部时序必须在提取前关闭。
- IP 块周边布置缓冲环，控制端口转换时间范围。
- 对输入使用 `set_driving_cell` 与适当 `set_load` 建模外部环境。
- 对输出使用保守 `load` 范围。
- 使用 `write_interface_timing/compare_interface_timing` 验证。
- 最终 signoff 考虑迁移到 HyperScale 或扁平分析确认。

22.6.18 定义快速时序模型

Defining a Quick Timing Model

定义 QTM 步骤：1. `create_qtm_model` 创建模型。2. `create_qtm_port` 定义输入/输出/时钟端口。3. `create_qtm_drive_type` 与 `set_qtm_port_drive` 设置驱动。4. `create_qtm_delay_arc` 定义组合/时序延迟弧。5. `create_qtm_constraint_arc` 定义 setup/hold 约束弧。6. `save_qtm_model` 保存为 .lib 或 .db。



图 22.28: 块的快速时序模型表示

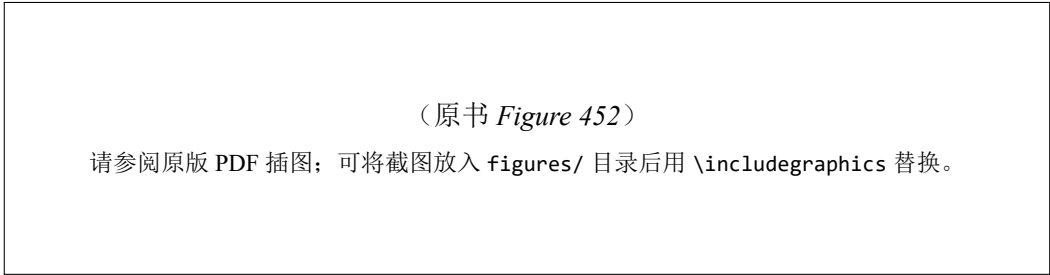


图 22.29: 快速时序模型分析

22.6.19 在设计中实例化 QTM

Instantiating a Quick Timing Model in a Design

```
read_qtm_model my_block_qtm.lib
set_link_library "*" my_block_qtm.lib"
link_design top
```

链接后块实例使用 QTM 替代完整网表，顶层可快速评估接口时序。

22.6.20 QTM 全局参数命令

QTM Global Model Parameters

表 22.8: QTM 全局模型参数命令

命令	全局模型参数
set_qtm_global_parameter	设置工艺、温度、电压等
report_qtm_model	报告已定义弧与端口
remove_qtm_model	删除 QTM 定义

22.6.21 HyperScale ECO 流程细节

HyperScale ECO Flow Details

PrimeTime ECO 与 HyperScale 集成支持顶层 fix_eco_* 与块级 ECO 协同。顶层 ECO 后须 write_hier_data 更新块 context；块级 ECO 使用最新 context 独立修复。MIM 实例须一致应用 ECO 变更。可

与 IC Compiler、StarRC 增量寄生接口做 signoff ECO。

并行 ECO：多块可同时运行 ECO 脚本以缩短收敛时间；各块 write_hier_data 后顶层统一 link_design 与 update_timing。

22.6.22 HyperScale 路径类型标记

HyperScale Path Type Markers

report_timing 路径点标记：“@”表示顶层施加的约束；“&”表示块内计算延时；“I”表示 ideal network；“H”表示 HyperScale context override。块级报告跨边界路径时，input external delay 行后的“@”表示该延时来自顶层分析。

22.6.23 跨块内电源域合并 ETM

Merging ETMs Across Block-Internal Power Supplies

多电源域块可按域分别 extract_model 再合并，或单次提取合并模型；合并时须保证各域接口弧与电平转换器行为正确抽象。UPF 意图须在块分析中正确加载（load_upf）后再提取。

22.6.24 QTM 约束弧类型

QTM Constraint Arc Types

create_qtm_constraint_arc 支持 setup、hold、recovery、removal、min_pulse_width、minimum_period 等 path_type。时钟端口与数据端口组合定义寄存器接口约束；可与 create_qtm_delay_arc 的 max/min 路径类型配合建模建立/保持时间。

22.6.25 Context 预算模式

Context Budget Modes

set_timing_budget -mode 支持 pin_slack（默认，按 pin 段 slack 比例分配）、path_slack（整条路径 slack 分配）等模式。update_budget 在 update_timing 之后运行一次，根据顶层违例路径调整块边界 input/output delay。

22.6.26 ETM 与 CCS 数据提取

Extracting Models With CCS Data

extract_model 可生成含 CCS 时序表的 ETM，提高顶层使用 CCS 延时计算时的精度。须块分析启用 CCS 且库含 CCS 数据；格式仍可为.db/.lib。与 NLDM-only ETM 相比，CCS ETM 文件更大但顶层 RC 分析更准确。

22.6.27 层次化分析选型指南

Hierarchical Analysis Selection Guide

流程选型参考：

- 早期 RTL/规划—QTM 或预算约束 + ETM
- 块布局后独立收敛—Context 特征化 + 块级 signoff
- 全芯片 signoff—HyperScale 自顶向下或扁平分析
- IP 交付—保守 ETM + 接口验证脚本
- 内存受限大芯片—HyperScale 多块分治

22.6.28 HyperScale 块 Context 数据类型

HyperScale Block Context Data Types

表 22.9: Context 数据类型与合并策略（表 66 摘要）

数据类型	MIM 合并取值
到达时间	各实例最早到达
要求时间	各实例最晚要求
转换时间	最坏转换
Case 值	一致则保留，否则 X
串扰 delta	各实例最坏
时钟映射	合并时钟定义

write_hier_data 写出全部 context 类型；read_context 在块级恢复。块级分析可禁用部分 context 调整：set_dont_override -include arc_delay/case_value/pin_transition。

22.6.29 时钟映射检查与修复

Clock Mapping Check and Fixes

report_constraint -boundary_check -all_violators 列出 clock_mapping、clock_attributes、clock_uncertainty 等可修复违例。常见修复：块级 create_clock 与顶层周期一致；set_propagated_clock 匹配 is_propagated 属性；添加 set_clock_uncertainty 匹配顶层。HyperScale 约束提取器可从扁平约束生成一致块级时钟定义，减少手工映射错误。

22.6.30 ETM 输入到寄存器与寄存器到输出路径

ETM Input-to-Register and Register-to-Output Paths

输入到寄存器：提取 setup 弧（最长数据路径 +setup）与 hold 弧（最短数据路径 +hold）。寄存器到输出：提取 max/min 两条延迟弧，延时为时钟到输出 pin 的延迟。输入到输出：组合路径提取 max/min 延迟弧，为输入转换与输出负载的函数。寄存器到寄存器路径不进入 ETM（仅遍历接口逻辑中的寄存器）。

22.6.31 ETM 透明锁存器与时间借用

ETM Transparent Latches and Time Borrowing

`extract_model -latch_levels 0/1/2` 控制接口透明锁存器级数（默认 2）。含锁存器接口时须用 `write_interface_timing` 与 `compare_interface_timing` 验证借用行为。顶层使用 ETM 时锁存器借用由模型内约束弧表达。

22.6.32 ETM 模型验证流程

ETM Model Validation Flow

推荐验证步骤：

1. 块网表 `update_timing` 后 `write_interface_timing` 作黄金参考
2. `extract_model -test_design` 生成含 ETM 实例的包装器
3. 包装器应用与块分析相同工作条件与约束
4. `compare_interface_timing` 比较各接口 `pin slack`（容差约 20 ps）

不匹配排查：工作条件/分析模式不一致；接口 `multicycle`；`set_max_delay` 不支持提取；包装器未零反标边界 `net`；`driving cell` 与集总电容差异；双重 `derating`。

22.6.33 QTM 驱动类型与负载

QTM Drive Types and Loads

`create_qtm_drive_type` 定义输入驱动（`-lib_cell`）或输出负载模板；`set_qtm_port_drive` 将驱动类型关联到端口。`create_qtm_port -type bidirectional` 建模 `inout`；时钟端口用于约束弧 `reference`。`save_qtm_model -format lib` 生成 Liberty 供 `link_design` 使用。

22.6.34 Context 预算自定义示例

Context Budget Customization Example

```
characterize_context -block block1 -instances BLK1
characterize_context -block block2 -instances BLK2
update_timing
set_timing_budget -mode pin_slack -slack_margin 0.1
set_timing_budget -through BLK1/Z -target_slack 0.5
update_budget
report_budget -through BLK1/Z
write_context -format ptsh -output BLK1_budgeted
```

22.6.35 HyperScale 与 SI 分析

HyperScale and SI Analysis

HyperScale 顶层与块级均支持 `update_noise` 与耦合延时分析。块 `context` 包含端口级串扰 `aggressor` 信息；块级读入 `context` 后 SI 分析与顶层一致。写 `context` 时保留无限窗口 `aggressor` 设置以确保保守 SI 边界。

22.6.36 report_hier_analysis 报告

Reporting HyperScale Configuration

`report_hier_analysis` 默认报告块名、时间戳、HyperScale 路径。可报告额外信息：链接状态、MIM 配置、`context` 时间戳等。若路径或数据错误，须退出会话、修正 `set_hier_config` 后重新 `link_design`。

22.6.37 块边界自动调整机制

Block Boundary Automatic Adjustment

顶层 HyperScale 分析后，块级 `read_context` 可自动调整用户边界约束：输入/输出 `delay`、`set_case_analysis`、端口转换时间、时钟不确定性等。不可解析的违例须用户修正块约束；`set_dont_override` 可抑制特定类型自动调整。该机制使块级在顶层真实环境下收敛，无需手工复制顶层到达/要求时间。

22.6.38 ETM 噪声与功耗选项

ETM Noise and Power Options

`extract_model -noise` 在模型中包含噪声特性；`-power` 包含功耗信息。`-format db` 供 Synopsys 工具直接使用；`-format lib` 便于审阅与第三方兼容。提取前 `check_timing` 与 `report_constraint` 确保无违例；多模式设计按 `mode` 分别提取后合并或分角使用。

22.6.39 QTM 与 HyperScale/ETM 选型

When to Use QTM Versus ETM Versus HyperScale

QTM 适用于早期架构探索与接口预算估算，创建快但精度低。ETM 适用于 IP 交付与顶层集成，须完整实现网表与保守边界约束。HyperScale 适用于 `signoff` 级层次化分析，保留块内部网表精度并降低顶层容量。成熟流程通常：QTM（早期）→ ETM（中期）→ HyperScale（`signoff`）或扁平分析。

无论采用何种块抽象，块内部时序必须在抽象前闭合；顶层仅对块接口施加环境约束，不能替代块内 `signoff`。层次化团队应约定 HyperScale 数据目录命名、时间戳检查与 `context` 版本管理规范。块级与顶层脚本应分离但共享 `characterize_context` 写出目录，避免时钟映射与约束漂移。`signoff` 前建议对关键跨块路径做扁平与 HyperScale 结果对比抽样验证，确认块模型与 `context` 未引入系统性悲观或乐观。对含多电源域与 DVFS 的设计，须在块与顶层使用一致的 SMVA 电压场景与二进制 `context` 格式。

Chapter 23

将 PrimeTime 与 SPICE 配合使用

Using PrimeTime With SPICE

您可以通过以下功能将 PrimeTime 结果与 SPICE 分析进行比较：

- **Simulation Link:** 关联 PrimeTime 与 SPICE 结果
- 使用 `write_spice_deck` 命令生成 SPICE 网表

23.1 Simulation Link: 关联 PrimeTime 与 SPICE 结果

Simulation Link to Correlate PrimeTime and SPICE Results

PrimeTime 静态时序分析基于门级网表表示，并结合库中门级单元的时序表征。在门级进行分析时，工具可在合理时间内对超大规模设计做穷举时序分析。

某些情况下，您可能希望用电路仿真进行晶体管级时序分析。这种高精度对以下目的可能有用：

- 验证 PrimeTime 在新工艺节点上的库表征方法
- 进一步分析信号质量，例如时钟网络中的高扇出网
- 进一步分析信号完整性效应，例如用于设计修复的双切换（double switching）

由于无法对含数百万门的网表做 SPICE 仿真，PrimeTime 提供 simulation link 以执行以下 SPICE 分析：

- Simulation Link 命令摘要
- 关联基于路径的非耦合 PrimeTime 与 SPICE 分析
- 关联基于弧的耦合 PrimeTime SI 与 SPICE 分析
- 关联 PrimeTime SI 与 SPICE 噪声分析
- 在多台机器上分布仿真

23.1.1 Simulation Link 命令摘要

Summary of Simulation Link Commands

以下 simulation link 命令便于在 PrimeTime 中进行 SPICE 分析。

表 23.1: Simulation Link 命令摘要

命令	任务
sim_setup_library	通过指定库名、子电路目录名与头文件名，设置用于仿真的 SPICE 模型。
sim_setup_simulator	指定 HSPICE 仿真器可执行文件路径，以及用于 PrimeTime 与仿真器比较的仿真器选项。
sim_validate_setup	通过在组合逻辑门上仿真来验证仿真设置。指定单元名、测试负载及仿真所用输入转换时间。
sim_validate_path	对指定路径段执行基于路径的非耦合 SPICE 分析，并将仿真结果与静态时序结果比较。
sim_enable_si_correlation	为耦合的一又二分之一级（one-and-a-half-stage）关联启用指定网。
sim_validate_stage	对指定级或一又二分之一级执行基于弧的耦合 SPICE 分析，并将仿真结果与静态时序结果比较。
sim_validate_noise	比较 PrimeTime SI 噪声分析结果与 SPICE 结果。

23.1.2 关联基于路径的非耦合 PrimeTime 与 SPICE 分析

Correlating Path-Based Uncoupled PrimeTime and SPICE Analysis

借助 PrimeTime simulation link，可按以下步骤执行基于路径的非耦合 SPICE 分析：

1. 使用以下命令配置 PrimeTime 静态时序分析：

```
# Disable use of nonconditional timing arcs between pins that have
# at least one conditional timing arc
set_app_var timing_disable_cond_default_arcs true

# Report all paths through parallel cell arcs
set_app_var timing_report_use_worst_parallel_cell_arc false

# Enable waveform propagation
set_app_var delay_calc_waveform_analysis_mode full_design

# Store waveform data when CCS waveform propagation is enabled
set_app_var timing_keep_waveform_on_points true

# Disable advanced on-chip variation (AOCV)
set_app_var timing_aocvm_enable_analysis false
```

```
# Disable AQCV for SPICE correlation at specific corners
remove_ocvm

# Reset user-specified derate factors
reset_timing_derate
```

2. 使用 `sim_setup_library` 设置 SPICE 仿真库，该命令指定库子电路与仿真头文件位置。例如：

```
pt_shell> sim_setup_library -library mylib \
    -sub_circuit my_subckt_dir -header my_header_file
```

3. 指定 SPICE 仿真器路径。例如：

```
pt_shell> sim_setup_simulator \
    -simulator /abc/tools/bin/hspice \
    -simulator_type hspice
```

4. 使用 `sim_validate_setup` 验证仿真设置，该命令对库中组合逻辑门执行仿真，确保 `simulation link` 的 SPICE 设置与库表征所用设置一致。例如：

```
pt_shell> sim_validate_setup -lib_cell k04_lib/inv7Q \
    -from A -to Y \
    -capacitance 0.35 -transition_time 0.12
```

PrimeTime 对指定单元调用 `simulation link` 并与静态时序分析结果比较。要检查 SPICE 环境与库表征环境是否一致，请指定与库索引点匹配的转换时间与负载电容值。

5. 使用带 `-pba_mode path` 的 `get_timing_paths` 创建用于仿真的基于路径分析路径集合。例如：

```
pt_shell> set my_paths \
    [get_timing_paths -pba_mode path -delay_type max]
```

6. 使用 `sim_validate_path` 提交路径进行 SPICE 分析，该命令仿真指定路径段并比较延时与转换时间。例如：

```
pt_shell> sim_validate_path -transition_time $my_paths
```

也可使用 `-from` 与 `-to` 比较路径段。例如：

```
pt_shell> sim_validate_path -from u2/A -to u5/Z \
    -transition_time $my_paths
```

验证报告显示 SPICE 仿真值（Ref）、PrimeTime 值（Val）、绝对差（Diff）与百分比差（%），分别针对延时与转换时间。例如：

Comparing path number 0 :										
Pin	Sense	Transition time				Delay				
		Ref	Val	Diff	%	Ref	Val	Diff		
U3/A	f	0.0197	0.01870	-0.00100	-5.12	0.017613	# 0.01749	-0.000117	-0.67	
U4/A	r	0.0175	0.01730	-0.00025	-1.45	0.014172	# 0.01404	-0.000129	-0.91	
U5/A	f	0.13	0.1371	0.0031	2.33	0.083447	# 0.08412	0.000681	0.82	

U5/Z	f	0.239	0.2431	0.00365	1.53	0.131	#	0.1345	0.00267	2.03

Path segment		0.0830	0.08385	0.000842	1.01	0.24713		0.2502	0.00310	1.26

注意

- 若路径起始于输入端口，请用 `set_driving_cell` 在输入端口上设置简单组合逻辑单元（如缓冲器或反相器）作为驱动单元。例如：
`pt_shell> set_driving_cell -lib_cell IBUF2 [get_ports in1]`
- 同一批路径上，SPICE 仿真运行时间通常长于 `report_timing`。
- 由于 SPICE 每一级都需时序窗口对齐，不支持基于路径的串扰 SPICE 仿真。也不支持电压与温度缩放。

23.1.3 关联基于弧的耦合 PrimeTime SI 与 SPICE 分析

Correlating Arc-Based Coupled PrimeTime SI and SPICE Analysis

借助 PrimeTime SI simulation link，可在 SPICE 中分析耦合延时的信号完整性行为，基于受害网弧写出 SPICE 网表，用 HSPICE 仿真，并自动比较 HSPICE 与 PrimeTime SI 结果。

使用 PrimeTime SI 时，可选择一个或多个网弧进行仿真。为关联目的，建议使用一又二分之一级延时。该功能仅当网具有标注的耦合 RC 网络时适用。计算中不考虑被滤除的 aggressor，其耦合电容在 SPICE 网表中被接地。

基于弧的耦合 SPICE 分析要求启用 PrimeTime SI。不支持电压与温度缩放。

按以下步骤执行基于弧的耦合 SPICE 分析：

1. 使用以下命令配置 PrimeTime SI 分析：

```
# Enable PrimeTime SI analysis
set_app_var si_enable_analysis true

# Disable use of nonconditional timing arcs between pins that have
# at least one conditional timing arc
set_app_var timing_disable_cond_default_arcs true

# Report all paths through parallel cell arcs
set_app_var timing_report_use_worst_parallel_cell_arc false

# Enable waveform propagation
set_app_var delay_calc_waveform_analysis_mode full_design

# Store waveform data when CCS waveform propagation is enabled
set_app_var timing_keep_waveform_on_points true

# Disable advanced on-chip variation (AOCV)
set_app_var timing_aocvm_enable_analysis false

# Disable AOCV for SPICE correlation at specific corners
```

```
remove_ocvm

# Reset user-specified derate factors
reset_timing_derate
```

2. 使用 `sim_setup_library` 设置 SPICE 仿真库（同路径分析）。
3. 使用 `sim_setup_simulator` 指定 SPICE 仿真器路径（同路径分析）。
4. 使用 `sim_validate_setup` 验证仿真设置（同路径分析）。
5. 使用 `get_timing_arcs` 创建用于仿真的耦合网弧集合。例如：

```
pt_shell> set my_arcs [get_timing_arcs -from U1/Z -to U2/A]
```

注意

不建议对由端口驱动的级执行基于弧的关联。

6. 为 `$net` 启用耦合的一又二分之一级关联，并执行时序更新。例如：

```
pt_shell> sim_enable_si_correlation \
           [get_nets -of [get_attribute $my_arcs from_pin]]
pt_shell> update_timing -full
```

7. 使用 `sim_validate_stage` 执行 SPICE 分析，该命令仿真指定一又二分之一级并比较级延时。验证报告显示 SPICE 仿真值（Ref）、PrimeTime 值（Val）、绝对差（Diff）与百分比差（%）。例如：

```
pt_shell> sim_validate_stage $my_arcs
```

Pin	Sense	Ref	Delay Val	Diff	%
U2/Z	f	0.59594 #	0.594371	-0.00156873	-0.26
Tolerance	MET			0	3.00

若弧包含接收级，则执行一又二分之一级分析；若不包含接收级，则报告单级关联。例如：

```
pt_shell> set my_arcs [get_timing_arcs -from U1/Z -to FF1/D]
pt_shell> sim_validate_stage $my_arcs
```

Pin	Sense	Ref	Delay Val	Diff	%
FF1/D	f	0.59594 #	0.594371	-0.00156873	-0.26
Tolerance	MET			0	3.00

23.1.4 关联 PrimeTime SI 与 SPICE 噪声分析

Correlating PrimeTime SI and SPICE Noise Analysis

要比较 PrimeTime SI 噪声分析结果与 SPICE 结果，运行 `sim_validate_noise` 命令。该命令比较两工具报告的噪声凸起高度与面积。

噪声关联流程如下：

```
# Specify the nets on which to compare noise analysis
sim_enable_si_correlation [get_nets -of_objects net_driver_pin]

# Run timing and noise analysis
update_timing -full
update_noise -full

# Specify the timing arcs and run noise correlation
set net_arc [get_timing_arcs -from net_driver_pin -to net_receiver_pin]
sim_validate_noise $net_arc -area [-verbose] \
    -analysis_type above_low | below_high
```

23.1.5 在多台机器上分布仿真

Distributing Simulations Across Multiple Machines

若需对大量集合对象运行仿真，可使用 `sim_setup_distributed` 将仿真分布到多台机器。例如：

```
pt_shell> sim_setup_distributed \
    -mode grd \
    -submit_command {qsub -V -cwd -P bnormal} \
    -sim_per_job 2

pt_shell> sim_validate_path $twelve_paths
```

`-sim_per_job` 指定每个提交作业中的集合对象仿真数量。提交作业总数取决于传给仿真命令的集合对象数量。（变分仿真的样本空间不会跨作业拆分。）

分布式仿真作业完成后，工具打印作业摘要：

```
Summary:
-----
Total number of simulations      : 12
Simulation per farm job         : 2
Total number of farm jobs       : 6
Completed number of farm jobs   : 4
Failed number of farm jobs      : 0
Timed out farm jobs             : 2
-----
```

以下命令支持分布式仿真：

- `sim_validate_noise`
- `sim_validate_path`

- `sim_validate_setup`
- `sim_validate_stage`
- `sim_analyze_path`

`sim_setup_distributed` 还提供限制排队作业与重试失败作业的选项。详见 man page。

注意

`sim_setup_distributed` 的限制：不支持 `sim_validate_path` 的 `-from` 与 `-to` 选项。

23.2 使用 `write_spice_deck` 命令生成 SPICE 网表

Generating a SPICE Deck With the `write_spice_deck` Command

要生成 SPICE 网表，运行 `write_spice_deck` 命令。该命令需要 PrimeTime SI 许可证。命令生成包含指定路径或弧上单元，以及时序路径或弧相关网上的电阻、对地电容与对 aggressor 网耦合电容的 SPICE 网表。命令还提供选项以生成激励受害路径与 aggressor 的激励。

关于生成与使用 SPICE 网表，请参阅：

- 为时序路径编写 SPICE 网表
- 为时序弧编写 SPICE 网表
- `write_spice_deck` 中的库激励（sensitization）
- 库驱动波形
- SPICE 仿真所需的其他信息
- `write_spice_deck` 输出示例
- 使用 `write_spice_deck` 进行 SPICE 关联的限制

23.2.1 为时序路径编写 SPICE 网表

Writing a SPICE Deck for a Timing Path

要分析时序路径的串扰延时效应（非静态噪声效应），生成表示该时序路径的 SPICE 网表。使用 `get_timing_paths` 收集路径供 `write_spice_deck` 使用。例如：

```
pt_shell> write_spice_deck -header header.spi \
    -output testcase.spi \
    -logic_one_voltage 1.5 \
    -logic_zero_voltage 0.0 \
    -sub_circuit_file ./SPICE/subckt.spi \
    [get_timing_paths -from A2 -to buf5/A]
```

要收集特定路径或转换，请结合 `-from`、`-to` 或 `-delay_type` 使用 `get_timing_paths`。若不使用这些选项，`get_timing_paths` 收集时序裕量最差的路径。

要为特定受害与 aggressor 转换生成电路激励波形，请使用 `get_timing_paths` 的 `-delay_type` 选项。设为 `max_rise`、`max_fall`、`min_rise` 或 `min_fall` 时，指定延时类型（最大或最小）及路径

端点的转换类型（上升或下降）。对路径端点处的串扰，该设置指定如下图所示的串扰条件。对路径上每条受害网，`write_spice_deck` 会适当激励 aggressor 网以测试指定的最大或最小延时转换。

（原书 *Figure 454: Specifying crosstalk delay transitions for paths*）
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 23.1: 为路径指定串扰延时转换

生成的 SPICE 网表包含整条路径上所有交叉耦合的 aggressor 网与电容。路径很长或很复杂且耦合电容很多时，可能产生过大数据文件，难以实际仿真。此时可尝试用 `get_timing_arcs` 仅选择含受害网及其邻近电路的路径片段。

23.2.2 为时序弧编写 SPICE 网表

Writing a SPICE Deck for a Timing Arc

要分析时序弧的串扰延时效应或静态噪声效应，生成表示该时序弧的 SPICE 网表。结合 `-from`、`-to` 与 `-of_objects` 使用 `get_timing_arcs` 为 `write_spice_deck` 指定弧：

```
pt_shell> write_spice_deck -header header.spi \
    -analysis_type above_high \
    -output ../SIMUL_BEYOND_HIGH/new_general.spi \
    -logic_one_voltage 1.5 -logic_zero_voltage 0.0 \
    -sub_circuit_file ./SPICE/subckt.spi \
    [get_timing_arcs -to buf5/A]
```

使用 `-analysis_type` 指定分析类型，取值如下：

- 串扰延时分析：`max_rise`、`max_fall`、`min_rise` 或 `min_fall`。

（原书 *Figure 455: Crosstalk delay transitions for arcs*）
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 23.2: 时序弧的串扰延时转换

- 静态噪声分析：`above_high`、`below_high`、`above_low` 或 `below_low`。

默认情况下，`write_spice_deck` 将 aggressor 转换放在到达时序窗口中间，而不一定是最坏串扰时刻。要与 PrimeTime SI 与 SPICE 一致，通常需通过一系列 SPICE 仿真“扫描”转换时间以找到最坏转换时间。

(原书 *Figure 456: Crosstalk noise transitions for arcs*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 23.3: 时序弧的串扰噪声转换

为减少仿真工作量，可使用 `write_spice_deck` 的 `-align_aggressors` 选项，将 `aggressor` 转换放在产生 `-analysis_type` 所指定条件下最坏串扰效应的位置。条件可为串扰延时分析的 `max_rise`、`max_fall`、`min_rise`、`min_fall`，或串扰噪声分析的 `above_high`、`below_high`、`above_low`、`below_low`。`aggressor` 对齐功能适用于 `get_timing_arcs` 获得的时序级，不适用于 `get_timing_paths` 获得的时序路径。

`aggressor` 对齐仅针对网时序弧（从单元输出引脚经网到另一单元输入引脚的弧），不针对单元时序弧（单元输入引脚到输出引脚的弧），尽管被驱动网的耦合 RC 网络仍会写出。

当 `write_spice_deck` 对网弧使用 `aggressor` 对齐时，选择与 PrimeTime 中 `update_timing` 期间相同的驱动单元弧。对直接用 `set_driving_cell` 驱动的 `aggressor`，`aggressor` 对齐不起作用。

23.2.3 `write_spice_deck` 中的库激励

Library Sensitization in `write_spice_deck`

`write_spice_deck` 可使用库中的激励信息来激励时序路径或级的逻辑单元，从而用与弧表征时相同的激励来激励时序弧。因此，耦合与非耦合分析的 PrimeTime 与 SPICE 关联更准确。

PrimeTime 使用库中包含的激励信息。若无库指定激励，则根据时序弧的 `when` 条件及组合逻辑函数或时序逻辑二进制状态表应用默认激励。

有关 Liberty 格式中单元激励语法的更多信息，见 *Library Compiler User Guide*。

以下示例说明 `write_spice_deck` 如何使用库定义的激励信息：

```
sensitization (2in_1out){
  pin_names (IN1, IN2, OUT);
  vector (0, "0 0 0") ;
  vector (1, "0 0 1") ;
  vector (2, "0 1 0") ;
  vector (3, "0 1 1") ;
  vector (4, "1 0 0") ;
  vector (5, "1 0 1") ;
  vector (6, "1 1 0") ;
  vector (7, "1 1 1") ;

  cell(my_cell){
    sensitization_master : 2in_1out;
    pin_name_map (A, B, Z);
    ...
    pin(Z) {
      ...
    }
  }
}
```



```

    timing() {
        related_pin : A;
        wave_rise (0, 4, 2, 6, 3);
        wave_fall (1, 5, 3, 6);
        wave_rise_sampling_index : 4;
        wave_fall_sampling_index : 3;
    }
}
}

```

激励模板 `2in_1out` 定义八个向量，每个向量按 `pin_names` 列表顺序定义单元引脚逻辑值。

`cell(my_cell)` 实例化模板 `2in_1out`，并将其引脚 A、B、Z 分别映射到 IN1、IN2、OUT。

对 A 到 Z 的时序弧，`wave_rise` 属性定义引脚 A 与 B 上产生 Z 上升沿的波形，使用 `2in_1out` 模板中先前定义的向量序列。`wave_rise_sampling_index` 定义在 `wave_rise` 的哪个转换上测量延时与转换时间；本例为第四次转换。输出下降转换的激励由 `wave_fall` 与 `wave_fall_sampling_index` 类似定义。

下图展示 PrimeTime SI 中对 `wave_rise` 属性的解释。向量序列 (0, 4, 2, 6, 3) 表示施加到引脚 A 与 B 以在 Z 产生上升沿的波形。A 到 Z 时序弧的延时与转换时间测量发生在第四次序列转换（本例为向量 ID 6 与 3 之间的转换）。

（原书 *Figure 457: Usage of wave_rise in library sensitization definition*）
 请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 23.4: 库激励定义中 `wave_rise` 的用法

PrimeTime SI 在转换之间分配固定时间间隔，使相关逻辑单元有足够时间稳定并初始化到正确状态。可用 `wave_rise_timing_interval` 属性指定不同时间间隔。以下示例将时间间隔设为 0.5 个库时间单位：

```
wave_rise_timing_interval : 0.5;
```

由于 SPICE 仿真器使用自适应时间步进，提供超过逻辑稳定所需时间的仿真时间不会显著增加仿真运行时间。

23.2.4 库驱动波形

Library Driver Waveform

当 `write_spice_deck` 激励单元输入时，使用与时序数据表征相同的波形。PrimeTime SI 在可用时从库获取预驱动（predriver）建模信息。

若库不含预驱动信息，默认使用标准 Synopsys 预驱动。要在 PrimeTime SI 中显式指定预驱动类型，使用 `set_library_driver_waveform` 命令。

`set_library_driver_waveform` 的 `-type` 设置指定预驱动类型，可为简单斜坡或标准 Synopsys 预驱动。若在命令中指定一个或多个库对象（库或库单元集合），则仅应用于这些对象；否则应用于整个设计。

库驱动设置不仅影响 `write_spice_deck`，也影响使用 CCS 模型进行高级延时计算时的预驱动。更多信息见 PrimeTime SI Crosstalk Delay Calculation Using CCS Models。

23.2.5 SPICE 仿真所需的其他信息

Additional Required Information for SPICE Simulation

使用 `write_spice_deck` 生成的网表前，必须添加以下信息：

- SPICE 网表中所有门使用的子电路定义；可使用 `.include` 语句。
- 门中所有晶体管及其他器件的 SPICE 模型。
- SPICE 网表中所有电源与地网的定义；网表头中的注释行说明如何将 VDD 定义为电源、VSS 定义为地。所有生成的对地电容连接到网 0，所有生成的分段线性（PWL）延时模型相对于网 0。

检查 SPICE 网表输出中的所有注释行以获取注释、警告与错误消息。典型错误包括缺少子电路文件、子电路文件中缺少定义、引脚顺序不兼容，或无法确定到达窗口的无约束路径。

PrimeTime SI 尝试为 SPICE 网表生成所有所需激励，但在某些情况下可能无法做到，例如缺少适当数据定义的 RAM。

23.2.6 write_spice_deck 输出示例

Example of write_spice_deck Output

以下是 `write_spice_deck` 命令示例：

```
pt_shell> write_spice_deck -output my_output \
    -sub_circuit_file spice_subckt \
    -logic_one_voltage 1.8 \
    [get_timing_paths]
```

该示例基于名为 `spice_subckt` 的 SPICE 子电路文件中的定义，生成名为 `my_output` 的 SPICE 网表文件。`-logic_one_voltage` 将门输入引脚的上摆幅电压设为 1.8 V，影响 PWL 模型生成。

以下示例展示该命令生成的输出文件及相应的输入 SPICE 子电路定义。

示例 90：SPICE 网表输出

```
MAX. critical path section: (falling) SecIn -> (falling)
SecondReg/D.
*.global vdd vss
*vdd vdd 0 1.8
*vss vss 0 0
.include "./spice/spi_deck.subckt"

*** critical path 0 has 7 pins.
***** Arrival Window Info. for pin 'SecIn' *****
* {myclock} pos_edge {min_r_f 0.1 0.1} {max_r_f 0.1 0.1}
* --clock-- {0 2}
```

```

*****
* !!! INFO: PrimeTime created the following critical path falling
input port 'SecIn' waveform.
* For rising pw1
* vSecIn SecIn 0 pw1(0.0ns 0 10.0995ns 0 10.1005ns 1.8)
* For falling pw1
vSecIn SecIn 0 pw1(0.0ns 1.8 10.0995ns 1.8 10.1005ns 0)
*****
* resistor(s) for net 'SecIn'.
r0      SecIn:4      SecIn:8      4.000000
r1      SecIn:3      SecIn:4      4.000000
r2      PreInv/A      SecIn:3      4.000000
r3      SecIn:5      SecIn:6      1.368000
r4      SecIn:6      SecIn:8      0.049228
r5      SecIn:5      SecIn      0.053363
* ground capacitors(s) for net 'SecIn'.
c0      PreInv/A      0      0.052000ff
* c1      SecIn:4      0      0.000000ff
c2      SecIn      0      0.081000ff
c3      SecIn:5      0      0.404000ff
c4      SecIn:6      0      0.009000ff
* c5      SecIn:8      0      0.000000ff
c6      SecIn:3      0      0.026000ff
* cross capacitance of net 'SecIn'.
cc0      SecIn:4      PreInv/Y      0.008000ff
cc1      SecIn:4      PreNet:14      0.026000ff
cc2      SecIn:4      PreNet:8      0.023000ff
cc3      SecIn:4      PreNet:4      0.023000ff
cc4      SecIn:8      PreInv/Y      0.009000ff
cc5      SecIn:8      PreNet:14      0.029000ff
cc6      SecIn:8      PreNet:8      0.026000ff
cc7      SecIn:8      PreNet:4      0.026000ff
cc8      SecIn:3      PreInv/Y      0.008000ff
cc9      SecIn:3      PreNet:14      0.026000ff
cc10     SecIn:3      PreNet:8      0.023000ff
cc11     SecIn:3      PreNet:4      0.023000ff

```

示例 91: SPICE 子电路输入文件

```

.SUBCKT buf1a3 A Y
MU11 N1N4 A VSS VSS n L=0.24 W=1.66
MU12 N1N4 A VDD VDD p L=0.24 W=2.50
MU21 Y N1N4 VSS VSS n L=0.24 W=1.66
MU22 Y N1N4 VDD VDD p L=0.24 W=2.50
.ENDS
.SUBCKT fdf1a6 CLK D Q
M1 N1N56 TP2 N1N119 VSS n L=0.24 W=1.00
M2 N1N119 D VSS VSS n L=0.24 W=1.00
M3 N1N56 TP3 N1N35 VSS n L=0.24 W=0.58
M4 N1N35 TP7 VSS VSS n L=0.24 W=0.58
M5 TP7 N1N56 VSS VSS n L=0.24 W=1.00

```

```

M6 TP7 TP3 N1N46 VSS n L=0.24 W=0.58
M7 N1N46 TP2 N1N37 VSS n L=0.24 W=0.58
M8 N1N37 N1N108 VSS VSS n L=0.24 W=0.58
M9 TP3 TP2 VDD VDD p L=0.24 W=0.78
.ENDS
.SUBCKT inv1a6 A Y
M1 Y A VSS VSS n L=0.24 W=3.32
M2 Y A VDD VDD p L=0.24 W=5.00
.ENDS
.SUBCKT or2c3 A B Y
M1I551 N1I551N10 B VSS VSS n L=0.24 W=1.66
M1I552 Y A N1I551N10 VSS n L=0.24 W=1.66
M1I553 Y A VDD VDD p L=0.24 W=2.50
M1I554 Y B VDD VDD p L=0.24 W=2.50
.ENDS

```

23.2.7 使用 `write_spice_deck` 进行 SPICE 关联的限制

Limitations of Using `write_spice_deck` for SPICE Correlation

`write_spice_deck` 有助于创建受害路径及其 aggressor 网的结构表示，但由于以下原因，不能期望运行单次 SPICE 网表仿真即可与 PrimeTime SI 结果做直接一对一比较：

- 所选路径可能很长且含大量交叉耦合电容，即使单次 SPICE 仿真也可能耗时过长而不切实际。
- PrimeTime SI 使用多次迭代进行计算，考虑受害路径的整体环境及多个开关周期；SPICE 网表生成器只能生成一条受害路径（及其 aggressor 网），且仅考虑路径的一个开关周期。
- 其他因素（如转换时间传播方案与库精度）也会影响 PrimeTime SI 与 SPICE 结果的比较。

Chapter 24

将 PrimeTime 与 Design Compiler 配合使用

Using PrimeTime With Design Compiler

PrimeTime 与 Design Compiler 综合工具配合良好。但两款工具在命令语法与行为上存在若干差异。要了解如何高效协同使用，请参阅：

- PrimeTime 特有功能
- 与 Design Compiler 的时序分析差异
- 共享 Design Compiler 与 PrimeTime 脚本
- Design Compiler 与 PrimeTime 中的路径组

24.1 PrimeTime 特有功能

Features Specific to PrimeTime

Design Compiler 不支持 PrimeTime 的某些功能，例如内部时钟、提取时序模型的某些特性，以及模式分析（即带有工作模式的时序模型）。

Design Compiler 不支持在叶单元内部引脚上创建时钟。必须在包含该内部引脚扇入或扇出路径上的输入或输出引脚上指定这些时钟。

Design Compiler 无法分析与优化使用模式分析的时序模型。要在特定工作模式下分析与优化模块，必须手动禁用非活动的时序弧。

24.2 与 Design Compiler 的时序分析差异

Timing Analysis Differences From Design Compiler

PrimeTime 与 Design Compiler 在时序分析行为上的差异，请参阅：

- 路径
- 转换时间
- `current_design` 命令

24.2.1 路径

Paths

以下差异适用于路径：

- 保持检查的 **time borrowing**

PrimeTime 默认禁用短路径借用（short-path borrowing，即在电平敏感锁存器上进行保持检查的 time borrowing）。Design Compiler 始终允许短路径借用。

若要在 PrimeTime 中模拟 Design Compiler 行为，将 `timing_allow_short_path_borrowing` 变量设为 `true`。关于路径借用的更多信息，见 *Synopsys Timing Constraints and Optimization User Guide*。

- 路径分段：负载相关延时

在 Design Compiler 中，若时序报告以分段路径的新起点为起点，Design Compiler 会把源门延时中与负载相关的部分转移到输出 net 延时中。该方法可能改善综合结果，但不适合静态时序分析；PrimeTime 不会把这部分负载相关延时加到 net 上。

24.2.2 转换时间

Transition Time

对双向（inout）引脚，PrimeTime 为引脚的驱动端与负载端分别维护转换时间信息。Design Compiler 保留驱动端与负载端转换时间的最大值。

24.2.3 `current_design` 命令

`current_design` Command

Design Compiler 要求切换当前设计以局部指定信息（如线负载模型）：

```
dc_shell> current_design BOTTOM
dc_shell> set_wire_load_model -name small
dc_shell> current_design MID
dc_shell> set_wire_load_model -name medium
dc_shell> current_design TOP
dc_shell> set_wire_load_model -name large
```

在 PrimeTime 中，应使用 `current_instance` 命令或指定层次单元名称来实现相同目的。不要为此目的使用 `current_design`。例如：

```
pt_shell> current_instance TOP
pt_shell> set_wire_load_model -name large
pt_shell> set_wire_load_model -name medium [get_cells U1]
pt_shell> set_wire_load_model -name small [get_cells U1/U2]
```

24.3 共享 Design Compiler 与 PrimeTime 脚本

Sharing Design Compiler and PrimeTime Scripts

对同一设计同时使用 Design Compiler 与 PrimeTime 时，可以共享脚本。最简便的方法是使用 Synopsys Design Constraints (SDC)，如「Synopsys Design Constraints Format (SDC) 脚本文件」一节所述。另一种方法是把包含共享命令的脚本与包含工具专用命令的脚本分开，如下例所示。

注意

无法将 dc_shell Tcl 脚本转换为 pt_shell 脚本，但可将部分 dc_shell Tcl 脚本直接 source 到 pt_shell。另一种方法是用 SDC 格式以 Tcl 编写共享约束文件。更多信息见「Synopsys Design Constraints Format (SDC) 脚本文件」。

以下 dc_shell 脚本包含名为 timing_assertions.scr 的另一脚本：

```
current_design MID
set_wire_load_model -name medium
current_design TOP
set_wire_load_mode enclosed
set_wire_load_model -name large
set_max_area 3000
set_dont_touch u13
source timing_assertions.scr
compile
```

以下脚本是 timing_assertions.scr 的一部分，其中命令对 Design Compiler 与 PrimeTime 均有效：

```
set_operating_conditions WCCOM
create_clock -period 10 CLK
set_input_delay ...
set_output_delay ...
```

要在 PrimeTime 中使用 Design Compiler 脚本，可使用时序断言脚本及含 PrimeTime 命令的脚本。部分 PrimeTime 命令语法与 Design Compiler 相同。但如前所述，工具专用命令需以不同方式指定。

本例中，pt_shell 脚本包含与前面 dc_shell 脚本等效的命令：

```
set_wire_load_mode enclosed
set_wire_load_model -name medium {u2 u4}
set_wire_load_model -name large
source timing_assertions.scr
set_max_area 3000
set_dont_touch u13
```

24.4 Synopsys Design Constraints Format (SDC) 脚本文件

Synopsys Design Constraints Format (SDC) Script Files

Synopsys Design Constraints (SDC) 格式脚本文件是使用 PrimeTime 与 Design Compiler 所支持命令子集的 Tcl 脚本。对部分受支持命令存在限制，例如某些命令并非所有选项均受支持。尽管 SDC 是 Tcl 的子集，但并非全功能脚本语言，其目的是在各 EDA 工具间统一时序约束描述，而非用于复杂脚本。

在 SDC 中只有一个设计：当前设计。无法用 SDC 更改当前设计。

SDC 文件用 `read_sdc` 读入 PrimeTime，用 `write_sdc` 从 PrimeTime 写出。

24.4.1 检查 SDC 文件语法

Checking the Syntax of the SDC File

要检查 SDC 文件并验证其语法与语义正确，运行 `read_sdc -syntax_only`。该命令在不改变设计的前提下验证脚本。关于 SDC 的更多信息，见 SolvNetPlus 上的 *Using the Synopsys Design Constraints Format Application Note*。

24.4.2 检查单位一致性

Checking the Consistency of Units

要检查 SDC 文件中的单位是否与主库单位一致，运行 `set_units`：

```
set_units -time 1ns -capacitance 1nF -current 1mA \  
-voltage 1V -resistance 1kOhm -power 1W
```

若指定单位与主库单位不同，PrimeTime 会发出警告。

`set_units` 可用于 Synopsys Design Constraints (SDC) 文件、命令行或脚本文件。该命令可选，但建议使用以识别来自不同约束文件的不一致单位。

24.5 Design Compiler 与 PrimeTime 中的路径组

Path Groups in Design Compiler and PrimeTime

在 Design Compiler 中，路径组用于修改优化的代价函数。可将路径或端点分配到命名组，每组可指定权重，贡献于设计的最大延时代价。

PrimeTime 检查每条时序路径的最大延时违例：分别把上升和下降信号的实际路径延时与目标路径延时比较。一个路径组中的最差违例，是该组中 slack 最小（负值绝对值最大）的路径。

可在 PrimeTime 中指定路径组。请求路径时序报告时，PrimeTime 为每个路径组列出一份报告。可用 `write_script` 或 `write_context` 将该信息导出到 Design Compiler，或对设计做预算（budgeting）。

创建显式路径组时，还可为路径组指定 critical range。critical range 定义 Design Compiler 优化期间路径组的延时裕量。非零值允许在最差违例不增大的前提下优化近临界路径。

附录 A

已弃用功能

Deprecated Features

以下功能当前已弃用，并将在未来版本中废止：

- HyperScale 分布式分析（HyperScale Distributed Analysis）
- 旧版基于路径的 SMVA（Legacy Path-Based SMVA）

A.1 HyperScale 分布式分析

HyperScale Distributed Analysis

本功能的文档章节包括：

- 从全芯片 Flat 分析到 HyperScale 分布式分析
- 使用恢复会话的类 Flat 顶层报告
- 从分布式分析到块级分析
- 分布式 HyperScale 分析中的 MIM 合并
- HyperScale 属性

A.1.1 从全芯片 Flat 分析到 HyperScale 分布式分析

From Full-Chip Flat to HyperScale Distributed Analysis

若已有执行全芯片 flat 分析的脚本，可轻松复用同一脚本执行 HyperScale 层次化分析。只需添加几行以启用层次化分析并指定主机。

PrimeTime 自动为每个较低层次块运行独立分析，生成各块的 HyperScale 模型，并使用生成的模型运行顶层分析。该方法称为 HyperScale 分布式分析。

以下为例。加粗行为添加到全芯片 flat 分析脚本中的行。

```
# Enable hierarchical analysis
set_app_var hier_enable_distributed_analysis true
```

```

# Set working directory for analysis data
set_app_var hier_distributed_working_directory $nfs_dir

# Optional: override default distributed hierarchical partitioning
set_hier_config ...

# Specify hosts for distributed analysis
set_host_options -num_processes $nProc1 -max_cores $mCore1 \
  -submit_command "..."
set_host_options -num_processes $nProc2 -max_cores $mCore2 \
  -submit_command "..."
...
start_hosts

# Run full-chip flat analysis script
  read_verilog blk_A.v
  read_verilog blk_B.v
  read_verilog top.v
  link_design TOP
  ...
  update_timing
  ...

# Flat-like reporting from top level
report_timing -path_type full_clock_expanded \
  -from IN1 -to U2/reg78 ...

```

启用层次化分析时（`hier_enable_distributed_analysis` 或 `hier_enable_analysis` 变量设为 `true`），每请求 32 个核心进行分析需要一枚 PrimeTime-ADV 许可证。需在链接设计之前设置主机与层次化配置。工具自动将较低层次块分配到可用主机上执行。

表示为 HyperScale 块模型的块（或其内部子块）不需要块级 Verilog 网表与寄生参数数据。但若现有脚本通过读入所有顶层与块级完整 Verilog 网表及寄生参数执行顶层全 flat 时序分析，仍可保留原脚本，无需删除读入块级 Verilog 与寄生参数的命令。

启用分布式分析流程（`set hier_enable_distributed_analysis true`）并正确设置 HyperScale 配置（使用 `set_hier_config`）后，工具自动链接 HyperScale 模型数据，即使已读入也不链接完整块级 Verilog；同时自动从 HyperScale 数据加载简化寄生参数，即使脚本指示读入完整块级寄生参数也会跳过。

为进一步改善运行时间与内存占用，可从脚本中删除读入块级 Verilog 的命令。

分布式分析的默认划分

Default Partitioning for Distributed Analysis 默认情况下，分布式分析方法将顶层每个块放入各自分区，如下图所示。

(原书 *Figure 458: Default Partitioning in Distributed Analysis*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 A.1: 分布式分析中的默认划分

覆盖分布式分析的默认划分

Overriding the Default Partitioning in Distributed Analysis 要覆盖默认划分, 使用 `set_hier_config` 命令。例如:

```
set_hier_config \
-enable_mim \           # Merge context for MIM blocks
-exclude {U1} \         # Keep U1 (blkA) with top level
-split {U6}             # Split U6 (blkD) one level down
```

下图展示对设计划分产生的变化。

(原书 *Figure 459: User-Directed Distributed Analysis Partitioning Example 1*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 A.2: 用户指定的分布式分析划分示例 1

若仅将某些大块指定为独立分析, 其余块保留在顶层, 请使用带 `-partitions` 的 `set_hier_config`。例如:

```
set_hier_config \
-partitions {U5 U6}      # Analyze only U5 and U6 as separate blocks
```

下图展示对设计划分产生的变化。

(原书 *Figure 460: User-Directed Partitioning Example 2*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 A.3: 用户指定的划分示例 2

不能将 `-partitions` 与 `-exclude` 或 `-split` 一起使用。但可将 `-partitions` 与 `-enable_mim` 一起使用, 以有选择地划分多实例模块 (MIM) 子集, 如下图所示。

(原书 *Figure 461: User-Directed Partitioning Example 3*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 A.4: 用户指定的划分示例 3

主机资源亲和性

Host Resource Affinity 可使用 `set_host_options` 与 `set_hier_config` 将较大块显式分配到具有更多 CPU 与内存资源的主机。例如，以下命令定义主机资源，并将块实例 P1、P2、P3 作为 HyperScale 块使用指定资源进行分析：

```
set_host_options -name RA -num_processes 1 -max_cores 2 ... host1
set_host_options -name RB -num_processes 1 -max_cores 1 ... host2
set_host_options -name RC -num_processes 1 -max_cores 1 ... host3
start_hosts
report_host_usage -verbose
...
set_hier_config -partitions {P1 P2} -affinity {RA RB} ...
set_hier_config -partitions {P3} -affinity {RC} ...
...
```

工具将分区 P1 与 P2 分配到资源 RA 与 RB 上分析，将分区 P3 分配到资源 RC 上分析。P1 与 P2 中较大的分区（按单元数）在 RA 与 RB 中较大资源（按分配内存）上分析。

较低层次块中的属性访问

Attribute Access in Lower-Level Blocks 在分布式分析中查询较低层次块对象的时序属性时，响应可能较慢，因为顶层无法立即获得这些属性。为加快响应，可将属性数据收集到缓存中。

例如，以下脚本高效收集并使用层次引脚的 `slack` 属性数据：

```
cache_distributed_attribute_data -class pin \
  -attributes {min_rise_slack max_rise_slack}

set mypins [get_pins -hierarchical *]
foreach_in_collection pin $mypins {
  set slack1 [get_attribute $mypins min_rise_slack]
  set slack2 [get_attribute $mypins max_rise_slack]
  ...
}
```

要删除缓存的属性数据：

```
purge_distributed_attribute_data -class pin \
  -attributes {min_rise_slack max_rise_slack}
```

分布式分析顶层流程

Distributed Analysis Top-Only Analysis Flow 仅顶层分布式分析流程在您只关心顶层时序结果时可减少计算资源占用。要选择此流程：

```
pt_shell> set_hier_config -enable_top_only_flow ...
```

在此流程中,HyperScale 分布式分析仅作用于设计的顶层部分。时序报告仅适用于“活动对象”(active objects), 即顶层设计的端口、单元、网与引脚。要判断对象是否活动, 从分布式分析管理器查询其 `is_active` 属性:

```
pt_shell> get_attribute [get_pins U123/ZN] is_active
true
```

在仅顶层分布式分析流程中, 时序更新后工具会释放块分区的计算资源, 从而阻止进一步的时序更新。要在仅顶层模式下时序更新后保留计算资源, 设置以下变量:

```
pt_shell> set hier_enable_release_resources_in_top_only_flow false
```

A.1.2 使用恢复会话的类 Flat 顶层报告

Flat-Like Top-Level Reporting Using Restored Sessions

若已在 HyperScale 分析会话中完成块级与顶层时序分析, 可将这些会话恢复到新的顶层会话, 并从该层级报告时序路径 (包括跨越层次边界的路径), 而无需再次承担顶层分析的开销。

例如, 假设先前分别为 blkA、blkB 与顶层运行了独立的 HyperScale 分析:

```
-----
# Analysis session for block blkA
...
read_context $topDir
...
save_session $blkASession

-----
# Analysis session for block blkB
...
read_context $topDir
...
save_session $blkBSession

-----
# Analysis session for top level
...
save_session $topSession
```

要恢复这些会话并从顶层生成时序报告, 执行以下操作:

```
...
set_app_var hier_enable_analysis true

# Read existing HyperScale session data
```

```

set_hier_config -session -block blkA -path $blkAsession
set_hier_config -session -block blkB -path $blkBsession
set_hier_config -session -top -path $topSession

# Generate full-chip timing report and noise report
report_timing -path_type full_clock_expanded -from IN1 -to BLKA/reg2 ...
report_noise ...
report_constraint ...
report_analysis_coverage ...
report_qor ...

```

恢复块级与顶层会话后，时序报告的工作方式与 HyperScale 分布式分析会话中相同。

A.1.3 从分布式分析到块级分析

From Distributed Analysis to Block-Level Analysis

HyperScale 分布式分析方法在后台自动为每个块运行块级分析。运行 HyperScale 分布式分析后，可使用自动保存的块级会话数据轻松重新运行块级分析。

例如，假设已运行分布式分析：

```

set_app_var hier_enable_distributed_analysis true
set_app_var hier_distributed_working_directory $nfs_dir
set_host_options -num_processes $nProc1 -max_cores $mCore1 \
  -submit_command "..."
set_host_options -num_processes $nProc2 -max_cores $mCore2 \
  -submit_command "..."
...
start_hosts

# Run full-chip flat analysis script
read_verilog ...
...
update_timing
...
save_session $DH_Session

```

保存分布式分析会话时，工具将顶层与块级数据保存到不同子目录。在新的 PrimeTime 会话中，可仅恢复一个块的会话数据或仅恢复顶层，并单独重新分析该部分设计。

例如：

```

-----
# Restore separate session for block blkA
set_app_var hier_enable_analysis true
restore_session $DH_Session/blkA
report_timing ...

-----
# Restore separate session for block blkB
set_app_var hier_enable_analysis true
restore_session $DH_Session/blkB

```

```
report_timing ...
```

```
-----
# Restore separate session for top level
set_app_var hier_enable_analysis true
restore_session $DH_Session/top
report_timing ...
```

默认情况下, `restore_session` 恢复原始会话的所有分区。为节省时间与内存, 可使用 `restore_session` 的 `-partitions` 选项有选择地仅恢复感兴趣的 HyperScale 分区。

默认情况下, 恢复的分布式分析会话使用原始会话中 `set_host_options` 命令定义的主机选项设置。但可在 `restore_session` 之前用新设置运行 `set_host_options` 覆盖原始主机设置。例如:

```
set_host_options -num_processes 2 -max_cores 4 myCore3 ...
restore_session -partitions {P2} my_session_dir
```

当前会话中任何 `set_host_options` 的使用都会使原始会话的所有主机选项设置失效。

A.1.4 分布式 HyperScale 分析中的 MIM 合并

MIM Merging in Distributed HyperScale Analysis

在全芯片分布式分析流程中, 可使用带 `-enable_mim` 的 `set_hier_config` 执行 MIM 合并, 如下图所示。

(原书 *Figure 462: MIM Merging for All Instances of a Block*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 A.5: 块的全部实例的 MIM 合并

若 MIM 集合中某块的层次化上下文与其余实例差异很大, 可能希望将该块与顶层一起分析。此时应使用带 `-partitions` 的 `set_hier_config` 显式指定划分, 如下图所示。

(原书 *Figure 463: MIM Merging for Some Instances of a Block*)
 请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 A.6: 块的若干实例的 MIM 合并

本例中, 向实例 U2 读入寄生参数并非必需, 但为防止完整 MIM 合并配置中可能出现的错误, 应向 MIM 块的所有实例读入寄生参数。

A.1.5 HyperScale 属性

HyperScale Attributes

表 A.1: 单元 (Cell) 对象的 HyperScale 属性

单元属性	说明
<code>is_active</code> (布尔)	在 HyperScale 仅顶层分布式分析 (<code>set_hier_config -enable_top_only_flow</code>) 中, 若该单元被视为活动对象则返回 <code>true</code> 。

表 A.2: 引脚 (Pin) 对象的 HyperScale 属性

引脚属性	说明
<code>is_active</code> (布尔)	在 HyperScale 仅顶层分布式分析 (<code>set_hier_config -enable_top_only_flow</code>) 中, 若该引脚被视为活动对象则返回 <code>true</code> 。

表 A.3: 端口 (Port) 对象的 HyperScale 属性

端口属性	说明
<code>is_active</code> (布尔)	在 HyperScale 仅顶层分布式分析 (<code>set_hier_config -enable_top_only_flow</code>) 中, 若该端口被视为活动对象则返回 <code>true</code> 。

A.2 旧版基于路径的 SMVA

Legacy Path-Based SMVA

注意

这是旧版仅基于路径的 SMVA 功能的原始文档, 在 `timing_cross_voltage_domain_analysis_mode` 变量设为 `legacy` 时使用。

当前 SMVA 功能 (见 SMVA Graph-Based Simultaneous Multivoltage Analysis) 同时支持 PBA 与 GBA 分析。

在多电压设计中, 某些时序路径可能从一个电源域跨越到另一个电源域。当电源域可在多种供电电压下工作时, PrimeTime 需要考虑给定路径的最坏可能工作电压组合。为确保检测到所有潜在时序违例, 默认分析考虑路径中每个单元的最坏供电电压。然而该分析偏悲观, 因为属于同一域的单元不能同时工作在不同供电电压下。

图 A.7 中, 从 U1/CK 到 U7/D 的时序路径跨越两个电源域 P1 与 P2。每个域可在低、高两种电压之一工作。每个单元的延时标于单元旁, 红色与蓝色值分别表示低、高供电电压下的延时。

(原书 *Figure 464: Timing Path Through Two Power Domains*)
请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 A.7: 跨越两个电源域的时序路径

对 setup 检查, 默认分析考虑数据路径上的最长延时与时钟路径上的最短延时, 因此使用图中所圈的最坏情况延时值。结果偏悲观, 例如单元 U2 不可能在低电压工作而单元 U3 在高电压工作——二者属于同一电源域, 必须同时工作在低或高电压。

若在此分析中发现时序违例, 可通过用每种固定供电电压组合 (如 V1/V2 设为 low/low、low/high、high/low、high/high) 多次分析设计来降低悲观性。但电源域数量很大时, 电压组合数量也非常大, 多次分析的成本可能过高。

同时多电压分析 (simultaneous multivoltage analysis, SMVA) 在单次基于路径的分析运行中消除跨域电压悲观性, 且运行时间开销很小。采用该策略, PrimeTime 考虑每条路径所跨越域的低、高电压的所有组合效应, 但仅针对跨域路径, 且仅使用 UPF 命令定义的各域允许供电电压组合。

图 A.8 展示使用同时多电压分析的路径 setup 时序计算。在基于路径的分析期间, PrimeTime 分别考虑来自电源域 P1 与 P2 对 setup slack 的贡献, 并考虑每个域的高、低供电电压, 但不在给定域内混合高、低电压。

(原书 *Figure 465: Simultaneous Multivoltage Slack Calculation*)
请参阅原版 PDF 插图; 可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 A.8: 同时多电压 Slack 计算

本例中, 最坏情况 slack 为 -5, 而非常规最坏情况时序分析得到的悲观值 -9。最坏供电电压组合为 P1 低、P2 高。注意, 同时多电压分析仅在基于路径的分析期间进行, 因为最坏供电电压组合可能因路径而异。

要在基于路径的分析期间执行同时多电压分析, 将 `timing_enable_cross_voltage_domain_analysis` 变量设为 `true`。默认为 `false`, 执行常规最坏情况时序分析。

若变量设为 `true`, 在下次时序更新时 PrimeTime 识别跨越多个电源域的路径, 并随后将报告限制为仅这些路径。若随后使用 `report_timing` 的 `-pba_mode path` 执行基于路径的分析, PrimeTime 对跨域路径执行高效的基于路径重算, 找出每条路径所跨越各域的最坏情况电压 (由 `set_voltage` 与 UPF 命令定义), 同时消除标准片上变异分析中的悲观性。

同时多电压分析期间, 工具执行以下功能:

- 识别并忽略在 CRPR 公共点之前跨越电源域的路径, 如图 A.9 所示。
- 识别重汇聚时钟路径, 如图 A.10 所示。

(原书 *Figure 466: Analyzing Only Paths Affected by Domain Crossings*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 A.9: 仅分析受域跨越影响的路径

(原书 *Figure 467: Clock Path Reconverges*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 A.10: 时钟路径重汇聚

- 当路径跨越电源域时，改善序贯时钟网络的悲观性降低，如图 A.11 所示。

(原书 *Figure 468: Sequential Clock Network Crosses Power Domains*)
 请参阅原版 PDF 插图；可将截图放入 figures/ 目录后用 \includegraphics 替换。

图 A.11: 序贯时钟网络跨越电源域

同时多电压分析使用流程

Simultaneous Multivoltage Analysis Usage Flow 同时多电压分析期间，默认情况下每个单元在各电压电平的延时由 PrimeTime 现有多电压延时计算方法确定。使用 CCS 库并通过 `define_scaling_lib_group` 调用库缩放组可获得最准确结果。若无这些模型，PrimeTime 使用可用的 NLDM 模型。

同时多电压分析旨在仅分析与报告跨越电源域边界的时序路径。使用此功能前，建议先按常规方法修复域内时序路径；这些运行中可忽略任何跨域时序路径。完成后，执行同时多电压分析运行以全面分析跨域路径。

当具有多电压表征数据的库可用时，在修复所有域内违例后，跨域路径同时多电压分析的典型使用流程如下：

```
set search_path
set link_path "*" mylib_1v.db
read_verilog ...
link_design
read_parasitics
```

```
load_upf ...
read_sdc ... #includes set_voltage commands

# Derate margins not including variation due to voltage level settings
set_timing_derate -early 0.92
set_timing_derate -late 1.08
define_scaling_lib_group "mylib_1V.db mylib_0.8V.db \
                        mylib_1.2V.db mylib_1.4V.db"

# Invoke accurate multivoltage analysis for cross-domain paths
set timing_enable_cross_voltage_domain_analysis true

# Conservative cross-domain-only timing report
report_timing ...
# Path-based cross-domain timing report with reduced pessimism
report_timing -pba_mode path ...
```

使用跨域 Derating 的分析

Analysis Using Cross-Domain Derating 若无关于电压变分的可靠库表征信息，可使用跨域 derating 分析不同供电电压的效应。为此，用 `set_cross_voltage_domain_analysis_guardband` 指定 derating 因子调整值。例如：

```
pt_shell> set_cross_voltage_domain_analysis_guardband -late 0.1
```

该 derating 会加到 `set_timing_derate` 指定的普通 derating 以及由 `timing_aocvm_enable_analysis` 变量调用的高级片上变异（AOCV）分析之上。本例中，若 `set_timing_derate` 设置的常规 late derating 为 1.05，额外 late guard band 0.1 使最终 late derating 为 $1.05 + 0.1 = 1.15$ 。derating 因子 1.15 建模跨越电源域的时序路径上由片上变异效应与变化电压引起的组合时序变异。

额外 guard band 0.1 旨在在无此电压变分库表征数据时建模供电电压变分的时序效应。若有多电压库表征数据，则不需要 `set_cross_voltage_domain_analysis_guardband` 命令。但 PrimeTime 仍接受该命令，且在启用跨域分析时应用额外 derating。

若无多电压库表征数据，在修复所有域内违例后，使用 derating 进行跨域路径同时多电压分析的典型流程如下：

```
set search_path
set link_path
read_verilog ...
link_design
read_parasitics
load_upf ...
read_sdc #includes set_voltage commands

# Example margins, not including V_OCV
set_timing_derate -early 0.92
set_timing_derate -late 1.08

# Invoke accurate multivoltage analysis for cross-domain paths
set timing_enable_cross_voltage_domain_analysis true
```

```
# Specify derating for voltage supply settings, +/- 10%
set_cross_voltage_domain_analysis_guardband -early 0.9
set_cross_voltage_domain_analysis_guardband -late 1.1

# Conservative cross-domain-only timing report with low-high derating
report_timing ...

# Path-based cross-domain timing report with reduced pessimism
report_timing -pba_mode path ...
```

附录 B

术语表

Glossary

B.1 术语条目

Glossary

以下条目按英文字母顺序排列。英文术语保留原文；定义已译为中文，命令名与技术标识符保持英文。

有效边沿 (*active edge*)

使数据锁存到触发器的时钟信号由低到高或由高到低的转换。

aggressor 网 (*aggressor net*)

因与另一网 (受害网, *victim net*) 之间的寄生交叉电容, 对受害网造成不良串扰效应的网。

标注 (*annotation*)

附着于对象的信息 (如附在网上的延时值); 或将信息附着到对象的过程。

AOCV (高级片上变异) (*AOCV*)

Advanced on-chip variation (AOCV) 是 Synopsys 的一种技术, 它根据所分析路径的位置与逻辑深度减少不必要的悲观度。

弧 (*arc*)

与特定路径相关的一组时序约束。

ASIC

Application-specific integrated circuit; 为执行特定任务而制造的专用集成电路。

ASIC 厂商 (*ASIC vendor*)

为他人制造集成电路的公司, 使用具有定义功能与时序特性的标准电路元件。

异步 (*asynchronous*)

两个不同时钟或信号之间无时序关系 (转换可相对于彼此在任意时刻发生)。

属性 (*attribute*)

与对象关联的字符串或值，携带该对象的信息。例如附在单元上的 `number_of_pins` 属性表示单元引脚数。可用 `get_attribute` 查询属性值。

反标 (*back-annotation*)

将详细寄生数据应用到设计的过程，以便更准确分析最终电路时序。数据来自布局完成后的外部工具。

基线映像 (*baseline image*)

在 DMSA 分析中，由网表映像与场景的公共数据文件合并产生的映像。

借用 (*borrowing*)

一种分析技术：当下一级为透明模式工作的电平敏感锁存器时，允许路径的一部分从设计的下一级借用时序裕量。

捕获 (*capture*)

在路径端点用时钟将数据锁存到锁存器或触发器的过程，从而获取由路径起点较早时钟边沿发射的数据。

单元 (*cell*) 集成电路中具有已知功能与时序特性的组件，可作为构建更大电路的元素。

单元延时 (*cell delay*)

路径中从单元（逻辑门）输入到输出的延时。

时钟 (*clock*)

将数据锁存到时序元件的周期信号。用 `create_clock` 或 `create_generated_clock` 定义信号为时钟并指定其时序特性。

时钟路径 (*clock path*)

从时钟输入端口或内部单元引脚开始、到时序元件时钟输入引脚结束的时序路径。

时钟门控 (*clock gating*)

用时钟信号上的组合逻辑元件（如多路选择器或 AND 门）控制时钟，以在选定时刻关断时钟或修改时钟脉冲特性。

时钟延迟 (*clock latency*)

见 `latency`（延迟）。

时钟重汇聚悲观度 (*clock reconvergence pessimism*)

静态时序分析中的一种不准确现象：两条不同时钟路径共享部分物理路径，而分析工具对一条路径采用共享段的最小延时、对另一条路径采用同一段的最大延时。纠正这种不准确现象称为时钟重汇聚悲观度消除（CRPR）。

时钟偏斜 (*clock skew*)

见 `skew`（偏斜）。

关闭边沿 (*closing edge*)

将数据锁存到电平敏感锁存器、结束透明模式并使数据输入不敏感的时钟边沿。

集合 (collection)

从已加载设计中选取的对象集。例如 `set mylist [get_clocks CLK*]` 创建时钟集合并将变量 `mylist` 设为该集合，随后可对集合使用如 `remove_clock $mylist` 的命令。

组合反馈环 (combinational feedback loop)

信号路径回到起点的组合逻辑网络。PrimeTime 必须在环内某点断开环（停止追踪信号）。

组合逻辑 (combinational logic)

仅含无记忆或内部状态元件的逻辑网络。可含 AND、OR、XOR、反相器，但不能含触发器、锁存器、寄存器或 RAM。

命令焦点 (command focus)

在 DMSA 分析中，分析命令所应用的当前场景集。命令焦点可包含会话中全部场景或仅子集。

公共点 (common point)

存在时钟重汇聚悲观性的路径中，发射与捕获时钟路径共享段上最后一个单元输出。

条件时序弧 (conditional timing arc)

延时值取决于某些条件（如输入逻辑状态）的时序弧。

保守 (conservative)

倾向于悲观（而非乐观）以确保检测到所有违例的分析算法或技术。

约束 (constraint)

应用于路径或设计的时序规格或要求。时序约束限制信号到达器件输入或于器件输出有效的允许时间范围。

上下文表征 (context characterization)

使用 `characterize_context` 捕获子设计在芯片级时序环境中时序上下文的过程，允许在 PrimeTime 中独立分析子设计或在 Design Compiler 中优化。实例的时序上下文包括时钟信息、输入到达时间、输出延时、时序例外、设计规则、传播常量、线负载模型、输入驱动与电容负载。

上下文无关 (context independent)

在不同上下文中均准确。该术语指时序模型的可用性而非行为。例如 `extract_model` 创建的模型是上下文无关的，即可在不同上下文中准确使用（其行为随使用上下文变化）。

CPU 时间 (CPU time)

计算机中央处理器完成任务所用时间（由工具报告）。衡量任务所需计算资源；CPU 时间小于实际经过时间。

关键路径 (critical path)

路径组中违例最大（或时序 slack 最小）的路径。

关键区域 (critical region)

在 HyperTrace 加速 PBA 中，设计中 slack 关键（违例）的引脚集合。slack 临界阈值通常为 0，但可配置。

串扰 (crosstalk)

网与物理相邻网之间寄生电容引起的不良效应。相邻网上的信号转换可在受影响网上引起噪声凸起或转换时间变化。

当前设计 (current design)

当前视为设计层次顶层的设计。可用 `current_design` 指定。

当前映像 (current image)

在 DMSA 分析中，当场景数多于主机数、工作进程需切换至另一场景时自动保存到磁盘的映像。

当前实例 (current instance)

设计层次中实例专用命令默认操作的实例（层次单元）。可用 `current_instance` 设置，类似 Linux 中的 `cd`。

当前会话 (current session)

在 DMSA 分析中，当前选定进行分析的场景集。

数据检查 (data check)

两个数据信号之间的时序检查，如握手接口信号或 `preset/clear` 引脚间的 `recovery/removal` 检查。被检查的两信号称为 `constrained` 与 `related` 信号；`related` 信号在时序关系中被视为时钟。例如 `setup` 数据检查验证 `constrained` 信号在 `related` 信号有效边沿之前稳定。

数据路径 (data path)

从数据发射点到数据捕获点的时序路径。“路径”通常指数据路径，另有时钟路径、`async_default` 路径等。

数据库 (.db) 格式 (database (.db) format)

Synopsys 用于存储设计、.lib 逻辑库、时钟信息、反标寄生信息与时序模型的文件格式。可用 `read_db` 将部分或全部信息读入 PrimeTime。Design Compiler 与 Formality 等工具也使用 .db 文件。

Design Compiler

Synopsys 工具，可将硬件描述语言定义的设计综合为优化的、工艺相关的门级设计。支持多种 flat 与层次风格，可针对速度、面积与功耗优化组合与时序设计。与 PrimeTime 配合良好，但是可独立使用的工具。

边沿敏感锁存器 (edge-sensitive latch)

触发器；在时钟输入每个有效边沿捕获数据的寄存器元件。有效边沿可为低到高或高到低，取决于器件类型。

端点 (endpoint)

时序路径在设计中结束的位置，即数据被时钟边沿捕获或必须在特定时刻可用的位置。每个端点必须是寄存器数据输入引脚或输出端口。

例外 (exception)

见 `timing exception`（时序例外）。

互斥时钟 (*exclusive clocks*)

永不同时活动的两个时钟（例如因多路复用）。

提取时序模型 (*extracted timing model*)

由 `extract_model` 从门级网表创建的时序模型，表示块的时序特性（弧值），但不含功能信息，无法综合。

伪路径 (*false path*)

设计中存在但不应做时序分析的路径，例如两个永不同时使能的多路复用块之间的路径。须在 PrimeTime 中用 `set_false_path` 定义为时序例外，或用 `set_disable_timing` 将相关对象排除在分析之外。

扇入 (*fanin*)

影响指定时序端点（sink）的端口与引脚集合。若存在经组合逻辑从引脚到该 sink 的时序路径，则该引脚在时序扇入中。扇入追踪在寄存器（时序单元）时钟引脚处停止。

扇出 (*fanout*)

在 PrimeTime 中通常指时序扇出（transitive fanout），即受指定源端口或引脚影响的时序端点上的端口与引脚集合。若存在经组合逻辑从源到引脚的时序路径，则该引脚在时序扇出中。扇出追踪在寄存器输入处停止。注意时序扇出不同于直接扇出（direct fanout，即连接到单元输出的单元输入数量）。

触发器 (*flip-flop*)

由边沿敏感时钟输入控制的存储元件。通常有输入 D、输出 Q、时钟输入，可能还有异步 `set/clear`。时钟有效边沿时锁存 D 的值并在 Q 保持。有效边沿可为上升或下降，取决于触发器类型。

门控时钟 (*gated clock*)

可被逻辑修改的时钟信号，例如为省电而可关断的时钟。

生成时钟 (*generated clock*)

由集成电路内部生成的时钟；非直接来自外部源。例如由系统时钟生成的二分频时钟。用 `create_generated_clock` 指定生成时钟特性。

毛刺 (*glitch*)

足以导致逻辑失败的噪声凸起。

基于图的精化 (*graph-based refinement*)

在 HyperTrace 加速 PBA 中，在关键区域内选择性信号合并以提高精度，同时保持基于图表示的运行时间与内存优势的过程。

保持约束 (*hold constraint*)

规定在器件锁存数据的时钟边沿之后，数据必须在器件输入端继续保持稳定多长时间的时序约束。它相对于时钟对时序路径施加最小延时要求。

HyperGrid

PrimeTime 功能，通过在多台主机上并行运行的工作进程分析超大设计。可用常规 PrimeTime 分析相同命令探索与报告整个设计。

HyperScale

PrimeTime 层次化分析方法，用独立运行分析块级与顶层部分，并准确处理跨层次边界的时序接口。

HyperTrace 加速穷举基于路径分析（PBA）的 PrimeTime 技术。识别时序图关键区域，基于图的精化在关键区域内应用选择性信号合并以提高精度，再用该精化时序驱动穷举 PBA 搜索。

I-V 特性（稳态）（*I-V characteristics (steady-state)*）

单元输出的电流-电压特性；输出保持逻辑 1 或 0 时电流随电压的变化。PrimeTime SI 用此信息计算串扰噪声效应。

岛（*island*） 见 voltage area（电压区域）。

inout 引脚（*inout pin*）

单元的双向引脚；可作为输入或输出工作。

输入延时（*input delay*）

规定从时钟边沿到信号到达指定输入端口的最小或最大延时的约束。PrimeTime 用此信息检查输入端口及该输入扇出中的时序违例。

知识产权（*intellectual property*）

一家公司拥有并授权另一家公司使用的信息。例如一家公司可授权在另一家公司的 ASIC 中使用芯片子模块设计（如微处理器核），提供版图、电气规格与时序模型而不公开子模块网表。

接口逻辑模型（ILM）（*interface logic model (ILM)*）

已被 HyperScale 层次化分析取代的过时时序模型类型。

锁存器（*latch*）

由电平敏感门控输入控制的存储元件。门控有效时输出跟随输入；门控由有效变为无效时锁存输入值，输出保持该值。门控信号由无效到有效的边沿称为 opening edge（打开边沿），由有效到无效称为 closing edge（关闭边沿）。

延迟（*latency*）

时钟信号从时钟源传播到设计内特定点所需时间。时钟信号在此时间内“隐藏”（潜伏）。

发射（*launch*）

在路径起点用时钟使数据从锁存器或触发器输出的过程；所释放的数据由路径端点处的另一时钟边沿捕获。

版图（*layout*）

生成集成电路元件与互连的几何位置、尺寸与形状的过程。版图信息可供外部工具生成元件与互连寄生电阻电容的准确信息，随后在 PrimeTime 中反标以进行更准确的时序分析。

叶级（*leaf level*）

设计层次中包含最低级库单元（如树的叶子）的层次。

电平敏感锁存器（*level-sensitive latch*）

门控输入有效时允许数据从输入传到输出的寄存器；门控无效时保持数据输出恒定。

逻辑库 (*logic library*)

含设计中电路元件功能与时序特性信息的数据库，以 Liberty 语法指定时也称.lib 逻辑库。通常由 ASIC 厂商提供。

Liberty

指定单元库信息（如延时、时序约束与功耗）的标准格式。SiliconSmart 库表征工具生成 Liberty (.lib) 文本文件；Library Compiler 读入 Liberty 并生成编译单元库 (.db) 文件。

Library Compiler

Synopsys 工具，为 PrimeTime、Design Compiler 等创建库数据库。从 Liberty (.lib) 文本文件读入 ASIC 库描述并编译为.db 格式数据库。

LVF (Liberty 变异格式) (*LVF (Liberty Variation Format)*)

Liberty 语法中指定单元变异参数（如单元延时均值与标准差 sigma）的标准格式。

管理进程 (*manager process*)

在 DMSA 与 HyperGrid 分布式分析中，由用户调用、编排一个或多个工作进程的 PrimeTime 进程。

多周期路径 (*multicycle path*)

设计为数据从起点到终点传播需超过一个时钟周期的路径。例如乘法器（慢组合逻辑）可能设计为四个时钟周期后捕获有效结果。须用 `set_multicycle_path` 定义为时序例外。

原生 (*native*)

完全在 PrimeTime 内工作、不需外部程序的命令或过程。

网弧 (*net arc*)

从驱动引脚到同一网上负载引脚的时序弧，因寄生网电容具有非零延时。

网延时 (*net delay*)

时序路径中从单元输出到下一单元输入的延时，由互连寄生电容引起。

网络延迟 (*network latency*)

时钟信号从时钟定义点传播到寄存器时钟引脚的时间。

非线性延时模型 (NLDM) (*nonlinear delay model (NLDM)*)

在.lib 逻辑库中指定单元延时的标准表格式。

噪声 (*noise*)

稳态网模拟电压的暂时偏离或变化，如 CMOS 门输出处的串扰感应电压凸起。

噪声凸起 (*noise bump*)

绘制成电压对时间时呈隆起状（常建模为三角形）的噪声事件。

噪声免疫曲线 (*noise immunity curve*)

规定单元输入允许的最大噪声凸起尺寸而不在输出产生传播噪声的函数，常近似为双曲线。

噪声裕量 (*noise margin*)

DC 噪声裕量是某逻辑电平最极端输出电压（如 V_{OLmax} ）与同逻辑电平最极端输入阈值（如 V_{ILmax} ）之差。AC 噪声裕量是单元输入允许而不在输出产生逻辑失败的最大噪声凸起（伏特）。

噪声 slack (*noise slack*)

输入出现噪声凸起时避免逻辑失败的量，等于失败阈值电压减凸起高度，再乘以凸起宽度。单位为库电压单位乘以库时间单位。

不变约束 (*no-change constraint*)

数据信号在控制信号脉冲之前、期间与之后必须保持不变的时间。例如地址信号在写脉冲期间必须稳定：在控制脉冲前沿前稳定 **setup** 时间、脉冲期间、以及后沿后稳定 **hold** 时间。

打开边沿 (*opening edge*)

断言电平敏感锁存器门控输入的时钟边沿。锁存器从打开边沿到关闭边沿处于透明状态。

工作条件 (*operating conditions*)

电路工作的工艺、电压与温度条件，影响单元时序特性。

乐观 (*optimistic*)

存在已知精度限制的分析算法或技术，不准确可能使电路看起来比实际有更多时序 **slack**，从而可能检测不到实际电路中的违例。

输出延时 (*output delay*)

从输出端口到捕获该端口数据的外部时序器件的最小或最大延时约束，规定信号在输出端口必须可用的时间以满足外部时序元件的 **setup** 与 **hold**。

寄生电容 (*parasitic capacitance*)

网与相邻互连或衬底物理邻近引起的网电容；或网中 **p-n** 结电荷存储效应。

寄生电阻 (*parasitic resistance*)

互连或网因互连材料有限电导率产生的电阻。

分区 (*partition*)

在 HyperGrid 分布式分析中，被拆分为多块以在多台机器上并行分析的大设计的一部分。

路径 (*path*) 从寄存器时钟引脚或输入端口开始、经任意数量组合逻辑、在寄存器数据输入引脚或输出端口结束的点到点序列。时钟边沿在路径起点发射的数据经组合逻辑传播到端点，由另一时钟边沿捕获。

路径端点 (*path endpoint*)

见 **endpoint**（端点）。

路径组 (*path group*)

相关路径的组，由 **create_clock** 隐式分组或 **group_path** 显式分组。默认情况下，由同一时钟定时的端点路径属于同一路径组。路径组影响 PrimeTime 中违例报告，也影响 Design Compiler 优化不同路径的相对力度。

路径检查器 (*path inspector*)

PrimeTime GUI 中显示时序路径详细信息的窗口，如路径原理图、路径元素表、时序波形、路径延时剖面与文本报告。

路径起点 (*path startpoint*)

见 **startpoint** (起点)。

悲观 (*pessimistic*)

存在已知精度限制的分析算法或技术，不准确可能使电路看起来比实际有更少的时序 slack，从而可能报告实际电路中不存在的违例。

引脚 (*pin*) 设计中单元实例的输入或输出。引脚时序约束在 .lib 逻辑库中指定。

端口 (*port*) 设计的输入或输出。用 **set_input_delay** 与 **set_output_delay** 为端口指定时序约束。

版图后 (*postlayout*)

完成版图之后、可获得详细寄生数据并用于设计反标的阶段。

过程 (Tcl) (*procedure (Tcl)*)

用户在 PrimeTime 中用 **proc** 创建的自定义命令。定义后可像标准命令一样运行，可带参数并使用局部与外部变量。

传播噪声 (*propagated noise*)

由驱动该网的门输入噪声引起的网上的噪声凸起。例如输入为 0、输出为 1 的反相器，输入正凸起可在输出产生负凸起。

pt_shell PrimeTime 的基于文本命令行界面，可输入命令、运行脚本并以文本查看结果。与之相对，PrimeTime GUI 还提供菜单、对话框、按钮及分析结果的图形展示。

快速时序模型 (*quick timing model*)

当无网表且无其他时序模型时，用 PrimeTime 命令为块创建的临时时序模型。使用 **create_qtm_model** 创建、**create_qtm_port** 与 **create_qtm_delay_arc** 指定特性、**save_qtm_model** 保存。

RC 电阻-电容，用单个电阻与电容表示网寄生的简单寄生模型。

重汇聚悲观性 (*reconvergence pessimism*)

见 **clock reconvergence pessimism** (时钟重汇聚悲观性)。

恢复约束 (*recovery constraint*)

异步控制信号（如触发器异步 **clear** 输入）变为无效与随后有效时钟边沿之间所需的最短时间。类似有效时钟边沿的 **setup** 检查。若有效时钟边沿在异步输入变为无效后过早发生，寄存器数据不确定，因为时钟输入数据可能无效。

寄存器 (*register*)

触发器（边沿敏感时钟）或锁存器（电平敏感门控）的叶级实例。

相关信号 (*related signal*)

在数据到数据时序检查中被视为时钟的数据信号。另一被检查信号称为 **constrained** 信号。

移除约束 (removal constraint)

异步输入仍有效时发生的时钟边沿与随后撤销异步控制信号之间所需的最短时间。类似移除异步控制信号的 hold 检查。若异步输入在时钟边沿后过早变为无效，寄存器数据不确定，因为时钟数据可能有效并与异步控制冲突。

RSPF Reduced Standard Parasitic Format, Cadence 定义的用于反标存储电路寄生信息的格式。

场景 (scenario)

在 DMSA 分析中，分析设计的特定工作条件与模式组合。

脚本 (script)

含一系列 `pt_shell` 命令的文本文件，可用 `source` 执行。

SDF Standard Delay Format, 存储电路信息用于反标的标准文件格式，包括延时（引脚到引脚单元延时与网延时）与时序检查（`setup`、`hold`、`recovery`、`removal` 等）。PrimeTime 用 `read_sdf` 与 `write_sdf` 读写 SDF。

敏化 (sensitize)

找到或施加使特定路径在逻辑上激活并传播数据的输入向量。

时序逻辑 (sequential logic)

含具有记忆或内部状态元件（如触发器、锁存器、寄存器或 RAM）的逻辑网络。

会话 (session)

一般指 PrimeTime 工具中当前运行的分析会话；也可指 `save_session` 创建、`restore_session` 读取的数据目录。

建立约束 (setup constraint)

规定在器件锁存数据的时钟边沿之前，数据必须提前多长时间在器件输入端有效的时序约束。它相对于时钟对时序路径施加最大延时要求。

信号完整性 (signal integrity)

信号或网对串扰效应的免疫；网不受物理相邻网上转换或噪声影响的特性。

信号引脚 (signal pin)

非 PG 引脚，如时钟、数据或复位引脚。

偏斜 (skew)

时钟转换相对标称预期时间的偏移或变化量；或时钟网络中两点偏移量的差异。

时序裕量 (slack) (slack)

避免违例的时间量。例如信号必须在不晚于 8 ns 到达单元输入而实际 5 ns 到达，则 slack 为 3 ns。slack 为 0 表示约束刚好满足；负 slack 表示时序违例。

转换时间 (slew)

信号从低到高或从高到低变化所需时间；亦称 `transition time`。

源延迟 (source latency)

时钟信号从理想波形原点到时钟定义点的传播时间。

<i>SPEF</i>	Standard Parasitic Exchange Format, OVI 定义的用于反标存储电路寄生信息的格式。
<i>SPICE</i>	Simulation program with integrated circuit emphasis; 在晶体管、电容、电阻级分析电路操作的时基仿真工具。有多种来源的不同实现。

起点 (*startpoint*)

时序路径在设计中开始的位置, 即数据由时钟边沿发射或预期在特定时刻可用的位置。每个起点必须是寄存器时钟引脚或输入端口。

静态时序分析 (*static timing analysis*)

通过考虑路径延时与时序约束, 高效判断电路是否违反任何时序要求的分析。与门级仿真相比更快、更全面且不需测试向量, 但不能替代仿真, 因为不检查电路功能。

静态噪声分析 (*static noise analysis*)

PrimeTime SI 执行的分析, 确定最坏噪声效应以便最小化或消除。考虑物理相邻 aggressor 与受害网之间的交叉耦合电容及受害网的稳态 (一或零) 负载特性。

同步 (*synchronous*)

两个时钟或信号之间的时序关系, 特定转换同时发生或相位差随时间保持固定。

任务 (*task*) 在 DMSA 与 HyperGrid 分布式分析中, 管理进程为工作进程定义的自包含工作单元。

Tcl Tool command language; *pt_shell* 接口所基于的标准命令脚本语言。

Tcl 过程 (*Tcl procedure*)

见 *procedure* (Tcl)。

多线程多核分析 (*threaded multicore analysis*)

使用多核提高性能的流程, 对时序更新进行线程化并优化其他主要流程区域的处理。

三态 (*three-state*)

器件输出 (如总线驱动器) 可处于三种逻辑状态之一: 低 (0)、高 (1) 或高阻 (Z)。

时间借用 (*time borrowing*)

见 *borrowing* (借用)。

时序弧 (*timing arc*)

见 *arc* (弧)。

时序例外 (*timing exception*)

PrimeTime 假定的默认 (单周期) 时序行为的例外。为正确分析电路, 须指定每条不符合默认行为的路径。时序例外包括伪路径、多周期路径及需要与默认计算时间不同的最小或最大延时的路径。

时序扇入与扇出 (*timing fanin and fanout*)

见 *fanin* 与 *fanout*。

时序模型 (*timing model*)

含块时序特性但不一定含功能信息的电路模型。常用于层次化时序分析, 因对整个超大芯片做单一 *flat* 分析可能过于耗时。

转换时间 (*transition time*)

信号从低到高或从高到低变化所需时间。

传递扇入与扇出 (*transitive fanin and fanout*)

见 fanin 与 fanout。

透明 (*transparent*)

电平敏感锁存器门控输入被断言时的状态。在此期间，信号从输入传播到输出，器件的行为如同 buffer。

真路径 (*true path*)

逻辑上可能工作的路径；可敏化并能传播数据的路径。

不确定性（时钟不确定性）(*uncertainty (clock uncertainty)*)

时钟边沿相对标称理想时钟时间的变化量。

向量 (*vector*)

为测试或分析施加到器件输入的特定值集，或器件输出的结果值集；或用于敏化设计中路径进行分析的此类值序列。

受害网 (*victim net*)

因与另一网（aggressor 网）之间的寄生交叉电容而产生不良串扰效应的网。

电压区域 (*voltage area*)

芯片中使用与其他部分不同供电电压的物理区域。

线负载模型 (*wire load model*)

布局前、无反标延时或寄生信息时用于时序分析的网电阻电容模型。PrimeTime 根据每条网的扇出与库指定参数估算线负载。

工作进程 (*worker process*)

在 DMSA 与 HyperGrid 分布式分析中，执行分布式分析工作的一组进程之一，由管理进程控制。